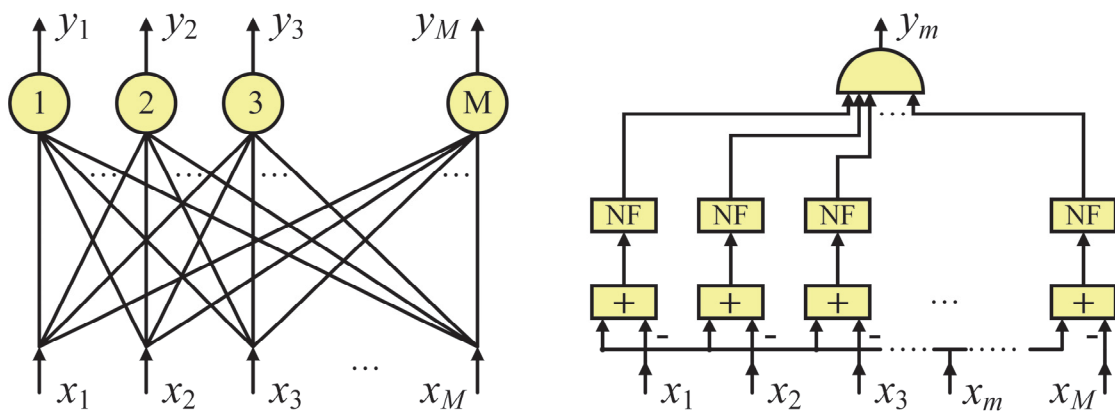


Matlab and Worked Problems in Intelligent Computing and System Design

Edition 1.4

Hon Keung Kwan



Matlab and Worked Problems in Intelligent Computing and System Design

Edition 1.4

Hon Keung Kwan

Professor
University of Windsor
Department of Electrical and Computer Engineering
401 Sunset Avenue, Windsor, Ontario, Canada N9B 3P4

Publication date: Edition 1.4, 24 January 2019.

ISBN13: 9781988307084

Copyright © 2019 by Hon Keung Kwan

All rights reserved. This work in whole or part cannot be translated or copied or reproduced or distributed in any form or by any means, or stored in a data base or retrieval system, without the prior written permission of the copyright holder.

dfisp.org

Preface

“Matlab and Worked Problems in Intelligent Computing and System Design” is an accompanying eBook for the “Intelligent Computing and System Design” eBook written by the same author and are designed to be used together to reinforce learning.

These two eBooks form a set of senior undergraduate and graduate textbooks on Intelligent Computing and System Design for students in engineering, computer science, science, and business, and other programs; and contain fundamental and advanced topics to meet the curriculum requirements of both undergraduate and graduate courses. These two eBooks are also suitable for engineers and computing professionals who want to self-learn this interesting and useful subject.

No specific prerequisites are required but a background of elementary courses in linear algebra and calculus would be useful.

The topics of Intelligent Computing and System Design covered by these two eBooks are clearly described and explained with the aid of step-by-step hand-calculation examples and problems, Matlab examples and problems, Matlab design examples and problems, equations, and self-explanatory figures and tables.

A complete list of all the chapters and problems and their titles are listed under the table of contents; notations for numbering equations, tables, and figures, and terms for examples and problems are explained immediate after the table of contents.

A clickable table of contents also comes with the eBook to allow a quick link of a specific chapter and problem.

The format of these two eBooks has been designed such that their PDF (Portable Document Format) files are suitable to be used directly by an instructor in lecture classes and tutorial classes without the needs for preparing PowerPoint presentation files.

Note that an “Indent” is used to indicate the start of a group of related short paragraphs for an easier view.

The contents of these eBooks were developed under classroom and research environments for courses taught by the author at the University of Windsor. Input and/or feedback regarding typo errors and improvements would be appreciated.

Hon Keung Kwan, 24 January 2019

<http://www.uwindsor.ca/isplab/>

Matlab and Worked Problems in Intelligent Computing and System Design

Edition 1.4

Hon Keung Kwan

	Pg
Front cover	1
Title	1
Preface	2
Table of Contents	6
Notations for Equations, Tables, and Figures	1
Terms for Problems and Examples	
Chapter 1 Introduction	-
Chapter 2 One-Layer Feedforward Neural Networks	14
P2.1 8-input to 8-output pattern associator with Hebb-rule learning and recall	
P2.2 4-input to 1-output pattern associator with Hebb-rule and delta-rule learnings and recalls	
P2.3 Identical weight matrix for Hebb rule and delta rule under zero initial weights and orthogonal patterns	
P2.4 Output vector change due to simultaneous K weight updates in delta-rule trained pattern associator	
P2.5 Repeated learning for difficult-to-learn pattern in Hebb-rule trained pattern associator	

Chapter 3	One-Layer Recurrent Neural Networks	16
P3.1	Input to output relationship of Hebb-rule trained autoassociator	
P3.2	Hebb-rule trained autoassociator learning and recalls	
P3.3	Matlab-programming of pattern rectification and pattern completion properties of delta-rule trained autoassociator	
P3.4	Delta-rule trained autoassociator learning and recalls	
P3.5	Weight change and learning properties of delta-rule trained autoassociator	
Chapter 4	Multilayer Neural Networks	-
Chapter 5	Associative Memories	7
P5.1	Original and complementary bipolar vector-pair storage	
P5.2	BAM learning and recall abilities	
P5.3	TAM learning and recall abilities	
Chapter 6	Self-Organizing Maps	33
P6.1	1-d Mexican hat function derivation from wavelet function	
P6.2	Bubble formation by SOM clustering	
P6.3	6 to 3 binary vector clustering by SOM	
P6.4	16 to 5 bipolar vectors clustering by SOM	
P6.5	2-D self-organizing map	
Chapter 7	Learning Vector Quantization	27
P7.1	2-input vectors clustering to 5 classes by LVQ	
P7.2	4 binary vectors clustering to 2 classes by LVQ	

P7.3 6 binary vectors clustering to 2 classes by LVQ

P7.4 6 binary vectors clustered to 3 classes by LVQ

P7.5 8 binary vectors clustering to 4 classes by LVQ

Chapter 8 Radial Basis Function Networks 8

P8.1 Interpolation RBF network learning and recall

P8.2 Generalized RBF network learning and recall

Chapter 9 Fuzzy Sets 7

P9.1 Derivation of fuzzy complement of fuzzy set

P9.2 Membership function terminologies

P9.3 Width of modified Gaussian membership function

P9.4 Derivatives of Gaussian membership function

P9.5 Complement, union, and interception of 2 fuzzy sets

Chapter 10 Fuzzy Logic and Reasoning 29

P10.1 Binary fuzzy relation matrix computation

P10.2 Properties of binary relations using Max-Min composition

P10.3 Properties of binary relations using Max-Product composition

P10.4 Verifying properties of binary relations using Max-Min composition

P10.5 Verifying properties of binary relations using Max-Product composition

P10.6 Computation and interpretation of fuzzy relation matrices

P10.7 Fuzzy relation matrix computation using different compositions

P10.8 Output MF derivation of 1-rule 1-input 1-output fuzzy reasoning system

P10.9	Output MF derivation of 1-rule 2-input 1-output fuzzy reasoning system	
P10.10	Output MF derivation of 3-rule 3-input 3-output fuzzy reasoning system	
P10.11	Centroid computation of aggregated output MF of 2-rule 2-input 1-output Mamdani fuzzy System	
Chapter 11	Fuzzy Systems	17
P11.1	Derivation of inverse Gaussian membership function	
P11.2	Derivation of inverse generalized bell membership function	
P11.3	Inverses and gradients of monotonic membership functions	
P11.4	Defuzzified output computation of Mamdani fuzzy system with fuzzy inputs	
Chapter 12	Fuzzy-Neural Networks	-
Chapter 13	Genetic Algorithms	24
P13.1	Average fitness value calculation of schemas	
P13.2	Formula for average fitness value calculation of schemas	
P13.3	Maximum number of distinct strings in population of N binary strings of L -bit length	
P13.4	Union and interception operations on schemas	
P13.5	GA with basic operators	
P13.6	Roulette-wheel selection method	
P13.7	Elite strategy by deterministic sampling method	
P13.8	Elite strategy by remainder stochastic sampling with replacement	
P13.9	Comparisons of improved reproduction methods	

- P13.10 Multiple-point crossover operation for retaining quality schemas
- P13.11 Inversion operator for better codes and better sets
- P13.12 Ranking procedure to limit uncontrolled spread of top chromosomes I
- P13.13 Ranking procedure to limit uncontrolled spread of top chromosomes II
- P13.14 Sharing function to limit uncontrolled growth of similar strings
- References

Chapter 14 Differential Evolution 16

- P14.1 2-variable sphere function minimization using DE
- P14.2 1-dimensional sine function interpolation using DE

Chapter 15 Particle Swarm Optimization 35

- P15.1 Second-order polynomial function interpolation using PSO
- P15.2 Third-order polynomial function interpolation using PSO

Chapter 16 Simulated Annealing -

Chapter 17 Coding, Initialization, and Constraints 4

- P17.1 Calculation of maximum 5-parameter codes of bipolar bits
- P17.2 Improved parameter initialization method with continuous and discrete values
- P17.3 Violating-index-based objective function

Chapter 18 Multiplierless Neural Networks 4

- P18.1 Second-order nonlinear function derivations I

P18.2 Second-order nonlinear function derivations II

Back cover	1
------------	---

Total page	253 pages (68 problems)
------------	-------------------------

Pg: Total page number of a chapter

The front cover shows the structure of the one-layer Max/Min Net¹ that requires the time of two-input adder, one hardlimiting function, and one AND gate to compute the maximum or minimum of any arbitrary set of M real values. ¹ H. K. Kwan, "One-layer feedforward neural network for fast maximum/minimum determination", Electronics Letters, 13 August 1992, Volume 28, Number 17, pages 1583-1585.

Notations for Equations, Tables, and Figures

Symbol	Description	Example
C	Chapter number	
X	Example number	
N	Table/Figure/Equation number	
P	Problem	
(PC.X.N)	Nth equation of Problem C.X	(P4.1.1)
Table PC.X.N	Nth table of Problem C.X	Table P4.2.2
Figure PC.X.N	Nth figure of Problem C.X	Figure P4.3.3

Terms for Examples and Problems

Term	Description
Example	As listed in main eBook
Design Example	As listed in main eBook
Problem	As listed in Matlab and Worked Problems eBook

Chapter 2

One-Layer Feedforward Neural Networks

Problem 2.1 (8-input to 8-output pattern associator with Hebb-rule learning and recall)

Assuming that a pattern associator is to be used for storing the three pattern-pairs as shown in Table P2.1.1:

Table P2.1.1

k	$\mathbf{X}_k$	$\mathbf{Y}_k$
1	[1 1 1 1 1 1 1 1]	[1 1 1 1 -1 -1 -1 -1]
2	[1 -1 1 -1 1 -1 1 -1]	[1 -1 1 -1 -1 1 -1 1]
3	[1 1 -1 -1 1 1 -1 -1]	[1 1 -1 -1 -1 -1 1 1]

(a) Determine the values of all the weights of the eight-input to eight-output pattern associator using the Hebb rule.

(b) Using the bipolar step function at the output of each neuron:

(i) Determine the output vector due to each of the following test input vectors, $\mathbf{X}_k$ for $k = 1$ to 3.

(ii) Determine the test output vectors of the following test input vectors as given in Table P2.1.2.

Table P2.1.2

$\mathbf{X}_{1t}$	[1 -1 -1 1 1 1 1 1]
$\mathbf{X}_{2t}$	[-1 1 1 -1 1 -1 1 -1]
$\mathbf{X}_{3t}$	[1 -1 -1 -1 1 1 -1 1]

Solution

(a)

$$\begin{aligned}
 \mathbf{W} &= \varepsilon \sum_{k=1}^3 \mathbf{X}_k^T \mathbf{Y}_k \\
 &= \varepsilon \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} +1 & +1 & +1 & +1 & -1 & -1 & -1 & -1 \end{bmatrix} \\
 &\quad + \varepsilon \begin{bmatrix} +1 \\ -1 \\ +1 \\ -1 \\ +1 \\ -1 \\ +1 \\ -1 \end{bmatrix} \begin{bmatrix} +1 & -1 & +1 & -1 & -1 & +1 & -1 & +1 \end{bmatrix} \\
 &\quad + \varepsilon \begin{bmatrix} +1 \\ +1 \\ -1 \\ -1 \\ +1 \\ +1 \\ -1 \\ -1 \end{bmatrix} \begin{bmatrix} +1 & +1 & -1 & -1 & -1 & -1 & +1 & +1 \end{bmatrix} \\
 &= \varepsilon \begin{bmatrix} +3 & +1 & +1 & -1 & -3 & -1 & -1 & +1 \\ +1 & +3 & -1 & +1 & -1 & -3 & +1 & -1 \\ +1 & -1 & +3 & +1 & -1 & +1 & -3 & -1 \\ -1 & +1 & +1 & +3 & +1 & -1 & -1 & -3 \\ +3 & +1 & +1 & -1 & -3 & -1 & -1 & +1 \\ +1 & +3 & -1 & +1 & -1 & -3 & +1 & -1 \\ +1 & -1 & +3 & +1 & -1 & +1 & -3 & -1 \\ -1 & +1 & +1 & +3 & +1 & -1 & -1 & -3 \end{bmatrix}
 \end{aligned}$$

(b)(i)

$$\begin{aligned}
\mathbf{Y}_{1t} &= G_h[\mathbf{X}_1 \quad \mathbf{W}] = G_h\left[\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \mathbf{W}\right] \\
&= G_h\left[\varepsilon\begin{bmatrix} 8 & 8 & 8 & 8 & -8 & -8 & -8 & -8 \end{bmatrix}\right] \\
&= \begin{bmatrix} 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 \end{bmatrix} = \mathbf{Y}_1 \\
\mathbf{Y}_{2t} &= G_h[\mathbf{X}_2 \quad \mathbf{W}] = G_h\left[\begin{bmatrix} 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \end{bmatrix} \mathbf{W}\right] \\
&= G_h\left[\varepsilon\begin{bmatrix} 8 & -8 & 8 & -8 & -8 & 8 & -8 & 8 \end{bmatrix}\right] \\
&= \begin{bmatrix} 1 & -1 & 1 & -1 & -1 & 1 & -1 & 1 \end{bmatrix} = \mathbf{Y}_2 \\
\mathbf{Y}_{3t} &= G_h[\mathbf{X}_3 \quad \mathbf{W}] = G_h\left[\begin{bmatrix} 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 \end{bmatrix} \mathbf{W}\right] \\
&= G_h\left[\varepsilon\begin{bmatrix} 8 & 8 & -8 & -8 & -8 & -8 & 8 & 8 \end{bmatrix}\right] \\
&= \begin{bmatrix} 1 & 1 & -1 & -1 & -1 & -1 & 1 & 1 \end{bmatrix} = \mathbf{Y}_3
\end{aligned}$$

(b)(ii)

$$\begin{aligned}
\mathbf{Y}_{1t} &= G_h[\mathbf{X}_{1t} \quad \mathbf{W}] = G_h\left[\begin{bmatrix} 1 & -1 & -1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \mathbf{W}\right] \\
&= G_h\left[\varepsilon\begin{bmatrix} 4 & 4 & 4 & 4 & -4 & -4 & -4 & -4 \end{bmatrix}\right] \\
&= \begin{bmatrix} 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 \end{bmatrix} = \mathbf{Y}_1 \\
\mathbf{Y}_{2t} &= G_h[\mathbf{X}_2 \quad \mathbf{W}] = G_h\left[\begin{bmatrix} -1 & 1 & 1 & -1 & 1 & -1 & 1 & -1 \end{bmatrix} \mathbf{W}\right] \\
&= G_h\left[\varepsilon\begin{bmatrix} 4 & -4 & 4 & -4 & -4 & 4 & -4 & 4 \end{bmatrix}\right] \\
&= \begin{bmatrix} 1 & -1 & 1 & -1 & -1 & 1 & -1 & 1 \end{bmatrix} = \mathbf{Y}_2 \\
\mathbf{Y}_{3t} &= G_h[\mathbf{X}_3 \quad \mathbf{W}] = G_h\left[\begin{bmatrix} 1 & -1 & -1 & -1 & 1 & 1 & -1 & 1 \end{bmatrix} \mathbf{W}\right] \\
&= G_h\left[\varepsilon\begin{bmatrix} 4 & 4 & -4 & -4 & -4 & -4 & 4 & 4 \end{bmatrix}\right] \\
&= \begin{bmatrix} 1 & 1 & -1 & -1 & -1 & -1 & 1 & 1 \end{bmatrix} = \mathbf{Y}_3
\end{aligned}$$

Problem 2.2 (4-input to 1-output pattern associator with Hebb-rule and delta-rule learnings and recalls)

Consider a four-input to one-output pattern associator for training the NAND Gate problem as shown in Table P2.2.1.

(a) Assuming the Hebb rule is used for training, the bipolar step function $G_h[\cdot]$ (with $L = 1$) is used for recall, zero initial weights, and

$\varepsilon = 1/N$ (N is the number of bipolar elements in an input pattern), calculate the weight matrix, and verify the recall correctness.

(b) Assuming the delta rule is used for training, the bipolar step function $G_h[.]$ (with $L = 1$) is used for training and recall, zero initial weights, weight update is carried out at the end of each pattern presentation, and all computations are rounded to two decimal places, obtain the weight matrix with a minimum number of epochs until the recall outputs of all original input patterns are correct.

Table P2.2.1 NAND Gate problem.

k	$\mathbf{X}_k$	$\mathbf{Y}_k$
1	$[-1 \ -1]$	$[1]$
2	$[-1 \ 1]$	$[1]$
3	$[1 \ -1]$	$[1]$
4	$[1 \ 1]$	$[-1]$

Solution

(a) Calculating the weight matrix as

$$\begin{aligned} \mathbf{W} &= \varepsilon \sum_{k=1}^4 \mathbf{X}_k^T \mathbf{Y}_k = \varepsilon \left\{ \begin{bmatrix} -1 \\ -1 \end{bmatrix} [1] + \begin{bmatrix} -1 \\ 1 \end{bmatrix} [1] + \begin{bmatrix} 1 \\ -1 \end{bmatrix} [1] + \begin{bmatrix} 1 \\ 1 \end{bmatrix} [-1] \right\} \\ &= \begin{bmatrix} -2\varepsilon \\ -2\varepsilon \end{bmatrix} = \begin{bmatrix} -1 \\ -1 \end{bmatrix} \end{aligned}$$

Using $G_h[.]$, the outputs are recalled as

$$\mathbf{Y}'_k = G_h[\mathbf{X}_k \mathbf{W}] \quad \text{for } k = 1, 2, 3, 4$$

$$k = 1,$$

$$\mathbf{Y}'_1 = G_h[\mathbf{X}_1 \mathbf{W}] = G_h \left[\begin{bmatrix} -1 & -1 \end{bmatrix} \begin{bmatrix} -1 \\ -1 \end{bmatrix} \right] = G_h[2] = 1$$

$$k = 2,$$

$$\mathbf{Y}'_2 = G_h[\mathbf{X}_2 \mathbf{W}] = G_h \left[\begin{bmatrix} -1 & 1 \end{bmatrix} \begin{bmatrix} -1 \\ -1 \end{bmatrix} \right] = G_h[0] = 1$$

$$k = 3,$$

$$\mathbf{Y}'_3 = G_h[\mathbf{X}_3 \mathbf{W}] = G_h \left[\begin{bmatrix} 1 & -1 \end{bmatrix} \begin{bmatrix} -1 \\ -1 \end{bmatrix} \right] = G_h[0] = 1$$

$$k = 4,$$

$$\mathbf{Y}'_4 = G_h[\mathbf{X}_4 \mathbf{W}] = G_h \left[\begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} -1 \\ -1 \end{bmatrix} \right] = G_h[-2] = -1$$

Hence,

$$\mathbf{Y}' = [1 \quad 1 \quad 1 \quad -1]$$

All the output can be recalled correctly.

(b) The network is started with zero initial weights. When the pattern 1 is presented, from Table P2.2.2, the output is

$$y'_{11} = G_h \left[\sum_{i=1}^2 x_{i1} w_{i1} \right] = G_h[(-1)(0) + (-1)(0)] = 1$$

The output error is

$$e_{11} = y_{11} - y'_{11} = 1 - 1 = 0$$

Hence,

$$\Delta w_{i1} = \varepsilon x_{i1} e_{11} = 0 \quad \text{for } i = 1 \text{ to } 2$$

$$w_{i1} = w_{i1} + \Delta w_{i1} \quad \text{for } i = 1 \text{ to } 2$$

$$[w_{11} \quad w_{21}]^T = [0 + \Delta w_{11} \quad 0 + \Delta w_{21}]^T = [0 \quad 0]^T$$

Then, the training pattern 2 is presented and so on. The computational steps are summarized in Table P2.2.2 as

Table 2.2.2

ep	k	x_{1k}	x_{2k}	y_{1k}	y'_{1k}	e_{1k}	Δw_{11}	Δw_{21}	w_{11}	w_{21}
1	1	-1	-1	1	1	0	0	0	0	0
1	2	-1	1	1	1	0	0	0	0	0
1	3	1	-1	1	1	0	0	0	0	0
1	4	1	1	-1	1	-2	-1	-1	-1	-1
		SSE = 4								
2	1	-1	-1	1	1	0	0	0	-1	-1
2	2	-1	1	1	1	0	0	0	-1	-1
2	3	1	-1	1	1	0	0	0	-1	-1
2	4	1	1	-1	-1	0	0	0	-1	-1
		SSE = 0								

The weight vector obtained is

$$\mathbf{W} = [-1.00 \quad -1.00]^T$$

$$\mathbf{Y}'_k = G_h[\mathbf{X}_k \mathbf{W}] \quad \text{for } k = 1, 2, 3, 4$$

$$\mathbf{Y}' = [1 \quad 1 \quad 1 \quad -1]^T$$

Thus, all the training patterns can be recalled correctly when using the weights obtained by the delta rule using the bipolar step function.

Problem 2.3 (Identical weight matrix for Hebb rule and delta rule under zero initial weights and orthogonal patterns)

Give the derivations to show that for zero initial weights and orthogonal input patterns, the weight matrix obtained by both the Hebb rule and the delta rule is identical after the first time presentation.

Solution

For zero initial weights, after the 1st pattern-pair, $\mathbf{X}_1$ and $\mathbf{Y}_1$, has been presented, from (2.17a)-(2.18a), we have

$$w_{ij} = \varepsilon x_{i1} (y_{j1} - y'_{j1}) \quad (\text{P2.3.1})$$

where y_{j1} and y'_{j1} represent, respectively, the ideal and obtained output at the j th output neuron during the presentation of the 1st pattern-pair.

Assuming the bipolar ramp function $G_r[.]$ is used for recall, and for zero initial weights, $y'_{j1} = 0$. Hence (P2.3.1) becomes

$$w_{ij} = \varepsilon x_{i1} y_{j1} \quad (\text{P2.3.2})$$

After the 2nd pattern-pair, $\mathbf{X}_2$ and $\mathbf{Y}_2$, has been presented, we have

$$\begin{aligned} w_{ij} &= \varepsilon x_{i1} y_{j1} + \varepsilon x_{i2} (y_{j2} - y'_{j2}) \\ &= \varepsilon x_{i1} y_{j1} + \varepsilon x_{i2} \left(y_{j2} - G_r \left[\sum_{i=1}^N (\varepsilon x_{i1} y_{j1}) x_{i2} \right] \right) \quad (\text{P2.3.3}) \\ &= \varepsilon x_{i1} y_{j1} + \varepsilon x_{i2} \left(y_{j2} - G_r \left[\varepsilon y_{j1} \sum_{i=1}^N (x_{i1} x_{i2}) \right] \right) \end{aligned}$$

If $\mathbf{X}_1$ and $\mathbf{X}_2$ are orthogonal, (P2.3.3) becomes

$$w_{ij} = \varepsilon \sum_{i=1}^2 x_{ik} y_{ik} \quad (\text{P2.3.4})$$

In general, for orthogonal input patterns and first time presentation, after K pattern-pairs training,

$$w_{ij} = \varepsilon \sum_{i=1}^K x_{ik} y_{ik} \quad (\text{P2.3.5a})$$

The corresponding weight matrix is

$$\mathbf{W} = \varepsilon \sum_{k=1}^K \mathbf{X}_k^T \mathbf{Y}_k \quad (\text{P2.3.5b})$$

Problem 2.4 (Output vector change due to simultaneous K weight updates in delta-rule trained pattern associator)

For the delta rule, give the derivations to show that the resultant change in the output vector due to simultaneous K weight updates is given by

$$\Delta \mathbf{Y}_t = G_r \left[c \sum_{k=1}^K \mathbf{E}_k (\mathbf{X}_k \mathbf{X}_t^T) / N \right]$$

Explain the implications of the derived expression.

Solution

From (2.16a), the resultant change in an output j due to a weight change Δw_{ij} between an input i and the output j is

$$\Delta y_{jt} = G_r \left[\sum_{i=1}^N x_{it} \Delta w_{ij} \right] \quad (\text{P2.4.1})$$

For K learning pattern-pairs and all the K weight updates are carried out at the end of a cycle, substituting Δw_{ij} of (2.18a) into (P2.4.1), we obtain

$$\begin{aligned} \Delta y_{jt} &= G_r \left[\sum_{i=1}^N x_{it} \sum_{k=1}^K \Delta w_{ij} \right] = G_r \left[\varepsilon \sum_{i=1}^N x_{it} \sum_{k=1}^K x_{ik} e_{jk} \right] \\ &= G_r \left[\varepsilon \sum_{k=1}^K e_{jk} \sum_{i=1}^N x_{ik} x_{it} \right] \\ &= G_r \left[c \sum_{k=1}^K e_{jk} (\mathbf{X}_k \mathbf{X}_t^T) / N \right] \end{aligned} \quad (\text{P2.4.2})$$

From (P2.4.2), the change in the output vector $\mathbf{Y}_t$

$$\Delta \mathbf{Y}_t = G_r \left[c \sum_{k=1}^K \mathbf{E}_k (\mathbf{X}_k \mathbf{X}_t^T) / N \right] \quad (\text{P2.4.3})$$

From (P2.4.3), for $\mathbf{X}_t = \mathbf{X}_k$, $\Delta \mathbf{Y}_t$ depends on $\mathbf{E}_k$. For $-1 < (\mathbf{X}_k \mathbf{X}_t^T) / N < 1$, $\Delta \mathbf{Y}_t$ depends on $\mathbf{E}_k$ and $(\mathbf{X}_k \mathbf{X}_t^T) / N$.

If each of the weight updates within a cycle is carried out separately, the above analysis holds for individual pattern-pair as if $K = 1$.

Problem 2.5 (Repeated learning for difficult-to-learn pattern in Hebb-rule trained pattern associator)

Given the definition of the bipolar ramp function $G_r(z)$ as

$$G_r(z) = \begin{cases} +1 & \text{for } z \geq L \\ \theta_g z & \text{for } -L \leq z < L \\ -1 & \text{for } z < -L \end{cases}$$

Assuming that $G_r(z)$ with $L = 0.4$, $\theta_g = 1/L$, and a learning rate $\varepsilon = 0.25$ are used to learn the four-input and one-output pattern associator as given in Table P2.5.1.

Table P2.5.1

k	$\mathbf{X}_k$	$\mathbf{Y}_k$
1	[-1 1 1 -1]	1
2	[1 -1 -1 1]	-1
3	[1 -1 1 -1]	1
4	[1 1 1 -1]	-1

- Use the Hebb rule to calculate the weight vector $\mathbf{W}_0$.
- Use the bipolar ramp function $G_r(z)$ at the outputs of the pattern associator and $\mathbf{W}_0$ to recall the outputs of the four pattern-pairs.
- State and explain your observations in the recall outputs obtained in (b).
- State the Hebb rule learning strategy that should be applied in a pattern associator when a difficult-to-learn pattern-pair emerges.

(e) Using the bipolar ramp function $G_r(z)$, apply the Hebb rule learning strategy as stated in (d) until all the four given pattern-pairs have been correctly learnt.

(f) State your observations by comparing different sets of internal outputs, z_k for $k = 1$ to 4 obtained in (e).

(g) After learning, state and explain the nonlinear function that should be used for robust recall.

Solution

(a) Using the Hebb rule, the weight vector $\mathbf{W}_0$ can be calculated as

$$\begin{aligned}
 \mathbf{W}_0 &= \varepsilon \sum_{k=1}^4 \mathbf{X}_t^T \mathbf{Y}_k \\
 &= \varepsilon \begin{bmatrix} -1 \\ 1 \\ 1 \\ -1 \end{bmatrix} [1] + \varepsilon \begin{bmatrix} 1 \\ -1 \\ -1 \\ 1 \end{bmatrix} [-1] + \varepsilon \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix} [1] + \varepsilon \begin{bmatrix} 1 \\ 1 \\ 1 \\ -1 \end{bmatrix} [-1] \\
 &= \varepsilon \begin{bmatrix} (-1) \times 1 + 1 \times (-1) + 1 \times 1 + 1 \times (-1) \\ 1 \times 1 + (-1) \times (-1) + (-1) \times 1 + 1 \times (-1) \\ 1 \times 1 + (-1) \times (-1) + 1 \times 1 + 1 \times (-1) \\ (-1) \times 1 + 1 \times (-1) + (-1) \times 1 + (-1) \times (-1) \end{bmatrix} \\
 &= \varepsilon [-2 \quad 0 \quad 2 \quad -2]^T
 \end{aligned}$$

(b) Using the bipolar ramp function $G_r(z)$ and the weight $\mathbf{W}_0$, all the outputs can be recalled as

$$\begin{aligned}
 \mathbf{Y}_{1t}^0 &= G_r[\mathbf{X}_1 \mathbf{W}_0] \\
 &= G_r[0.25((-1) \times (-2) + 1 \times 0 + 1 \times 2 + (-1) \times (-2))] = G_r[1.5] = 1 = \mathbf{Y}_1
 \end{aligned}$$

$$\begin{aligned}
 \mathbf{Y}_{2t}^0 &= G_r[\mathbf{X}_2 \mathbf{W}_0] \\
 &= G_r[0.25(1 \times (-2) + (-1) \times 0 + (-1) \times 2 + 1 \times (-2))] = G_r[-1.5] = -1 = \mathbf{Y}_2
 \end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{3t}^0 &= G_r[\mathbf{X}_3 \mathbf{W}_0] \\
&= G_r[0.25(1 \times (-2) + (-1) \times 0 + 2 \times (-1) + (-1) \\
&\quad \times (-2))] = G_r[0.5] = 1 = \mathbf{Y}_3
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{4t}^0 &= G_r[\mathbf{X}_4 \mathbf{W}_0] \\
&= G_r[0.25(1 \times (-2) + 1 \times 0 + 1 \times 2 + (-1) \\
&\quad \times (-2))] = G_r[0.5] = 1 \neq \mathbf{Y}_4
\end{aligned}$$

(c) Observation: It can be observed that the first three pattern-pairs, $\mathbf{X}_k - \mathbf{Y}_k$ for $k = 1$ to 3 can be correctly recalled whereas the fourth pattern $\mathbf{X}_4 - \mathbf{Y}_4$ cannot be correctly recalled.

Explanation: Among the four given pattern-pairs, $\mathbf{X}_1 - \mathbf{Y}_1$ and $\mathbf{X}_2 - \mathbf{Y}_2$ are complementary to each other; $\mathbf{X}_3$ is orthogonal to $\mathbf{X}_1$ and $\mathbf{X}_2$; the normalized correlations of $\mathbf{X}_4$ to $\mathbf{X}_1$, $\mathbf{X}_2$, and $\mathbf{X}_3$ are respectively, 0.5, -0.5, and 0.5.

In other words, $\mathbf{X}_4$ is a difficult to learn pattern with a 50% similarity to all other input patterns $\mathbf{X}_1$, $\mathbf{X}_2$, and $\mathbf{X}_3$.

(d) When a difficult-to-learn pattern pair emerges in a pattern associator, the repeated learning strategy in the Hebb rule should be applied.

(e) Step 1: Applying repeated learning to $\mathbf{X}_4 - \mathbf{Y}_4$ pair, we obtain

$$\mathbf{W}_1 = \varepsilon \sum_{k=1}^4 \mathbf{X}_t^T \mathbf{Y}_k + \varepsilon \mathbf{X}_4^T \mathbf{Y}_4 = \varepsilon \sum_{k=1}^4 \mathbf{X}_t^T \mathbf{Y}_k + \varepsilon \begin{bmatrix} 1 \\ 1 \\ 1 \\ -1 \end{bmatrix} [-1] = \varepsilon \begin{bmatrix} -3 \\ -1 \\ 1 \\ -1 \end{bmatrix}$$

Using the bipolar ramp function $G_r(z)$, all the outputs can be recalled from the weight vector $\mathbf{W}_1$ as

$$\begin{aligned}
\mathbf{Y}_{1t}^1 &= G_r[\mathbf{X}_1 \mathbf{W}_1] \\
&= G_r[0.25\{(-1) \times (-3) + 1 \times (-1) + 1 \times 1 + (-1) \\
&\quad \times (-1)\}] = G_r[1.0] = 1 = \mathbf{Y}_1
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{2t}^1 &= G_r[\mathbf{X}_2 \mathbf{W}_1] \\
&= G_r[0.25\{1 \times (-3) + (-1) \times (-1) + (-1) \times 1 + 1 \\
&\quad \times (-1)\}] = G_r[-1.0] = -1 = \mathbf{Y}_2
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{3t}^1 &= G_r[\mathbf{X}_3 \mathbf{W}_1] \\
&= G_r[0.25\{1 \times (-3) + (-1) \times (-1) + 1 \times 1 + (-1) \\
&\quad \times (-1)\}] = G_r[0.0] = 0 \neq \mathbf{Y}_3
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{4t}^1 &= G_r[\mathbf{X}_4 \mathbf{W}_1] \\
&= G_r[0.25\{1 \times (-3) + 1 \times (-1) + 1 \times 1 + (-1) \\
&\quad \times (-1)\}] = G_r[-0.5] = -1 = \mathbf{Y}_4
\end{aligned}$$

Step 2: To learn the difficult-to-learn pattern-pair $\mathbf{X}_3 - \mathbf{Y}_3$, repeated learning is applied as

$$\begin{aligned}
\mathbf{W}_2 &= \left\{ \varepsilon \sum_{k=1}^4 \mathbf{X}_t^T \mathbf{Y}_k + \varepsilon \mathbf{X}_4^T \mathbf{Y}_4 \right\} + \varepsilon \mathbf{X}_3^T \mathbf{Y}_3 = \varepsilon \begin{bmatrix} -3 \\ -1 \\ 1 \\ -1 \end{bmatrix} + \varepsilon \begin{bmatrix} 1 \\ -1 \\ 1 \\ -1 \end{bmatrix} [1] \\
&= \varepsilon \begin{bmatrix} -2 \\ -2 \\ 2 \\ -2 \end{bmatrix}
\end{aligned}$$

Using the bipolar ramp function $G_r(z)$, all the outputs can be recalled from the weight vector $\mathbf{W}_2$ as

$$\begin{aligned}
\mathbf{Y}_{1t}^2 &= G_r[\mathbf{X}_1 \mathbf{W}_2] \\
&= G_r[0.25\{(-1) \times (-2) + 1 \times (-2) + 1 \times 2 + (-1) \\
&\quad \times (-2)\}] = G_r[1.0] = 1 = \mathbf{Y}_1
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{2t}^2 &= G_r[\mathbf{X}_2 \mathbf{W}_2] \\
&= G_r[0.25\{1 \times (-2) + (-1) \times (-2) + (-1) \times 2 + 1 \\
&\quad \times (-2)\}] = G_r[-1.0] = -1 = \mathbf{Y}_2
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{3t}^2 &= G_r[\mathbf{X}_3 \mathbf{W}_2] \\
&= G_r[0.25\{1 \times (-2) + (-1) \times (-2) + 1 \times 2 + (-1) \\
&\quad \times (-2)\}] = G_r[1.0] = 1 = \mathbf{Y}_3
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{4t}^2 &= G_r[\mathbf{X}_4 \mathbf{W}_2] \\
&= G_r[0.25\{1 \times (-2) + 1 \times (-2) + 1 \times 2 + (-1) \\
&\quad \times (-2)\}] = G_r[0.0] = 0 \neq \mathbf{Y}_4
\end{aligned}$$

Step 3: To learn the difficult-to-learn pattern-pair $\mathbf{X}_4 - \mathbf{Y}_4$, repeated learning is applied as

$$\begin{aligned}
\mathbf{W}_3 \left\{ \varepsilon \sum_{k=1}^4 \mathbf{X}_k^T \mathbf{Y}_k + \varepsilon \mathbf{X}_4^T \mathbf{Y}_4 + \varepsilon \mathbf{X}_3^T \mathbf{Y}_3 \right\} + \varepsilon \mathbf{X}_4^T \mathbf{Y}_4 &= \varepsilon \begin{bmatrix} -2 \\ -2 \\ 2 \\ -2 \end{bmatrix} + \varepsilon \begin{bmatrix} 1 \\ 1 \\ 1 \\ -1 \end{bmatrix} [-1] \\
&= \varepsilon \begin{bmatrix} -3 \\ -3 \\ 1 \\ -1 \end{bmatrix}
\end{aligned}$$

Using the bipolar ramp function $G_r(z)$, all the outputs can be recalled from the weight vector $\mathbf{W}_3$ as

$$\begin{aligned}
\mathbf{Y}_{1t}^3 &= G_r[\mathbf{X}_1 \mathbf{W}_3] \\
&= G_r[0.25\{(-1) \times (-3) + 1 \times (-3) + 1 \times 1 + (-1) \\
&\quad \times (-1)\}] = G_r[0.5] = 1 = \mathbf{Y}_1
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{2t}^3 &= G_r[\mathbf{X}_2 \mathbf{W}_3] \\
&= G_r[0.25\{1 \times (-3) + (-1) \times (-3) + (-1) \times 1 + 1 \\
&\quad \times (-1)\}] = G_r[-0.5] = -1 = \mathbf{Y}_2
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{3t}^3 &= G_r[\mathbf{X}_3 \mathbf{W}_3] \\
&= G_r[0.25\{1 \times (-3) + (-1) \times (-3) + 1 \times 1 + (-1) \\
&\quad \times (-1)\}] = G_r[0.5] = 1 = \mathbf{Y}_3
\end{aligned}$$

$$\begin{aligned}
\mathbf{Y}_{4t}^3 &= G_r[\mathbf{X}_4 \mathbf{W}_3] \\
&= G_r[0.25\{1 \times (-3) + 1 \times (-3) + 1 \times 1 + (-1) \\
&\quad \times (-1)\}] = G_r[-1] = -1 = \mathbf{Y}_4
\end{aligned}$$

(f) By comparing the three different sets of internal outputs, z_k for $k = 1$ to 4 obtained in (e) using $\mathbf{W}_k$ for $k = 1$ to 3, the noise margins for the recall outputs of -1 and +1 before applying $G_r(z)$ are $[-0.5, 0.0]$ for $\mathbf{W}_1$, $[-1.0, 0.0]$ or $[0.0, 1.0]$ for $\mathbf{W}_2$, and $[-0.5, 0.5]$ for $\mathbf{W}_3$.

The noise margins $[-0.5, 0.5]$ obtained at the end by $\mathbf{W}_3$ exhibit equal spread on its left and right sides and therefore $\mathbf{W}_3$ has the best noise tolerant capability.

(g) After learning, the bipolar ramp function with $L = 0$ which is now equivalent to the bipolar step function should be used for robust recall.

It is because the noise margin of the bipolar ramp function with $L \neq 0$ can then be released to accommodate possible deviations that may arise in noisy input recall.

Chapter 3

One-Layer Recurrent Neural Networks

Problem 3.1 (Input to output relationship of Hebb-rule trained autoassociator)

Assuming all initial weights are zero, give the step-by-step derivation that the obtained test output vector, $\mathbf{Y}_t$, due to a test input vector, $\mathbf{X}_t$, of a Hebb-rule trained autoassociator is given by

$$\mathbf{Y}_t(n) = G \left[c \sum_{k=1}^K \mathbf{X}_k \{ \mathbf{X}_k \mathbf{Y}_t(n-1)^T / N \} \right]$$

Where $G[.]$ represents a nonlinear quantization function at each output; $\mathbf{X}_k$ for $k = 1$ to K represents the K patterns that have been trained in the autoassociator; N represents the number of elements in each pattern; and $c = N\epsilon$ in which ϵ denotes the learning rate parameter.

Solution

For a Hebb rule trained autoassociator, the weight matrix is

$$\mathbf{W} = \epsilon \sum_{k=1}^K (\mathbf{X}_k^T)(\mathbf{X}_k)$$

Given a test input vector $\mathbf{X}_t = \mathbf{Y}_t(1)$, the output vector at any iteration n is

$$\mathbf{Y}_t(n) = G [\mathbf{Y}_t(n-1) \mathbf{W}] = G \left[\epsilon \sum_{k=1}^K (\mathbf{Y}_t(n-1)) (\mathbf{X}_k^T) \mathbf{X}_k \right]$$

Thus

$$\begin{aligned}
 \mathbf{Y}_t(n) &= G [\mathbf{W}^T \mathbf{Y}_t(n-1)^T] = G \left[\epsilon \sum_{k=1}^K (\mathbf{X}_k^T) \mathbf{X}_k (\mathbf{Y}_t(n-1)^T) \right] \\
 &= G \left[N \epsilon \sum_{k=1}^K (\mathbf{X}_k^T) \{ (\mathbf{X}_k) (\mathbf{Y}_t(n-1)^T) / N \} \right] \\
 &= G \left[C \sum_{k=1}^K (\mathbf{X}_k^T) \{ (\mathbf{X}_k) (\mathbf{Y}_t(n-1)^T) / N \} \right]
 \end{aligned}$$

Problem 3.2 (Hebb-rule trained autoassociator learning and recalls)

Given the following three patterns which are to be stored in a Hebb rule trained autoassociator as

$$\begin{aligned}
 \mathbf{X}_1 &= [1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1] \\
 \mathbf{X}_2 &= [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1] \\
 \mathbf{X}_3 &= [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1]
 \end{aligned}$$

Assuming that the learning rate parameter, ϵ , is 0.125, the bipolar step function is used for recall, and the initial weights are zero:

- Calculate the weight matrix of the autoassociator.
- Check the correctness of the recall of each of the three patterns.
- Recall the corrupted version of each of $\mathbf{X}_1$, $\mathbf{X}_2$, and $\mathbf{X}_3$ as given by

$$\begin{aligned}
 \mathbf{X}_{t1} &= [-1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1] \\
 \mathbf{X}_{t2} &= [-1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1] \\
 \mathbf{X}_{t3} &= [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1]
 \end{aligned}$$

to illustrate the pattern rectification capability of the autoassociator.

- Recall the incomplete version of each of $\mathbf{X}_1$, $\mathbf{X}_2$, and $\mathbf{X}_3$ as given by

$$\begin{aligned}
 \mathbf{X}_{t4} &= [0 \quad 0 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1] \\
 \mathbf{X}_{t5} &= [0 \quad 0 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1] \\
 \mathbf{X}_{t6} &= [0 \quad 0 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1]
 \end{aligned}$$

to illustrate the pattern completion capability of the autoassociator.

Solution

(a)

$$\begin{aligned}
\mathbf{W} &= \epsilon \sum_{k=1}^3 \mathbf{x}_k^T \mathbf{x}_k \\
&= \epsilon \begin{bmatrix} 3 & 1 & 1 & -1 & 3 & 1 & 1 & -1 \\ 1 & 3 & -1 & 1 & 1 & 3 & -1 & 1 \\ 1 & -1 & 3 & 1 & 1 & -1 & 3 & 1 \\ -1 & 1 & 1 & 3 & -1 & 1 & 1 & 3 \\ 3 & 1 & 1 & -1 & 3 & 1 & 1 & -1 \\ 1 & 3 & -1 & 1 & 1 & 3 & -1 & 1 \\ 1 & -1 & 3 & 1 & 1 & -1 & 3 & 1 \\ -1 & 1 & 1 & 3 & -1 & 1 & 1 & 3 \end{bmatrix} \\
&= \begin{bmatrix} .375 & .125 & .125 & -.125 & .375 & .125 & .125 & -.125 \\ .125 & .375 & -.125 & .125 & .125 & .375 & -.125 & .125 \\ .125 & -.125 & .375 & .125 & .125 & -.125 & .375 & .125 \\ -.125 & .125 & .125 & .375 & -.125 & .125 & .125 & .375 \\ .375 & .125 & .125 & -.125 & .375 & .125 & .125 & -.125 \\ .125 & .375 & -.125 & .125 & .125 & .375 & -.125 & .125 \\ .125 & -.125 & .375 & .125 & .125 & -.125 & .375 & .125 \\ -.125 & .125 & .125 & .375 & -.125 & .125 & .125 & .375 \end{bmatrix}
\end{aligned}$$

(b)

$$\mathbf{X}_t = \mathbf{X}_1 = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_1$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1) = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}_1$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2) \mathbf{W} = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1]$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}_1$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3) \mathbf{W} = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1]$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}_1$$

...

$$\mathbf{X}_t = \mathbf{X}_2 = [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_2$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1) \mathbf{W} = [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1] \\ = \mathbf{X}_2$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2) \mathbf{W} = [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1]$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1] \\ = \mathbf{X}_2$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3) \mathbf{W} = [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1]$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1] \\ = \mathbf{X}_2$$

...

$$\mathbf{X}_t = \mathbf{X}_3 = [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_3$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1) \mathbf{W} = [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1] \\ = \mathbf{X}_3$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2) \mathbf{W} = [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1]$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1] \\ = \mathbf{X}_3$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3) \mathbf{W} = [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1]$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1] \\ = \mathbf{X}_3$$

...

The outputs of $\mathbf{X}_1$, $\mathbf{X}_2$, and $\mathbf{X}_3$ can be recalled correctly.

(c)

$$\mathbf{X}_t = \mathbf{X}_{t1} = [-1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_{t1}$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1)$$

$$= [.250 \quad .750 \quad .750 \quad 1.250 \quad .250 \quad .750 \quad .750 \quad 1.250]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1] = \mathbf{X}_1$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2) \mathbf{W}$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1] = \mathbf{X}_1$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3) \mathbf{W}$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1] = \mathbf{X}_1$$

...

$$\mathbf{X}_t = \mathbf{X}_{t2} = [-1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_{t2}$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1) =$$

$$[.250 \quad -1.250 \quad .750 \quad -.750 \quad .250 \quad -1.250 \quad .750 \quad -.750]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1] \\ = \mathbf{X}_2$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2) \mathbf{W}$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1] \\ = \mathbf{X}_2$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3) \mathbf{W}$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1] \\ = \mathbf{X}_2$$

...

$$\mathbf{X}_t = \mathbf{X}_{t3} = [-1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_{t3}$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1) =$$

$$[.250 \quad .750 \quad -1.250 \quad -.750 \quad .250 \quad .750 \quad -1.250 \quad -.750]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1] \\ = \mathbf{X}_3$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2) \mathbf{W}$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1] \\ = \mathbf{X}_3$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3) \mathbf{W}$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1] \\ = \mathbf{X}_3$$

...

Corrupted versions of $\mathbf{X}_1$, $\mathbf{X}_2$, and $\mathbf{X}_3$ can be recalled correctly.

(d)

$$\mathbf{X}_t = \mathbf{X}_{t4} = [0 \quad 0 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_{t4}$$

$$\mathbf{Z}_t(2) = \mathbf{Y}_t(1) =$$

$$[.500 \quad .500 \quad 1.000 \quad 1.000 \quad .500 \quad .500 \quad 1.000 \quad 1.000]$$

$$\mathbf{Y}_t(2) = G_h[\mathbf{Z}_t(2)] = [1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1] = \mathbf{X}_1$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2) \mathbf{W}$$

$$\mathbf{Y}_t(3) = G_h[\mathbf{Z}_t(3)] = [1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1] = \mathbf{X}_1$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3) \mathbf{W}$$

$$\mathbf{Y}_t(4) = G_h[\mathbf{Z}_t(4)] = [1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1] = \mathbf{X}_1$$

...

$$\mathbf{X}_t = \mathbf{X}_{t5} = [0 \quad 0 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_{t5}$$

$$\begin{aligned}\mathbf{Z}_t(2) &= \mathbf{Y}_t(1) \\ &= [.750 \quad -0.750 \quad .750 \quad -.750 \quad .750 \quad -0.750 \quad .750 \quad -.750]\end{aligned}$$

$$\begin{aligned}\mathbf{Y}_t(2) &= G_h[\mathbf{Z}_t(2)] = [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1] \\ &= \mathbf{X}_2\end{aligned}$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2) \mathbf{W}$$

$$\begin{aligned}\mathbf{Y}_t(3) &= G_h[\mathbf{Z}_t(3)] = [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1] \\ &= \mathbf{X}_2\end{aligned}$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3) \mathbf{W}$$

$$\begin{aligned}\mathbf{Y}_t(4) &= G_h[\mathbf{Z}_t(4)] = [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1] \\ &= \mathbf{X}_2\end{aligned}$$

...

$$\mathbf{X}_t = \mathbf{X}_{t6} = [0 \quad 0 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1]$$

$$\mathbf{Y}_t(1) = \mathbf{X}_{t6}$$

$$\begin{aligned}\mathbf{Z}_t(2) &= \mathbf{Y}_t(1) \\ &= [.500 \quad .500 \quad -1.000 \quad -1.0000 \quad .500 \quad .500 \quad -1.000 \quad -1.000]\end{aligned}$$

$$\begin{aligned}\mathbf{Y}_t(2) &= G_h[\mathbf{Z}_t(2)] = [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1] \\ &= \mathbf{X}_3\end{aligned}$$

$$\mathbf{Z}_t(3) = \mathbf{Y}_t(2) \mathbf{W}$$

$$\begin{aligned}\mathbf{Y}_t(3) &= G_h[\mathbf{Z}_t(3)] = [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1] \\ &= \mathbf{X}_3\end{aligned}$$

$$\mathbf{Z}_t(4) = \mathbf{Y}_t(3) \mathbf{W}$$

$$\begin{aligned}\mathbf{Y}_t(4) &= G_h[\mathbf{Z}_t(4)] = [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1] \\ &= \mathbf{X}_3\end{aligned}$$

...

Incomplete versions of $\mathbf{X}_1$, $\mathbf{X}_2$, and $\mathbf{X}_3$ can be recalled correctly.

Problem 3.3 (Matlab-programming of pattern rectification and pattern completion properties of delta-rule trained autoassociator)

Write a Matlab program to illustrate the pattern rectification and the pattern completion properties of delta-rule trained AA as described in Example 3.9.

Problem 3.4 (Delta-rule trained autoassociator learning and recalls)

A delta-rule trained autoassociator with nonzero diagonal weight matrix is to be used to learn the following three patterns:

$$\mathbf{X}_1 = \begin{bmatrix} -1 & -1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

$$\mathbf{X}_2 = \begin{bmatrix} 1 & -1 & -1 & -1 & -1 & 1 \end{bmatrix}$$

$$\mathbf{X}_3 = \begin{bmatrix} 1 & 1 & 1 & 1 & -1 & -1 \end{bmatrix}$$

Assuming that the learning rate parameter, ε , is 0.1666, the bipolar ramp function is used for training and the bipolar step function is used for recall, and the initial weights are zero. All computations are to be rounded to 4 decimal places.

(a) Show the step-by-step training procedure until the sum of squared output errors, SSE is zero.

(b) Check the correctness of the recalls of each of the three patterns.

(c) Recall the corrupted version of each of $\mathbf{X}_1$, $\mathbf{X}_2$, and $\mathbf{X}_3$ as given by

$$\mathbf{X}_{1t} = \begin{bmatrix} -1 & -1 & 1 & -1 & 1 & 1 \end{bmatrix}$$

$$\mathbf{X}_{2t} = \begin{bmatrix} 1 & -1 & 1 & -1 & -1 & 1 \end{bmatrix}$$

$$\mathbf{X}_{3t} = \begin{bmatrix} 1 & 1 & 1 & -1 & -1 & -1 \end{bmatrix}$$

to illustrate the pattern rectification capability of the autoassociator.

(d) Recall the incomplete version of each of $\mathbf{X}_1$, $\mathbf{X}_2$, and $\mathbf{X}_3$ as given by

$$\mathbf{X}_{4t} = \begin{bmatrix} -1 & -1 & 1 & 0 & 1 & 1 \end{bmatrix}$$

$$\mathbf{X}_{5t} = \begin{bmatrix} 1 & -1 & -1 & -1 & 0 & 1 \end{bmatrix}$$

$$\mathbf{X}_{6t} = \begin{bmatrix} 1 & 0 & 1 & 1 & -1 & -1 \end{bmatrix}$$

to illustrate the pattern completion capability of the autoassociator.

Solution

(a)

Cycle 1

$$\mathbf{W} = \begin{bmatrix} .0000 & .0000 & .0000 & .0000 & .0000 & .0000 \\ .0000 & .0000 & .0000 & .0000 & .0000 & .0000 \\ .0000 & .0000 & .0000 & .0000 & .0000 & .0000 \\ .0000 & .0000 & .0000 & .0000 & .0000 & .0000 \\ .0000 & .0000 & .0000 & .0000 & .0000 & .0000 \\ .0000 & .0000 & .0000 & .0000 & .0000 & .0000 \end{bmatrix}$$

$$\mathbf{X}_k = \begin{bmatrix} -1.0000 & -1.0000 & 1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & -1.0000 & -1.0000 & -1.0000 & -1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & 1.0000 & -1.0000 & -1.0000 \end{bmatrix}$$

$$\mathbf{Z}_k = \mathbf{X}_k \mathbf{W} = \begin{bmatrix} .0000 & .0000 & .0000 & .0000 & .0000 & .0000 \\ .0000 & .0000 & .0000 & .0000 & .0000 & .0000 \\ .0000 & .0000 & .0000 & .0000 & .0000 & .0000 \end{bmatrix}$$

$$\mathbf{Y}_k = G_r[\mathbf{Z}_k] = \begin{bmatrix} .0000 & .0000 & .0000 & .0000 & .0000 & .0000 \\ .0000 & .0000 & .0000 & .0000 & .0000 & .0000 \\ .0000 & .0000 & .0000 & .0000 & .0000 & .0000 \end{bmatrix}$$

$$\mathbf{E}_k = \mathbf{X}_k - \mathbf{Y}_k = \begin{bmatrix} -1.0000 & -1.0000 & 1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & -1.0000 & -1.0000 & -1.0000 & -1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & 1.0000 & -1.0000 & -1.0000 \end{bmatrix}$$

$$SSE = 18.000000$$

$$\delta \mathbf{W} = \sum \delta \mathbf{W}_k = \begin{bmatrix} .4998 & .1666 & -.1666 & -.1666 & -.4998 & -.1666 \\ .1666 & .4998 & .1666 & .1666 & -.1666 & -.4998 \\ -.1666 & .1666 & .4998 & .4998 & .1666 & -.1666 \\ -.1666 & .1666 & .4998 & .4998 & .1666 & -.1666 \\ -.4998 & -.1666 & .1666 & .1666 & .4998 & .1666 \\ -.1666 & -.4998 & -.1666 & -.1666 & .1666 & .4998 \end{bmatrix}$$

$$\mathbf{W} = \mathbf{W} + \delta\mathbf{W} =$$

$$\begin{bmatrix} .4998 & .1666 & -.1666 & -.1666 & -.4998 & -.1666 \\ .1666 & .4998 & .1666 & .1666 & -.1666 & -.4998 \\ -.1666 & .1666 & .4998 & .4998 & .1666 & -.1666 \\ -.1666 & .1666 & .4998 & .4998 & .1666 & -.1666 \\ -.4998 & -.1666 & .1666 & .1666 & .4998 & .1666 \\ -.1666 & -.4998 & -.1666 & -.1666 & .1666 & .4998 \end{bmatrix}$$

Cycle 2

$$\mathbf{W} = \begin{bmatrix} .4998 & .1666 & -.1666 & -.1666 & -.4998 & -.1666 \\ .1666 & .4998 & .1666 & .1666 & -.1666 & -.4998 \\ -.1666 & .1666 & .4998 & .4998 & .1666 & -.1666 \\ -.1666 & .1666 & .4998 & .4998 & .1666 & -.1666 \\ -.4998 & -.1666 & .1666 & .1666 & .4998 & .1666 \\ -.1666 & -.4998 & -.1666 & -.1666 & .1666 & .4998 \end{bmatrix}$$

$$\mathbf{X}_k =$$

$$\begin{bmatrix} -1.0000 & -1.0000 & 1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & -1.0000 & -1.0000 & -1.0000 & -1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & 1.0000 & -1.0000 & -1.0000 \end{bmatrix}$$

$$\mathbf{Z}_k = \mathbf{X}_k \cdot \mathbf{W} =$$

$$\begin{bmatrix} -1.6660 & -.9996 & .9996 & .9996 & 1.6660 & .9996 \\ .9996 & -.9996 & -1.6660 & -1.6660 & -.9996 & .9996 \\ .9996 & 1.6660 & .9996 & .9996 & -.9996 & -1.6660 \end{bmatrix}$$

$$\mathbf{Y}_k = G_r[\mathbf{Z}_k] =$$

$$\begin{bmatrix} -1.0000 & -.9996 & .9996 & .9996 & 1.0000 & .9996 \\ .9996 & -.9996 & -1.0000 & -1.0000 & -.9996 & .9996 \\ .9996 & 1.0000 & .9996 & .9996 & -.9996 & -1.0000 \end{bmatrix}$$

$$\mathbf{E}_k = \mathbf{X}_k - \mathbf{Y}_k =$$

$$\begin{bmatrix} .0000 & -.0004 & .0004 & .0004 & .0000 & .0004 \\ .0004 & -.0004 & .0000 & .0000 & -.0004 & .0004 \\ .0004 & .0000 & .0004 & .0004 & -.0004 & .0000 \end{bmatrix}$$

$$SSE = 1.920000E - 06$$

$$\delta \mathbf{W} = \sum \delta \mathbf{W}_k =$$

$$\begin{bmatrix} .0000 & .0000 & .0000 & .0000 & -.0000 & .0000 \\ .0000 & .0000 & .0000 & .0000 & .0000 & -.0000 \\ .0000 & .0000 & .0000 & .0000 & .0000 & .0000 \\ .0000 & .0000 & .0000 & .0000 & .0000 & .0000 \\ -.0000 & .0000 & .0000 & .0000 & .0000 & .0000 \\ .0000 & -.0000 & .0000 & .0000 & .0000 & .0000 \end{bmatrix}$$

$$\mathbf{W} = \mathbf{W} + \delta \mathbf{W} =$$

$$\begin{bmatrix} .4999 & .1666 & -.1666 & -.1666 & -.4999 & -.1666 \\ .1666 & .4999 & .1666 & .1666 & -.1666 & -.4999 \\ -.1666 & .1666 & .4999 & .4999 & .1666 & -.1666 \\ -.1666 & .1666 & .4999 & .4999 & .1666 & -.1666 \\ -.4999 & -.1666 & .1666 & .1666 & .4999 & .1666 \\ -.1666 & -.4999 & -.1666 & -.1666 & .1666 & .4999 \end{bmatrix}$$

Cycle 3

$$\mathbf{W} = \begin{bmatrix} .4999 & .1666 & -.1666 & -.1666 & -.4999 & -.1666 \\ .1666 & .4999 & .1666 & .1666 & -.1666 & -.4999 \\ -.1666 & .1666 & .4999 & .4999 & .1666 & -.1666 \\ -.1666 & .1666 & .4999 & .4999 & .1666 & -.1666 \\ -.4999 & -.1666 & .1666 & .1666 & .4999 & .1666 \\ -.1666 & -.4999 & -.1666 & -.1666 & .1666 & .4999 \end{bmatrix}$$

$$\mathbf{X}_k =$$

$$\begin{bmatrix} -1.0000 & -1.0000 & 1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & -1.0000 & -1.0000 & -1.0000 & -1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & 1.0000 & -1.0000 & -1.0000 \end{bmatrix}$$

$$\mathbf{Z}_k = \mathbf{X}_k \mathbf{W} =$$

$$\begin{bmatrix} -1.6662 & -.9998 & .9998 & .9998 & 1.6662 & .9998 \\ .9998 & -.9998 & -1.6662 & -1.6662 & -.9998 & .9998 \\ .9998 & 1.6662 & .9998 & .9998 & -.9998 & -1.6662 \end{bmatrix}$$

$$\mathbf{Y}_k = G_r[\mathbf{Z}_k] =$$

$$\begin{bmatrix} -1.0000 & -.9998 & .9998 & .9998 & 1.0000 & .9998 \\ .9998 & -.9998 & -1.0000 & -1.0000 & -.9998 & .9998 \\ .9998 & 1.0000 & .9998 & .9998 & -.9998 & -1.0000 \end{bmatrix}$$

$$\mathbf{E}_k = \mathbf{X}_k - \mathbf{Y}_k =$$

$$\begin{bmatrix} .0000 & -.0002 & .0002 & .0002 & .0000 & .0002 \\ .0002 & -.0002 & .0000 & .0000 & -.0002 & .0002 \\ .0002 & .0000 & .0002 & .0002 & -.0002 & .0000 \end{bmatrix}$$

$$SSE = 4.800000E - 07$$

$$\delta \mathbf{W} = \sum \delta \mathbf{W}_k =$$

$$\begin{bmatrix} .0000 & .0000 & .0000 & .0000 & -.0000 & .0000 \\ .0000 & .0000 & .0000 & .0000 & .0000 & -.0000 \\ .0000 & .0000 & .0000 & .0000 & .0000 & .0000 \\ .0000 & .0000 & .0000 & .0000 & .0000 & .0000 \\ -.0000 & .0000 & .0000 & .0000 & .0000 & .0000 \\ .0000 & -.0000 & .0000 & .0000 & .0000 & .0000 \end{bmatrix}$$

$$\mathbf{W} = \mathbf{W} + \delta \mathbf{W} =$$

$$\begin{bmatrix} .5000 & .1666 & -.1666 & -.1666 & -.5000 & -.1666 \\ .1666 & .5000 & .1666 & .1666 & -.1666 & -.5000 \\ -.1666 & .1666 & .5000 & .5000 & .1666 & -.1666 \\ -.1666 & .1666 & .5000 & .5000 & .1666 & -.1666 \\ -.5000 & -.1666 & .1666 & .1666 & .5000 & .1666 \\ -.1666 & -.5000 & -.1666 & -.1666 & .1666 & .5000 \end{bmatrix}$$

Cycle 4

$$\mathbf{W} = \begin{bmatrix} .5000 & .1666 & -.1666 & -.1666 & -.5000 & -.1666 \\ .1666 & .5000 & .1666 & .1666 & -.1666 & -.5000 \\ -.1666 & .1666 & .5000 & .5000 & .1666 & -.1666 \\ -.1666 & .1666 & .5000 & .5000 & .1666 & -.1666 \\ -.5000 & -.1666 & .1666 & .1666 & .5000 & .1666 \\ -.1666 & -.5000 & -.1666 & -.1666 & .1666 & .5000 \end{bmatrix}$$

$$\mathbf{X}_k =$$

$$\begin{bmatrix} -1.0000 & -1.0000 & 1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & -1.0000 & -1.0000 & -1.0000 & -1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & 1.0000 & -1.0000 & -1.0000 \end{bmatrix}$$

$$\mathbf{Z}_k = \mathbf{X}_k \mathbf{W} =$$

$$\begin{bmatrix} -1.6664 & -1.0000 & 1.0000 & 1.0000 & 1.6664 & 1.0000 \\ 1.0000 & -1.0000 & -1.6664 & -1.6664 & -1.0000 & 1.0000 \\ 1.0000 & 1.6664 & 1.0000 & 1.0000 & -1.0000 & -1.6664 \end{bmatrix}$$

$$\mathbf{Y}_k = G_r[\mathbf{Z}_k] = \begin{bmatrix} -1.0000 & -1.0000 & 1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & -1.0000 & -1.0000 & -1.0000 & -1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & 1.0000 & -1.0000 & -1.0000 \end{bmatrix}$$

$$\mathbf{E}_k = \mathbf{X}_k - \mathbf{Y}_k = \begin{bmatrix} .0000 & .0000 & .0000 & .0000 & .0000 & .0000 \\ .0000 & .0000 & .0000 & .0000 & .0000 & .0000 \\ .0000 & .0000 & .0000 & .0000 & .0000 & .0000 \end{bmatrix}$$

$$SSE = 0.000000E + 00$$

(b)

Iteration 1

$$\mathbf{W} = \begin{bmatrix} .5000 & .1666 & -.1666 & -.1666 & -.5000 & -.1666 \\ .1666 & .5000 & .1666 & .1666 & -.1666 & -.5000 \\ -.1666 & .1666 & .5000 & .5000 & .1666 & -.1666 \\ -.1666 & .1666 & .5000 & .5000 & .1666 & -.1666 \\ -.5000 & -.1666 & .1666 & .1666 & .5000 & .1666 \\ -.1666 & -.5000 & -.1666 & -.1666 & .1666 & .5000 \end{bmatrix}$$

$$\mathbf{X}_t = \begin{bmatrix} -1.0000 & -1.0000 & 1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & -1.0000 & -1.0000 & -1.0000 & -1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & 1.0000 & -1.0000 & -1.0000 \end{bmatrix}$$

$$\mathbf{Z}_t = \mathbf{X}_t \mathbf{W} = \begin{bmatrix} -1.6664 & -1.0000 & 1.0000 & 1.0000 & 1.6664 & 1.0000 \\ 1.0000 & -1.0000 & -1.6664 & -1.6664 & -1.0000 & 1.0000 \\ 1.0000 & 1.6664 & 1.0000 & 1.0000 & -1.0000 & -1.6664 \end{bmatrix}$$

$$\mathbf{Y}_t = G_h[\mathbf{Z}_t] = \begin{bmatrix} -1.0000 & -1.0000 & 1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & -1.0000 & -1.0000 & -1.0000 & -1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & 1.0000 & -1.0000 & -1.0000 \end{bmatrix}$$

$$SSE = 0.000000E + 00$$

(c)

Iteration 1

$$\mathbf{W} = \begin{bmatrix} .5000 & .1666 & -.1666 & -.1666 & -.5000 & -.1666 \\ .1666 & .5000 & .1666 & .1666 & -.1666 & -.5000 \\ -.1666 & .1666 & .5000 & .5000 & .1666 & -.1666 \\ -.1666 & .1666 & .5000 & .5000 & .1666 & -.1666 \\ -.5000 & -.1666 & .1666 & .1666 & .5000 & .1666 \\ -.1666 & -.5000 & -.1666 & -.1666 & .1666 & .5000 \end{bmatrix}$$

$$\mathbf{X}_t =$$

$$\begin{bmatrix} -1.0000 & -1.0000 & 1.0000 & -1.0000 & 1.0000 & 1.0000 \\ 1.0000 & -1.0000 & 1.0000 & -1.0000 & -1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & -1.0000 & -1.0000 & -1.0000 \end{bmatrix}$$

$$\mathbf{Z}_t = \mathbf{X}_t \mathbf{W} =$$

$$\begin{bmatrix} -1.3332 & -1.3332 & .0000 & .0000 & 1.3332 & 1.3332 \\ .6668 & -.6668 & -.6664 & -.6664 & -.6668 & .6668 \\ 1.3332 & 1.3332 & .0000 & .0000 & -1.3332 & -1.3332 \end{bmatrix}$$

$$\mathbf{Y}_t = G_h[\mathbf{Z}_t] =$$

$$\begin{bmatrix} -1.0000 & -1.0000 & 1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & -1.0000 & -1.0000 & -1.0000 & -1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & 1.0000 & -1.0000 & -1.0000 \end{bmatrix}$$

$$SSE = 0.000000E + 00$$

(d)

Iteration 1

$$\mathbf{W} = \begin{bmatrix} .5000 & .1666 & -.1666 & -.1666 & -.5000 & -.1666 \\ .1666 & .5000 & .1666 & .1666 & -.1666 & -.5000 \\ -.1666 & .1666 & .5000 & .5000 & .1666 & -.1666 \\ -.1666 & .1666 & .5000 & .5000 & .1666 & -.1666 \\ -.5000 & -.1666 & .1666 & .1666 & .5000 & .1666 \\ -.1666 & -.5000 & -.1666 & -.1666 & .1666 & .5000 \end{bmatrix}$$

$$\mathbf{X}_t =$$

$$\begin{bmatrix} -1.0000 & -1.0000 & 1.0000 & .0000 & 1.0000 & 1.0000 \\ 1.0000 & -1.0000 & -1.0000 & -1.0000 & .0000 & 1.0000 \\ 1.0000 & .0000 & 1.0000 & 1.0000 & -1.0000 & -1.0000 \end{bmatrix}$$

$$\mathbf{Z}_t = \mathbf{X}_t \mathbf{W} = \begin{bmatrix} -1.4998 & -1.1666 & .5000 & .5000 & 1.4998 & 1.1666 \\ .5000 & -1.1666 & -1.4998 & -1.4998 & -.5000 & 1.1666 \\ .8334 & 1.1664 & .8334 & .8334 & -.8334 & -1.1664 \end{bmatrix}$$

$$\mathbf{Y}_t = G_h[\mathbf{Z}_t] = \begin{bmatrix} -1.0000 & -1.0000 & 1.0000 & 1.0000 & 1.0000 & 1.0000 \\ 1.0000 & -1.0000 & -1.0000 & -1.0000 & -1.0000 & 1.0000 \\ 1.0000 & 1.0000 & 1.0000 & 1.0000 & -1.0000 & -1.0000 \end{bmatrix}$$

$$SSE = 0.000000E + 00$$

Problem 3.5 (Weight change and learning properties of delta-rule trained autoassociator)

For a delta-rule trained autoassociator, if weight update is carried out once in a cycle:

- Derive the total weight change at the end of the first cycle.
- Comment on the weight learning after the first cycle.

Solution

(a)

From (3.12)-(3.13), we have

$$\delta w_{ij} = \varepsilon \sum_{k=1}^K x_{ik}(x_{jk} - y_{jk}) \quad \text{for } i, j = 1 \text{ to } N \quad (\text{P3.5.1})$$

or

$$\delta \mathbf{W} = \varepsilon \sum_{k=1}^K \mathbf{X}_k (\mathbf{X}_k - \mathbf{Y}_k) \quad (\text{P3.5.2})$$

where

$$y_{jk} = G_r \left[\sum_{i=1}^N w_{ij} x_{ik} \right] \quad (\text{P3.5.3})$$

or

$$\mathbf{Y}_k = G_r[\mathbf{W} \mathbf{X}_k] \quad (\text{P3.5.4})$$

For zero initial weights, we obtain

$$\delta w_{ij} = \varepsilon \sum_{k=1}^K x_{ik} x_{jk} \quad \text{for } i, j = 1 \text{ to } N \quad (\text{P3.5.5})$$

or

$$\delta \mathbf{W} = \varepsilon \sum_{k=1}^K \mathbf{X}_k^T \mathbf{X}_k \quad (\text{P3.5.6})$$

Hence, the updated weight at the end of the first cycle is

$$\mathbf{W} = \varepsilon \sum_{k=1}^K \mathbf{X}_k^T \mathbf{X}_k \quad (\text{P3.5.7})$$

(b)

After the first cycle:

- a. The delta rule (with nonzero diagonal weight elements) will stop learning for orthogonal patterns training.
- b. The delta rule (with diagonal elements set to 0) will keep on adapting weights and hence the output so as to produce exact target patterns for both orthogonal and non-orthogonal patterns trainings.

Chapter 5

Associative Memories

Problem 5.1 (Original and complementary bipolar vector-pair storage)

Prove that a BAM stores a bipolar vector-pair $\mathbf{X}_k$ $\mathbf{Y}_k$ as well as its complementary bipolar vector-pair $\mathbf{X}_k^c$ and $\mathbf{Y}_k^c$.

Solution

For a BAM, we have

$$\mathbf{Y}_k = G_h[\mathbf{X}_k \mathbf{W}]$$

$$\mathbf{X}_k = G_h[\mathbf{Y}_k \mathbf{W}^T]$$

and for the complementary bipolar vector-pair $\mathbf{X}_k^c$ and $\mathbf{Y}_k^c$, we have

$$\mathbf{X}_k^c = -\mathbf{X}_k \text{ and } \mathbf{Y}_k^c = -\mathbf{Y}_k$$

Therefore

$$G_h[\mathbf{X}_k^c \mathbf{W}] = G_h[-\mathbf{X}_k \mathbf{W}] = -G_h[\mathbf{X}_k \mathbf{W}] = -\mathbf{Y}_k = \mathbf{Y}_k^c$$

$$G_h[\mathbf{Y}_k^c \mathbf{W}^T] = G_h[-\mathbf{Y}_k \mathbf{W}^T] = -G_h[\mathbf{Y}_k \mathbf{W}^T] = -\mathbf{X}_k = \mathbf{X}_k^c$$

We conclude that a BAM stores a bipolar vector-pair $\mathbf{X}_k$ and $\mathbf{Y}_k$ as well as its complementary bipolar vector-pair $\mathbf{X}_k^c$ and $\mathbf{Y}_k^c$.

Problem 5.2 (BAM learning and recall abilities)

Given the following two vector-pairs as

$$\mathbf{X}_1 = [1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1]$$

$$\mathbf{Y}_1 = [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1]$$

$$\mathbf{X}_2 = [1 \quad 1 \quad 1 \quad 1 \quad -1 \quad -1 \quad -1 \quad -1]$$

$$\mathbf{Y}_2 = [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1]$$

which are to be stored in a BAM.

- Determine the weight matrix $\mathbf{W}$ of the BAM.
- Check the correctness of forward recalls and backward recalls.
- Check the correctness of complementary forward recalls and complementary backward recalls.
- Given the following two noisy vectors as

$$\mathbf{X}_t = [1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad -1 \quad -1]$$

$$\mathbf{Y}_t = [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad -1 \quad -1 \quad 1]$$

Compute the steady-state vector-pair of the recall in each of the two cases.

Solution

(a)

$$\mathbf{W} = \mathbf{X}_1^T \mathbf{Y}_1 + \mathbf{X}_2^T \mathbf{Y}_2 = \begin{bmatrix} 2 & 0 & 0 & -2 & 2 & 0 & 0 & -2 \\ 2 & 0 & 0 & -2 & 2 & 0 & 0 & -2 \\ 2 & 0 & 0 & -2 & 2 & 0 & 0 & -2 \\ 2 & 0 & 0 & -2 & 2 & 0 & 0 & -2 \\ 0 & 2 & -2 & 0 & 0 & 2 & -2 & 0 \\ 0 & 2 & -2 & 0 & 0 & 2 & -2 & 0 \\ 0 & 2 & -2 & 0 & 0 & 2 & -2 & 0 \\ 0 & 2 & -2 & 0 & 0 & 2 & -2 & 0 \end{bmatrix}$$

(b) For the original pattern pairs recalls

Forward recalls:

$$\begin{aligned} G_h[\mathbf{X}_1 \mathbf{W}] &= G_h[[1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1] \mathbf{W}] \\ &= G_h[[8 \quad 8 \quad -8 \quad -8 \quad 8 \quad 8 \quad -8 \quad -8]] \\ &= [1 \quad 1 \quad -1 \quad -1 \quad 1 \quad 1 \quad -1 \quad -1] = \mathbf{Y}_1 \end{aligned}$$

$$\begin{aligned}
G_h[\mathbf{X}_2 \mathbf{W}] &= G_h \begin{bmatrix} 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 \end{bmatrix} \mathbf{W} \\
&= G_h \begin{bmatrix} 8 & -8 & 8 & -8 & 8 & -8 & 8 & -8 \end{bmatrix} \\
&= \begin{bmatrix} 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \end{bmatrix} = \mathbf{Y}_2
\end{aligned}$$

Backward recalls:

$$\begin{aligned}
G_h[\mathbf{Y}_1 \mathbf{W}^T] &= G_h \begin{bmatrix} 1 & 1 & -1 & -1 & 1 & 1 & -1 & -1 \end{bmatrix} \mathbf{W}^T \\
&= G_h \begin{bmatrix} 8 & 8 & 8 & 8 & 8 & 8 & 8 & 8 \end{bmatrix} \\
&= \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} = \mathbf{X}_1
\end{aligned}$$

$$\begin{aligned}
G_h[\mathbf{Y}_2 \mathbf{W}^T] &= G_h \begin{bmatrix} 1 & -1 & 1 & -1 & 1 & -1 & 1 & -1 \end{bmatrix} \mathbf{W}^T \\
&= G_h \begin{bmatrix} 8 & 8 & 8 & 8 & -8 & -8 & -8 & -8 \end{bmatrix} \\
&= \begin{bmatrix} 1 & 1 & 1 & 1 & -1 & -1 & -1 & -1 \end{bmatrix} = \mathbf{X}_2
\end{aligned}$$

(c) For the complementary pattern pairs recalls

Forward recalls:

$$\begin{aligned}
G_h[\mathbf{X}_1^c \mathbf{W}] &= G_h \begin{bmatrix} -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 \end{bmatrix} \mathbf{W} \\
&= G_h \begin{bmatrix} -8 & -8 & 8 & 8 & -8 & -8 & 8 & 8 \end{bmatrix} \\
&= \begin{bmatrix} -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 \end{bmatrix} = \mathbf{Y}_1^c
\end{aligned}$$

$$\begin{aligned}
G_h[\mathbf{X}_2^c \mathbf{W}] &= G_h \begin{bmatrix} -1 & -1 & -1 & -1 & 1 & 1 & 1 & 1 \end{bmatrix} \mathbf{W} \\
&= G_h \begin{bmatrix} -8 & 8 & -8 & 8 & -8 & 8 & -8 & 8 \end{bmatrix} \\
&= \begin{bmatrix} -1 & 1 & -1 & 1 & -1 & 1 & -1 & 1 \end{bmatrix} = \mathbf{Y}_2^c
\end{aligned}$$

Backward recalls:

$$\begin{aligned}
G_h[\mathbf{Y}_1^c \mathbf{W}^T] &= G_h \begin{bmatrix} -1 & -1 & 1 & 1 & -1 & -1 & 1 & 1 \end{bmatrix} \mathbf{W}^T \\
&= G_h \begin{bmatrix} -8 & -8 & -8 & -8 & -8 & -8 & -8 & -8 \end{bmatrix} \\
&= \begin{bmatrix} -1 & -1 & -1 & -1 & -1 & -1 & -1 & -1 \end{bmatrix} = \mathbf{X}_1^c
\end{aligned}$$

$$\begin{aligned}
G_h[\mathbf{Y}_2^c \mathbf{W}^T] &= G_h \begin{bmatrix} -1 & 1 & -1 & 1 & -1 & 1 & -1 & 1 \end{bmatrix} \mathbf{W}^T \\
&= G_h \begin{bmatrix} -8 & -8 & -8 & -8 & 8 & 8 & 8 & 8 \end{bmatrix} \\
&= \begin{bmatrix} -1 & -1 & -1 & -1 & 1 & 1 & 1 & 1 \end{bmatrix} = \mathbf{X}_2^c
\end{aligned}$$

(d) For the recall of the noisy vector

$$\mathbf{X}_t = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & -1 & -1 \end{bmatrix}$$

$$\begin{aligned}
G_h[\mathbf{X}_t \mathbf{W}] &= G_h[[1 \ 1 \ 1 \ 1 \ 1 \ 1 \ -1 \ -1] \mathbf{W}] \\
&= G_h[8 \ 0 \ 0 \ -8 \ 8 \ 0 \ 0 \ -8] \\
&= [1 \ 1 \ 1 \ -1 \ 1 \ 1 \ 1 \ -1] = \mathbf{Y}_t
\end{aligned}$$

$$\begin{aligned}
G_h[\mathbf{Y}_t \mathbf{W}^T] &= G_h[[1 \ 1 \ 1 \ -1 \ 1 \ 1 \ 1 \ -1] \mathbf{W}^T] \\
&= G_h[8 \ 8 \ 8 \ 8 \ 0 \ 0 \ 0 \ 0] \\
&= [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] = \mathbf{X}_1
\end{aligned}$$

The steady-state vector-pair is $\mathbf{X}_1$ and $\mathbf{Y}_1$.

For the recall of the noisy vector:

$$\mathbf{Y}_t = [1 \ -1 \ 1 \ -1 \ 1 \ -1 \ -1 \ 1]$$

$$\begin{aligned}
G_h[\mathbf{Y}_t \mathbf{W}^T] &= G_h[[1 \ -1 \ 1 \ -1 \ 1 \ -1 \ -1 \ 1] \mathbf{W}^T] \\
&= G_h[4 \ 4 \ 4 \ 4 \ -4 \ -4 \ -4 \ -4] \\
&= [1 \ 1 \ 1 \ 1 \ -1 \ -1 \ -1 \ -1] = \mathbf{X}_2
\end{aligned}$$

The steady-state vector-pair is $\mathbf{X}_2$ and $\mathbf{Y}_2$.

Problem 5.3 (TAM learning and recall abilities)

Given a complete set of temporal patterns $\mathbf{X}_1, \mathbf{X}_2, \mathbf{X}_3, \mathbf{X}_1$, as

$$\mathbf{X}_1 = [1 \ 1 \ 1 \ 1 \ 1 \ 1]$$

$$\mathbf{X}_2 = [1 \ 1 \ -1 \ 1 \ -1 \ -1]$$

$$\mathbf{X}_3 = [1 \ -1 \ 1 \ 1 \ -1 \ 1]$$

which are to be stored in a TAM.

- Determine the weight matrix $\mathbf{W}$ of the TAM.
- Check the correctness of forward recalls and backward recalls.
- Check the correctness of complementary forward recalls and complementary backward recalls.
- Given the following noisy vector as

$$\mathbf{X}_t = [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad 1]$$

Identify the original vector of $\mathbf{X}_t$ by performing forward and backward recalls.

Solution

(a)

$$\mathbf{W} = \mathbf{X}_1^T \mathbf{X}_2 + \mathbf{X}_2^T \mathbf{X}_3 + \mathbf{X}_3^T \mathbf{X}_1 = \begin{bmatrix} 3 & 1 & 1 & 3 & -1 & 1 \\ 1 & -1 & -1 & 1 & -3 & -1 \\ 1 & 3 & -1 & 1 & 1 & -1 \\ 3 & 1 & 1 & 3 & -1 & 1 \\ -1 & 1 & -3 & -1 & -1 & -3 \\ 1 & 3 & -1 & 1 & 1 & -1 \end{bmatrix}$$

(b) For the original pattern sequence recalls

Forward recalls:

$$\begin{aligned} G_h[\mathbf{X}_1 \mathbf{W}] &= G_h[[1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1] \mathbf{W}] \\ &= G_h[8 \quad 8 \quad -4 \quad 8 \quad -4 \quad -4] \\ &= [1 \quad 1 \quad -1 \quad 1 \quad -1 \quad -1] = \mathbf{X}_2 \end{aligned}$$

$$\begin{aligned} G_h[\mathbf{X}_2 \mathbf{W}] &= G_h[[1 \quad 1 \quad -1 \quad 1 \quad -1 \quad -1] \mathbf{W}] \\ &= G_h[6 \quad -6 \quad 6 \quad 6 \quad -6 \quad 6] \\ &= [1 \quad -1 \quad 1 \quad 1 \quad -1 \quad 1] = \mathbf{X}_3 \end{aligned}$$

$$\begin{aligned} G_h[\mathbf{X}_3 \mathbf{W}] &= G_h[[1 \quad -1 \quad 1 \quad 1 \quad -1 \quad 1] \mathbf{W}] \\ &= G_h[8 \quad 8 \quad 4 \quad 8 \quad 4 \quad 4] \\ &= [1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1] = \mathbf{X}_1 \end{aligned}$$

Backward recalls:

$$\begin{aligned} G_h[\mathbf{X}_3 \mathbf{W}^T] &= G_h[[1 \quad -1 \quad 1 \quad 1 \quad -1 \quad 1] \mathbf{W}^T] \\ &= G_h[8 \quad 4 \quad -4 \quad 8 \quad -8 \quad -4] \\ &= [1 \quad 1 \quad -1 \quad 1 \quad -1 \quad -1] = \mathbf{X}_2 \end{aligned}$$

$$\begin{aligned} G_h[\mathbf{X}_2 \mathbf{W}^T] &= G_h \begin{bmatrix} 1 & 1 & -1 & 1 & -1 & -1 \end{bmatrix} \mathbf{W}^T \\ &= G_h \begin{bmatrix} 6 & 6 & 6 & 6 & 6 & 6 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} = \mathbf{X}_1 \end{aligned}$$

$$\begin{aligned} G_h[\mathbf{X}_1 \mathbf{W}^T] &= G_h \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \mathbf{W}^T \\ &= G_h \begin{bmatrix} 8 & -4 & 4 & 8 & -8 & 4 \end{bmatrix} \\ &= \begin{bmatrix} 1 & -1 & 1 & 1 & -1 & 1 \end{bmatrix} = \mathbf{X}_3 \end{aligned}$$

(c) For the complementary pattern sequence recalls

Forward recalls:

$$\begin{aligned} G_h[\mathbf{X}_1^c \mathbf{W}] &= G_h \begin{bmatrix} -1 & -1 & -1 & -1 & -1 & -1 \end{bmatrix} \mathbf{W} \\ &= G_h \begin{bmatrix} -8 & -8 & 4 & -8 & 4 & 4 \end{bmatrix} \\ &= \begin{bmatrix} -1 & -1 & 1 & -1 & 1 & 1 \end{bmatrix} = \mathbf{X}_2^c \end{aligned}$$

$$\begin{aligned} G_h[\mathbf{X}_2^c \mathbf{W}] &= G_h \begin{bmatrix} -1 & -1 & 1 & -1 & 1 & 1 \end{bmatrix} \mathbf{W} \\ &= G_h \begin{bmatrix} -6 & 6 & -6 & -6 & 6 & -6 \end{bmatrix} \\ &= \begin{bmatrix} -1 & 1 & -1 & -1 & 1 & -1 \end{bmatrix} = \mathbf{X}_3^c \end{aligned}$$

$$\begin{aligned} G_h[\mathbf{X}_3^c \mathbf{W}] &= G_h \begin{bmatrix} -1 & 1 & -1 & -1 & 1 & -1 \end{bmatrix} \mathbf{W} \\ &= G_h \begin{bmatrix} -8 & -8 & -4 & -8 & -4 & -4 \end{bmatrix} \\ &= \begin{bmatrix} -1 & -1 & -1 & -1 & -1 & -1 \end{bmatrix} = \mathbf{X}_1^c \end{aligned}$$

Backward recalls:

$$\begin{aligned} G_h[\mathbf{X}_3^c \mathbf{W}^T] &= G_h \begin{bmatrix} -1 & 1 & -1 & -1 & 1 & -1 \end{bmatrix} \mathbf{W}^T \\ &= G_h \begin{bmatrix} -8 & -4 & 4 & -8 & 8 & 4 \end{bmatrix} \\ &= \begin{bmatrix} -1 & -1 & 1 & -1 & 1 & 1 \end{bmatrix} = \mathbf{X}_2^c \end{aligned}$$

$$\begin{aligned} G_h[\mathbf{X}_2^c \mathbf{W}^T] &= G_h \begin{bmatrix} -1 & -1 & 1 & -1 & 1 & 1 \end{bmatrix} \mathbf{W}^T \\ &= G_h \begin{bmatrix} -6 & -6 & -6 & -6 & -6 & -6 \end{bmatrix} \\ &= \begin{bmatrix} -1 & -1 & -1 & -1 & -1 & -1 \end{bmatrix} = \mathbf{X}_1^c \end{aligned}$$

$$\begin{aligned} G_h[\mathbf{X}_1^c \mathbf{W}^T] &= G_h \begin{bmatrix} -1 & -1 & -1 & -1 & -1 & -1 \end{bmatrix} \mathbf{W}^T \\ &= G_h \begin{bmatrix} -8 & 4 & -4 & -8 & 8 & -4 \end{bmatrix} \\ &= \begin{bmatrix} -1 & 1 & -1 & -1 & 1 & -1 \end{bmatrix} = \mathbf{X}_3^c \end{aligned}$$

(d) For the recall of the noisy vector $\mathbf{X}_t = [1 \quad -1 \quad 1 \quad -1 \quad 1 \quad 1]$

$$\begin{aligned} G_h[\mathbf{X}_t \mathbf{W}] &= G_h[[1 \quad -1 \quad 1 \quad -1 \quad 1 \quad 1] \mathbf{W}] \\ &= G_h[0 \quad 8 \quad -4 \quad 0 \quad 4 \quad -4] \\ &= [1 \quad 1 \quad -1 \quad 1 \quad 1 \quad -1] = \mathbf{X}_t' \end{aligned}$$

$$\begin{aligned} G_h[\mathbf{X}_t' \mathbf{W}] &= G_h[[1 \quad 1 \quad -1 \quad 1 \quad 1 \quad -1] \mathbf{W}] \\ &= G_h[4 \quad -4 \quad 0 \quad 4 \quad -8 \quad 0] \\ &= [1 \quad -1 \quad 1 \quad 1 \quad -1 \quad 1] = \mathbf{X}_3 \end{aligned}$$

$$\begin{aligned} G_h[\mathbf{X}_t \mathbf{W}^T] &= G_h[[1 \quad -1 \quad 1 \quad -1 \quad 1 \quad 1] \mathbf{W}^T] \\ &= G_h[0 \quad -4 \quad -4 \quad 0 \quad -8 \quad -4] \\ &= [1 \quad -1 \quad -1 \quad 1 \quad -1 \quad -1] = \mathbf{X}_t'' \end{aligned}$$

$$\begin{aligned} G_h[\mathbf{X}_t'' \mathbf{W}^T] &= G_h[[1 \quad -1 \quad -1 \quad 1 \quad -1 \quad -1] \mathbf{W}^T] \\ &= G_h[4 \quad 8 \quad 0 \quad 4 \quad 4 \quad 0] \\ &= [1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1] = \mathbf{X}_1 \end{aligned}$$

The original vector of $\mathbf{X}_t$ is $\mathbf{X}_2$.

Chapter 6

Self-Organizing Maps

Problem 6.1 (1-d Mexican hat function derivation from wavelet function)

Show the step-by-step derivation of the one-dimensional Mexican hat function as given by

$$h(x) = d - \frac{2a}{c^2} e^{\frac{-(x-b)^2}{2c^2}} \times \left\{ 1 - \frac{(x-b)^2}{c^2} \right\} \quad (\text{P6.1.1})$$

Solution

Differentiating a one-dimensional Gaussian function of (P6.1.2) as (P6.1.3), we have

$$f(x) = a e^{\frac{-(x-b)^2}{2c^2}} \quad (\text{P6.1.2})$$

$$f'(x) = a \times \frac{-2(x-b)}{2c^2} e^{\frac{-(x-b)^2}{2c^2}} \quad (\text{P6.1.3})$$

Taking the second-order differentiation of (P6.1.3), we have

$$\begin{aligned} f''(x) &= a \left[\frac{d}{dx} \left(e^{\frac{-(x-b)^2}{2c^2}} \right) \times \frac{-(x-b)}{c^2} + e^{\frac{-(x-b)^2}{2c^2}} \times \frac{d}{dx} \left(\frac{-(x-b)}{c^2} \right) \right] \\ &= a \left[e^{\frac{-(x-b)^2}{2c^2}} \times \frac{-2(x-b)}{2c^2} \times \frac{-(x-b)}{c^2} + e^{\frac{-(x-b)^2}{2c^2}} \times \frac{-1}{c^2} \right] \\ &= \frac{a}{c^2} e^{\frac{-(x-b)^2}{2c^2}} \times \left\{ \frac{(x-b)^2}{c^2} - 1 \right\} \end{aligned} \quad (\text{P6.1.4})$$

By taking the negative and normalized version of the second-order derivative derived in (P6.1.4), we obtain

$$f''(x) = \frac{2a}{c^2} e^{\frac{-(x-b)^2}{2c^2}} \times \left\{ \frac{(x-b)^2}{c^2} - 1 \right\} \quad (\text{P6.1.5})$$

In (P6.1.5), the factor 2 introduced serves the purpose of scaling the second-order derivative function obtained in (P6.1.4) to obtain a set of desired amplitudes for both the center excitation region and the surrounding inhibition region in the Mexican hat function. The resulting one-dimensional Mexican hat function is obtained by shifting up (P6.1.5) by d as

$$h(x) = d - \frac{2a}{c^2} e^{\frac{-(x-b)^2}{2c^2}} \times \left\{ 1 - \frac{(x-b)^2}{c^2} \right\} \quad (\text{P6.1.6})$$

Problem 6.2 (Bubble formation by SOM clustering)

Show the step-by-step computation of Example 6.1 for $k = 1$ and $j = 0$ to 10.

Solution

The step-by-step computation for $k = 1$ and $j = 0$ to 10 are shown below.

First of all, the lateral feedback weights of the Mexican hat function are given as

$$f_{-2j} = f_{-1j} = f_{0j} = f_{1j} = f_{2j} = 0.06 \text{ for } j = 0 \text{ to } 10$$

$$f_{-5j} = f_{-4j} = f_{-3j} = f_{3j} = f_{4j} = f_{5j} = -0.04 \text{ for } j = 0 \text{ to } 10$$

$y_j(0)$ (at $k = 0$) and any j is given as

$$y_j(0) = F \left[A \sin^2 \left(\frac{j\pi}{N-1} \right) \right] \text{ for } j = 0 \text{ to } N-1$$

Hence, for $j = 0$ to $N-1 = 10$, we have

$$y_0(0) = F \left[0.1 \sin^2 \left(\frac{0 \times \pi}{11 - 1} \right) \right] = 0.00$$

$$y_1(0) = F \left[0.1 \sin^2 \left(\frac{1 \times \pi}{11 - 1} \right) \right] = 0.01$$

$$y_2(0) = F \left[0.1 \sin^2 \left(\frac{2 \times \pi}{11 - 1} \right) \right] = 0.0345 \cong 0.03$$

$$y_3(0) = F \left[0.1 \sin^2 \left(\frac{3 \times \pi}{11 - 1} \right) \right] = 0.07$$

$$y_4(0) = F \left[0.1 \sin^2 \left(\frac{4 \times \pi}{11 - 1} \right) \right] = 0.09$$

$$y_5(0) = F \left[0.1 \sin^2 \left(\frac{5 \times \pi}{11 - 1} \right) \right] = 0.10$$

$$y_6(0) = F \left[0.1 \sin^2 \left(\frac{6 \times \pi}{11 - 1} \right) \right] = 0.09$$

$$y_7(0) = F \left[0.1 \sin^2 \left(\frac{7 \times \pi}{11 - 1} \right) \right] = 0.07$$

$$y_8(0) = F \left[0.1 \sin^2 \left(\frac{8 \times \pi}{11 - 1} \right) \right] = 0.0345 \cong 0.03$$

$$y_9(0) = F \left[0.1 \sin^2 \left(\frac{9 \times \pi}{11 - 1} \right) \right] = 0.01$$

$$y_{10}(0) = F \left[0.1 \sin^2 \left(\frac{10 \times \pi}{11 - 1} \right) \right] = 0.00$$

Computation of $y_j(k)$ for $j = 0$ to 10 and $k > 0$ is

$$\begin{aligned} y_j(k) &= F \left[7 \sum_{m=-5}^5 f_{mj} y_{j+m}(k-1) \right] \\ &= F [7 \{ f_{-5,j} y_{j-5}(k-1) + f_{-4,j} y_{j-4}(k-1) + f_{-3,j} y_{j-3}(k-1) \\ &\quad + f_{-2,j} y_{j-2}(k-1) + f_{-1,j} y_{j-1}(k-1) + f_{0,j} y_j(k-1) \\ &\quad + f_{1,j} y_{j+1}(k-1) + f_{2,j} y_{j+2}(k-1) + f_{3,j} y_{j+3}(k-1) \\ &\quad + f_{4,j} y_{j+4}(k-1) + f_{5,j} y_{j+5}(k-1) \}] \end{aligned}$$

For $k = 1$ and $j = 0$, we have

$$\begin{aligned}
 y_0(1) &= F \left[7 \sum_{m=-5}^5 f_{m0} y_m(0) \right] \\
 &= F [7 \{ f_{-5,0} y_{-5}(0) + f_{-4,0} y_{-4}(0) + f_{-3,0} y_{-3}(0) \\
 &\quad + f_{-2,0} y_{-2}(0) + f_{-1,0} y_{-1}(0) + f_{0,0} y_0(0) + f_{1,0} y_1(0) \\
 &\quad + f_{2,0} y_2(0) + f_{3,0} y_3(0) + f_{4,0} y_4(0) + f_{5,0} y_5(0) \}] \\
 &= F [7 \{ f_{-5,0} \times (0.0) + f_{-4,0} \times (0.0) + f_{-3,0} \times (0.0) \\
 &\quad + f_{-2,0} \times (0.0) + f_{-1,0} \times (0.0) + f_{0,0} \times (0.0) + f_{1,0} \times (0.01) \\
 &\quad + f_{2,0} \times (0.03) + f_{3,0} \times (0.07) + f_{4,0} \times (0.09) + f_{5,0} \times (0.10) \}] \\
 &= F [7 \{ -0.04 \times (0.0) - 0.04 \times (0.0) - 0.04 \times (0.0) \\
 &\quad + 0.06 \times (0.0) + 0.06 \times (0.0) + 0.06 \times (0.0) + 0.06 \times (0.01) \\
 &\quad + 0.06 \times (0.03) - 0.04 \times (0.07) - 0.04 \times (0.09) - 0.04 \times (0.10) \}] \\
 &= F [-0.056] = 0.00
 \end{aligned}$$

For $k = 1$ and $j = 1$, we have

$$\begin{aligned}
 y_1(1) &= F \left[7 \sum_{m=-5}^5 f_{m1} y_{1+m}(0) \right] \\
 &= F [7 \{ f_{-5,1} y_{1-5}(0) + f_{-4,1} y_{1-4}(0) + f_{-3,1} y_{1-3}(0) \\
 &\quad + f_{-2,1} y_{1-2}(0) + f_{-1,1} y_{1-1}(0) + f_{0,1} y_1(0) + f_{1,1} y_{1+1}(0) \\
 &\quad + f_{2,1} y_{1+2}(0) + f_{3,1} y_{1+3}(0) + f_{4,1} y_{1+4}(0) + f_{5,1} y_{1+5}(0) \}] \\
 &= F [7 \{ f_{-5,1} y_{-4}(0) + f_{-4,1} y_{-3}(0) + f_{-3,1} y_{-2}(0) \\
 &\quad + f_{-2,1} y_{-1}(0) + f_{-1,1} y_0(0) + f_{0,1} y_1(0) + f_{1,1} y_2(0) \\
 &\quad + f_{2,1} y_3(0) + f_{3,1} y_4(0) + f_{4,1} y_5(0) + f_{5,1} y_6(0) \}] \\
 &= F [7 \{ f_{-5,1} (0.0) + f_{-4,1} (0.0) + f_{-3,1} (0.0) \\
 &\quad + f_{-2,1} (0.0) + f_{-1,1} (0.0) + f_{0,1} (0.01) + f_{1,1} (0.03) \\
 &\quad + f_{2,1} (0.07) + f_{3,1} (0.09) + f_{4,1} (0.10) + f_{5,1} (0.09) \}] \\
 &= F [7 \{ -0.04(0.0) - 0.04(0.0) - 0.04(0.0) \\
 &\quad + 0.06(0.0) + 0.06(0.0) + 0.06(0.01) + 0.06(0.03) \\
 &\quad + 0.06(0.07) - 0.04(0.09) - 0.04(0.10) - 0.04(0.09) \}]
 \end{aligned}$$

$$= F[-0.0322] = 0.00$$

For $k = 1$ and $j = 2$, we have

$$\begin{aligned} y_2(1) &= F \left[7 \sum_{m=-5}^5 f_{m2} y_{2+m}(0) \right] \\ &= F [7 \{ f_{-5,2} y_{2-5}(0) + f_{-4,2} y_{2-4}(0) + f_{-3,2} y_{2-3}(0) \\ &\quad + f_{-2,2} y_{2-2}(0) + f_{-1,2} y_{2-1}(0) + f_{0,2} y_2(0) + f_{1,2} y_{2+1}(0) \\ &\quad + f_{2,2} y_{2+2}(0) + f_{3,2} y_{2+3}(0) + f_{4,2} y_{2+4}(0) + f_{5,2} y_{2+5}(0) \}] \\ &= F [7 \{ f_{-5,2} y_{-3}(0) + f_{-4,2} y_{-2}(0) + f_{-3,2} y_{-1}(0) \\ &\quad + f_{-2,2} y_0(0) + f_{-1,2} y_1(0) + f_{0,2} y_2(0) + f_{1,2} y_3(0) \\ &\quad + f_{2,2} y_4(0) + f_{3,2} y_5(0) + f_{4,2} y_6(0) + f_{5,2} y_7(0) \}] \\ &= F [7 \{ f_{-5,2}(0.0) + f_{-4,2}(0.0) + f_{-3,2}(0.0) \\ &\quad + f_{-2,2}(0.0) + f_{-1,2} y_1(0.01) + f_{0,2} y_2(0.03) + f_{1,2} y_3(0.07) \\ &\quad + f_{2,2} y_4(0.09) + f_{3,2} y_5(0.1) + f_{4,2} y_6(0.09) + f_{5,2} y_7(0.07) \}] \\ &= F [7 \{ -0.04(0.0) - 0.04(0.0) - 0.04(0.0) \\ &\quad + 0.06(0.0) + 0.06(0.01) + 0.06(0.03) + 0.06(0.07) \\ &\quad + 0.06(0.09) - 0.04(0.1) - 0.04(0.09) - 0.04(0.07) \}] \\ &= F[0.0122] = 0.0122 \cong 0.01 \end{aligned}$$

For $k = 1$ and $j = 3$, we have

$$\begin{aligned} y_3(1) &= F \left[7 \sum_{m=-5}^5 f_{m3} y_{3+m}(0) \right] \\ &= F [7 \{ f_{-5,3} y_{3-5}(0) + f_{-4,3} y_{3-4}(0) + f_{-3,3} y_{3-3}(0) \\ &\quad + f_{-2,3} y_{3-2}(0) + f_{-1,3} y_{3-1}(0) + f_{0,3} y_3(0) + f_{1,3} y_{3+1}(0) \\ &\quad + f_{2,3} y_{3+2}(0) + f_{3,3} y_{3+3}(0) + f_{4,3} y_{3+4}(0) + f_{5,3} y_{3+5}(0) \}] \\ &= F [7 \{ f_{-5,3} y_{-2}(0) + f_{-4,3} y_{-1}(0) + f_{-3,3} y_0(0) \\ &\quad + f_{-2,3} y_1(0) + f_{-1,3} y_2(0) + f_{0,3} y_3(0) + f_{1,3} y_4(0) \\ &\quad + f_{2,3} y_5(0) + f_{3,3} y_6(0) + f_{4,3} y_7(0) + f_{5,3} y_8(0) \}] \end{aligned}$$

$$\begin{aligned}
&= F[7\{f_{-5,3}(0.0) + f_{-4,3}(0.0) + f_{-3,3}(0.0) \\
&\quad + f_{-2,3}(0.01) + f_{-1,3}y_2(0.03) + f_{0,3}y_3(0.07) + f_{1,3}y_4(0.09) \\
&\quad + f_{2,3}y_5(0.1) + f_{3,3}y_6(0.09) + f_{4,7}y_7(0.07) + f_{5,3}y_8(0.03)\}] \\
&= F[7\{-0.04(0.0) - 0.04(0.0) - 0.04(0.0) \\
&\quad + 0.06(0.01) + 0.06(0.03) + 0.06(0.07) + 0.06(0.09) \\
&\quad + 0.06(0.1) - 0.04(0.09) - 0.04(0.07) - 0.04(0.03)\}] \\
&= F[0.0728] = 0.0728 \cong 0.07
\end{aligned}$$

For $k = 1$ and $j = 4$, we have

$$\begin{aligned}
y_4(1) &= F\left[7 \sum_{m=-5}^5 f_{m4}y_{4+m}(0)\right] \\
&= F[7\{f_{-5,4}y_{4-5}(0) + f_{-4,4}y_{4-4}(0) + f_{-3,4}y_{4-3}(0) \\
&\quad + f_{-2,4}y_{4-2}(0) + f_{-1,4}y_{4-1}(0) + f_{0,4}y_{4-0}(0) + f_{1,4}y_{4+1}(0) \\
&\quad + f_{2,4}y_{4+2}(0) + f_{3,4}y_{4+3}(0) + f_{4,4}y_{4+4}(0) + f_{5,4}y_{4+5}(0)\}] \\
&= F[7\{f_{-5,4}y_{-1}(0) + f_{-4,4}y_0(0) + f_{-3,4}y_1(0) \\
&\quad + f_{-2,4}y_2(0) + f_{-1,4}y_3(0) + f_{0,4}y_4(0) + f_{1,4}y_5(0) \\
&\quad + f_{2,4}y_6(0) + f_{3,4}y_7(0) + f_{4,4}y_8(0) + f_{5,4}y_9(0)\}] \\
&= F[7\{f_{-5,4}(0.0) + f_{-4,4}(0.0) + f_{-3,4}(0.01) \\
&\quad + f_{-2,4}(0.03) + f_{-1,4}(0.07) + f_{0,4}(0.09) + f_{1,4}(0.10) \\
&\quad + f_{2,4}(0.09) + f_{3,4}(0.07) + f_{4,4}(0.03) + f_{5,4}(0.01)\}] \\
&= F[7\{-0.04(0.0) - 0.04(0.0) - 0.04(0.01) \\
&\quad + 0.06(0.03) + 0.06(0.07) + 0.06(0.09) + 0.06(0.10) \\
&\quad + 0.06(0.09) - 0.04(0.07) - 0.04(0.03) - 0.04(0.01)\}] \\
&= F[0.126] = 0.126 \cong 0.13
\end{aligned}$$

For $k = 1$ and $j = 5$, we have

$$\begin{aligned}
 y_5(1) &= F \left[7 \sum_{m=-5}^5 f_{m5} y_{5+m}(0) \right] \\
 &= F [7 \{ f_{-5,5} y_{5-5}(0) + f_{-4,5} y_{5-4}(0) + f_{-3,5} y_{5-3}(0) \\
 &\quad + f_{-2,5} y_{5-2}(0) + f_{-1,5} y_{5-1}(0) + f_{0,5} y_{5-0}(0) + f_{1,5} y_{5+1}(0) \\
 &\quad + f_{2,5} y_{5+2}(0) + f_{3,5} y_{5+3}(0) + f_{4,5} y_{5+4}(0) + f_{5,5} y_{5+5}(0) \}] \\
 &= F [7 \{ f_{-5,5} y_0(0) + f_{-4,5} y_1(0) + f_{-3,5} y_2(0) \\
 &\quad + f_{-2,5} y_3(0) + f_{-1,5} y_4(0) + f_{0,5} y_5(0) + f_{1,5} y_6(0) \\
 &\quad + f_{2,5} y_7(0) + f_{3,5} y_8(0) + f_{4,5} y_9(0) + f_{5,5} y_{10}(0) \}] \\
 &= F [7 \{ f_{-5,5}(0.0) + f_{-4,5}(0.01) + f_{-3,5}(0.03) \\
 &\quad + f_{-2,5}(0.07) + f_{-1,5}(0.09) + f_{0,5}(0.1) + f_{1,5}(0.09) \\
 &\quad + f_{2,5}(0.07) + f_{3,5}(0.03) + f_{4,5}(0.01) + f_{5,5}(0.0) \}] \\
 &= F [7 \{ -0.04(0.0) - 0.04(0.01) - 0.04(0.03) \\
 &\quad + 0.06(0.07) + 0.06(0.09) + 0.06(0.1) + 0.06(0.09) \\
 &\quad + 0.06(0.07) - 0.04(0.03) - 0.04(0.01) - 0.04(0) \}] \\
 &= F[0.1540] = 0.154 \cong 0.15
 \end{aligned}$$

For $k = 1$ and $j = 6$, we have

$$\begin{aligned}
 y_6(1) &= F \left[7 \sum_{m=-5}^5 f_{m6} y_{6+m}(0) \right] \\
 &= F [7 \{ f_{-5,6} y_{6-5}(0) + f_{-4,6} y_{6-4}(0) + f_{-3,6} y_{6-3}(0) \\
 &\quad + f_{-2,6} y_{6-2}(0) + f_{-1,6} y_{6-1}(0) + f_{0,6} y_{6-0}(0) + f_{1,6} y_{6+1}(0) \\
 &\quad + f_{2,6} y_{6+2}(0) + f_{3,6} y_{6+3}(0) + f_{4,6} y_{6+4}(0) + f_{5,6} y_{6+5}(0) \}] \\
 &= F [7 \{ f_{-5,6} y_1(0) + f_{-4,6} y_2(0) + f_{-3,6} y_3(0) \\
 &\quad + f_{-2,6} y_4(0) + f_{-1,6} y_5(0) + f_{0,6} y_6(0) + f_{1,6} y_7(0) \\
 &\quad + f_{2,6} y_8(0) + f_{3,6} y_9(0) + f_{4,6} y_{10}(0) + f_{5,6} y_{11}(0) \}] \\
 &= F [7 \{ f_{-5,6}(0.01) + f_{-4,6}(0.03) + f_{-3,6}(0.07) \\
 &\quad + f_{-2,6}(0.09) + f_{-1,6}(0.1) + f_{0,6}(0.09) + f_{1,6}(0.07) \\
 &\quad + f_{2,6}(0.03) + f_{3,6}(0.01) + f_{4,6}(0.0) + f_{5,6}(0.0) \}]
 \end{aligned}$$

$$\begin{aligned}
&= F[7\{-0.04(0.01) - 0.04(0.03) - 0.04(0.07) \\
&\quad + 0.06(0.09) + 0.06(0.1) + 0.06(0.09) + 0.06(0.07) \\
&\quad + 0.06(0.03) - 0.04(0.01) - 0.04(0.0) - 0.04(0.0)\}] \\
&= F[0.1260] = 0.126 \cong 0.13
\end{aligned}$$

For $k = 1$ and $j = 7$, we have

$$\begin{aligned}
y_5(1) &= F\left[7 \sum_{m=-5}^5 f_{m7} y_{7+m}(0)\right] \\
&= F[7\{f_{-5,7} y_{7-5}(0) + f_{-4,7} y_{7-4}(0) + f_{-3,7} y_{7-3}(0) \\
&\quad + f_{-2,7} y_{7-2}(0) + f_{-1,7} y_{7-1}(0) + f_{0,7} y_{7-0}(0) + f_{1,7} y_{7+1}(0) \\
&\quad + f_{2,7} y_{7+2}(0) + f_{3,7} y_{7+3}(0) + f_{4,7} y_{7+4}(0) + f_{5,7} y_{7+5}(0)\}] \\
&= F[7\{f_{-5,7} y_2(0) + f_{-4,7} y_3(0) + f_{-3,7} y_4(0) \\
&\quad + f_{-2,7} y_5(0) + f_{-1,7} y_6(0) + f_{0,7} y_7(0) + f_{1,7} y_8(0) \\
&\quad + f_{2,7} y_9(0) + f_{3,7} y_{10}(0) + f_{4,7} y_{11}(0) + f_{5,7} y_{12}(0)\}] \\
&= F[7\{f_{-5,7}(0.03) + f_{-4,7}(0.07) + f_{-3,7}(0.09) \\
&\quad + f_{-2,7}(0.1) + f_{-1,7}(0.09) + f_{0,7}(0.07) + f_{1,7}(0.03) \\
&\quad + f_{2,7}(0.01) + f_{3,7}(0.0) + f_{4,7}(0.0) + f_{5,7}(0.0)\}] \\
&= F[7\{-0.04(0.03) - 0.04(0.07) - 0.04(0.09) \\
&\quad + 0.06(0.1) + 0.06(0.09) + 0.06(0.07) + 0.06(0.03) \\
&\quad + 0.06(0.01) - 0.04(0.0) - 0.04(0.0) - 0.04(0.0)\}] \\
&= F[0.0728] = 0.0728 \cong 0.07
\end{aligned}$$

For $k = 1$ and $j = 8$, we have

$$\begin{aligned}
y_8(1) &= F\left[7 \sum_{m=-5}^5 f_{m8} y_{8+m}(0)\right] \\
&= F[7\{f_{-5,8} y_{8-5}(0) + f_{-4,8} y_{8-4}(0) + f_{-3,8} y_{8-3}(0) \\
&\quad + f_{-2,8} y_{8-2}(0) + f_{-1,8} y_{8-1}(0) + f_{0,8} y_{8-0}(0) + f_{1,8} y_{8+1}(0) \\
&\quad + f_{2,8} y_{8+2}(0) + f_{3,8} y_{8+3}(0) + f_{4,8} y_{8+4}(0) + f_{5,8} y_{8+5}(0)\}]
\end{aligned}$$

$$\begin{aligned}
&= F[7\{f_{-5,8}y_3(0) + f_{-4,8}y_4(0) + f_{-3,8}y_5(0) \\
&\quad + f_{-2,8}y_6(0) + f_{-1,8}y_7(0) + f_{0,8}y_8(0) + f_{1,8}y_9(0) \\
&\quad + f_{2,8}y_{10}(0) + f_{3,8}y_{11}(0) + f_{4,8}y_{12}(0) + f_{5,8}y_{13}(0)\}] \\
&= F[7\{f_{-5,8}(0.07) + f_{-4,8}(0.09) + f_{-3,8}(0.1) \\
&\quad + f_{-2,8}(0.09) + f_{-1,8}(0.07) + f_{0,8}(0.03) + f_{1,8}(0.01) \\
&\quad + f_{2,8}(0.0) + f_{3,8}(0.0) + f_{4,8}(0.0) + f_{5,8}(0.0)\}] \\
&= F[7\{-0.04(0.07) - 0.04(0.09) - 0.04(0.01) \\
&\quad + 0.06(0.09) + 0.06(0.07) + 0.06(0.03) + 0.06(0.01) \\
&\quad + 0.06(0.0) - 0.04(0.0) - 0.04(0.0) - 0.04(0.0)\}] \\
&= F[0.0112] = 0.0112 \cong 0.01
\end{aligned}$$

For $k = 1$ and $j = 9$, we have

$$\begin{aligned}
y_9(1) &= F\left[7 \sum_{m=-5}^5 f_{m9}y_{9+m}(0)\right] \\
&= F[7\{f_{-5,9}y_{9-5}(0) + f_{-4,9}y_{9-4}(0) + f_{-3,9}y_{9-3}(0) \\
&\quad + f_{-2,9}y_{9-2}(0) + f_{-1,9}y_{9-1}(0) + f_{0,9}y_{9-0}(0) + f_{1,9}y_{9+1}(0) \\
&\quad + f_{2,9}y_{9+2}(0) + f_{3,9}y_{9+3}(0) + f_{4,9}y_{9+4}(0) + f_{5,9}y_{9+5}(0)\}] \\
&= F[7\{f_{-5,9}y_4(0) + f_{-4,9}y_5(0) + f_{-3,9}y_6(0) \\
&\quad + f_{-2,9}y_7(0) + f_{-1,9}y_8(0) + f_{0,9}y_9(0) + f_{1,9}y_{10}(0) \\
&\quad + f_{2,9}y_{11}(0) + f_{3,9}y_{12}(0) + f_{4,9}y_{13}(0) + f_{5,9}y_{14}(0)\}] \\
&= F[7\{f_{-5,9}(0.09) + f_{-4,9}(0.1) + f_{-3,9}(0.09) \\
&\quad + f_{-2,9}(0.07) + f_{-1,9}(0.03) + f_{0,9}(0.01) + f_{1,9}(0.0) \\
&\quad + f_{2,9}(0.0) + f_{3,9}(0.0) + f_{4,9}(0.0) + f_{5,9}(0.0)\}] \\
&= F[7\{-0.04(0.09) - 0.04(0.1) - 0.04(0.09) \\
&\quad + 0.06(0.07) + 0.06(0.03) + 0.06(0.01) + 0.06(0.0) \\
&\quad + 0.06(0.0) - 0.04(0.0) - 0.04(0.0) - 0.04(0.0)\}] \\
&= F[-0.0322] = 0.0
\end{aligned}$$

For $k = 1$ and $j = 10$, we have

$$\begin{aligned}
&= F[7\{f_{-5,10}y_{10-5}(0) + f_{-4,10}y_{10-4}(0) + f_{-3,10}y_{10-3}(0) \\
&+ f_{-2,10}y_{10-2}(0) + f_{-1,10}y_{10-1}(0) + f_{0,10}y_{10-0}(0) + f_{1,10}y_{10+1}(0) \\
&+ f_{2,10}y_{10+2}(0) + f_{3,10}y_{10+3}(0) + f_{4,10}y_{10+4}(0) + f_{5,10}y_{10+5}(0)\}] \\
&= F[7\{f_{-5,10}y_5(0) + f_{-4,10}y_6(0) + f_{-3,10}y_7(0) \\
&+ f_{-2,10}y_8(0) + f_{-1,10}y_9(0) + f_{0,10}y_{10}(0) + f_{1,10}y_{11}(0) \\
&+ f_{2,10}y_{12}(0) + f_{3,10}y_{13}(0) + f_{4,10}y_{14}(0) + f_{5,10}y_{15}(0)\}] \\
&= F[7\{f_{-5,10}(0.1) + f_{-4,10}(0.09) + f_{-3,10}(0.07) \\
&+ f_{-2,10}(0.03) + f_{-1,10}(0.01) + f_{0,10}(0.0) + f_{1,10}(0.0) \\
&+ f_{2,10}(0.0) + f_{3,10}(0.0) + f_{4,10}(0.0) + f_{5,10}(0.0)\}] \\
&= F[7\{-0.04(0.1) - 0.04(0.09) - 0.04(0.07) \\
&+ 0.06(0.03) + 0.06(0.01) + 0.06(0.0) + 0.06(0.0) \\
&+ 0.06(0.0) - 0.04(0.0) - 0.04(0.0) - 0.04(0.0)\}] \\
&= F[-0.0560] = 0.0
\end{aligned}$$

Problem 6.3 (6 to 3 binary vector clustering by SOM)

A self-organizing map (SOM) is used to cluster 6 vectors as given by

$$\mathbf{X}_1 = [1 \ 1 \ 1 \ 0 \ 0 \ 0]$$

$$\mathbf{X}_2 = [0 \ 1 \ 1 \ 1 \ 0 \ 0]$$

$$\mathbf{X}_3 = [0 \ 0 \ 1 \ 1 \ 1 \ 0]$$

$$\mathbf{X}_4 = [0 \ 0 \ 0 \ 1 \ 1 \ 1]$$

$$\mathbf{X}_5 = [1 \ 0 \ 0 \ 0 \ 1 \ 1]$$

$$\mathbf{X}_6 = [1 \ 1 \ 0 \ 0 \ 0 \ 1]$$

The maximum number of clusters to be formed is $C = 3$.

In the network of this example, the number of inputs is 6 and the number of outputs is 3.

Suppose the learning rate parameter is $g(0) = 0.5$ and $g(k + 1) = 0.5 g(k)$, and the initial weight matrix $\mathbf{W}(0)$ is given as

$$\mathbf{W}(0) = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 0 & 0 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

Perform the SOM learning by:

- (a) Showing the step-by-step computations of the first 2 iterations;
- (b) Writing a Matlab program to compute the final weight matrix $g(k + 1) = 0.99 g(k)$; and
- (c) Verifying the recall correctness of clustering using the final weight matrix.

Solution

- (a) Hand-calculation

Iteration 1

$$g(0) = 0.5;$$

Present the input vector $\mathbf{X}_1 = [1 \ 1 \ 1 \ 0 \ 0 \ 0]$:

$$\begin{aligned} d_1 &= (1-1.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (0-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 \\ &= 0.00 \end{aligned}$$

$$\begin{aligned} d_2 &= (1-0.00)^2 + (1-0.00)^2 + (1-1.00)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-0.00)^2 \\ &= 4.00 \end{aligned}$$

$$\begin{aligned} d_3 &= (1-0.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-1.00)^2 \\ &= 6.00 \end{aligned}$$

d_1 is the smallest, hence, the weight vector $\mathbf{W}_1$ will be updated:

$$\mathbf{W}_1 = [1 \ 1 \ 1 \ 0 \ 0 \ 0] + 0.5 * ([1 \ 1 \ 1 \ 0 \ 0 \ 0] - [1 \ 1 \ 1 \ 0 \ 0 \ 0])$$

$$= [1.00 \ 1.00 \ 1.00 \ 0.00 \ 0.00 \ 0.00]$$

Present the Input vector $\mathbf{X}_2 = [0 \ 1 \ 1 \ 1 \ 0 \ 0]$:

$$\begin{aligned} d_1 &= (0-1.00)^2+(1-1.00)^2+(1-1.00)^2+(1-0.00)^2+(0-0.00)^2+(0-0.00)^2 \\ &= 2.00 \end{aligned}$$

$$\begin{aligned} d_2 &= (0-0.00)^2+(1-0.00)^2+(1-1.00)^2+(1-1.00)^2+(0-1.00)^2+(0-0.00)^2 \\ &= 2.00 \end{aligned}$$

$$\begin{aligned} d_3 &= (0-0.00)^2+(1-0.00)^2+(1-0.00)^2+(1-1.00)^2+(0-1.00)^2+(0-1.00)^2 \\ &= 4.00 \end{aligned}$$

d_1 is the smallest, hence, the weight vector $\mathbf{W}_1$ will be updated;

d_2 is the smallest, hence, the weight vector $\mathbf{W}_2$ will be updated:

$$\begin{aligned} \mathbf{W}_1 &= [1 \ 1 \ 1 \ 0 \ 0 \ 0] + 0.5*([0 \ 1 \ 1 \ 1 \ 0 \ 0] - [1 \ 1 \ 1 \ 0 \ 0 \ 0]) \\ &= [0.50 \ 1.00 \ 1.00 \ 0.50 \ 0.00 \ 0.00] \end{aligned}$$

$$\begin{aligned} \mathbf{W}_2 &= [0 \ 0 \ 1 \ 1 \ 1 \ 0] + 0.5*([0 \ 1 \ 1 \ 1 \ 0 \ 0] - [0 \ 0 \ 1 \ 1 \ 1 \ 0]) \\ &= [0.00 \ 0.50 \ 1.00 \ 1.00 \ 0.50 \ 0.00] \end{aligned}$$

Present the input vector $\mathbf{X}_3 = [0 \ 0 \ 1 \ 1 \ 1 \ 0]$:

$$\begin{aligned} d_1 &= (0-0.50)^2+(0-1.00)^2+(1-1.00)^2+(1-0.50)^2+(1-0.00)^2+(0-0.00)^2 \\ &= 2.50 \end{aligned}$$

$$\begin{aligned} d_2 &= (0-0.00)^2+(0-0.50)^2+(1-1.00)^2+(1-1.00)^2+(1-0.50)^2+(0-0.00)^2 \\ &= 0.50 \end{aligned}$$

$$\begin{aligned} d_3 &= (0-0.00)^2+(0-0.00)^2+(1-0.00)^2+(1-1.00)^2+(1-1.00)^2+(0-1.00)^2 \\ &= 2.00 \end{aligned}$$

d_2 is the smallest, hence, the weight vector $\mathbf{W}_2$ will be updated:

$$\mathbf{W}_2 = [0 \ 0.5 \ 1 \ 1 \ 0.5 \ 0] + 0.5*([0 \ 0 \ 1 \ 1 \ 1 \ 0] - [0 \ 0.5 \ 1 \ 1 \ 0.5 \ 0])$$

$$= [0.00 \ 0.25 \ 1.00 \ 1.00 \ 0.75 \ 0.00]$$

Present the input vector $\mathbf{X}_4 = [0 \ 0 \ 0 \ 1 \ 1 \ 1]$:

$$\begin{aligned} d_1 &= (0-0.50)^2+(0-1.00)^2+(0-1.00)^2+(1-0.50)^2+(1-0.00)^2+(1-0.00)^2 \\ &= 4.50 \end{aligned}$$

$$\begin{aligned} d_2 &= (0-0.00)^2+(0-0.25)^2+(0-1.00)^2+(1-1.00)^2+(1-0.75)^2+(1-0.00)^2 \\ &= 2.13 \end{aligned}$$

$$\begin{aligned} d_3 &= (0-0.00)^2+(0-0.00)^2+(0-0.00)^2+(1-1.00)^2+(1-1.00)^2+(1-1.00)^2 \\ &= 0.00 \end{aligned}$$

d_3 is the smallest, hence, the weight vector $\mathbf{W}_3$ will be updated:

$$\begin{aligned} \mathbf{W}_3 &= [0 \ 0 \ 0 \ 1 \ 1 \ 1] + 0.5*([0 \ 0 \ 0 \ 1 \ 1 \ 1] - [0 \ 0 \ 0 \ 1 \ 1 \ 1]) \\ &= [0.00 \ 0.00 \ 0.00 \ 1.00 \ 1.00 \ 1.00] \end{aligned}$$

Present the input vector $\mathbf{X}_5 = [1 \ 0 \ 0 \ 0 \ 1 \ 1]$:

$$\begin{aligned} d_1 &= (1-0.50)^2+(0-1.00)^2+(0-1.00)^2+(0-0.50)^2+(1-0.00)^2+(1-0.00)^2 \\ &= 4.50 \end{aligned}$$

$$\begin{aligned} d_2 &= (1-0.00)^2+(0-0.25)^2+(0-1.00)^2+(0-1.00)^2+(1-0.75)^2+(1-0.00)^2 \\ &= 4.13 \end{aligned}$$

$$\begin{aligned} d_3 &= (1-0.00)^2+(0-0.00)^2+(0-0.00)^2+(0-1.00)^2+(1-1.00)^2+(1-1.00)^2 \\ &= 2.00 \end{aligned}$$

d_3 is the smallest, hence, the weight vector $\mathbf{W}_3$ will be updated:

$$\begin{aligned} \mathbf{W}_3 &= [0 \ 0 \ 0 \ 1 \ 1 \ 1] + 0.5*([1 \ 0 \ 0 \ 0 \ 1 \ 1] - [0 \ 0 \ 0 \ 1 \ 1 \ 1]) \\ &= [0.50 \ 0.00 \ 0.00 \ 0.50 \ 1.00 \ 1.00] \end{aligned}$$

Present the input vector $\mathbf{X}_6 = [1 \ 1 \ 0 \ 0 \ 0 \ 1]$:

$$d_1 = (1-0.50)^2+(1-1.00)^2+(0-1.00)^2+(0-0.50)^2+(0-0.00)^2+(1-0.00)^2$$

$$= 2.50$$

$$d_2 = (1-0.00)^2 + (1-0.25)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-0.75)^2 + (1-0.00)^2 \\ = 5.13$$

$$d_3 = (1-0.50)^2 + (1-0.00)^2 + (0-0.00)^2 + (0-0.50)^2 + (0-1.00)^2 + (1-1.00)^2 \\ = 2.50$$

d_1 is the smallest, hence, the weight vector $\mathbf{W}_1$ will be updated;

d_3 is the smallest, hence, the weight vector $\mathbf{W}_3$ will be updated:

$$\mathbf{W}_1 = [0.5 \ 1 \ 1 \ 0.5 \ 0 \ 0] + 0.5 * ([1 \ 1 \ 0 \ 0 \ 0 \ 1] - [0.5 \ 1 \ 1 \ 0.5 \ 0 \ 0]) \\ = [0.75 \ 1.00 \ 0.50 \ 0.25 \ 0.00 \ 0.50]$$

$$\mathbf{W}_3 = [0.5 \ 0 \ 0 \ 0.5 \ 1 \ 1] + 0.5 * ([1 \ 1 \ 0 \ 0 \ 0 \ 1] - [0.5 \ 0 \ 0 \ 0.5 \ 1 \ 1]) \\ = [0.75 \ 0.50 \ 0.00 \ 0.25 \ 0.50 \ 1.00]$$

After Iteration 1, the weight vectors are:

$$\mathbf{W}_1 = [0.75 \ 1.00 \ 0.50 \ 0.25 \ 0.00 \ 0.50]$$

$$\mathbf{W}_2 = [0.00 \ 0.25 \ 1.00 \ 1.00 \ 0.75 \ 0.00]$$

$$\mathbf{W}_3 = [0.75 \ 0.50 \ 0.00 \ 0.25 \ 0.50 \ 1.00]$$

Computing $g(2) = 0.5 * g(1) = 0.25$.

Iteration 2

Present the input vector $\mathbf{X}_1 = [1 \ 1 \ 1 \ 0 \ 0 \ 0]$:

$$d_1 = (1-0.75)^2 + (1-1.00)^2 + (1-0.50)^2 + (0-0.25)^2 + (0-0.00)^2 + (0-0.50)^2 \\ = 0.63$$

$$d_2 = (1-0.00)^2 + (1-0.25)^2 + (1-1.00)^2 + (0-1.00)^2 + (0-0.75)^2 + (0-0.00)^2 \\ = 3.13$$

$$d_3 = (1-0.75)^2 + (1-0.50)^2 + (1-0.00)^2 + (0-0.25)^2 + (0-0.50)^2 + (0-1.00)^2$$

$$= 2.63$$

d_1 is the smallest, hence, the weight vector $\mathbf{W}_1$ will be updated:

$$\begin{aligned}\mathbf{W}_1 &= [0.75 \ 1 \ 0.5 \ 0.25 \ 0 \ 0.5] \\ &+ 0.25 * ([1 \ 1 \ 1 \ 0 \ 0 \ 0] - [0.75 \ 1 \ 0.5 \ 0.25 \ 0 \ 0.5]) \\ &= [0.81 \ 1.00 \ 0.63 \ 0.19 \ 0.00 \ 0.38]\end{aligned}$$

Present the input vector $\mathbf{X}_2 = [0 \ 1 \ 1 \ 1 \ 0 \ 0]$:

$$\begin{aligned}d_1 &= (0-0.81)^2 + (1-1.00)^2 + (1-0.63)^2 + (1-0.19)^2 + (0-0.00)^2 + (0-0.38)^2 \\ &= 1.60\end{aligned}$$

$$\begin{aligned}d_2 &= (0-0.00)^2 + (1-0.25)^2 + (1-1.00)^2 + (1-1.00)^2 + (0-0.75)^2 + (0-0.00)^2 \\ &= 1.13\end{aligned}$$

$$\begin{aligned}d_3 &= (0-0.75)^2 + (1-0.50)^2 + (1-0.00)^2 + (1-0.25)^2 + (0-0.50)^2 + (0-1.00)^2 \\ &= 3.63\end{aligned}$$

d_2 is the smallest, hence, the weight vector $\mathbf{W}_2$ will be updated:

$$\begin{aligned}\mathbf{W}_2 &= [0 \ 0.25 \ 1 \ 1 \ 0.75 \ 0] + 0.25 * ([0 \ 1 \ 1 \ 1 \ 0 \ 0] - [0 \ 0.25 \ 1 \ 1 \ 0.75 \ 0]) \\ &= [0.00 \ 0.44 \ 1.00 \ 1.00 \ 0.56 \ 0.00]\end{aligned}$$

Present the input vector $\mathbf{X}_3 = [0 \ 0 \ 1 \ 1 \ 1 \ 0]$:

$$\begin{aligned}d_1 &= (0-0.81)^2 + (0-1.00)^2 + (1-0.63)^2 + (1-0.19)^2 + (1-0.00)^2 + (0-0.38)^2 \\ &= 3.60\end{aligned}$$

$$\begin{aligned}d_2 &= (0-0.00)^2 + (0-0.44)^2 + (1-1.00)^2 + (1-1.00)^2 + (1-0.56)^2 + (0-0.00)^2 \\ &= 0.38\end{aligned}$$

$$\begin{aligned}d_3 &= (0-0.75)^2 + (0-0.50)^2 + (1-0.00)^2 + (1-0.25)^2 + (1-0.50)^2 + (0-1.00)^2 \\ &= 3.63\end{aligned}$$

d_2 is the smallest, hence, the weight vector $\mathbf{W}_2$ will be updated:

$$\mathbf{W}_2 = [0 \ 0.4375 \ 1 \ 1 \ 0.5625 \ 0] + 0.25*([0 \ 0 \ 1 \ 1 \ 1 \ 0] - [0 \ 0.4375 \ 1 \ 1 \ 0.5625 \ 0]) = [0.00 \ 0.33 \ 1.00 \ 1.00 \ 0.67 \ 0.00]$$

Present the input vector $\mathbf{X}_4 = [0 \ 0 \ 0 \ 1 \ 1 \ 1]$:

$$\begin{aligned} d_1 &= (0-0.81)^2 + (0-1.00)^2 + (0-0.63)^2 + (1-0.19)^2 + (1-0.00)^2 + (1-0.38)^2 \\ &= 4.10 \end{aligned}$$

$$\begin{aligned} d_2 &= (0-0.00)^2 + (0-0.33)^2 + (0-1.00)^2 + (1-1.00)^2 + (1-0.67)^2 + (1-0.00)^2 \\ &= 2.22 \end{aligned}$$

$$\begin{aligned} d_3 &= (0-0.75)^2 + (0-0.50)^2 + (0-0.00)^2 + (1-0.25)^2 + (1-0.50)^2 + (1-1.00)^2 \\ &= 1.63 \end{aligned}$$

d_3 is the smallest, hence, the weight vector $\mathbf{W}_3$ will be updated:

$$\mathbf{W}_3 = [0.75 \ 0.5 \ 0 \ 0.25 \ 0.5 \ 1] + 0.25*([0 \ 0 \ 0 \ 1 \ 1 \ 1] - [0.75 \ 0.5 \ 0 \ 0.25 \ 0.5 \ 1]) = [0.56 \ 0.38 \ 0.00 \ 0.44 \ 0.63 \ 1.00]$$

Present the input vector $\mathbf{X}_5 = [1 \ 0 \ 0 \ 0 \ 1 \ 1]$:

$$\begin{aligned} d_1 &= (1-0.81)^2 + (0-1.00)^2 + (0-0.63)^2 + (0-0.19)^2 + (1-0.00)^2 + (1-0.38)^2 \\ &= 2.85 \end{aligned}$$

$$\begin{aligned} d_2 &= (1-0.00)^2 + (0-0.33)^2 + (0-1.00)^2 + (0-1.00)^2 + (1-0.67)^2 + (1-0.00)^2 \\ &= 4.22 \end{aligned}$$

$$\begin{aligned} d_3 &= (1-0.56)^2 + (0-0.38)^2 + (0-0.00)^2 + (0-0.44)^2 + (1-0.63)^2 + (1-1.00)^2 \\ &= 0.66 \end{aligned}$$

d_3 is the smallest, hence, the weight vector $\mathbf{W}_3$ will be updated:

$$\mathbf{W}_3 = [0.5625 \ 0.375 \ 0 \ 0.4375 \ 0.625 \ 1] + 0.25*([1 \ 0 \ 0 \ 0 \ 1 \ 1] - [0.5625 \ 0.375 \ 0 \ 0.4375 \ 0.625 \ 1]) = [0.67 \ 0.28 \ 0.00 \ 0.33 \ 0.72 \ 1.00]$$

Present the input vector $\mathbf{X}_6 = [1 \ 1 \ 0 \ 0 \ 0 \ 1]$:

$$d_1 = (1-0.81)^2 + (1-1.00)^2 + (0-0.63)^2 + (0-0.19)^2 + (0-0.00)^2 + (1-0.38)^2$$

$$= 0.85$$

$$d_2 = (1-0.00)^2 + (1-0.33)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-0.67)^2 + (1-0.00)^2 \\ = 4.90$$

$$d_3 = (1-0.67)^2 + (1-0.28)^2 + (0-0.00)^2 + (0-0.33)^2 + (0-0.72)^2 + (1-1.00)^2 \\ = 1.25$$

d_1 is the smallest, hence, the weight vector $\mathbf{W}_1$ will be updated:

$$\mathbf{W}_1 = [0.8125 \ 1 \ 0.625 \ 0.1875 \ 0 \ 0.375] + 0.25 * ([1 \ 1 \ 0 \ 0 \ 0 \ 1] - [0.8125 \ 1 \ 0.625 \ 0.1875 \ 0 \ 0.375]) = [0.86 \ 1.00 \ 0.47 \ 0.14 \ 0.00 \ 0.53]$$

After Iteration 2, the weight vectors are:

$$\mathbf{W}_1 = [0.86 \ 1.00 \ 0.47 \ 0.14 \ 0.00 \ 0.53]$$

$$\mathbf{W}_2 = [0.00 \ 0.33 \ 1.00 \ 1.00 \ 0.67 \ 0.00]$$

$$\mathbf{W}_3 = [0.67 \ 0.28 \ 0.00 \ 0.33 \ 0.72 \ 1.00]$$

(b)

Matlab program

```
% Problem 6.3 (som_P6_3.m)
% 1. SOM is used to cluster 6 input vectors Xk into a maximum of
% C=3 clusters.
% 2. A SOM net with three output neurons is first initialized, so 3
% or less number of clusters will be formed at the end.
% 3. For small number of clusters like 3, neighboring updates are not
% effective.
% 4. Learning parameter g(k) is updated (x0.99) at the end of
% each iteration.
clear all;
gk = 0.5; % Set initial value of learning rate g().
X = [1 1 1 0 0 0; 0 1 1 1 0 0; 0 0 1 1 1 0; ... % Input vectors
     0 0 0 1 1 1; 1 0 0 0 1 1; 1 1 0 0 0 1];
% 3 distinct vectors are selected for 3 initial weight vectors.
W = [X(1,:);X(3,:);X(4,:)];
```

```

% Perform iterations according to SOM weight update algorithm.
for iter=1:100          % Cycle through iterations
    for k = 1:size(X,1) % Cycle through input vectors (patterns)
        % STEP 1:
        % B = repmat(A,M,N) creates a matrix B consisting of an M-by-N
        % tiling of copies of A. The size of B[size(A,1)*M,size(A,2)*N].
        % Example: For size(A)=[1,6] & size(W,1)=3, size(B)=[3,6].
        % d(3,1) gives the sum of squared differences between X(k,:) with
        % W(1:3,:).
        d = sum((repmat(X(k,:),size(W,1),1) - W).^2,2);
        % STEP 2: Find weight vector that has the smallest distance
        % from X(k) which gives the index of the winning neuron.
        bmu = find(d == min(d)); % finding index of winning neuron(s)
        % STEP 3: Update weight matrix. Notice that the weight matrix
        % is only updated at column "bmu", which corresponds to the
        % index of the winning neuron.
        for j=1:length(bmu) % Updating the winning weight vectors
            W(bmu(j),:) = W(bmu(j),:) + gk*(X(k,:) - W(bmu(j),:));
        end
    end
    gk = 0.99*gk; % update g(k) after each iteration.
end
W = W'; % Transpose the weight vector
fprintf('The weight vector after 100 iterations\n\n');
disp(W)
% Recall output using: Yk = Xk*WT
for k=1:size(X,1)
    Y(k,:) = X(k,:)*W;
end
fprintf('The output vector after 100 iterations\n\n');
disp(Y) % Display output Yk

```

(c) Verification

The weight vector after 100 iterations

$$\mathbf{W}_1 = [1.0000 \quad 1.0000 \quad 0.4480 \quad 0.0000 \quad 0.0000 \quad 0.5520]$$

$$\mathbf{W}_2 = [0.0000 \quad 0.4480 \quad 1.0000 \quad 1.0000 \quad 0.5520 \quad 0.0000]$$

$$\mathbf{W}_3 = [0.5520 \quad 0.0000 \quad 0.0000 \quad 0.4480 \quad 1.0000 \quad 1.0000]$$

The output vector after 100 iterations

$$\mathbf{Y}_1 = [2.4480 \quad 1.4480 \quad 0.5520]$$

$$\mathbf{Y}_2 = [1.4480 \quad 2.4480 \quad 0.4480]$$

$$\mathbf{Y}_3 = [0.4480 \quad 2.5520 \quad 1.4480]$$

$$\mathbf{Y}_4 = [0.5520 \quad 1.5520 \quad 2.4480]$$

$$\mathbf{Y}_5 = [1.5520 \quad 0.5520 \quad 2.5520]$$

$$\mathbf{Y}_6 = [2.5520 \quad 0.4480 \quad 1.5520]$$

Note that three distinct clusters, namely $\mathbf{X}_1$ and $\mathbf{X}_6$, $\mathbf{X}_2$ and $\mathbf{X}_3$, $\mathbf{X}_4$ and $\mathbf{X}_5$ have been formed.

Problem 6.4 (16 to 5 bipolar vectors clustering by SOM)

In this problem, 16 bipolar vectors $\mathbf{X}$ are to be clustered into 5 clusters using the SOM learning.

Given the initial learning rate is $g(0) = 0.5$ and $g(k+1) = 0.99 g(k)$ for $k \geq 1$. The neighborhood width is 2 for the initial 100 iterations and is decreased to 1 after 100 iterations.

The first 5 bipolar vectors are to be used as the initial weight vectors. Write a Matlab program to plot the clustering results obtained after 500 iterations.

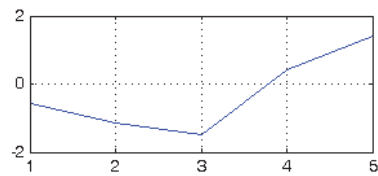
$$\begin{aligned} \mathbf{X} = & [-1 \ -1 \ -1 \ -1; -1 \ -1 \ -1 \ 1; -1 \ -1 \ 1 \ -1; -1 \ -1 \ 1 \ 1; \\ & -1 \ 1 \ -1 \ -1; -1 \ 1 \ -1 \ 1; -1 \ 1 \ 1 \ -1; -1 \ 1 \ 1 \ 1; \\ & 1 \ -1 \ -1 \ -1; 1 \ -1 \ -1 \ 1; 1 \ -1 \ 1 \ -1; 1 \ -1 \ 1 \ 1; \\ & 1 \ 1 \ -1 \ -1; 1 \ 1 \ -1 \ 1; 1 \ 1 \ 1 \ -1; 1 \ 1 \ 1 \ 1]; \end{aligned}$$

Solution

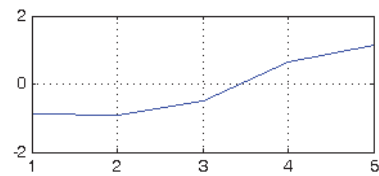
In this Worked Problem, the recall output shape can be used to distinguish different clusters from each other.

For the cluster 1, input patterns 5, 6, 7, and 13 have the same cluster.

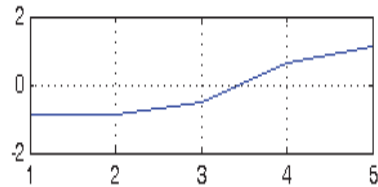
Cluster 1



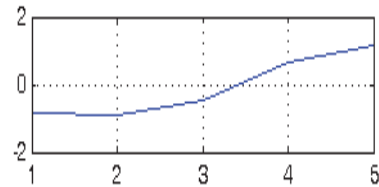
Y5



Y6

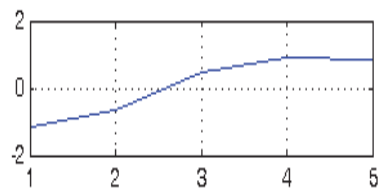


Y7

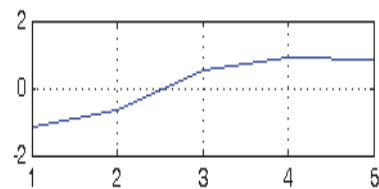


Y13

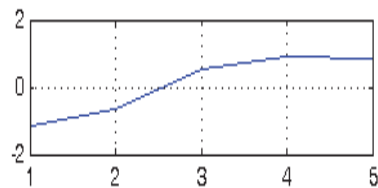
Cluster 2



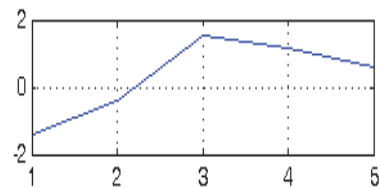
Y8



Y14

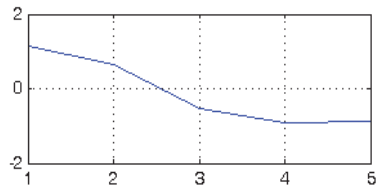


Y15

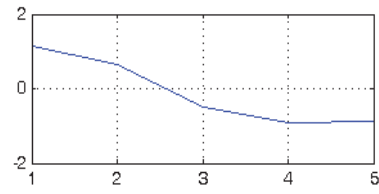


Y16

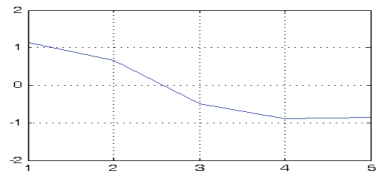
Cluster 3



Y2

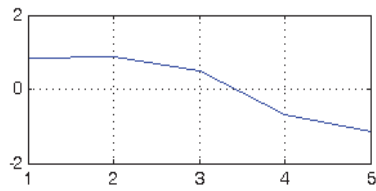


Y3

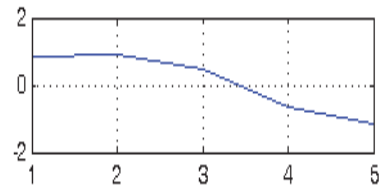


Y9

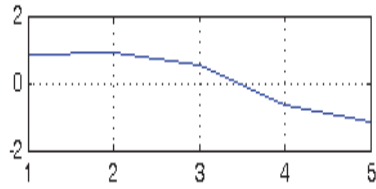
Cluster 4



Y4

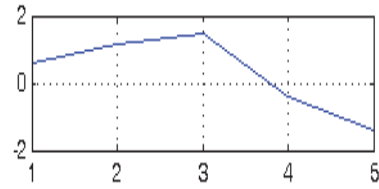


Y10

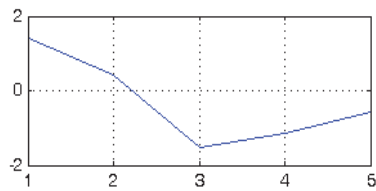


Y11

Cluster 5



Y12



Y1

Matlab program

```

% Problem 6.4 (som_P6_4.m)
% C clustering problem
% K = No of input vectors; I = No of elements per input vector;
% 1. SOM is used to cluster K input vectors Xk into a maximum of
% C clusters.
% 2. A SOM net with C output neurons is first initialized, so C or less
% number of clusters will be formed at the end.
% 3. Neighboring updates are to be used for large C.
% 4. The learning parameter gk is updated at the end of each iteration.
close all; clear all;
% Nb = Initial Neighbourhood width;
Nb = 2;
gk = 0.5; % Initial value of learning rate g(0).
X = [-1 -1 -1 -1; -1 -1 -1 1; -1 -1 1 -1; -1 -1 1 1; ... % Input vectors
     -1 1 -1 -1; -1 1 -1 1; -1 1 1 -1; -1 1 1 1; ...
     1 -1 -1 -1; 1 -1 -1 1; 1 -1 1 -1; 1 -1 1 1; ...
     1 1 -1 -1; 1 1 -1 1; 1 1 1 -1; 1 1 1 1];
C = 5; % C = Max no of clusters;
W = [X(1,:);X(2,:);X(3,:);X(4,:);X(5,:)]; % Set initial weight vectors.
% Perform iterations according to the SOM weight update algorithm.
for iter=1:500
    for k = 1:size(X,1)
        % STEP 1: K = 16; I = 4;
        % B = repmat(A,M,N) creates a matrix B of an M-by-N tiling of
        % copies of A.
        % The size of B is [size(A,1)*C, size(A,2)*1].
        % Example: For size(A)=[1,I] & size(W,1)=C, size(B)=[C,I].
        % d(C,1) gives the sum of squared differences between X(k,:) and
        % W(1:C,:).
        d = sum((repmat(X(k,:),size(W,1),1) - W).^2,2);
        % STEP 2: Find weight vector that has the smallest distance
        % from X(k), which yields the index of the winning neuron(s).
        bmu = find(d == min(d));
        % STEP 3: Update weight matrix. Note that the weight matrix is

```

```

% only updated at column "bmu", which corresponds to the index
% of the winning neuron.
for j=1:length(bmu) % Update the winning weight vectors
    W(bmu(j),:) = W(bmu(j),:) + gk*(X(k,:) - W(bmu(j),:));
    % Update left & right neighbors for neighborhood of 1
    for Nbt = 1:Nb
        if (bmu(j)-Nbt) >= 1
            W(bmu(j)-Nbt,:)=W(bmu(j)-Nbt,:)...
                +gk*(X(k,:)-W(bmu(j)-Nbt,:));
        end
        if (bmu(j)+Nbt) <= C
            W(bmu(j)+Nbt,:)=W(bmu(j)+Nbt,:)...
                +gk*(X(k,:)-W(bmu(j)+Nbt,:));
        end
    end
end
end
gk = 0.99*gk;    % Halve g(k) after each iteration.
if iter >= 100; Nb=1; end;
end
%W = W';
fprintf('Weight Vectors\n\n');
disp(W)
% Recall output: Yk = Xk*WT
XMin = 1; XMax = C; YMin = -2; YMax = 2;
for k=1:size(X,1)
    Y(k,:) = X(k,:)*W';
    figure; plot(Y(k,:)); axis([XMin XMax YMin YMax]); grid;
end
fprintf('Output Vectors\n\n');
disp(Y) %Display output Yk

```

Problem 6.5 (2-D self-organizing map)

Consider a two-dimensional self-organization network consisting of 4×4 outputs, with 2 random inputs $[x_1(k), x_2(k)]$ and 4×4 initial random outputs $[u_{ij}(k), v_{ij}(k)]$.

The value of each input/output ranges from 0.0 to 1.0.

The learning rate parameter $g(k)$ and the square neighborhood width are given in Table P6.5.1.

The maximum number of iterations is 500. Show the self-organization results at $k = 0$ to 5, 20, 40, and 500.

Table P6.5.1

Iteration, k	$g(k)$	Square Neighborhood Width
0	0.6	2
20	0.4	1
40	0.2	1

Solution

$$k = 0: x_1(k) = 0.6208, x_2(k) = 0.6062$$

$$[u_{ij}(k), v_{ij}(k)] =$$

[0.24, 0.56]	[0.47, 0.11]	[0.10, 0.45]	[0.44, 0.37]
[0.87, 0.48]	[0.33, 0.72]	[0.73, 0.09]	[0.86, 0.03]
[0.12, 0.00]	[0.85, 0.81]	[0.45, 1.00]	[1.00, 0.03]
[0.85, 0.64]	[0.58, 0.81]	[0.95, 0.03]	[0.25, 0.19]

$$r_{ij}(k) \angle \theta_{ij}(k) =$$

0.6107 $\angle$ 66.80°	0.4854 $\angle$ 13.17°	0.4574 $\angle$ 77.47°	0.5711 $\angle$ 40.06°
0.9903 $\angle$ 28.88°	0.7937 $\angle$ 65.37°	0.7379 $\angle$ 7.02°	0.8567 $\angle$ 1.99°
0.1225 $\angle$ 0°	1.1788 $\angle$ 43.61°	1.0943 $\angle$ 65.77°	0.9962 $\angle$ 1.71°
1.0578 $\angle$ 36.97°	0.9935 $\angle$ 54.39°	0.9503 $\angle$ 1.80°	0.3129 $\angle$ 37.23°

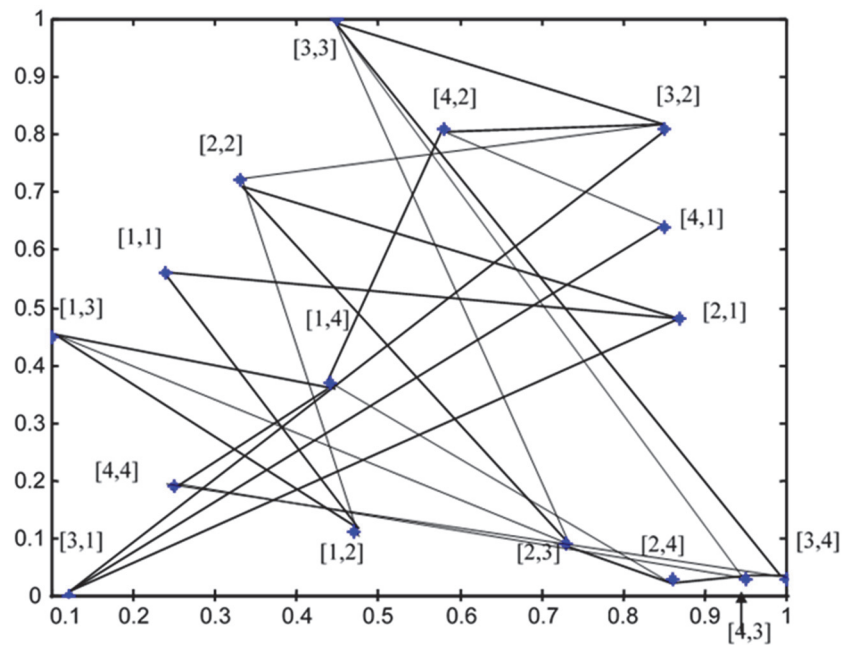


Figure P6.5.1 Two-dimensional self-organization results at $k = 0$.

$$k = 1: x_1(k) = 0.7770, x_2(k) = 0.8840$$

$$[u_{ij}(k), v_{ij}(k)] =$$

[0.24, 0.56]	[0.47, 0.11]	[0.10, 0.45]	[0.44, 0.37]
[0.72, 0.56]	[0.50, 0.65]	[0.67, 0.40]	[0.72, 0.38]
[0.42, 0.37]	[0.71, 0.69]	[0.55, 0.76]	[0.77, 0.38]
[0.71, 0.62]	[0.60, 0.69]	[0.75, 0.38]	[0.47, 0.44]

$$r_{ij}(k) \angle \theta_{ij}(k) =$$

0.6107 $\angle$ 66.80°	0.4854 $\angle$ 13.17°	0.4574 $\angle$ 77.47°	0.5711 $\angle$ 40.06°
0.9088 $\angle$ 37.87°	0.8243 $\angle$ 52.43°	0.7768 $\angle$ 30.83°	0.8075 $\angle$ 27.82°
0.5588 $\angle$ 41.37°	0.9921 $\angle$ 44.18°	0.9414 $\angle$ 54.10°	0.8579 $\angle$ 26.26°
0.9418 $\angle$ 41.12°	0.9146 $\angle$ 48.99°	0.8417 $\angle$ 26.86°	0.6449 $\angle$ 43.11°

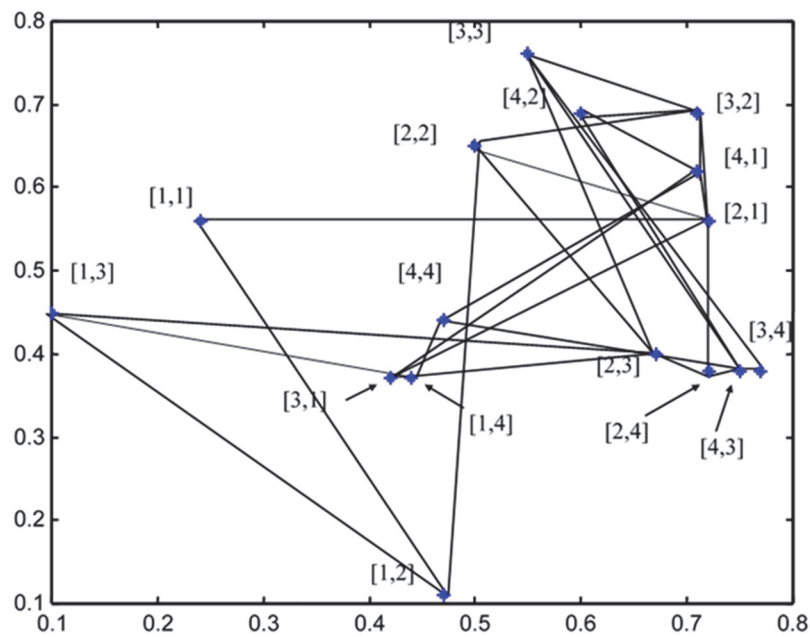


Figure P6.5.2 Two-dimensional self-organization results at $k = 1$.

$$k = 2: x_1(k) = 0.7055, x_2(k) = 0.8131$$

$$[u_{ij}(k), v_{ij}(k)] =$$

[0.56, 0.76]	[0.66, 0.57]	[0.51, 0.71]	[0.64, 0.68]
[0.75, 0.75]	[0.67, 0.79]	[0.73, 0.69]	[0.75, 0.68]
[0.63, 0.68]	[0.75, 0.81]	[0.69, 0.84]	[0.77, 0.68]
[0.75, 0.78]	[0.71, 0.81]	[0.77, 0.68]	[0.66, 0.71]

$$r_{ij}(k) \angle \theta_{ij}(k) =$$

0.9413 $\angle$ 53.62°	0.8708 $\angle$ 40.82°	0.8708 $\angle$ 54.31°	0.9327 $\angle$ 46.74°
1.0653 $\angle$ 45.00°	1.0354 $\angle$ 49.70°	1.0060 $\angle$ 43.39°	0.0143 $\angle$ 42.20°
0.9282 $\angle$ 47.19°	1.1021 $\angle$ 47.20°	1.0816 $\angle$ 50.60°	1.0314 $\angle$ 41.45°
1.0807 $\angle$ 46.12°	1.0720 $\angle$ 48.76°	1.0260 $\angle$ 41.45°	0.9631 $\angle$ 47.09°

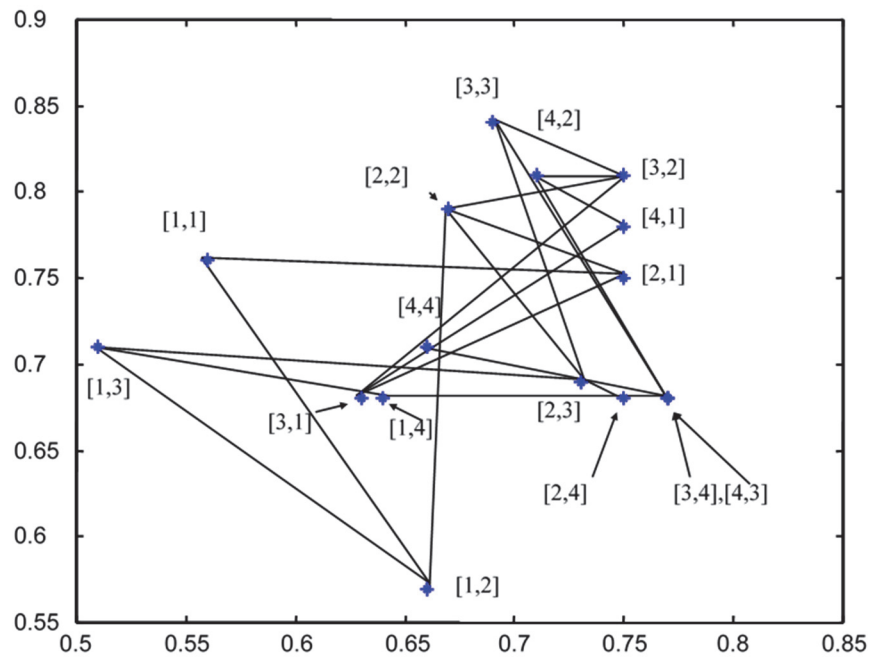


Figure P6.5.3 Two-dimensional self-organization results at $k = 2$.

$$k = 3: x_1(k) = 0.7736, x_2(k) = 0.0424$$

$$[u_{ij}(k), v_{ij}(k)] =$$

[0.56, 0.76]	[0.66, 0.57]	[0.51, 0.71]	[0.64, 0.68]
[0.72, 0.79]	[0.69, 0.80]	[0.72, 0.76]	[0.72, 0.76]
[0.68, 0.76]	[0.72, 0.81]	[0.70, 0.82]	[0.73, 0.76]
[0.72, 0.80]	[0.71, 0.81]	[0.73, 0.76]	[0.69, 0.77]

$$r_{ij}(k) \angle \theta_{ij}(k) =$$

0.9413 $\angle$ 53.62°	0.8708 $\angle$ 40.82°	0.8708 $\angle$ 54.31°	0.9327 $\angle$ 46.74°
1.0714 $\angle$ 47.65°	1.0600 $\angle$ 49.22°	1.0473 $\angle$ 46.55°	1.0498 $\angle$ 46.55°
1.0170 $\angle$ 48.18°	1.0866 $\angle$ 48.37°	1.0784 $\angle$ 49.51°	1.0562 $\angle$ 46.15°
1.0778 $\angle$ 48.01°	1.0747 $\angle$ 48.76°	1.0542 $\angle$ 46.15°	1.0310 $\angle$ 48.14°

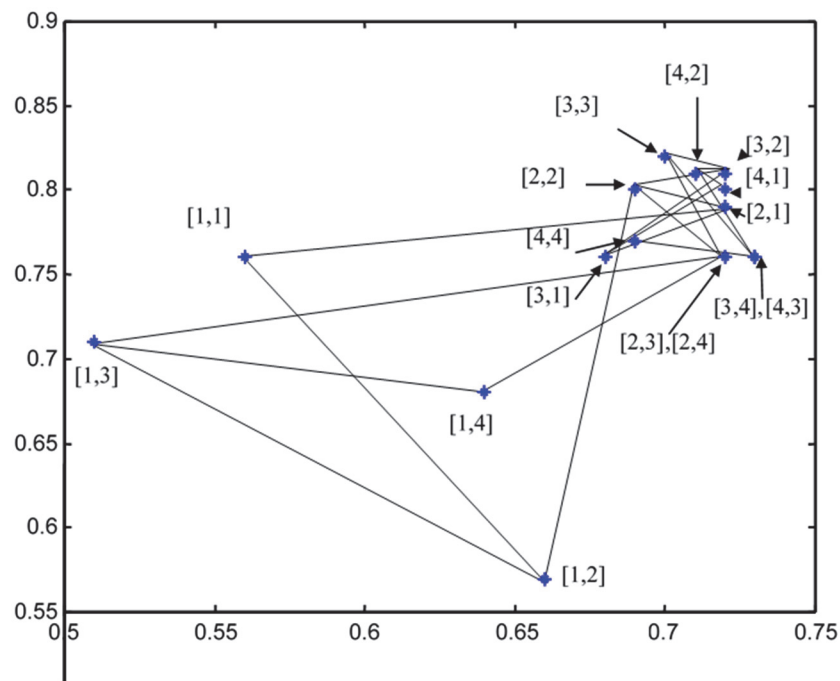


Figure P6.5.4 Two-dimensional self-organization results at $k = 3$.

$$k = 4: x_1(k) = 0.7848, x_2(k) = 0.9112$$

$$[u_{ij}(k), v_{ij}(k)] =$$

[0.69, 0.33]	[0.73, 0.25]	[0.67, 0.31]	[0.72, 0.30]
[0.75, 0.34]	[0.74, 0.35]	[0.75, 0.33]	[0.75, 0.33]
[0.74, 0.33]	[0.75, 0.35]	[0.74, 0.35]	[0.76, 0.33]
[0.72, 0.80]	[0.71, 0.81]	[0.73, 0.76]	[0.69, 0.77]

$$r_{ij}(k) \angle \theta_{ij}(k) =$$

0.7629 $\angle$ 25.56°	0.7698 $\angle$ 18.90°	0.7346 $\angle$ 24.83°	0.7790 $\angle$ 22.62°
0.8276 $\angle$ 24.39°	0.8177 $\angle$ 25.31°	0.8205 $\angle$ 23.75°	0.8227 $\angle$ 23.75°
0.8053 $\angle$ 24.03°	0.8309 $\angle$ 25.02°	0.8235 $\angle$ 25.31°	0.8260 $\angle$ 23.47°
1.0778 $\angle$ 48.01°	1.0747 $\angle$ 48.76°	1.0542 $\angle$ 46.15°	1.0310 $\angle$ 48.14°

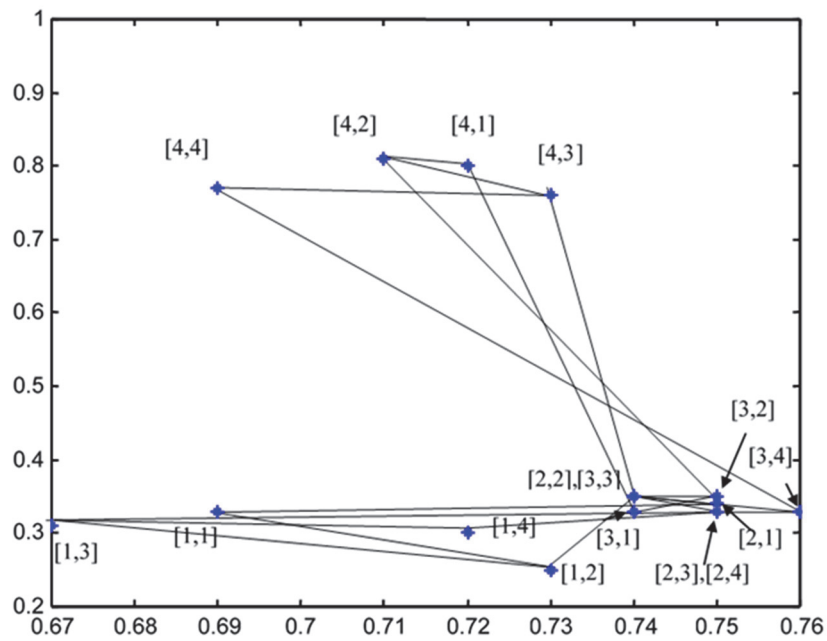


Figure P6.5.5 Two-dimensional self-organization results at $k = 4$.

$$k = 5: x_1(k) = 0.4714, x_2(k) = 0.9726$$

$$[u_{ij}(k), v_{ij}(k)] =$$

[0.69, 0.33]	[0.73, 0.25]	[0.67, 0.31]	[0.72, 0.30]
[0.77, 0.68]	[0.77, 0.69]	[0.77, 0.68]	[0.75, 0.33]
[0.76, 0.68]	[0.77, 0.69]	[0.77, 0.69]	[0.76, 0.33]
[0.76, 0.87]	[0.75, 0.87]	[0.76, 0.85]	[0.69, 0.77]

$$r_{ij}(k) \angle \theta_{ij}(k) =$$

0.7629 $\angle$ 25.56°	0.7698 $\angle$ 18.90°	0.7346 $\angle$ 24.83°	0.7790 $\angle$ 22.62°
1.0313 $\angle$ 41.45°	1.0288 $\angle$ 41.86°	1.0276 $\angle$ 41.45°	0.8227 $\angle$ 23.75°
1.0223 $\angle$ 41.82°	1.0334 $\angle$ 41.86°	1.0316 $\angle$ 41.86°	0.8260 $\angle$ 23.47°
1.1526 $\angle$ 48.86°	1.1514 $\angle$ 49.24°	1.1428 $\angle$ 48.20°	1.0310 $\angle$ 48.14°

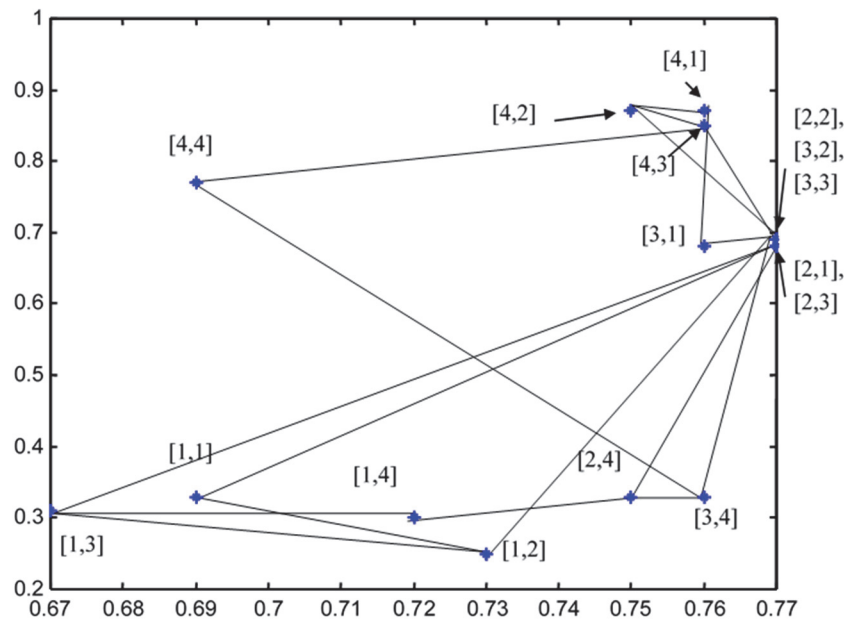


Figure P6.5.6 Two-dimensional self-organization results at $k = 5$.

$$k = 20: x_1(k) = 0.2937, x_2(k) = 0.5721$$

$$[u_{ij}(k), v_{ij}(k)] =$$

[0.73, 0.75]	[0.58, 0.59]	[0.58, 0.59]	[0.26, 0.09]
[0.73, 0.86]	[0.63, 0.84]	[0.63, 0.84]	[0.11, 0.56]
[0.73, 0.86]	[0.63, 0.84]	[0.63, 0.84]	[0.11, 0.56]
[0.75, 0.88]	[0.64, 0.87]	[0.64, 0.87]	[0.14, 0.80]

$$r_{ij}(k) \angle \theta_{ij}(k) =$$

1.0494 $\angle$ 45.77°	0.8236 $\angle$ 45.49°	0.8236 $\angle$ 45.49°	0.2752 $\angle$ 19.09°
1.1256 $\angle$ 49.67°	1.0526 $\angle$ 53.13°	1.0526 $\angle$ 53.13°	0.5693 $\angle$ 78.89°
1.1256 $\angle$ 49.67°	1.0526 $\angle$ 53.13°	1.0526 $\angle$ 53.13°	0.5693 $\angle$ 78.89°
1.1514 $\angle$ 49.56°	1.0830 $\angle$ 53.66°	1.0830 $\angle$ 53.66°	0.8164 $\angle$ 80.07°

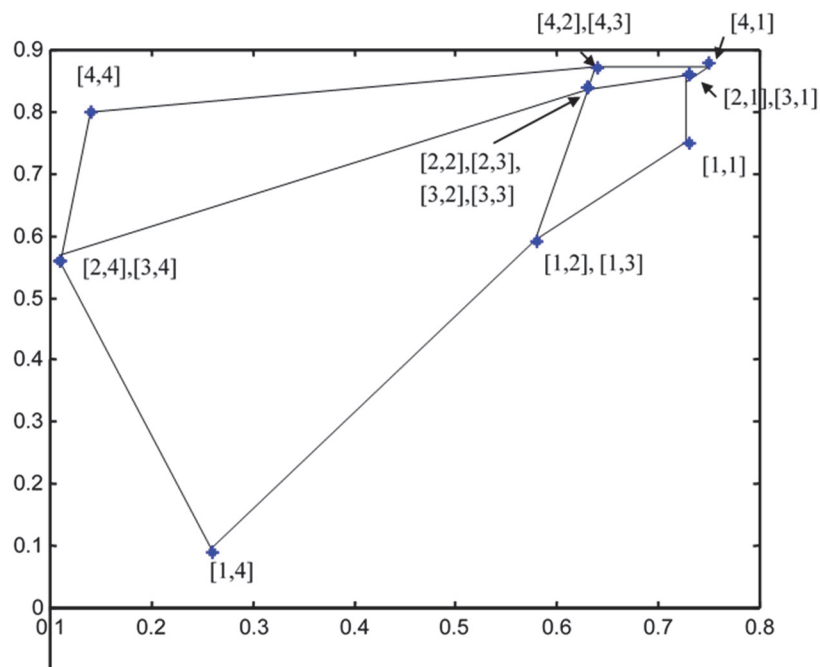


Figure P6.5.7 Two-dimensional self-organization results at $k = 20$.

$$k = 40: x_1(k) = 0.4194, x_2(k) = 0.6793$$

$$[u_{ij}(k), v_{ij}(k)] =$$

[0.75, 0.39]	[0.72, 0.37]	[0.24, 0.30]	[0.20, 0.28]
[0.75, 0.39]	[0.72, 0.37]	[0.24, 0.30]	[0.20, 0.30]
[0.80, 0.80]	[0.75, 0.80]	[0.25, 0.54]	[0.20, 0.54]
[0.84, 0.92]	[0.75, 0.91]	[0.29, 0.80]	[0.18, 0.78]

$$r_{ij}(k) \angle \theta_{ij}(k) =$$

0.8430 $\angle$ 27.47°	0.8134 $\angle$ 27.19°	0.3833 $\angle$ 51.34°	0.3448 $\angle$ 54.46°
0.8438 $\angle$ 27.47°	0.8151 $\angle$ 27.19°	0.3834 $\angle$ 51.34°	0.3609 $\angle$ 56.30°
1.1277 $\angle$ 45.00°	1.1010 $\angle$ 46.84°	0.5958 $\angle$ 65.15°	0.5702 $\angle$ 69.67°
1.2479 $\angle$ 47.60°	1.1754 $\angle$ 50.50°	0.8469 $\angle$ 70.07°	0.8016 $\angle$ 77.00°

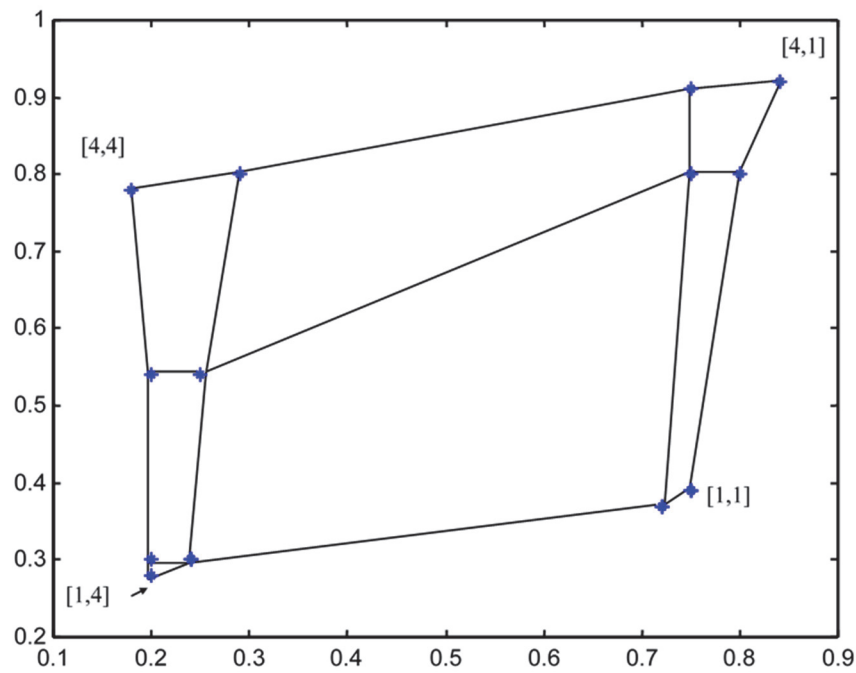


Figure P6.5.8 Two-dimensional self-organization results at $k = 40$.

$$k = 500: x_1(k) = 0.3211, x_2(k) = 0.7124$$

$$[u_{ij}(k), v_{ij}(k)] =$$

[0.72, 0.22]	[0.71, 0.22]	[0.31, 0.31]	[0.18, 0.29]
[0.73, 0.29]	[0.64, 0.38]	[0.32, 0.45]	[0.20, 0.43]
[0.71, 0.59]	[0.57, 0.65]	[0.34, 0.68]	[0.24, 0.67]
[0.67, 0.71]	[0.50, 0.75]	[0.37, 0.76]	[0.26, 0.75]

$$r_{ij}(k) \angle \theta_{ij}(k) =$$

0.7533 $\angle$ 16.99°	0.7415 $\angle$ 17.22°	0.4407 $\angle$ 45.00°	0.3413 $\angle$ 58.17°
0.7900 $\angle$ 21.67°	0.7450 $\angle$ 30.70°	0.5550 $\angle$ 54.58°	0.4739 $\angle$ 65.06°
0.9176 $\angle$ 39.73°	0.8663 $\angle$ 48.75°	0.7566 $\angle$ 63.43°	0.7135 $\angle$ 70.29°
0.9730 $\angle$ 46.66°	0.9000 $\angle$ 56.31°	0.8448 $\angle$ 64.04°	0.7969 $\angle$ 70.88°

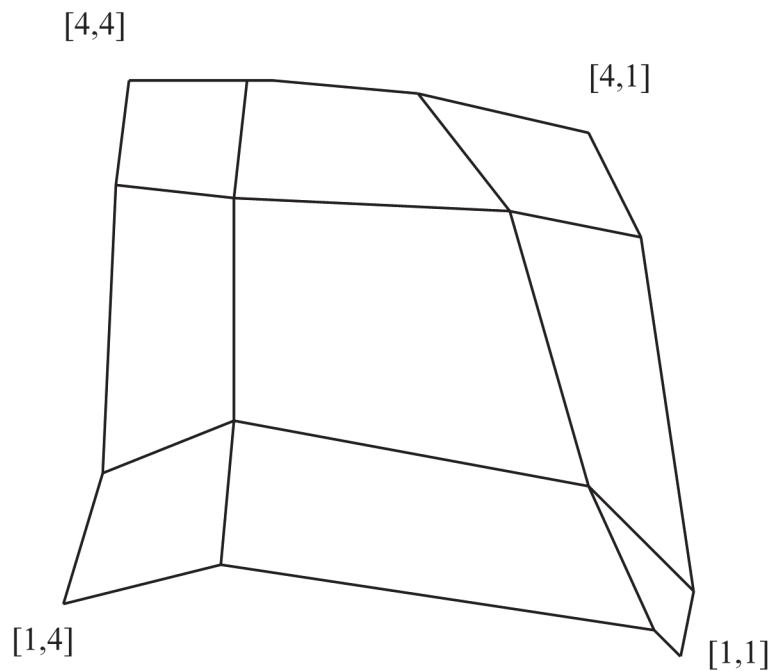


Figure P6.5.9 Two-dimensional self-organization results at $k = 500$.

Chapter 7

Learning Vector Quantization

Problem 7.1 (2-input vectors clustering to 5 classes by LVQ)

Consider a LVQ network with two input units $[x_{1k}, x_{2k}]$ and five target classes: C_1, C_2, C_3, C_4 , and C_5 .

There are twenty-five classification units, with initial weight vectors indicated by the coordinates $[x_{1k}, x_{2k}]$ on Table P7.1.1.

For example, the classification units for Class C_1 have initial weight vectors of $[-1.0, -1.0]$, $[-0.5, -0.5]$, $[0.0, 0.0]$, $[0.5, 0.5]$, and $[1.0, 1.0]$.

Using the Euclidean distance (or simply the geometry of the diagram to give an initial estimation of the shortest distance) and $g(k) = 0.1$, determine the changes that occur in each of the following steps:

Table P7.1.1

x_2						
1.0	C2	C3	C4	C5	C1	
0.5	C3	C4	C5	C1	C2	
0.0	C4	C5	C1	C2	C3	
-0.5	C5	C1	C2	C3	C4	
-1.0	C1	C2	C3	C4	C5	
	-1.0	-0.5	0.0	0.5	1.0	x_1

(a) Present an input vector $\mathbf{X}_1 = [0.6, -0.5]$ of $T_1 = 3$. Determine its new weight vector.

(b) Present an input vector $\mathbf{X}_2 = [0.0, 0.1]$ of $T_2 = 5$. Determine its new weight vector.

(c) Present an input vector $\mathbf{X}_3 = [0.4, 0.6]$ of $T_3 = 1$. Determine its new weight vector.

Solution

(a)

Since $[0.6, -0.5]$ is closest to $[0.5, -0.5]$ and $J = 3$, $T_1 = J$, update $\mathbf{W}_3$ according to (7.1) as

$$\mathbf{W}_3 = [0.5, -0.5] + 0.1 \{ [0.6, -0.5] - [0.5, -0.5] \} = [0.51, -0.50]$$

(b)

Since $[0.0, 0.1]$ is closest to $[0.0, 0.0]$ and $J = 1$, $T_2 \neq J$, update $\mathbf{W}_1$ according to (7.2) as

$$\mathbf{W}_1 = [0.0, 0.0] - 0.1 \{ [0.0, 0.1] - [0.0, 0.0] \} = [0.00, -0.01]$$

(c)

Since $[0.4, 0.6]$ is closest to $[0.5, 0.5]$ and $J = 1$, $T_3 = J$, update $\mathbf{W}_1$ according to (7.1) as

$$\mathbf{W}_1 = [0.5, 0.5] + 0.1 \{ [0.4, 0.6] - [0.5, 0.5] \} = [0.49, 0.51]$$

Problem 7.2 (4 binary vectors clustering to 2 classes by LVQ)

A LVQ is used to cluster 4 vectors into 2 classes as specified in Table P7.2.1.

Assuming $\mathbf{X}_1$ and $\mathbf{X}_3$ are selected as the initial weights of two different clusters.

With $g(0) = 0.5$, $g(k + 1) = 0.99 * g(k)$, write a Matlab program to obtain the weight vectors and recall the outputs after 100 iterations.

Table P7.2.1

k	$\mathbf{X}_k$	Class
1	[1, 0, 0, 0]	1
2	[0, 1, 0, 0]	1
3	[0, 0, 1, 0]	2
4	[0, 0, 0, 1]	2

Solution

Using the Matlab program listed at the end, the weight matrix after 100 iterations becomes

$$\mathbf{W} = \begin{bmatrix} 0.4480 & 0.5520 & 0 & 0 \\ 0 & 0 & 0.4480 & 0.5520 \end{bmatrix}$$

Matlab program

```
% Problem 7.2 (lvq_P7_2.m)
% 1. LVQ is used to cluster 4 input vectors Xk into C=2 clusters.
% 2. A LVQ net with 4 inputs and 2 outputs is defined.
% 4. The learning parameter g(k) is updated at the end of each
% iteration.
clear all;
gk = 0.5; % Set initial value of learning rate g().
X = [1 0 0 0; 0 1 0 0; 0 0 1 0; 0 0 0 1];
W = [X(1,:);X(2,:)]; % Set initial arbitrary weight vectors.
xc = [1 1 2 2]; %The desired output class vector.
% Perform iterations according to the SOM weight update algorithm.
for iter=1:100 % Cycle through the iterations
    for k = 1:size(X,1) % Cycle through the input vectors (patterns)
        % STEP 1:
        % B = repmat(A,M,N) creates a matrix B consisting of an M-by-N
        % tiling of copies of A.
        % The size of B is [size(A,1)*M, size(A,2)*N].
        % Example: For size(A)=[1,4] & size(W,1)=2, size(B)=[2,4].
```

```

% d(2,1) gives the sum of squared differences between X(k,:) with
% W(1:2,:).
d = sum(( repmat(X(k,:),size(W,1),1) - W).^2,2);
% STEP 2: Find weight vector that has the smallest distance from
% X(k) which gives the index of the winning neuron.
bmu = find(d==min(d)); % finding index of winning neuron(s)
% STEP 3: Update weight matrix. Notice that the weight matrix is
% only updated along the column of "bmu", which corresponds to
% the index of the winning neuron(s).
for j=1:length(bmu) % Update the winning weight vectors
    if xc(1,k) == bmu(j)
        W(bmu(j),:) = W(bmu(j),:) + gk*(X(k,:) - W(bmu(j),:));
    else
        W(bmu(j),:) = W(bmu(j),:) - gk*(X(k,:) - W(bmu(j),:));
    end
end
end
gk = 0.99*gk; % Update g(k) after each iteration.
end
fprintf('The weight vector after 100 iterations\n\n');
disp(W)
% Recall output using  $Y_k = X_k * W^T$ 
for k=1:size(X,1)
    Y(k,:) = X(k,:)*W';
end
fprintf('The output vector after 100 iterations\n\n');
disp(Y) % Display output Yk

```

Problem 7.3 (6 binary vectors clustering to 2 classes by LVQ)

In the LVQ network as shown in Figure P7.3.1, six vectors are assigned to two classes (classes 1 and 2) as shown in Table P7.3.1.

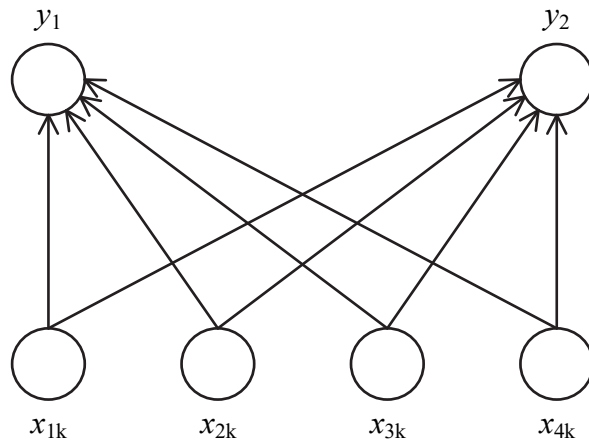


Figure P7.3.1 4-input 2-output LVQ network.

Table P7.3.1

k	$\mathbf{X}_k$	Class
1	[1, 0, 0, 0]	1
2	[0, 0, 0, 1]	2
3	[1, 1, 0, 0]	1
4	[0, 0, 1, 1]	2
5	[1, 0, 1, 0]	1
6	[0, 1, 0, 1]	2

Given $g(0) = 0.1$ and $g(k+1) = 0.99 * g(k)$:

- Show the step by step calculations within the first iteration.
- Determine the weight vectors and the recalled output vectors after 100 iterations.

Solution

- The first 2 vectors are used to initialize the 2 reference weight vectors. The first output unit represents class 1 ($C_1 = 1$), and the second output unit represents class 2 ($C_2 = 2$).

This leaves vectors $\mathbf{X}_3$, $\mathbf{X}_4$, $\mathbf{X}_5$, and $\mathbf{X}_6$ as the remaining training vectors for the first iteration. The procedure of the iteration 1 is shown as follows:

Step 0: Initialize weights:

$$\mathbf{W}_1 = [1, 0, 0, 0], \mathbf{W}_2 = [0, 0, 0, 1].$$

Initialize $g(0) = 0.1$.

Step 1: Begin computation:

Step 2: For input vector $\mathbf{X}_3 = [1, 1, 0, 0]$ with $T_3 = 1$, do Steps 3-4.

Step 3: Comparing $\mathbf{X}_3$ to $\mathbf{W}_1$ and $\mathbf{W}_2$, the winner is $\mathbf{W}_1$ and $J = 1$, since $\mathbf{X}_3$ is closer to $\mathbf{W}_1$ than to $\mathbf{W}_2$.

$$d_{31} = \|\mathbf{X}_3 - \mathbf{W}_1\| = [0 + 1 + 0 + 0]^{1/2} = 1$$

$$d_{32} = \|\mathbf{X}_3 - \mathbf{W}_2\| = [1 + 1 + 0 + 1]^{1/2} = \sqrt{3}$$

Step 4: Since $T_3 = 1$ and $C_J = C_1 = 1$, update $\mathbf{W}_1$ according to (7.1) as follows

$$\mathbf{W}_1 = [1, 0, 0, 0] + 0.1 \{[1, 1, 0, 0] - [1, 0, 0, 0]\} = [1, 0.1, 0, 0]$$

Step 2: For input vector $\mathbf{X}_4 = [0, 0, 1, 1]$ with $T_4 = 2$, do Steps 3-4.

Step 3: Comparing $\mathbf{X}_4$ to $\mathbf{W}_1$ and $\mathbf{W}_2$, the winner is $\mathbf{W}_2$ and $J = 2$ (implying $\mathbf{X}_4$ is closer to $\mathbf{W}_2$).

$$d_{41} = \|\mathbf{X}_4 - \mathbf{W}_1\| = [1 + 0.01 + 1 + 1]^{1/2} = \sqrt{3.01}$$

$$d_{42} = \|\mathbf{X}_4 - \mathbf{W}_2\| = [0 + 0 + 1 + 0]^{1/2} = 1$$

Step 4: Since $T_4 = 2$ and $C_J = C_2 = 2$, update $\mathbf{W}_2$ according to (7.1) as follows

$$\mathbf{W}_2 = [0, 0, 0, 1] + 0.1 \{[0, 0, 1, 1] - [0, 0, 0, 1]\} = [0, 0, 0.1, 1]$$

Step 2: For input vector $\mathbf{X}_5 = [1, 0, 1, 0]$ with $T_5 = 1$, do Steps 3-4.

Step 3: Comparing $\mathbf{X}_5$ to $\mathbf{W}_1$ and $\mathbf{W}_2$, the winner is $\mathbf{W}_1$ and $J = 1$ (implying $\mathbf{X}_5$ is closer to $\mathbf{W}_1$).

$$d_{51} = \|\mathbf{X}_5 - \mathbf{W}_1\| = [0 + 0.01 + 1 + 0]^{1/2} = \sqrt{1.01}$$

$$d_{52} = \|\mathbf{X}_5 - \mathbf{W}_2\| = [1 + 0 + 0.81 + 1]^{1/2} = \sqrt{2.81}$$

Step 4: Since $T_5 = 1$ and $C_J = C_1 = 1$, update $\mathbf{W}_1$ according to (7.1) as follows

$$\mathbf{W}_1 = [1, 0.1, 0, 0] + 0.1 \{[1, 0, 1, 0] - [1, 0.1, 0, 0]\} = [1, 0.09, 0.1, 0]$$

Step 2: For input vector $\mathbf{X}_6 = [0, 1, 0, 1]$ with $T_6 = 2$, do Steps 3-4.

Step 3: Comparing $\mathbf{X}_6$ to $\mathbf{W}_1$ and $\mathbf{W}_2$, the winner is $\mathbf{W}_2$ and $J = 2$ (implying $\mathbf{X}_6$ is closer to C_2).

$$d_{61} = \|\mathbf{X}_6 - \mathbf{W}_1\| = [1 + 0.8281 + 0.01 + 1]^{1/2} = \sqrt{2.8381}$$

$$d_{62} = \|\mathbf{X}_6 - \mathbf{W}_2\| = [0 + 1 + 0.01 + 0]^{1/2} = \sqrt{1.01}$$

Step 4: Since $T_6 = 2$ and $C_J = C_2 = 2$, update $\mathbf{W}_2$ according to (7.1) as follows

$$\mathbf{W}_2 = [0, 0, 0.1, 1] + 0.1 \{[0, 1, 0, 1] - [0, 0, 0.1, 1]\} = [0, 0.1, 0.09, 1]$$

Step 5: This complete one iteration of training. Reduce the learning rate.

Step 6: Test the stopping condition.

(b) Repeat the above with $\mathbf{X}_1$, $\mathbf{X}_2$, $\mathbf{X}_3$, $\mathbf{X}_4$, $\mathbf{X}_5$, and $\mathbf{X}_6$ for the second iteration and so on until either all the 5 input vectors have been learned and classified correctly or until the specified stopping condition is met.

With $g(0)=0.1$ and $g(k+1)=0.99*g(k)$, the weight vectors after 100 iterations are

$$\mathbf{W}_1 = [1.0000 \quad 0.3331 \quad 0.3471 \quad 0.0000]$$

$$\mathbf{W}_2 = [0.0000 \quad 0.3471 \quad 0.3331 \quad 1.0000]$$

The recalled output vectors after 100 iterations are

$$\mathbf{Y}_1 = [1.0000 \quad 0.0000]$$

$$\mathbf{Y}_2 = [0.0000 \quad 1.0000]$$

$$\mathbf{Y}_3 = [1.3331 \quad 0.3471]$$

$$\mathbf{Y}_4 = [0.3471 \quad 1.3331]$$

$$\mathbf{Y}_5 = [1.3471 \quad 0.3331]$$

$$\mathbf{Y}_6 = [0.3331 \quad 1.3471]$$

From the above, it can be observed that each of the six input vectors has been clustered correctly at the output.

Problem 7.4 (6 binary vectors clustered to 3 classes by LVQ)

A LVQ is used to cluster 6 vectors into 3 classes as specified by Table P7.4.1.

Assuming $\mathbf{X}_1$, $\mathbf{X}_3$ and $\mathbf{X}_5$ are selected as the initial weights of three different clusters, with a learning rate $g(1) = 0.5$ and $g(k) = 0.5 * g(k - 1)$ for $k \geq 2$:

(a) Show the calculation steps to obtain the weights in the first 2 iterations and rounded the results in two decimal places.

(b) Repeat the weight learning iterations up to 100 using a Matlab program and verify the classification correctness.

Table P7.4.1

k	$\mathbf{X}_k$	Class
1	[1 1 1 0 0 0]	1
2	[0 1 1 1 0 0]	1
3	[0 0 1 1 1 0]	2
4	[0 0 0 1 1 1]	2
5	[1 0 0 0 1 1]	3
6	[1 1 0 0 0 1]	3

Solution

The initial weights of three different clusters are

$$\mathbf{W}_1 = [1 \ 1 \ 1 \ 0 \ 0 \ 0], C=1$$

$$\mathbf{W}_2 = [0 \ 0 \ 1 \ 1 \ 1 \ 0], C=2$$

$$\mathbf{W}_3 = [1 \ 0 \ 0 \ 0 \ 1 \ 1], C=3$$

Iteration 1

Presenting the input $\mathbf{X}_1$:

$$\begin{aligned} d_1 &= (1-1.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (0-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 \\ &= 0.00 \end{aligned}$$

$$\begin{aligned} d_2 &= (1-0.00)^2 + (1-0.00)^2 + (1-1.00)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-0.00)^2 \\ &= 4.00 \end{aligned}$$

$$\begin{aligned} d_3 &= (1-1.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (0-0.00)^2 + (0-1.00)^2 + (0-1.00)^2 \\ &= 4.00 \end{aligned}$$

d_1 is smaller, the weight vector $\mathbf{W}_1$ of class C1 will be updated.

The input $\mathbf{X}_1$ and the weight $\mathbf{W}_1$ are of the same class, consequently

$$\mathbf{W}_1 = [1.00 \ 1.00 \ 1.00 \ 0.00 \ 0.00 \ 0.00] + 0.5 * ([1 \ 1 \ 1 \ 0 \ 0 \ 0] - [1.00 \ 1.00 \ 1.00 \ 0.00 \ 0.00 \ 0.00]) = [1.00 \ 1.00 \ 1.00 \ 0.00 \ 0.00 \ 0.00]$$

Presenting the input $\mathbf{X}_2$:

$$\begin{aligned} d_1 &= (0-1.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (1-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 \\ &= 2.00 \end{aligned}$$

$$\begin{aligned} d_2 &= (0-0.00)^2 + (1-0.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (0-1.00)^2 + (0-0.00)^2 \\ &= 2.00 \end{aligned}$$

$$\begin{aligned} d_3 &= (0-1.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (0-1.00)^2 + (0-1.00)^2 \\ &= 6.00 \end{aligned}$$

d_1 is smaller, the weight vector $\mathbf{W}_1$ of class C1 will be updated. d_2 is smaller, the weight vector $\mathbf{W}_2$ of class C2 will be updated.

The input $\mathbf{X}_2$ and the weight $\mathbf{W}_1$ are of the same class, consequently

$$\mathbf{W}_1 = [1.00 \ 1.00 \ 1.00 \ 0.00 \ 0.00 \ 0.00] + 0.5*([0 \ 1 \ 1 \ 1 \ 0 \ 0] - [1.00 \ 1.00 \ 1.00 \ 0.00 \ 0.00 \ 0.00]) = [0.50 \ 1.00 \ 1.00 \ 0.50 \ 0.00 \ 0.00]$$

The input $\mathbf{X}_2$ and the weight $\mathbf{W}_2$ are of the different class, consequently

$$\mathbf{W}_2 = [0.00 \ 0.00 \ 1.00 \ 1.00 \ 1.00 \ 0.00] - 0.5*([0 \ 1 \ 1 \ 1 \ 0 \ 0] - [0.00 \ 0.00 \ 1.00 \ 1.00 \ 1.00 \ 0.00]) = [0.00 \ -0.50 \ 1.00 \ 1.00 \ 1.50 \ 0.00]$$

Presenting the input $\mathbf{X}_3$:

$$\begin{aligned} d_1 &= (0-0.50)^2 + (0-1.00)^2 + (1-1.00)^2 + (1-0.50)^2 + (1-0.00)^2 + (0-0.00)^2 \\ &= 2.50 \end{aligned}$$

$$\begin{aligned} d_2 &= (0-0.00)^2 + (0-(-0.50))^2 + (1-1.00)^2 + (1-1.00)^2 + (1-1.50)^2 + (0-0.00)^2 \\ &= 0.50 \end{aligned}$$

$$\begin{aligned} d_3 &= (0-1.00)^2 + (0-0.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (1-1.00)^2 + (0-1.00)^2 \\ &= 4.00 \end{aligned}$$

d_2 is smaller, the weight vector $\mathbf{W}_2$ of class C2 will be updated.

The input $\mathbf{X}_3$ and the weight $\mathbf{W}_2$ are of the same class, consequently

$$\mathbf{W}_2 = [0.00 \ -0.50 \ 1.00 \ 1.00 \ 1.50 \ 0.00] + 0.5*([0 \ 0 \ 1 \ 1 \ 1 \ 0] - [0.00 \ -0.50 \ 1.00 \ 1.00 \ 1.50 \ 0.00]) = [0.00 \ -0.25 \ 1.00 \ 1.00 \ 1.25 \ 0.00]$$

Presenting the input $\mathbf{X}_4$:

$$\begin{aligned} d_1 &= (0-0.50)^2 + (0-1.00)^2 + (0-1.00)^2 + (1-0.50)^2 + (1-0.00)^2 + (1-0.00)^2 \\ &= 4.50 \end{aligned}$$

$$\begin{aligned} d_2 &= (0-0.00)^2 + (0-(-0.25))^2 + (0-1.00)^2 + (1-1.00)^2 + (1-1.25)^2 + (1-0.00)^2 \\ &= 2.13 \end{aligned}$$

$$d_3 = (0-1.00)^2 + (0-0.00)^2 + (0-0.00)^2 + (1-0.00)^2 + (1-1.00)^2 + (1-1.00)^2$$

$$= 2.00$$

d_3 is smaller, the weight vector $\mathbf{W}_3$ of class C3 will be updated.

The input $\mathbf{X}_4$ and the weight $\mathbf{W}_3$ are of the different class, consequently

$$\mathbf{W}_3 = [1.00 \ 0.00 \ 0.00 \ 0.00 \ 1.00 \ 1.00] - 0.5 * ([0 \ 0 \ 0 \ 1 \ 1 \ 1] - [1.00 \ 0.00 \ 0.00 \ 0.00 \ 1.00 \ 1.00]) = [1.50 \ 0.00 \ 0.00 \ -0.50 \ 1.00 \ 1.00]$$

Presenting the input $\mathbf{X}_5$:

$$d_1 = (1-0.50)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-0.50)^2 + (1-0.00)^2 + (1-0.00)^2$$

$$= 4.50$$

$$d_2 = (1-0.00)^2 + (0-(-0.25))^2 + (0-1.00)^2 + (0-1.00)^2 + (1-1.25)^2 + (1-0.00)^2$$

$$= 4.13$$

$$d_3 = (1-1.50)^2 + (0-0.00)^2 + (0-0.00)^2 + (0-(-0.50))^2 + (1-1.00)^2 + (1-1.00)^2$$

$$= 0.50$$

d_3 is smaller, the weight vector $\mathbf{W}_3$ of class C3 will be updated.

The input $\mathbf{X}_5$ and the weight $\mathbf{W}_3$ are of the same class, consequently

$$\mathbf{W}_3 = [1.50 \ 0.00 \ 0.00 \ -0.50 \ 1.00 \ 1.00] + 0.5 * ([1 \ 0 \ 0 \ 0 \ 1 \ 1] - [1.50 \ 0.00 \ 0.00 \ -0.50 \ 1.00 \ 1.00]) = [1.25 \ 0.00 \ 0.00 \ -0.25 \ 1.00 \ 1.00]$$

Presenting the input $\mathbf{X}_6$:

$$d_1 = (1-0.50)^2 + (1-1.00)^2 + (0-1.00)^2 + (0-0.50)^2 + (0-0.00)^2 + (1-0.00)^2$$

$$= 2.50$$

$$d_2 = (1-0.00)^2 + (1-(-0.25))^2 + (0-1.00)^2 + (0-1.00)^2 + (0-1.25)^2 + (1-0.00)^2$$

$$= 7.13$$

$$d_3 = (1-1.25)^2 + (1-0.00)^2 + (0-0.00)^2 + (0-(-0.25))^2 + (0-1.00)^2 + (1-1.00)^2$$

$$= 2.13$$

d_3 is smaller, the weight vector $\mathbf{W}_3$ of class C3 will be updated.

The input $\mathbf{X}_6$ and the weight $\mathbf{W}_3$ are of the same class, consequently

$$\mathbf{W}_3 = [1.25 \ 0.00 \ 0.00 \ -0.25 \ 1.00 \ 1.00] + 0.5 * ([1 \ 1 \ 0 \ 0 \ 0 \ 1] - [1.25 \ 0.00 \ 0.00 \ -0.25 \ 1.00 \ 1.00]) = [1.13 \ 0.50 \ 0.00 \ -0.13 \ 0.50 \ 1.00]$$

After Iteration 1, the weight vectors are

$$\mathbf{W}_1 = [0.50 \ 1.00 \ 1.00 \ 0.50 \ 0.00 \ 0.00]$$

$$\mathbf{W}_2 = [0.00 \ -0.25 \ 1.00 \ 1.00 \ 1.25 \ 0.00]$$

$$\mathbf{W}_3 = [1.13 \ 0.50 \ 0.00 \ -0.13 \ 0.50 \ 1.00]$$

Computing $g(2) = 0.5 * g(1) = 0.25$.

Iteration 2

Presenting the input $\mathbf{X}_1$:

$$\begin{aligned} d_1 &= (1-0.50)^2 + (1-1.00)^2 + (1-1.00)^2 + (0-0.50)^2 + (0-0.00)^2 + (0-0.00)^2 \\ &= 0.50 \end{aligned}$$

$$\begin{aligned} d_2 &= (1-0.00)^2 + (1-(-0.25))^2 + (1-1.00)^2 + (0-1.00)^2 + (0-1.25)^2 + (0-0.00)^2 \\ &= 5.13 \end{aligned}$$

$$\begin{aligned} d_3 &= (1-1.13)^2 + (1-0.50)^2 + (1-0.00)^2 + (0-(-0.13))^2 + (0-0.50)^2 + (0-1.00)^2 \\ &= 2.53 \end{aligned}$$

d_1 is smaller, the weight vector $\mathbf{W}_1$ of class C1 will be updated.

The input $\mathbf{X}_1$ and the weight $\mathbf{W}_1$ are of the same class, consequently

$$\mathbf{W}_1 = [0.50 \ 1.00 \ 1.00 \ 0.50 \ 0.00 \ 0.00] + 0.25 * ([1 \ 1 \ 1 \ 0 \ 0 \ 0] - [0.50 \ 1.00 \ 1.00 \ 0.50 \ 0.00 \ 0.00]) = [0.63 \ 1.00 \ 1.00 \ 0.38 \ 0.00 \ 0.00]$$

Presenting the input $\mathbf{X}_2$:

$$d_1 = (0-0.63)^2 + (1-1.00)^2 + (1-1.00)^2 + (1-0.38)^2 + (0-0.00)^2 + (0-0.00)^2$$

$$= 0.78$$

$$d_2 = (0-0.00)^2 + (1-(-0.25))^2 + (1-1.00)^2 + (1-1.00)^2 + (0-1.25)^2 + (0-0.00)^2 \\ = 3.13$$

$$d_3 = (0-1.13)^2 + (1-0.50)^2 + (1-0.00)^2 + (1-(-0.13))^2 + (0-0.50)^2 + (0-1.00)^2 \\ = 5.03$$

d_1 is smaller, the weight vector $\mathbf{W}_1$ of class C1 will be updated.

The input $\mathbf{X}_2$ and the weight $\mathbf{W}_1$ are of the same class, consequently

$$\mathbf{W}_1 = [0.63 \ 1.00 \ 1.00 \ 0.38 \ 0.00 \ 0.00] + 0.25 * ([0 \ 1 \ 1 \ 1 \ 0 \ 0] - [0.63 \ 1.00 \ 1.00 \ 0.38 \ 0.00 \ 0.00]) = [0.47 \ 1.00 \ 1.00 \ 0.54 \ 0.00 \ 0.00]$$

Presenting the input $\mathbf{X}_3$:

$$d_1 = (0-0.47)^2 + (0-1.00)^2 + (1-1.00)^2 + (1-0.54)^2 + (1-0.00)^2 + (0-0.00)^2 \\ = 2.43$$

$$d_2 = (0-0.00)^2 + (0-(-0.25))^2 + (1-1.00)^2 + (1-1.00)^2 + (1-1.25)^2 + (0-0.00)^2 \\ = 0.13$$

$$d_3 = (0-1.13)^2 + (0-0.50)^2 + (1-0.00)^2 + (1-(-0.13))^2 + (1-0.50)^2 + (0-1.00)^2 \\ = 5.03$$

d_2 is smaller, the weight vector $\mathbf{W}_2$ of class C2 will be updated.

The input $\mathbf{X}_3$ and the weight $\mathbf{W}_2$ are of the same class, consequently

$$\mathbf{W}_2 = [0.00 \ -0.25 \ 1.00 \ 1.00 \ 1.25 \ 0.00] + 0.25 * ([0 \ 0 \ 1 \ 1 \ 1 \ 0] - [0.00 \ -0.25 \ 1.00 \ 1.00 \ 1.25 \ 0.00]) = [0.00 \ -0.19 \ 1.00 \ 1.00 \ 1.19 \ 0.00]$$

Presenting the input $\mathbf{X}_4$:

$$d_1 = (0-0.47)^2 + (0-1.00)^2 + (0-1.00)^2 + (1-0.54)^2 + (1-0.00)^2 + (1-0.00)^2 \\ = 4.43$$

$$d_2 = (0-0.00)^2 + (0-(-0.19))^2 + (0-1.00)^2 + (1-1.00)^2 + (1-1.19)^2 + (1-0.00)^2$$

$$= 2.07$$

$$d_3 = (0-1.13)^2 + (0-0.50)^2 + (0-0.00)^2 + (1-(-0.13))^2 + (1-0.50)^2 + (1-1.00)^2 \\ = 3.03$$

d_2 is smaller, the weight vector $\mathbf{W}_2$ of class C2 will be updated.

The input $\mathbf{X}_4$ and the weight $\mathbf{W}_2$ are of the same class, consequently

$$\mathbf{W}_2 = [0.00 \ 0.19 \ 1.00 \ 1.00 \ 1.19 \ 0.00] + 0.25 * ([0 \ 0 \ 0 \ 1 \ 1 \ 1] - [0.00 \ 0.19 \ 1.00 \ 1.00 \ 1.19 \ 0.00]) = [0.00 \ 0.14 \ 0.75 \ 1.00 \ 1.14 \ 0.25]$$

Presenting the input $\mathbf{X}_5$:

$$d_1 = (1-0.47)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-0.54)^2 + (1-0.00)^2 + (1-0.00)^2 \\ = 4.57$$

$$d_2 = (1-0.00)^2 + (0-(-0.14))^2 + (0-0.75)^2 + (0-1.00)^2 + (1-1.14)^2 + (1-0.25)^2 \\ = 3.16$$

$$d_3 = (1-1.13)^2 + (0-0.50)^2 + (0-0.00)^2 + (0-(-0.13))^2 + (1-0.50)^2 + (1-1.00)^2 \\ = 0.53$$

d_3 is smaller, the weight vector $\mathbf{W}_3$ of class C3 will be updated.

The input $\mathbf{X}_5$ and the weight $\mathbf{W}_3$ are of the same class, consequently

$$\mathbf{W}_3 = [1.13 \ 0.50 \ 0.00 \ -0.13 \ 0.50 \ 1.00] + 0.25 * ([1 \ 0 \ 0 \ 0 \ 1 \ 1] - [1.13 \ 0.50 \ 0.00 \ -0.13 \ 0.50 \ 1.00]) = [1.10 \ 0.38 \ 0.00 \ -0.10 \ 0.63 \ 1.00]$$

Presenting the input $\mathbf{X}_6$:

$$d_1 = (1-0.47)^2 + (1-1.00)^2 + (0-1.00)^2 + (0-0.53)^2 + (0-0.00)^2 + (1-0.00)^2 \\ = 2.56$$

$$d_2 = (1-0.00)^2 + (1-(-0.14))^2 + (0-0.75)^2 + (0-1.00)^2 + (0-1.14)^2 + (1-0.25)^2 \\ = 5.73$$

$$d_3 = (1-1.10)^2 + (1-0.38)^2 + (0-0.00)^2 + (0-(-0.10))^2 + (0-0.63)^2 + (1-1.00)^2$$

$$= 0.80$$

d_3 is smaller, the weight vector $\mathbf{W}_3$ of class C3 will be updated.

The input $\mathbf{X}_6$ and the weight $\mathbf{W}_3$ are of the same class, consequently

$$\mathbf{W}_3 = [1.09 \ 0.38 \ 0.00 \ -0.09 \ 0.63 \ 1.00] + 0.25 * ([1 \ 1 \ 0 \ 0 \ 0 \ 1] - [1.10 \ 0.38 \ 0.00 \ -0.10 \ 0.63 \ 1.00]) = [1.07 \ 0.54 \ 0.00 \ -0.07 \ 0.47 \ 1.00]$$

After Iteration 2, the weight vectors are:

$$\mathbf{W}_1 = [0.47 \ 1.00 \ 1.00 \ 0.53 \ 0.00 \ 0.00]$$

$$\mathbf{W}_2 = [0.00 \ -0.14 \ 0.75 \ 1.00 \ 1.14 \ 0.25]$$

$$\mathbf{W}_3 = [1.07 \ 0.54 \ 0.00 \ -0.07 \ 0.47 \ 1.00]$$

Computing $g(3) = 0.5 * g(2) = 0.125$.

(b) Using $g(k) = 0.5 * g(k - 1)$ for $k \geq 0$, after Iteration 100, the weight vectors are

$$\mathbf{W}_1 = [0.47 \ 1.00 \ 1.00 \ 0.53 \ 0.00 \ 0.00]$$

$$\mathbf{W}_2 = [0.00 \ -0.08 \ 0.64 \ 1.00 \ 1.08 \ 0.36]$$

$$\mathbf{W}_3 = [1.04 \ 0.53 \ 0.00 \ -0.04 \ 0.47 \ 1.00]$$

The recall outputs obtained verify the classification correctness as shown in Table P7.4.2.

Table P7.4.2

k	$\mathbf{X}_k$	$\mathbf{Y}_k$	Class
1	[1 1 1 0 0 0]	[2.47 , 0.56, 1.57]	1
2	[0 1 1 1 0 0]	[2.53 , 1.56, 0.49]	1
3	[0 0 1 1 1 0]	[1.53, 2.74 , 0.43]	2
4	[0 0 0 1 1 1]	[0.53, 2.44 , 1.43]	2
5	[1 0 0 0 1 1]	[0.47, 1.44, 2.51]	3
6	[1 1 0 0 0 1]	[1.47, 0.28, 2.57]	3

Problem 7.5 (8 binary vectors clustering to 4 classes by LVQ)

A LVQ is used to cluster 8 vectors into 4 classes as specified by Table P7.5.1.

Assuming $\mathbf{X}_1$, $\mathbf{X}_3$, $\mathbf{X}_5$ and $\mathbf{X}_7$ are selected as the initial weights of three different clusters, with a learning rate $g(1) = 0.5$ and $g(k) = 0.5 * g(k-1)$ for $k \geq 2$:

(a) Show the calculation steps to obtain the weights in the first 2 iterations and rounded the results in two decimal places.

(b) Repeat the weight learning iterations up to 100 using a Matlab program and verify the classification correctness.

Table P7.5.1

k	$\mathbf{X}_k$	Class
1	[1 1 1 1 0 0 0 0]	1
2	[0 1 1 1 1 0 0 0]	1
3	[0 0 1 1 1 1 0 0]	2
4	[0 0 0 1 1 1 1 0]	2
5	[0 0 0 0 1 1 1 1]	3
6	[1 1 0 0 0 0 1 1]	3
7	[1 1 0 0 0 0 1 1]	4
8	[1 1 1 0 0 0 0 1]	4

Solution

The initial weights of three different clusters are

$$\mathbf{W}_1 = [1 1 1 1 0 0 0 0], C=1$$

$$\mathbf{W}_2 = [0 0 1 1 1 1 0 0], C=2$$

$$\mathbf{W}_3 = [0 0 0 0 1 1 1 1], C=3$$

$$\mathbf{W}_4 = [1 1 0 0 0 0 1 1], C=4$$

Iteration 1

Presenting the input $\mathbf{X}_1$:

$$d_1 = (1-1.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (0-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 = 0.00$$

$$d_2 = (1-0.00)^2 + (1-0.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-0.00)^2 + (0-0.00)^2 = 4.00$$

$$d_3 = (1-0.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-1.00)^2 = 8.00$$

$$d_4 = (1-1.00)^2 + (1-1.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 + (0-1.00)^2 + (0-1.00)^2 = 4.00$$

d_1 is smaller, the weight vector $\mathbf{W}_1$ of class C1 will be updated.

The input $\mathbf{X}_1$ and the weight $\mathbf{W}_1$ are of the same class, consequently

$$\begin{aligned} \mathbf{W}_1 &= [1.00 \ 1.00 \ 1.00 \ 1.00 \ 0.00 \ 0.00 \ 0.00 \ 0.00] + 0.5 * ([1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0] - [1.00 \ 1.00 \ 1.00 \ 1.00 \ 0.00 \ 0.00 \ 0.00 \ 0.00]) \\ &= [1.00 \ 1.00 \ 1.00 \ 1.00 \ 0.00 \ 0.00 \ 0.00 \ 0.00] \end{aligned}$$

Presenting the input $\mathbf{X}_2$:

$$d_1 = (0-1.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (1-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 = 2.00$$

$$d_2 = (0-0.00)^2 + (1-0.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (0-1.00)^2 + (0-0.00)^2 + (0-0.00)^2 = 2.00$$

$$d_3 = (0-0.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (1-1.00)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-1.00)^2 = 6.00$$

$$d_4 = (0-1.00)^2 + (1-1.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (0-0.00)^2 + (0-1.00)^2 + (0-1.00)^2 = 6.00$$

d_1 is smaller, the weight vector $\mathbf{W}_1$ of class C1 will be updated.

d_2 is smaller, the weight vector $\mathbf{W}_2$ of class C2 will be updated.

The input $\mathbf{X}_2$ and the weight $\mathbf{W}_1$ are of the same class, consequently

$$\begin{aligned}\mathbf{W}_1 &= [1.00 \ 1.00 \ 1.00 \ 1.00 \ 0.00 \ 0.00 \ 0.00 \ 0.00] + 0.5*([0 \ 1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \\ &0] - [1.00 \ 1.00 \ 1.00 \ 1.00 \ 0.00 \ 0.00 \ 0.00 \ 0.00]) \\ &= [0.50 \ 1.00 \ 1.00 \ 1.00 \ 0.50 \ 0.00 \ 0.00 \ 0.00]\end{aligned}$$

The input $\mathbf{X}_2$ and the weight $\mathbf{W}_2$ are of the different class:

$$\begin{aligned}\mathbf{W}_2 &= [0.00 \ 0.00 \ 1.00 \ 1.00 \ 1.00 \ 1.00 \ 0.00 \ 0.00] - 0.5*([0 \ 1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \\ &0] - [0.00 \ 0.00 \ 1.00 \ 1.00 \ 1.00 \ 1.00 \ 0.00 \ 0.00]) \\ &= [0.00 \ -0.50 \ 1.00 \ 1.00 \ 1.00 \ 1.50 \ 0.00 \ 0.00]\end{aligned}$$

Presenting the input $\mathbf{X}_3$:

$$\begin{aligned}d_1 &= (0-0.50)^2 + (0-1.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (1-0.50)^2 + (1-0.00)^2 + (0-0.00)^2 \\ &+ (0-0.00)^2 = 2.50\end{aligned}$$

$$\begin{aligned}d_2 &= (0-0.00)^2 + (0-(-0.50))^2 + (1-1.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (1-1.50)^2 \\ &+ (0-0.00)^2 + (0-0.00)^2 = 0.50\end{aligned}$$

$$\begin{aligned}d_3 &= (0-0.00)^2 + (0-0.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (0-1.00)^2 \\ &+ (0-1.00)^2 = 4.00\end{aligned}$$

$$\begin{aligned}d_4 &= (0-1.00)^2 + (0-1.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (0-1.00)^2 \\ &+ (0-1.00)^2 = 8.00\end{aligned}$$

d_2 is smaller, the weight vector $\mathbf{W}_2$ of class C2 will be updated.

The input $\mathbf{X}_3$ and the weight $\mathbf{W}_2$ are of the same class, consequently:

$$\begin{aligned}\mathbf{W}_2 &= [0.00 \ -0.50 \ 1.00 \ 1.00 \ 1.00 \ 1.50 \ 0.00 \ 0.00] + 0.5*([0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 0 \ 0 \\ &0] - [0.00 \ -0.50 \ 1.00 \ 1.00 \ 1.00 \ 1.50 \ 0.00 \ 0.00]) \\ &= [0.00 \ -0.25 \ 1.00 \ 1.00 \ 1.00 \ 1.25 \ 0.00 \ 0.00]\end{aligned}$$

Presenting the input $\mathbf{X}_4$:

$$d_1 = (0-0.50)^2 + (0-1.00)^2 + (0-1.00)^2 + (1-1.00)^2 + (1-0.50)^2 + (1-0.00)^2 + (1-0.00)^2 + (0-0.00)^2 = 4.50$$

$$d_2 = (0-0.00)^2 + (0-(-0.25))^2 + (0-1.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (1-1.25)^2 + (1-0.00)^2 + (0-0.00)^2 = 2.13$$

$$d_3 = (0-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 + (1-0.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (0-1.00)^2 = 2.00$$

$$d_4 = (0-1.00)^2 + (0-1.00)^2 + (0-0.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (1-1.00)^2 + (0-1.00)^2 = 6.00$$

d_3 is smaller, the weight vector $\mathbf{W}_3$ of class C3 will be updated.

The input $\mathbf{X}_4$ and the weight $\mathbf{W}_3$ are of the different class, consequently

$$\begin{aligned}\mathbf{W}_3 &= [0.00 \ 0.00 \ 0.00 \ 0.00 \ 1.00 \ 1.00 \ 1.00 \ 1.00] - 0.5 * ([0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 0] - [0.00 \ 0.00 \ 0.00 \ 0.00 \ 1.00 \ 1.00 \ 1.00 \ 1.00]) \\ &= [0.00 \ 0.00 \ 0.00 \ -0.50 \ 1.00 \ 1.00 \ 1.00 \ 1.50]\end{aligned}$$

Presenting the input $\mathbf{X}_5$:

$$d_1 = (0-0.50)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-1.00)^2 + (1-0.50)^2 + (1-0.00)^2 + (1-0.00)^2 + (1-0.00)^2 = 6.50$$

$$d_2 = (0-0.00)^2 + (0-0.25)^2 + (0-1.00)^2 + (0-1.00)^2 + (1-1.00)^2 + (1-1.25)^2 + (1-0.00)^2 + (1-0.00)^2 = 4.13$$

$$d_3 = (0-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 + (0-(-0.50))^2 + (1-1.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (1-1.50)^2 = 0.50$$

$$d_4 = (0-1.00)^2 + (0-1.00)^2 + (0-0.00)^2 + (0-0.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (1-1.00)^2 + (1-1.00)^2 = 4.00$$

d_3 is smaller, the weight vector $\mathbf{W}_3$ of class C3 will be updated.

The input $\mathbf{X}_3$ and the weight $\mathbf{W}_3$ are of the same class, consequently

$$\begin{aligned}\mathbf{W}_3 &= [0.00 \ 0.00 \ 0.00 \ -0.50 \ 1.00 \ 1.00 \ 1.00 \ 1.50] + 0.5*([0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1 \\ &1] - [0.00 \ 0.00 \ 0.00 \ -0.50 \ 1.00 \ 1.00 \ 1.00 \ 1.50]) \\ &= [0.00 \ 0.00 \ 0.00 \ -0.25 \ 1.00 \ 1.00 \ 1.00 \ 1.25]\end{aligned}$$

Presenting the input $\mathbf{X}_6$:

$$\begin{aligned}d_1 &= (1-0.50)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-0.50)^2 + (1-0.00)^2 + (1-0.00)^2 \\ &+ (1-0.00)^2 = 6.50\end{aligned}$$

$$\begin{aligned}d_2 &= (1-0.00)^2 + (0-0.25)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-1.00)^2 + (1-1.25)^2 + (1-0.00)^2 \\ &+ (1-0.00)^2 = 6.13\end{aligned}$$

$$\begin{aligned}d_3 &= (1-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 + (0-(-0.25))^2 + (0-1.00)^2 + (1-1.00)^2 \\ &+ (1-1.00)^2 + (1-1.25)^2 = 2.13\end{aligned}$$

$$\begin{aligned}d_4 &= (1-1.00)^2 + (0-1.00)^2 + (0-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 + (1-0.00)^2 + (1-1.00)^2 \\ &+ (1-1.00)^2 = 2.00\end{aligned}$$

d_4 is smaller, the weight vector $\mathbf{W}_4$ of class C4 will be updated.

The input $\mathbf{X}_6$ and the weight $\mathbf{W}_4$ are of the different class, consequently

$$\begin{aligned}\mathbf{W}_4 &= [1.00 \ 1.00 \ 0.00 \ 0.00 \ 0.00 \ 0.00 \ 1.00 \ 1.00] - 0.5*([1 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \\ &1] - [1.00 \ 1.00 \ 0.00 \ 0.00 \ 0.00 \ 0.00 \ 1.00 \ 1.00]) \\ &= [1.00 \ 1.50 \ 0.00 \ 0.00 \ 0.00 \ -0.50 \ 1.00 \ 1.00]\end{aligned}$$

Presenting the input $\mathbf{X}_7$:

$$\begin{aligned}d_1 &= (1-0.50)^2 + (1-1.00)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-0.50)^2 + (0-0.00)^2 + (1-0.00)^2 \\ &+ (1-0.00)^2 = 4.50\end{aligned}$$

$$\begin{aligned}d_2 &= (1-0.00)^2 + (1-0.25)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-1.25)^2 + (1-0.00)^2 \\ &+ (1-0.00)^2 = 9.13\end{aligned}$$

$$d_3 = (1-0.00)^2 + (1-0.00)^2 + (0-0.00)^2 + (0-(-0.25))^2 + (0-1.00)^2 + (0-1.00)^2$$

$$+(1-1.00)^2+(1-1.25)^2 = 4.13$$

$$d_4 = (1-1.00)^2 + (1-1.50)^2 + (0-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 + (0-(-0.50))^2 + (1-1.00)^2 \\ + (1-1.00)^2 = 0.50$$

d_4 is smaller, the weight vector $\mathbf{W}_4$ of class C4 will be updated.

The input $\mathbf{X}_7$ and the weight $\mathbf{W}_4$ are of the same class, consequently

$$\mathbf{W}_4 = [1.00 \ 1.50 \ 0.00 \ 0.00 \ 0.00 \ -0.50 \ 1.00 \ 1.00] + 0.5 * ([1 \ 1 \ 0 \ 0 \ 0 \ 0 \ 1 \\ 1] - [1.00 \ 1.50 \ 0.00 \ 0.00 \ 0.00 \ -0.50 \ 1.00 \ 1.00]) \\ = [1.00 \ 1.25 \ 0.00 \ 0.00 \ 0.00 \ -0.25 \ 1.00 \ 1.00]$$

Presenting the input $\mathbf{X}_8$:

$$d_1 = (1-0.50)^2 + (1-1.00)^2 + (1-1.00)^2 + (0-1.00)^2 + (0-0.50)^2 + (0-0.00)^2 + (0-0.00)^2 \\ + (1-0.00)^2 = 2.50$$

$$d_2 = (1-0.00)^2 + (1-0.25)^2 + (1-1.00)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-1.25)^2 + (0-0.00)^2 \\ + (1-0.00)^2 = 7.13$$

$$d_3 = (1-0.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (0-(-0.25))^2 + (0-1.00)^2 + (0-1.00)^2 \\ + (0-1.00)^2 + (1-1.25)^2 = 6.13$$

$$d_4 = (1-1.00)^2 + (1-1.25)^2 + (1-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 + (0-(-0.25))^2 \\ + (0-1.00)^2 + (1-1.00)^2 = 2.13$$

d_4 is smaller, the weight vector $\mathbf{W}_4$ of class C4 will be updated.

The input $\mathbf{X}_8$ and the weight $\mathbf{W}_4$ are of the same class, consequently

$$\mathbf{W}_4 = [1.00 \ 1.25 \ 0.00 \ 0.00 \ 0.00 \ -0.25 \ 1.00 \ 1.00] + 0.5 * ([1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0 \\ 1] - [1.00 \ 1.25 \ 0.00 \ 0.00 \ 0.00 \ -0.25 \ 1.00 \ 1.00]) \\ = [1.00 \ 1.13 \ 0.50 \ 0.00 \ 0.00 \ -0.13 \ 0.50 \ 1.00]$$

After Iteration 1, the weight vectors are:

$$\mathbf{W}_1 = [0.50 \ 1.00 \ 1.00 \ 1.00 \ 0.50 \ 0.00 \ 0.00 \ 0.00]$$

$$\mathbf{W}_2 = [0.00 \ 0.25 \ 1.00 \ 1.00 \ 1.00 \ 1.25 \ 0.00 \ 0.00]$$

$$\mathbf{W}_3 = [0.00 \ 0.00 \ 0.00 \ 0.25 \ 1.00 \ 1.00 \ 1.00 \ 1.25]$$

$$\mathbf{W}_4 = [1.00 \ 1.13 \ 0.50 \ 0.00 \ 0.00 \ 0.13 \ 0.50 \ 1.00]$$

Computing $g(2) = 0.5 * g(1) = 0.25$.

Iteration 2

Presenting the input $\mathbf{X}_1$:

$$d_1 = (1-0.50)^2 + (1-1.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (0-0.50)^2 + (0-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 = 0.50$$

$$d_2 = (1-0.00)^2 + (1-0.25)^2 + (1-1.00)^2 + (1-1.00)^2 + (0-1.00)^2 + (0-1.25)^2 + (0-0.00)^2 + (0-0.00)^2 = 5.13$$

$$d_3 = (1-0.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (1-(-0.25))^2 + (0-1.00)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-1.25)^2 = 9.13$$

$$d_4 = (1-1.00)^2 + (1-1.13)^2 + (1-0.50)^2 + (1-0.00)^2 + (0-0.00)^2 + (0-(-0.13))^2 + (0-0.50)^2 + (0-1.00)^2 = 2.53$$

d_1 is smaller, the weight vector $\mathbf{W}_1$ of class C1 will be updated.

The input $\mathbf{X}_1$ and the weight $\mathbf{W}_1$ are of the same class, consequently

$$\begin{aligned} \mathbf{W}_1 &= [0.50 \ 1.00 \ 1.00 \ 1.00 \ 0.50 \ 0.00 \ 0.00 \ 0.00] + 0.25 * ([1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0] - [0.50 \ 1.00 \ 1.00 \ 1.00 \ 0.50 \ 0.00 \ 0.00 \ 0.00]) \\ &= [0.63 \ 1.00 \ 1.00 \ 1.00 \ 0.38 \ 0.00 \ 0.00 \ 0.00] \end{aligned}$$

Presenting the input $\mathbf{X}_2$:

$$d_1 = (0-0.63)^2 + (1-1.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (1-0.38)^2 + (0-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 = 0.78$$

$$d_2 = (0-0.00)^2 + (1-0.25)^2 + (1-1.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (0-1.25)^2 + (0-0.00)^2 + (0-0.00)^2 = 3.13$$

$$d_3 = (0-0.00)^2 + (1-0.00)^2 + (1-0.00)^2 + (1-(-0.25))^2 + (1-1.00)^2 + (0-1.00)^2 \\ + (0-1.00)^2 + (0-1.25)^2 = 7.13$$

$$d_4 = (0-1.00)^2 + (1-1.13)^2 + (1-0.50)^2 + (1-0.00)^2 + (1-0.00)^2 + (0-(-0.13))^2 \\ + (0-0.50)^2 + (0-1.00)^2 = 4.53$$

d_1 is smaller, the weight vector $\mathbf{W}_1$ of class C1 will be updated.

The input $\mathbf{X}_2$ and the weight $\mathbf{W}_1$ are of the same class, consequently

$$\mathbf{W}_1 = [0.63 \ 1.00 \ 1.00 \ 1.00 \ 0.38 \ 0.00 \ 0.00 \ 0.00] + 0.25 * ([0 \ 1 \ 1 \ 1 \ 1 \ 0 \\ 0 \ 0] - [0.63 \ 1.00 \ 1.00 \ 1.00 \ 0.38 \ 0.00 \ 0.00 \ 0.00]) \\ = [0.47 \ 1.00 \ 1.00 \ 1.00 \ 0.54 \ 0.00 \ 0.00 \ 0.00]$$

Presenting the input $\mathbf{X}_3$:

$$d_1 = (0-0.47)^2 + (0-1.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (1-0.54)^2 + (1-0.00)^2 + (0-0.00)^2 \\ + (0-0.00)^2 = 2.43$$

$$d_2 = (0-0.00)^2 + (0-0.25)^2 + (1-1.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (1-1.25)^2 + (0-0.00)^2 \\ + (0-0.00)^2 = 0.13$$

$$d_3 = (0-0.00)^2 + (0-0.00)^2 + (1-0.00)^2 + (1-(-0.25))^2 + (1-1.00)^2 + (1-1.00)^2 \\ + (0-1.00)^2 + (0-1.25)^2 = 5.13$$

$$d_4 = (0-1.00)^2 + (0-1.13)^2 + (1-0.50)^2 + (1-0.00)^2 + (1-0.00)^2 + (1-(-0.13))^2 \\ + (0-0.50)^2 + (0-1.00)^2 = 7.03$$

d_2 is smaller, the weight vector $\mathbf{W}_2$ of class C2 will be updated.

The input $\mathbf{X}_3$ and the weight $\mathbf{W}_2$ are of the same class, consequently

$$\mathbf{W}_2 = [0.00 \ -0.25 \ 1.00 \ 1.00 \ 1.00 \ 1.25 \ 0.00 \ 0.00] + 0.25 * ([0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 1 \\ 0 \ 0] - [0.00 \ -0.25 \ 1.00 \ 1.00 \ 1.00 \ 1.25 \ 0.00 \ 0.00]) \\ = [0.00 \ -0.19 \ 1.00 \ 1.00 \ 1.00 \ 1.19 \ 0.00 \ 0.00]$$

Presenting the input $\mathbf{X}_4$:

$$d_1 = (0-0.47)^2 + (0-1.00)^2 + (0-1.00)^2 + (1-1.00)^2 + (1-0.54)^2 + (1-0.00)^2 + (1-0.00)^2 + (0-0.00)^2 = 4.43$$

$$d_2 = (0-0.00)^2 + (0-0.19)^2 + (0-1.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (1-1.19)^2 + (1-0.00)^2 + (0-0.00)^2 = 2.07$$

$$d_3 = (0-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 + (1-(-0.25))^2 + (1-1.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (0-1.25)^2 = 3.13$$

$$d_4 = (0-1.00)^2 + (0-1.13)^2 + (0-0.50)^2 + (1-0.00)^2 + (1-0.00)^2 + (1-(-0.13))^2 + (1-0.50)^2 + (0-1.00)^2 = 7.03$$

d_2 is smaller, the weight vector $\mathbf{W}_2$ of class C2 will be updated.

The input $\mathbf{X}_4$ and the weight $\mathbf{W}_2$ are of the same class, consequently

$$\begin{aligned} \mathbf{W}_2 &= [0.00 \ 0.19 \ 1.00 \ 1.00 \ 1.00 \ 1.19 \ 0.00 \ 0.00] + 0.25 * ([0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 0] - [0.00 \ 0.19 \ 1.00 \ 1.00 \ 1.00 \ 1.19 \ 0.00 \ 0.00]) \\ &= [0.00 \ 0.14 \ 0.75 \ 1.00 \ 1.00 \ 1.14 \ 0.25 \ 0.00] \end{aligned}$$

Presenting the input $\mathbf{X}_5$:

$$d_1 = (0-0.47)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-1.00)^2 + (1-0.54)^2 + (1-0.00)^2 + (1-0.00)^2 + (1-0.00)^2 = 6.43$$

$$d_2 = (0-0.00)^2 + (0-(-0.14))^2 + (0-0.75)^2 + (0-1.00)^2 + (1-1.00)^2 + (1-1.14)^2 + (1-0.25)^2 + (1-0.00)^2 = 3.16$$

$$d_3 = (0-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 + (0-(-0.25))^2 + (1-1.00)^2 + (1-1.00)^2 + (1-1.00)^2 + (1-1.25)^2 = 0.13$$

$$d_4 = (0-1.00)^2 + (0-1.13)^2 + (0-0.50)^2 + (0-0.00)^2 + (1-0.00)^2 + (1-(-0.13))^2 + (1-0.50)^2 + (1-1.00)^2 = 5.03$$

d_3 is smaller, the weight vector $\mathbf{W}_3$ of class C3 will be updated.

The input $\mathbf{X}_5$ and the weight $\mathbf{W}_3$ are of the same class, consequently

$$\begin{aligned}\mathbf{W}_3 &= [0.00 \ 0.00 \ 0.00 \ -0.25 \ 1.00 \ 1.00 \ 1.00 \ 1.25] + 0.25*([0 \ 0 \ 0 \ 0 \ 1 \ 1 \\ &1 \ 1] - [0.00 \ 0.00 \ 0.00 \ -0.25 \ 1.00 \ 1.00 \ 1.00 \ 1.25]) \\ &= [0.00 \ 0.00 \ 0.00 \ -0.19 \ 1.00 \ 1.00 \ 1.00 \ 1.19]\end{aligned}$$

Presenting the input $\mathbf{X}_6$:

$$\begin{aligned}d_1 &= (1-0.47)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-0.54)^2 + (1-0.00)^2 + (1-0.00)^2 \\ &+ (1-0.00)^2 = 6.57\end{aligned}$$

$$\begin{aligned}d_2 &= (1-0.00)^2 + (0-(-0.14))^2 + (0-0.75)^2 + (0-1.00)^2 + (0-1.00)^2 + (1-1.14)^2 \\ &+ (1-0.25)^2 + (1-0.00)^2 = 5.16\end{aligned}$$

$$\begin{aligned}d_3 &= (1-0.00)^2 + (0-0.00)^2 + (0-0.00)^2 + (0-(-0.19))^2 + (0-1.00)^2 + (1-1.00)^2 \\ &+ (1-1.00)^2 + (1-1.19)^2 = 2.07\end{aligned}$$

$$\begin{aligned}d_4 &= (1-1.00)^2 + (0-1.13)^2 + (0-0.50)^2 + (0-0.00)^2 + (0-0.00)^2 + (1-(-0.13))^2 \\ &+ (1-0.50)^2 + (1-1.00)^2 = 3.03\end{aligned}$$

d_3 is smaller, the weight vector $\mathbf{W}_3$ of class C3 will be updated.

The input $\mathbf{X}_6$ and the weight $\mathbf{W}_3$ are of the same class, consequently

$$\begin{aligned}\mathbf{W}_3 &= [0.00 \ 0.00 \ 0.00 \ -0.19 \ 1.00 \ 1.00 \ 1.00 \ 1.19] + 0.25*([1 \ 0 \ 0 \ 0 \ 0 \ 1 \\ &1 \ 1] - [0.00 \ 0.00 \ 0.00 \ -0.19 \ 1.00 \ 1.00 \ 1.00 \ 1.19]) \\ &= [0.25 \ 0.00 \ 0.00 \ -0.14 \ 0.75 \ 1.00 \ 1.00 \ 1.14]\end{aligned}$$

Presenting the input $\mathbf{X}_7$:

$$\begin{aligned}d_1 &= (1-0.47)^2 + (1-1.00)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-0.54)^2 + (0-0.00)^2 + (1-0.00)^2 \\ &+ (1-0.00)^2 = 4.57\end{aligned}$$

$$\begin{aligned}d_2 &= (1-0.00)^2 + (1-(-0.14))^2 + (0-0.75)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-1.14)^2 \\ &+ (1-0.25)^2 + (1-0.00)^2 = 7.73\end{aligned}$$

$$\begin{aligned}d_3 &= (1-0.25)^2 + (1-0.00)^2 + (0-0.00)^2 + (0-(-0.14))^2 + (0-0.75)^2 + (0-1.00)^2 \\ &+ (1-1.00)^2 + (1-1.14)^2 = 3.16\end{aligned}$$

$$d_4 = (1-1.00)^2 + (1-1.13)^2 + (0-0.50)^2 + (0-0.00)^2 + (0-0.00)^2 + (0-(-0.13))^2 \\ + (1-0.50)^2 + (1-1.00)^2 = 0.53$$

d_4 is smaller, the weight vector $\mathbf{W}_4$ of class C4 will be updated.

The input $\mathbf{X}_7$ and the weight $\mathbf{W}_4$ are of the same class, consequently

$$\mathbf{W}_4 = [1.00 \ 1.13 \ 0.50 \ 0.00 \ 0.00 \ -0.13 \ 0.50 \ 1.00] + 0.25 * ([1 \ 1 \ 0 \ 0 \ 0 \ 0 \\ 1 \ 1] - [1.00 \ 1.13 \ 0.50 \ 0.00 \ 0.00 \ -0.13 \ 0.50 \ 1.00]) \\ = [1.00 \ 1.10 \ 0.38 \ 0.00 \ 0.00 \ -0.10 \ 0.63 \ 1.00]$$

Presenting the input $\mathbf{X}_8$:

$$d_1 = (1-0.47)^2 + (1-1.00)^2 + (1-1.00)^2 + (0-1.00)^2 + (0-0.53)^2 + (0-0.00)^2 + (0-0.00)^2 \\ + (1-0.00)^2 = 2.56$$

$$d_2 = (1-0.00)^2 + (1-(-0.14))^2 + (1-0.75)^2 + (0-1.00)^2 + (0-1.00)^2 + (0-1.14)^2 \\ + (0-0.25)^2 + (1-0.00)^2 = 6.73$$

$$d_3 = (1-0.25)^2 + (1-0.00)^2 + (1-0.00)^2 + (0-(-0.14))^2 + (0-0.75)^2 + (0-1.00)^2 \\ + (0-1.00)^2 + (1-1.14)^2 = 5.16$$

$$d_4 = (1-1.00)^2 + (1-1.10)^2 + (1-0.38)^2 + (0-0.00)^2 + (0-0.00)^2 + (0-(-0.10))^2 \\ + (0-0.63)^2 + (1-1.00)^2 = 0.80$$

d_4 is smaller, the weight vector $\mathbf{W}_4$ of class C4 will be updated.

The input $\mathbf{X}_8$ and the weight $\mathbf{W}_4$ are of the same class, consequently

$$\mathbf{W}_4 = [1.00 \ 1.10 \ 0.38 \ 0.00 \ 0.00 \ -0.10 \ 0.63 \ 1.00] + 0.25 * ([1 \ 1 \ 1 \ 0 \ 0 \ 0 \\ 0 \ 1] - [1.00 \ 1.09 \ 0.38 \ 0.00 \ 0.00 \ -0.09 \ 0.63 \ 1.00]) \\ = [1.00 \ 1.08 \ 0.54 \ 0.00 \ 0.00 \ -0.08 \ 0.47 \ 1.00]$$

After Iteration 2, the weight vectors are:

$$\mathbf{W}_1 = [0.47 \ 1.00 \ 1.00 \ 1.00 \ 0.53 \ 0.00 \ 0.00 \ 0.00]$$

$$\mathbf{W}_2 = [0.00 \ -0.14 \ 0.75 \ 1.00 \ 1.00 \ 1.14 \ 0.25 \ 0.00]$$

$$\mathbf{W}_3 = [0.25 \ 0.00 \ 0.00 \ -0.14 \ 0.75 \ 1.00 \ 1.00 \ 1.14]$$

$$\mathbf{W}_4 = [1.00 \ 1.08 \ 0.54 \ 0.00 \ 0.00 \ -0.08 \ 0.47 \ 1.00]$$

Computing $g(3) = 0.5 * g(2) = 0.125$.

(b) After Iteration 100: The weight vectors are Using $g(k) = 0.5 * g(k - 1)$ for $k \geq 0$, after Iteration 100, the weight vectors are

$$\mathbf{W}_1 = [0.47 \ 1.00 \ 1.00 \ 1.00 \ 0.53 \ 0.00 \ 0.00 \ 0.00]$$

$$\mathbf{W}_2 = [0.00 \ -0.08 \ 0.64 \ 1.00 \ 1.00 \ 1.08 \ 0.36 \ 0.00]$$

$$\mathbf{W}_3 = [0.36 \ 0.00 \ 0.00 \ -0.08 \ 0.64 \ 1.00 \ 1.00 \ 1.08]$$

$$\mathbf{W}_4 = [1.00 \ 1.04 \ 0.53 \ 0.00 \ 0.00 \ -0.04 \ 0.47 \ 1.00]$$

The recall outputs obtained verify the classification correctness as shown in Table P7.5.1.

Table P7.5.1

k	$\mathbf{X}_k$	$\mathbf{Y}_k$	Class
1	[1 1 1 1 0 0 0 0]	[3.47 , 1.56, 0.28, 2.57]	1
2	[0 1 1 1 1 0 0 0]	[3.53 , 2.56, 0.56, 1.57]	1
3	[0 0 1 1 1 1 0 0]	[2.53, 3.72 , 1.56, 0.49]	2
4	[0 0 0 1 1 1 1 0]	[1.53, 3.44 , 2.56, 0.43]	2
5	[0 0 0 0 1 1 1 1]	[0.53, 2.44, 3.72 , 1.43]	3
6	[1 0 0 0 0 1 1 1]	[0.47, 1.44, 3.44 , 2.43]	3
7	[1 1 0 0 0 0 1 1]	[1.47, 0.28, 2.44, 3.51]	4
8	[1 1 1 0 0 0 0 1]	[2.47, 0.56, 1.44, 3.57]	4

Chapter 8

Radial Basis Function Networks

Problem 8.1 (Interpolation RBF network learning and recall)

In an Interpolation RBF network, consider one hidden unit corresponds to an exact solution of each input pattern.

For the pattern-pairs defined in Table P8.1.1, do the following:

- (a) Draw a labelled Interpolation RBF network for the example under consideration.
- (b) Construct the interpolation matrix Φ for the resulting Interpolation RBF network and compute its inverse.
- (c) Compute the linear weights of the output layer of the Interpolation RBF network.
- (d) Using the values of weights obtained in (c), compute all the outputs to verify the correctness of the results.

Table P8.1.1 NAND Gate problem.

k	$\mathbf{X}_k$	y_k	$\mathbf{T}_p$
1	[0, 0]	1	[0, 0]
2	[0, 1]	1	[0, 1]
3	[1, 0]	1	[1, 0]
4	[1, 1]	0	[1, 1]

Solution

(a) From Table P8.1.1, the interpolation network will have: 2 inputs (same as number of elements in $\mathbf{X}_k$); 4 centers, hence 4 neurons in the hidden layer; 1 output (same as number of elements in $\mathbf{Y}_k$).

The interpolation RBF network of the NAND Gate problem is shown in Figure P8.1.1.

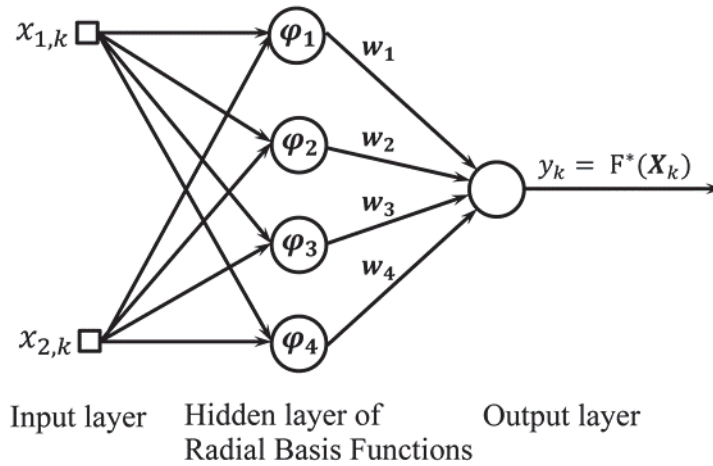


Figure P8.1.1

(b) The 4 RBF inputs and the 4 RBF centers are

$$\mathbf{X}_1 = \mathbf{T}_1 = [0,0] \qquad \mathbf{X}_2 = \mathbf{T}_2 = [0,1]$$

$$\mathbf{X}_3 = \mathbf{T}_3 = [1,0] \qquad \mathbf{X}_4 = \mathbf{T}_4 = [1,1]$$

In general, each element of the matrix $\mathbf{G}$ will have the form:

$$\varphi_{kp} = \exp\left(-\|\mathbf{X}_k - \mathbf{T}_p\|^2\right).$$

$$\begin{aligned} \varphi_{11} &= \exp(-\|\mathbf{X}_1 - \mathbf{T}_1\|^2) = \exp(-\|[0,0] - [0,0]\|^2) \\ &= \exp(-(0)^2) = 1.0000 \end{aligned}$$

$$\begin{aligned} \varphi_{12} &= \exp(-\|\mathbf{X}_1 - \mathbf{T}_2\|^2) = \exp(-\|[0,0] - [0,1]\|^2) \\ &= \exp(-(1)^2) = 0.3679 \end{aligned}$$

$$\varphi_{13} = \exp(-\|\mathbf{X}_1 - \mathbf{T}_3\|^2) = \exp(-\|[0,0] - [1,0]\|^2)$$

$$= \exp(-(1)^2) = 0.3679$$

$$\varphi_{14} = \exp(-\|\mathbf{X}_1 - \mathbf{T}_4\|^2) = \exp(-\|[0,0] - [1,1]\|^2)$$

$$= \exp(-(1.414)^2) = 0.1353$$

$$\varphi_{21} = \exp(-\|\mathbf{X}_2 - \mathbf{T}_1\|^2) = \exp(-\|[0,1] - [0,0]\|^2)$$

$$= \exp(-(1)^2) = 0.3679$$

$$\varphi_{22} = \exp(-\|\mathbf{X}_2 - \mathbf{T}_2\|^2) = \exp(-\|[0,1] - [0,1]\|^2)$$

$$= \exp(-(0)^2) = 1.0000$$

$$\varphi_{23} = \exp(-\|\mathbf{X}_2 - \mathbf{T}_3\|^2) = \exp(-\|[0,1] - [1,0]\|^2)$$

$$= \exp(-(1.414)^2) = 0.1353$$

$$\varphi_{24} = \exp(-\|\mathbf{X}_2 - \mathbf{T}_4\|^2) = \exp(-\|[0,1] - [1,1]\|^2)$$

$$= \exp(-(1)^2) = 0.3679$$

$$\varphi_{31} = \exp(-\|\mathbf{X}_3 - \mathbf{T}_1\|^2) = \exp(-\|[1,0] - [0,0]\|^2)$$

$$= \exp(-(1)^2) = 0.3679$$

$$\varphi_{32} = \exp(-\|\mathbf{X}_3 - \mathbf{T}_2\|^2) = \exp(-\|[1,0] - [0,1]\|^2)$$

$$= \exp(-(1.414)^2) = 0.1353$$

$$\varphi_{33} = \exp(-\|\mathbf{X}_3 - \mathbf{T}_3\|^2) = \exp(-\|[1,0] - [1,0]\|^2)$$

$$= \exp(-(0)^2) = 1.0000$$

$$\varphi_{34} = \exp(-\|\mathbf{X}_3 - \mathbf{T}_4\|^2) = \exp(-\|[1,0] - [1,1]\|^2)$$

$$= \exp(-(1)^2) = 0.3679$$

$$\varphi_{41} = \exp(-\|\mathbf{X}_4 - \mathbf{T}_1\|^2) = \exp(-\|[1,1] - [0,0]\|^2)$$

$$= \exp(-(1.414)^2) = 0.1353$$

$$\varphi_{42} = \exp(-\|\mathbf{X}_4 - \mathbf{T}_2\|^2) = \exp(-\|[1,1] - [0,1]\|^2)$$

$$= \exp(-(1)^2) = 0.3679$$

$$\phi_{43} = \exp(-\|\mathbf{X}_4 - \mathbf{T}_3\|^2) = \exp(-\|[1,1] - [1,0]\|^2)$$

$$= \exp(-(1)^2) = 0.3679$$

$$\phi_{44} = \exp(-\|\mathbf{X}_4 - \mathbf{T}_4\|^2) = \exp(-\|[1,1] - [1,1]\|^2)$$

$$= \exp(-(0)^2) = 1.0000$$

Hence, the interpolation matrix Φ is

$$\Phi = \begin{bmatrix} 1.0000 & 0.3679 & 0.3679 & 0.1353 \\ 0.3679 & 1.0000 & 0.1353 & 0.3679 \\ 0.3679 & 0.1353 & 1.0000 & 0.3679 \\ 0.1353 & 0.3679 & 0.3679 & 1.0000 \end{bmatrix}$$

The inverse interpolation matrix, Φ^{-1} is

$$\Phi^{-1} = \begin{bmatrix} +1.3376 & -0.4922 & -0.4922 & +0.1812 \\ -0.4922 & +1.3376 & +0.1812 & -0.4922 \\ -0.4922 & +0.1812 & +1.3376 & -0.4922 \\ +0.1812 & -0.4922 & -0.4922 & +1.3376 \end{bmatrix}$$

Note: In Matlab, the inverse $\mathbf{B}$ of a matrix $\mathbf{A}$ can be computed using the command $\mathbf{B} = \text{inv}(\mathbf{A})$.

(c) From the NAND problem, the desired output vector is $\mathbf{D} = [1 \ 1 \ 1 \ 0]^T$; and from equation (8.14), $\mathbf{W} = \Phi^{-1}\mathbf{D}$. Hence, the weight matrix is obtained as

$$\mathbf{W} = \Phi^{-1}\mathbf{D} = [0.3533 \ 1.0266 \ 1.0266 \ -0.8032]^T$$

(d) From the interpolation matrix Φ and the weight matrix $\mathbf{W}$ obtained, the outputs of all the 4 input patterns can be verified as

$$\Phi\mathbf{W} = [1.0000 \ 1.0000 \ 1.0000 \ 0.0000]^T = \mathbf{D}$$

Problem 8.2 (Generalized RBF network learning and recall)

Consider a Generalized RBF network with two inputs ($\mathbf{X}_k$), two radial basis function centers ($\mathbf{T}_p$), two weights (w_1 w_2) and one output (y_k) for the four input-output pattern-pair NAND problem as defined in Table P8.2.1.

(a) Draw a labelled Generalized RBF network with a bias b of fixed unity input for the NAND problem.

(b) Define a normalized RBF as $\exp(-\|\mathbf{X}_k - \mathbf{T}_p\|^2)$

$$\mathbf{G}(\|\mathbf{X}_k - \mathbf{T}_p\|) = \exp\left(-(P/d_{max}^2)\|\mathbf{X}_k - \mathbf{T}_p\|^2\right)$$

for $k = 1$ to K and $p = 1$ to P

where K is the number of pattern-pairs, P is the number of hidden units, and d_{max} is the maximum Euclidean norm distance between any two chosen RBF centers.

For the NAND problem with the two given RBF centers $\mathbf{T}_p$, compute the corresponding matrices $\mathbf{G}$ (of the hidden outputs) and $\mathbf{H}$ (of the bias and hidden outputs), and show the calculation steps that the pseudo-inverse $\mathbf{H}^+$ (for computing $\mathbf{W}_c = [b \ w_1 \ w_2]^T$) is given by

$$\mathbf{H}^+ = \begin{bmatrix} -0.9209 & +1.4209 & +1.4209 & -0.9209 \\ +1.8298 & -1.2516 & -1.2516 & +0.6733 \\ +0.6733 & -1.2516 & -1.2516 & +1.8298 \end{bmatrix}$$

(c) Compute the weights and the bias value of the Generalized RBF network.

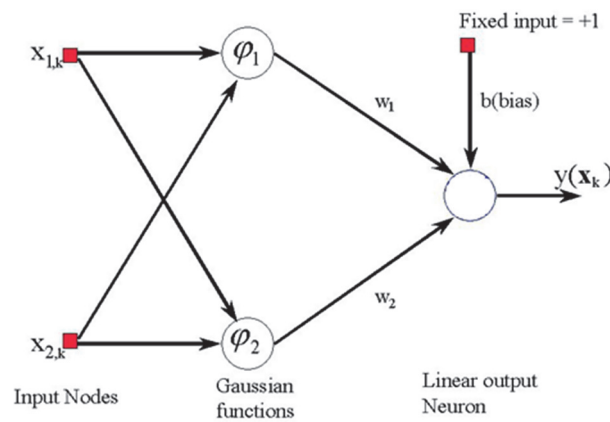
(d) Using the values of weights and bias obtained in (c), recall all the outputs to verify correctness.

Table P8.2.1 NAND Gate Problem.

k	$\mathbf{X}_k$	y_k	$\mathbf{T}_p$
1	[0, 0]	1	[0, 0]
2	[0, 1]	1	
3	[1, 0]	1	
4	[1, 1]	0	[1, 1]

Solution

(a) The labelled Generalized RBF network is shown in Figure P8.2.1.


Figure P8.2.1

(b) The 4 RBF inputs and the 2 RBF centers are

$$\mathbf{X}_1 = [0, 0], \mathbf{X}_2 = [0, 1], \mathbf{X}_3 = [1, 0], \mathbf{X}_4 = [1, 1].$$

$$\mathbf{T}_1 = [0, 0], \mathbf{T}_2 = [1, 1].$$

Note that $P = 2$ and $d_{max}^2 = \|[1, 1] - (0, 0)\|^2 = 2$, so $P/d_{max}^2 = 1$.

In general, each element of the matrix $\mathbf{G}$ will have the form

$$\varphi_{kp} = \exp\left(-\|\mathbf{X}_k - \mathbf{T}_p\|^2\right) = g_{kp}$$

$$g_{11} = \exp(-\|\mathbf{X}_1 - \mathbf{T}_1\|^2) = \exp(-\|[0, 0] - [0, 0]\|^2)$$

$$= \exp(-(0)^2) = 1.0000$$

$$g_{12} = \exp(-\| \mathbf{X}_1 - \mathbf{T}_2 \|^2) = \exp(-\| [0,0] - [1,1] \|^2)$$

$$= \exp(-(1.414)^2) = 0.1353$$

$$g_{21} = \exp(-\| \mathbf{X}_2 - \mathbf{T}_1 \|^2) = \exp(-\| [0,1] - [0,0] \|^2)$$

$$= \exp(-(1)^2) = 0.3679$$

$$g_{22} = \exp(-\| \mathbf{X}_2 - \mathbf{T}_2 \|^2) = \exp(-\| [0,1] - [1,1] \|^2)$$

$$= \exp(-(1)^2) = 0.3679$$

$$g_{31} = \exp(-\| \mathbf{X}_3 - \mathbf{T}_1 \|^2) = \exp(-\| [1,0] - [0,0] \|^2)$$

$$= \exp(-(1)^2) = 0.3679$$

$$g_{32} = \exp(-\| \mathbf{X}_3 - \mathbf{T}_2 \|^2) = \exp(-\| [1,0] - [1,1] \|^2)$$

$$= \exp(-(1)^2) = 0.3679$$

$$g_{41} = \exp(-\| \mathbf{X}_4 - \mathbf{T}_1 \|^2) = \exp(-\| [1,1] - [0,0] \|^2)$$

$$= \exp(-(1.414)^2) = 0.1353$$

$$g_{42} = \exp(-\| \mathbf{X}_4 - \mathbf{T}_2 \|^2) = \exp(-\| [1,1] - [1,1] \|^2)$$

$$= \exp(-(0)^2) = 1.0000$$

Hence

$$\mathbf{G} = \begin{bmatrix} g_{11} & g_{12} \\ g_{21} & g_{22} \\ g_{31} & g_{32} \\ g_{41} & g_{42} \end{bmatrix} = \begin{bmatrix} 1.0000 & 0.1353 \\ 0.3679 & 0.3679 \\ 0.3679 & 0.3679 \\ 0.1353 & 1.0000 \end{bmatrix}$$

The combined weight vector $\mathbf{W}_c$ is defined as

$$\mathbf{W}_c = [b \ w_1 \ w_2]^T$$

Since the desired output vector $\mathbf{D} = [1 \ 1 \ 1 \ 0]^T$, $\mathbf{W}_c$ can be solved from the following matrix equation

$$\mathbf{W}_c = \mathbf{H}^+ \mathbf{D}$$

where $\mathbf{H}^+$ is the pseudo-inverse of the matrix $\mathbf{H}$ that can be derived from $\mathbf{G}$ as

$$\mathbf{H} = \begin{bmatrix} 1 & g_{11} & g_{12} \\ 1 & g_{21} & g_{22} \\ 1 & g_{31} & g_{32} \\ 1 & g_{41} & g_{42} \end{bmatrix} = \begin{bmatrix} 1.0000 & 1.0000 & 0.1353 \\ 1.0000 & 0.3679 & 0.3679 \\ 1.0000 & 0.3679 & 0.3679 \\ 1.0000 & 0.1353 & 1.0000 \end{bmatrix}$$

Hence, we obtain

$$\mathbf{H}^+ = (\mathbf{H}^T \mathbf{H})^{-1} \mathbf{H}^T$$

and

$$\mathbf{H}^+ = \begin{bmatrix} -0.9209 & +1.4209 & +1.4209 & -0.9209 \\ +1.8298 & -1.2516 & -1.2516 & +0.6733 \\ +0.6733 & -1.2516 & -1.2516 & +1.8298 \end{bmatrix}$$

where

$$(\mathbf{H}^T \mathbf{H})^{-1} = \begin{bmatrix} 5.3740 & -5.8618 & -5.8618 \\ -5.8618 & 6.9344 & 5.5969 \\ -5.8618 & 5.5969 & 6.9344 \end{bmatrix}$$

(c) Solving $\mathbf{W}_c = \mathbf{H}^+ \mathbf{D}$, we obtain

$$\mathbf{W}_c = \begin{bmatrix} +1.9209 \\ -0.6733 \\ -1.8298 \end{bmatrix}$$

where $b = 1.9209$, $w_1 = -0.6733$, $w_2 = -1.8298$.

(d) Using the designed values of weights and bias, the recalled output vector $\mathbf{Y}$ is

$$\mathbf{Y} = \mathbf{H} \mathbf{W}_c = \begin{bmatrix} 1.0000 & 1.0000 & 0.1353 \\ 1.0000 & 0.3679 & 0.3679 \\ 1.0000 & 0.3679 & 0.3679 \\ 1.0000 & 0.1353 & 1.0000 \end{bmatrix} \begin{bmatrix} +1.9209 \\ -0.6733 \\ -1.8298 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 0 \end{bmatrix}$$

From the above computation, $\mathbf{Y}$ has been verified to be the same as the desired output vector.

Chapter 9

Fuzzy Sets

Problem 9.1 (Derivation of fuzzy complement of fuzzy set)

Derive the general form of the classical fuzzy complement of a fuzzy set A as described by $\mu_A(x) = e^{\frac{-1}{2}\left(\frac{x-c}{\sigma}\right)^2}$.

Solution

The fuzzy set A is described by the Gaussian function

$$\mu_A(x) = e^{\frac{-1}{2}\left(\frac{x-c}{\sigma}\right)^2}$$

The classical fuzzy complement of A is defined as

$$\mu_{\bar{A}}(x) = 1 - \mu_A(x)$$

which can be derived as

$$\mu_{\bar{A}}(x) = 1 - e^{\frac{-1}{2}\left(\frac{x-c}{\sigma}\right)^2} = 1 - \frac{1}{e^{\frac{1}{2}\left(\frac{x-c}{\sigma}\right)^2}} = \frac{e^{\frac{1}{2}\left(\frac{x-c}{\sigma}\right)^2} - 1}{e^{\frac{1}{2}\left(\frac{x-c}{\sigma}\right)^2}}$$

Problem 9.2 (Membership function terminologies)

- (a) Define the α -cut (or α -level) set of a fuzzy set A .
- (b) Based on the definition in Q9.2(a), determine Core (A).
- (c) Define the triangular membership function $\text{TriMF}(x, [a, b, c])$ in each of its distinct regions.
- (d) Given a fuzzy set A defined by a triangular membership function $\mu_A(x) = \text{TriMF}(x, [a, b, c])$, with the aid of a labelled diagram, derive

an expression for each of the α -cut set A_α and the width of the α -cut set.

(e) Define the strong α -cut (or strong α -level) set of a fuzzy set A .

(f) Based on the definition in Q9.2(d), determine Support (A).

Solution

(a) The α -cut (or α -level) set of a fuzzy set A is a crisp set defined by

$$A_\alpha = \{x | \mu_A(x) \geq \alpha\}$$

(b) The Core (A) is given by $\text{Core}(A) = A_1$ for $\alpha = 1$.

(c) The triangular membership function is defined by

$$\text{TriMF}(x, [a, b, c]) = \begin{cases} 0 & \text{if } x < a \\ \frac{x-a}{b-a} & \text{if } a \leq x < b \\ \frac{c-x}{c-b} & \text{if } b \leq x < c \\ 0 & \text{if } c \leq x \end{cases}$$

(d) Consider the triangular membership function $\mu_A(x)$ as drawn in Figure P9.2.1,

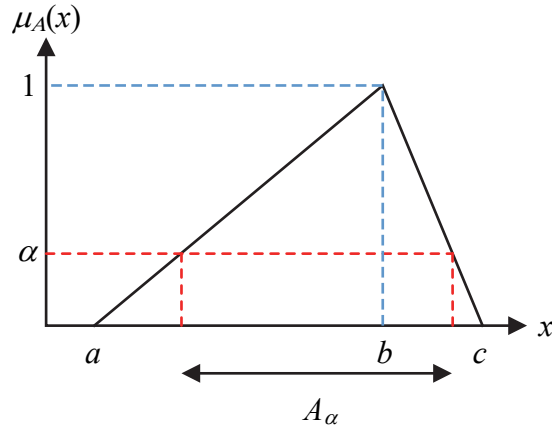


Figure P9.2.1

To find x for $\mu_A(x) = \alpha$, we have

$$(x_1 - a)/(b - a) = \alpha, \text{ hence } x_1 = a(1 - \alpha) + b\alpha$$

$$(c - x_2)/(c - b) = \alpha, \text{ hence } x_2 = c(1 - \alpha) + b\alpha$$

Therefore, the expression for the α -cut set is

$$A_\alpha = [x_1, x_2] = [a(1 - \alpha) + b\alpha, c(1 - \alpha) + b\alpha]$$

Also, the expression for the width of A_α is

$$c(1 - \alpha) + b\alpha - [a(1 - \alpha) + b\alpha] = (c - a)(1 - \alpha)$$

(e) The strong α -cut (or strong α -level) set of a fuzzy set A is a crisp set defined as

$$A'_\alpha = \{x | \mu_A(x) > \alpha\}$$

(f) The Support (A) is given by

$$\text{Support}(A) = A'_0 \text{ for } \alpha = 0$$

Problem 9.3 (Width of modified Gaussian membership function)

Let the fuzzy set A be defined by a modified Gaussian MF as

$$\mu_A(x) = e^{\frac{-1}{2}\left(\frac{x-c}{\sigma}\right)^{2b}}$$

Show that for all $\alpha \in (0, 1)$,

$$\lim_{b \rightarrow \infty} \text{width}(A_\alpha) = 2\sigma$$

Solution

The two end points of the fuzzy set A_α can be derived as $-2 \log_e \alpha$

$$\begin{aligned} \left\{x \mid e^{\frac{-1}{2}\left(\frac{x-c}{\sigma}\right)^{2b}} = \alpha\right\} &\Rightarrow \left(\frac{x-c}{\sigma}\right)^{2b} \\ &= -2 \ln \alpha \Rightarrow x = c \pm \sigma \sqrt{(-2 \ln \alpha)^{1/b}} \end{aligned}$$

Hence,

$$\begin{aligned}\text{width}(A_\alpha) &= c + \sigma(-2 \ln \alpha)^{1/2b} - \left[c - \sigma(-2 \ln \alpha)^{\frac{1}{2b}} \right] \\ &= 2\sigma(-2 \ln \alpha)^{\frac{1}{2b}}\end{aligned}$$

such that

$$\lim_{b \rightarrow \infty} \text{width}(A_\alpha) = 2\sigma(-2 \ln \alpha)^0 = 2\sigma$$

The above limit is valid for $\alpha \in (0, 1)$ since the two interval ends result in the undefined terms $2\sigma[\infty]^0$ and $2\sigma[0]^0$ respectively.

Problem 9.4 (Derivatives of Gaussian membership function)

Derive the partial derivatives of a Gaussian MF $y = \text{GaussMF}(x, c, \sigma)$ with respect to its argument x , and parameters c and σ , to verify the equations given in Table 9.7.1.

Solution

The Gaussian MF is defined as

$$y = \text{GaussMF}(x, a, c) = e^{\frac{-1}{2}\left(\frac{x-c}{\sigma}\right)^2}$$

By using the chain rule, the partial derivative of y with respect to x is

$$\frac{\partial y}{\partial x} = e^{\frac{-1}{2}\left(\frac{x-c}{\sigma}\right)^2} \left(-\frac{x-c}{\sigma} \right) \frac{1}{\sigma} = -\frac{x-c}{\sigma^2} y$$

Based on the fact that x and c appear in the same location of the Gaussian MF except for their sign difference, the partial derivative of y with respect to c is

$$\frac{\partial y}{\partial c} = -\frac{\partial y}{\partial x} = \frac{x-c}{\sigma^2} y$$

The partial derivative of y with respect to σ is

$$\frac{\partial y}{\partial \sigma} = e^{\frac{-1}{2}\left(\frac{x-c}{\sigma}\right)^2} \left(-\frac{x-c}{\sigma}\right) \left(-\frac{x-c}{\sigma^2}\right) = -\frac{(x-c)^2}{\sigma^3} y$$

Problem 9.5 (Complement, union, and interception of 2 fuzzy sets)

Consider the following two fuzzy sets

$$A = 0.1/-2 + 0.5/-1 + 1/0 + 0.6/1 + 0.2/2$$

$$B = 0.2/-2 + 0.4/-1 + 0/0 + 0.7/1 + 0.1/2$$

Write a Matlab program to perform:

- (a) Complement of a fuzzy set A ;
- (b) Complement of a fuzzy set B ;
- (c) Union of two fuzzy sets A and B ; and
- (d) Interception of two fuzzy sets A and B .

Solution

(a) Calculate the Complement (Negation) of the fuzzy set A denoted by $\bar{A}$.

Enter 1 for Complement; 2 for Union; 3 for Interception: 1

Enter fuzzy set A : [0.1,0.5,1,0.6,0.2]

$$\text{Complement } (A) = 0.9/-2 + 0.5/-1 + 0/0 + 0.4/1 + 0.8/2$$

Calculate the Complement (Negation) of the fuzzy set B denoted by $\bar{B}$.

Enter 1 for Complement; 2 for Union; 3 for Interception: 1

Enter fuzzy set B : [0.2,0.4,0,0.7,0.1]

$$\text{Complement } (B) = 0.8/-2 + 0.6/-1 + 1/0 + 0.3/1 + 0.9/2$$

(b) Calculate the Union (Disjunction) of the fuzzy set A and the fuzzy set B denoted by $A \cup B$.

Enter 1 for Complement; 2 for Union; 3 for Intersection: 2

Enter fuzzy set A: [0.1,0.5,1,0.6,0.2]

Enter fuzzy set B: [0.2,0.4,0,0.7,0.1]

Union(A, B)= $0.2/-2 + 0.5/-1 + 1/0 + 0.7/1 + 0.2/2$

(c) Calculate the Intersection (Conjunction) of the fuzzy set A and the fuzzy set B denoted by $A \cap B$.

Enter 1 for Complement; 2 for Union; 3 for Intersection: 3

Enter fuzzy set A: [0.1,0.5,1,0.6,0.2]

Enter fuzzy set B: [0.2,0.4,0,0.7,0.1]

Intersection (A, B)= $0.1/-2 + 0.4/-1 + 0/0 + 0.6/1 + 0.1/2$

Matlab program

%Problem 9.5

clear all; clc;

option=input('Enter 1 for Complement; 2 for Union; 3 for Intersection: ');

% Example

% For fuzzy set A= $0.1/-2+0.5/-1+1/0+0.6/1+0.2/2$,

% enter [0.1,0.5,1,0.6,0.2]

% For fuzzy set B= $0.2/-2+0.4/-1+0/0+0.7/1+0.1/2$,

% enter [0.2,0.4,0,0.7,0.1]

if option==1

 A=input('Enter fuzzy set A: ');

 [Am,n]=size(A);

else

 A=input('Enter fuzzy set A: ');

 B=input('Enter fuzzy set B: ');

 [Am,n]=size(A);

 [Bm,n]=size(B);

end

if option==1

 Ac=ones(Am)-A;

 disp('Complement(A)=');

 disp(Ac)

```
elseif option==2
    U=max(A,B);
    disp('Union(A,B)=');
    disp(U)
elseif option==3
    [m,n]=size(A);
    I=min(A,B);
    disp('Interception(A,B)=');
    disp(I)
end
```

Chapter 10

Fuzzy Logic and Reasoning

Problem 10.1 (Binary fuzzy relation matrix computation)

Let $X = Y = \mathbf{R}^+$ and $R(X, Y) = \text{"}x \text{ is close to } y\text{"}$. The membership function of the fuzzy relation $R(X, Y)$ can be subjectively defined as

$$\mu_R(x, y) = e^{-\sqrt{(y-x)^2}}$$

Given $X = \{3, 4, 5\}$ and $Y = \{3, 4, 5, 6, 7\}$:

(a) calculate the element values of the binary fuzzy relation matrix $\mathbf{R}$ where $R(X, Y) = \{((x, y), \mu_R(x, y)) | (x, y) \in X \times Y\}$;

(b) Plot and explain (with notations adopted) the corresponding sagittal diagram.

Solution

(a) The element values of $\mathbf{R}$ can be calculated from the given $\mu_R(x, y)$ at each (x, y) as

$$\mathbf{R}(1,1) = \mu_R(x_1, y_1) = 1$$

$$\mathbf{R}(1,2) = \mu_R(x_1, y_2) = 0.368$$

$$\mathbf{R}(1,3) = \mu_R(x_1, y_3) = 0.135$$

$$\mathbf{R}(1,4) = \mu_R(x_1, y_4) = 0.050$$

$$\mathbf{R}(1,5) = \mu_R(x_1, y_5) = 0.018$$

$$\mathbf{R}(2,1) = \mu_R(x_2, y_1) = 0.368$$

$$\mathbf{R}(2,2) = \mu_R(x_2, y_2) = 1$$

$$\mathbf{R}(2,3) = \mu_R(x_2, y_3) = 0.368$$

$$\mathbf{R}(2,4) = \mu_R(x_2, y_4) = 0.135$$

$$\mathbf{R}(2,5) = \mu_R(x_2, y_5) = 0.050$$

$$\mathbf{R}(3,1) = \mu_R(x_3, y_1) = 0.135$$

$$\mathbf{R}(3,2) = \mu_R(x_3, y_2) = 0.368$$

$$\mathbf{R}(3,3) = \mu_R(x_3, y_3) = 1$$

$$\mathbf{R}(3,4) = \mu_R(x_3, y_4) = 0.368$$

$$\mathbf{R}(3,5) = \mu_R(x_3, y_5) = 0.135$$

Therefore the binary fuzzy relation matrix $\mathbf{R}$ is given by

$$\mathbf{R} = \begin{bmatrix} 1.000 & 0.368 & 0.135 & 0.050 & 0.018 \\ 0.368 & 1.000 & 0.368 & 0.135 & 0.050 \\ 0.135 & 0.368 & 1.000 & 0.368 & 0.135 \end{bmatrix}$$

The (i, j) th element for $1 \leq i \leq 3$ and $1 \leq j \leq 5$ of the fuzzy relation matrix $\mathbf{R}$ expresses the degree of membership of $R(x, y) = "x \text{ is close to } y"$.

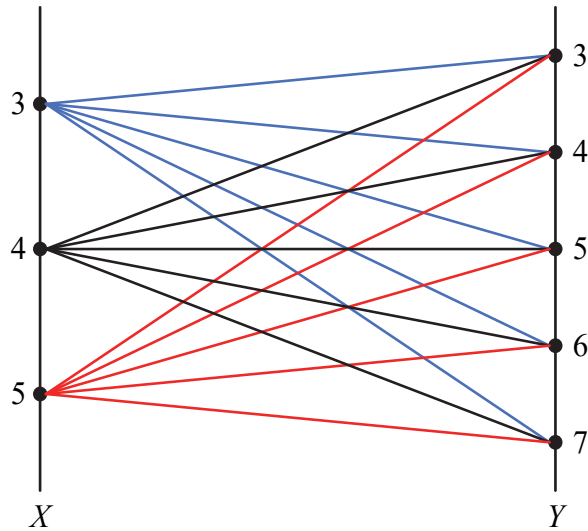


Figure P10.1.1 Sagittal diagram of Problem 10.1.

(b) The sagittal diagram for Problem 10.1 is shown in Figure P10.1.1 where the black solid line connecting $x = 4$ and $y = 3$ has a value of $R(4,3) = 0.368$; and similarly, $R(4,4) = 1$, $R(4,5) = 0.368$, $R(4,6) = 0.135$, $R(4,7) = 0.050$.

The same notations apply to all the 5 blue solid line connections from $x = 3$ to $Y = \{3, 4, 5, 6, 7\}$; as well as all the 5 red solid connections from $x = 5$ to $Y = \{3, 4, 5, 6, 7\}$.

Problem 10.2 (Properties of binary relations using Max-Min composition)

Let P, Q , and R be binary fuzzy relations on $X \times Y$, $Y \times Z$, and $Z \times W$, give the step-by-step proofs of the following properties of binary relations using Max-Min composition:

- (a) $P \circ (Q \circ R) = (P \circ Q) \circ R$
- (b) $P \circ (Q \cup R) = (P \circ Q) \cup (P \circ R)$
- (c) $P \circ (Q \cap R) \subseteq (P \circ Q) \cap (P \circ R)$
- (d) $Q \supseteq R \Rightarrow (P \circ Q) \supseteq (P \circ R)$

Solution

Let $P = [p_{xy}]$, $Q = [q_{yz}]$, $R = [r_{zw}]$ denote, respectively, binary fuzzy relations on $X \times Y$, $Y \times Z$, $Z \times W$, and for Max-Min composition, we have

(a)

$$[(P \circ Q) \circ R] = \vee_z [(P \circ Q)_{xz} \wedge r_{zw}]$$

(Max-Min composition of $(P \circ Q)$ & R along z)

$$= \vee_z \{ [\vee_y (p_{xy} \wedge q_{yz})] \wedge r_{zw} \}$$

(Max-Min composition of P & Q along y)

$$= \vee_z [\vee_y (p_{xy} \wedge q_{yz} \wedge r_{zw})]$$

$$\begin{aligned}
 &= \vee_y [\vee_z (p_{xy} \wedge q_{yz} \wedge r_{zw})] \quad (\text{Swap Max along } y \text{ with Max along } z) \\
 &= \vee_y \{p_{xy} \wedge [\vee_z (q_{yz} \wedge r_{zw})]\} \\
 &= \vee_y [p_{xy} \wedge (Q \circ R)_{yw}] \\
 &= [P \circ (Q \circ R)]
 \end{aligned}$$

Therefore

$$(P \circ Q) \circ R = P \circ (Q \circ R)$$

(b)

$$\begin{aligned}
 &[P \circ (Q \cup R)] \\
 &= \vee_y [p_{xy} \wedge (q_{yz} \vee r_{zw})] \\
 &= \vee_y [(p_{xy} \wedge q_{yz}) \vee (p_{xy} \wedge r_{zw})] \\
 &= [\vee_y [(p_{xy} \wedge q_{yz})] \vee [\vee_{\max(Y,Z)} (p_{xy} \wedge r_{zw})]] \quad (\text{take } \max(Y, Z) \text{ for } \wedge) \\
 &= (P \circ Q)_{xz} \vee (P \circ R)_{xw} \\
 &= [(P \circ Q) \cup (P \circ R)]_{xn} \quad (n = \max(Z, W) \text{ for } \cup)
 \end{aligned}$$

Therefore

$$P \circ (Q \cup R) = (P \circ Q) \cup (P \circ R)$$

(c)

$$\begin{aligned}
 &[P \circ (Q \cap R)] \\
 &= \vee_y [p_{xy} \wedge (q_{yz} \wedge r_{zw})] \\
 &= \vee_y [(p_{xy} \wedge q_{yz}) \wedge (p_{xy} \wedge r_{zw})] \quad (\text{see (b)}) \\
 &\leq [\vee_y (p_{xy} \wedge q_{yz})] \wedge [\vee_{\max(Y,Z)} (p_{xy} \wedge r_{zw})] \quad (\text{see proof below}) \\
 &= [\vee_y (p_{xy} \wedge q_{yz})] \wedge [\vee_m (p_{xm} \wedge r_{mw})] \\
 &= (P \circ Q)_{xz} \wedge (P \circ R)_{xw}
 \end{aligned}$$

$$= [(P \circ Q) \cap (P \circ R)]_{xn} \quad (n = \min(Z, W) \text{ for } \cap)$$

Therefore

$$P \circ (Q \cap R) \subseteq (P \circ Q) \cap (P \circ R)$$

Auxiliary inequality

$$\vee_z (s_z \wedge t_z) \leq (\vee_z s_z) \wedge (\vee_z t_z)$$

Proof

$$t_z \leq \vee_w t_w \quad \text{for } Z \leq W$$

$$\Rightarrow s_z \wedge t_z \leq s_z \wedge (\vee_w t_w)$$

$$\Rightarrow \vee_z (s_z \wedge t_z) \leq \vee_z [s_z \wedge (\vee_w t_w)] = (\vee_z s_z) \wedge (\vee_z t_z)$$

$$\Rightarrow \vee_z (s_z \wedge t_z) \leq (\vee_z s_z) \wedge (\vee_z t_z)$$

(d)

$$Q \supseteq R$$

$$\Rightarrow q_{yz} \geq r_{zw}$$

$$\Rightarrow (p_{xy} \wedge q_{yz}) \geq (p_{xy} \wedge r_{zw})$$

$$\Rightarrow [\vee_y (p_{xy} \wedge q_{yz}) \geq \vee_{\max(Y, Z)} (p_{xy} \wedge r_{zw})] \quad (\text{take } \max(Y, Z) \text{ for } \wedge)$$

$$= (P \circ Q)_{xz} \geq (P \circ R)_{xw}$$

Therefore

$$Q \supseteq R \Rightarrow P \circ Q \supseteq P \circ R$$

Problem 10.3 (Properties of binary relations using Max-Product composition)

Let P , Q , and R are binary fuzzy relations on $X \times Y$, $Y \times Z$ and $Z \times W$, give the proofs of the following properties of binary relations using Max-Product composition:

$$(a) P \circ (Q \circ R) = (P \circ Q) \circ R$$

$$(b) P \circ (Q \cup R) = (P \circ Q) \cup (P \circ R)$$

$$(c) Q \supseteq R \Rightarrow (P \circ Q) \supseteq (P \circ R)$$

Solution

Let $P = [p_{xy}]$, $Q = [q_{yz}]$, $R = [r_{zw}]$ denote, respectively, binary fuzzy relations on $X \times Y$, $Y \times Z$, $Z \times W$, and for Max-Product composition, we have

(a)

$$\begin{aligned} & [(P \circ Q) \circ R] \\ &= \vee_z [(P \circ Q)_{xz} \cdot r_{zw}] \\ &= \vee_z \{ [\vee_y (p_{xy} \cdot q_{yz})] \cdot r_{zw} \} \\ &= \vee_z [\vee_y (p_{xy} \cdot q_{yz} \cdot r_{zw})] \\ &= \vee_y [\vee_z (p_{xy} \cdot q_{yz} \cdot r_{zw})] \\ &= \vee_y \{ p_{xy} \cdot [\vee_z (q_{yz} \cdot r_{zw})] \} \\ &= \vee_y [p_{xy} \cdot (Q \circ R)_{yw}] \\ &= [P \circ (Q \circ R)] \end{aligned}$$

Therefore

$$(P \circ Q) \circ R = P \circ (Q \circ R).$$

(b)

$$\begin{aligned}
& [P \circ (Q \cup R)] \\
&= \vee_y [p_{xy} \cdot (q_{yz} \vee r_{zw})] \\
&= \vee_y [(p_{xy} \cdot q_{yz}) \vee (p_{xy} \cdot r_{zw})] \\
&= [\vee_y [(p_{xy} \cdot q_{yz})] \vee [\vee_{\min(Y,Z)} (p_{xy} \cdot r_{zw})]] \text{ (take } \min(Y, Z) \text{ for } \cdot) \\
&= (P \circ Q)_{xw} \vee (P \circ R)_{xw} \\
&= [(P \circ Q) \cup (P \circ R)]
\end{aligned}$$

Therefore

$$P \circ (Q \cup R) = (P \circ Q) \cup (P \circ R)$$

(c)

$$\begin{aligned}
& Q \supseteq R \\
&\Rightarrow q_{yz} \geq r_{zw} \\
&\Rightarrow p_{xy} \cdot q_{yz} \geq p_{xy} \cdot r_{zw} \\
&\Rightarrow \vee_y (p_{xy} \cdot q_{yz}) \geq \vee_{\min(Y,Z)} (p_{xy} \cdot r_{zw}) \text{ (take } \min(Y, Z) \text{ for } \cdot) \\
&= (P \circ Q)_{xz} \geq (P \circ R)_{xw}
\end{aligned}$$

Therefore

$$Q \supseteq R \Rightarrow P \circ Q \supseteq P \circ R$$

Problem 10.4 (Verifying properties of binary relations using Max-Min composition)

Assume that P, Q, R can be expressed as the following relation matrices

$$P = \begin{bmatrix} 0.7 & 0.8 & 0.9 \\ 0.8 & 0.7 & 0.6 \end{bmatrix}, Q = \begin{bmatrix} 0.8 & 0.8 & 0.7 \\ 0.7 & 0.6 & 0.7 \\ 0.9 & 0.8 & 0.9 \end{bmatrix}, R = \begin{bmatrix} 0.8 & 0.7 \\ 0.7 & 0.6 \\ 0.9 & 0.8 \end{bmatrix}$$

Verify the binary relation between the left and right hand sides of each of the four properties of Problem 10.2 using the above three fuzzy relation matrices for Max-Min composition.

Solution

Max-Min composition

Property 1

$$\begin{aligned} \text{LHS} &= [(P \circ Q) \circ R] = \vee_z [(P \circ Q)_{xz} \wedge r_{zw}] \\ &= \vee_z \{ [\vee_y (p_{xy} \wedge q_{yz})] \wedge r_{zw} \} = \vee_z \left\{ \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \end{bmatrix} \wedge r_{zw} \right\} \end{aligned}$$

where

$$m_{11} = \max(0.7 \wedge 0.8, 0.8 \wedge 0.7, 0.9 \wedge 0.9)$$

$$m_{12} = \max(0.7 \wedge 0.8, 0.8 \wedge 0.6, 0.9 \wedge 0.8)$$

$$m_{13} = \max(0.7 \wedge 0.7, 0.8 \wedge 0.7, 0.9 \wedge 0.9)$$

$$m_{21} = \max(0.8 \wedge 0.8, 0.7 \wedge 0.7, 0.6 \wedge 0.9)$$

$$m_{22} = \max(0.8 \wedge 0.8, 0.7 \wedge 0.6, 0.6 \wedge 0.8)$$

$$m_{23} = \max(0.8 \wedge 0.7, 0.7 \wedge 0.7, 0.6 \wedge 0.9)$$

Hence

$$[(P \circ Q) \circ R] = \vee_z \left\{ \begin{bmatrix} 0.9 & 0.8 & 0.9 \\ 0.8 & 0.8 & 0.7 \end{bmatrix} \wedge r_{zw} \right\}$$

$$\begin{aligned}
&= \begin{bmatrix} \max(0.9 \wedge 0.8, 0.8 \wedge 0.7, 0.9 \wedge 0.9) & \max(0.9 \wedge 0.7, 0.8 \wedge 0.6, 0.9 \wedge 0.8) \\ \max(0.8 \wedge 0.8, 0.8 \wedge 0.7, 0.7 \wedge 0.9) & \max(0.8 \wedge 0.7, 0.8 \wedge 0.6, 0.7 \wedge 0.8) \end{bmatrix} \\
&= \begin{bmatrix} 0.9 & 0.8 \\ 0.8 & 0.7 \end{bmatrix}
\end{aligned}$$

$$\text{RHS} = [P \circ (Q \circ R)] = \vee_y [p_{xy} \wedge (Q \circ R)_{yw}]$$

$$= \vee_y \{ [p_{xy} \wedge [\vee_z (q_{yz} \wedge r_{zw})]] \} = \vee_y \left\{ p_{xy} \wedge \begin{bmatrix} n_{11} & n_{12} \\ n_{21} & n_{22} \\ n_{31} & n_{32} \end{bmatrix} \right\}$$

where

$$n_{11} = \max(0.8 \wedge 0.8, 0.8 \wedge 0.7, 0.7 \wedge 0.9)$$

$$n_{12} = \max(0.8 \wedge 0.7, 0.8 \wedge 0.6, 0.7 \wedge 0.8)$$

$$n_{21} = \max(0.7 \wedge 0.8, 0.6 \wedge 0.7, 0.7 \wedge 0.9)$$

$$n_{22} = \max(0.7 \wedge 0.7, 0.6 \wedge 0.6, 0.7 \wedge 0.8)$$

$$n_{31} = \max(0.9 \wedge 0.8, 0.8 \wedge 0.7, 0.9 \wedge 0.9)$$

$$n_{32} = \max(0.9 \wedge 0.7, 0.8 \wedge 0.6, 0.9 \wedge 0.8)$$

$$\begin{aligned}
[P \circ (Q \circ R)] &= \vee_y \left\{ p_{xy} \wedge \begin{bmatrix} 0.8 & 0.7 \\ 0.7 & 0.7 \\ 0.9 & 0.8 \end{bmatrix} \right\} \\
&= \begin{bmatrix} \max(0.7 \wedge 0.8, 0.8 \wedge 0.7, 0.9 \wedge 0.9) & \max(0.7 \wedge 0.7, 0.8 \wedge 0.7, 0.9 \wedge 0.8) \\ \max(0.8 \wedge 0.8, 0.7 \wedge 0.7, 0.6 \wedge 0.9) & \max(0.8 \wedge 0.7, 0.7 \wedge 0.7, 0.6 \wedge 0.8) \end{bmatrix} \\
&= \begin{bmatrix} 0.9 & 0.8 \\ 0.8 & 0.7 \end{bmatrix}
\end{aligned}$$

Hence, LHS = RHS, i.e., $(P \circ Q) \circ R = P \circ (Q \circ R)$.

Property 2

$$\begin{aligned}
 \text{LHS} &= P \circ (Q \cup R) = \vee_y [p_{xy} \wedge (q_{yz} \vee r_{zw})] \\
 &= \vee_y \left\{ p_{xy} \wedge \begin{bmatrix} 0.8 \vee 0.8 & 0.8 \vee 0.7 & 0.7 \vee 0.0 \\ 0.7 \vee 0.7 & 0.6 \vee 0.6 & 0.7 \vee 0.0 \\ 0.9 \vee 0.9 & 0.8 \vee 0.8 & 0.9 \vee 0.0 \end{bmatrix} \right\} \\
 &= \vee_z \left\{ p_{xy} \wedge \begin{bmatrix} 0.8 & 0.8 & 0.7 \\ 0.7 & 0.6 & 0.7 \\ 0.9 & 0.8 & 0.9 \end{bmatrix} \right\} \\
 &= \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \end{bmatrix}
 \end{aligned}$$

where

$$m_{11} = \max(0.7 \wedge 0.8, 0.8 \wedge 0.7, 0.9 \wedge 0.9)$$

$$m_{12} = \max(0.7 \wedge 0.8, 0.8 \wedge 0.6, 0.9 \wedge 0.8)$$

$$m_{13} = \max(0.7 \wedge 0.7, 0.8 \wedge 0.7, 0.9 \wedge 0.9)$$

$$m_{21} = \max(0.8 \wedge 0.8, 0.7 \wedge 0.7, 0.6 \wedge 0.9)$$

$$m_{22} = \max(0.8 \wedge 0.8, 0.7 \wedge 0.6, 0.6 \wedge 0.8)$$

$$m_{23} = \max(0.8 \wedge 0.7, 0.7 \wedge 0.7, 0.6 \wedge 0.9)$$

Hence

$$P \circ (Q \cup R) = \begin{bmatrix} 0.9 & 0.8 & 0.9 \\ 0.8 & 0.8 & 0.7 \end{bmatrix}$$

$$\text{RHS} = (P \circ Q) \cup (P \circ R)$$

$$\begin{aligned}
 &= [\vee_y (p_{xy} \wedge q_{yz})] \vee [\vee_{\max(Y,Z)} (p_{xy} \wedge r_{zw})] \\
 &= \left\{ \vee_z \left(\begin{bmatrix} 0.7 & 0.8 & 0.9 \\ 0.8 & 0.7 & 0.6 \end{bmatrix} \wedge \begin{bmatrix} 0.8 & 0.8 & 0.7 \\ 0.7 & 0.6 & 0.7 \\ 0.9 & 0.8 & 0.9 \end{bmatrix} \right) \right\} \\
 &\quad \vee \left\{ \vee_z \left(\begin{bmatrix} 0.7 & 0.8 & 0.9 \\ 0.8 & 0.7 & 0.6 \end{bmatrix} \wedge \begin{bmatrix} 0.8 & 0.7 \\ 0.7 & 0.6 \\ 0.9 & 0.8 \end{bmatrix} \right) \right\}
 \end{aligned}$$

$$\begin{aligned}
&= \begin{bmatrix} \max(0.7 \wedge 0.8, 0.8 \wedge 0.7, 0.9 \wedge 0.9) & \max(0.7 \wedge 0.8, 0.8 \wedge 0.6, 0.9 \wedge 0.8) & \max(0.7 \wedge 0.7, 0.8 \wedge 0.7, 0.9 \wedge 0.9) \\ \max(0.8 \wedge 0.8, 0.7 \wedge 0.7, 0.6 \wedge 0.9) & \max(0.8 \wedge 0.8, 0.7 \wedge 0.6, 0.6 \wedge 0.8) & \max(0.8 \wedge 0.7, 0.7 \wedge 0.7, 0.6 \wedge 0.9) \end{bmatrix} \\
&\vee \begin{bmatrix} \max(0.7 \wedge 0.8, 0.8 \wedge 0.7, 0.9 \wedge 0.9) & \max(0.7 \wedge 0.7, 0.8 \wedge 0.6, 0.9 \wedge 0.8) \\ \max(0.8 \wedge 0.8, 0.7 \wedge 0.7, 0.6 \wedge 0.9) & \max(0.8 \wedge 0.7, 0.7 \wedge 0.6, 0.6 \wedge 0.8) \end{bmatrix} \\
&= \begin{bmatrix} 0.9 & 0.8 & 0.9 \\ 0.8 & 0.8 & 0.7 \end{bmatrix} \vee \begin{bmatrix} 0.9 & 0.8 \\ 0.8 & 0.7 \end{bmatrix} = \begin{bmatrix} 0.9 & 0.8 & 0.9 \\ 0.8 & 0.8 & 0.7 \end{bmatrix}
\end{aligned}$$

Hence, LHS = RHS, i.e., $P \circ (Q \cup R) = (P \circ Q) \cup (P \circ R)$.

Property 3

$$\begin{aligned}
\text{LHS} &= P \circ (Q \cap R) = \vee_y [p_{xy} \wedge (q_{yz} \wedge r_{zw})] \\
&= \vee_y \left\{ p_{xy} \wedge \begin{bmatrix} 0.8 \wedge 0.8 & 0.8 \wedge 0.7 & 0.7 \wedge 0.0 \\ 0.7 \wedge 0.7 & 0.6 \wedge 0.6 & 0.7 \wedge 0.0 \\ 0.9 \wedge 0.9 & 0.8 \wedge 0.8 & 0.9 \wedge 0.0 \end{bmatrix} \right\} \\
&= \vee_z \left\{ p_{xy} \wedge \begin{bmatrix} 0.8 & 0.7 & 0.0 \\ 0.7 & 0.6 & 0.0 \\ 0.9 & 0.8 & 0.0 \end{bmatrix} \right\} \\
&= \begin{bmatrix} \max(0.7 \wedge 0.8, 0.8 \wedge 0.7, 0.9 \wedge 0.9) & \max(0.7 \wedge 0.7, 0.8 \wedge 0.6, 0.9 \wedge 0.8) & \max(0.7 \wedge 0.0, 0.8 \wedge 0.0, 0.9 \wedge 0.0) \\ \max(0.8 \wedge 0.8, 0.7 \wedge 0.7, 0.6 \wedge 0.9) & \max(0.8 \wedge 0.7, 0.7 \wedge 0.6, 0.6 \wedge 0.8) & \max(0.8 \wedge 0.0, 0.7 \wedge 0.0, 0.6 \wedge 0.0) \end{bmatrix} \\
&= \begin{bmatrix} 0.9 & 0.8 & 0 \\ 0.8 & 0.7 & 0 \end{bmatrix} \\
\text{RHS} &= (P \circ Q) \cap (P \circ R) = \vee_y [(p_{xy} \wedge q_{yz})] \wedge [\vee_{\max(Y,Z)} (p_{xy} \wedge r_{zw})] \\
&= \left\{ \vee_z \left(\begin{bmatrix} 0.7 & 0.8 & 0.9 \\ 0.8 & 0.7 & 0.6 \end{bmatrix} \wedge \begin{bmatrix} 0.8 & 0.8 & 0.7 \\ 0.7 & 0.6 & 0.7 \\ 0.9 & 0.8 & 0.9 \end{bmatrix} \right) \right\} \\
&\quad \wedge \left\{ \vee_z \left(\begin{bmatrix} 0.7 & 0.8 & 0.9 \\ 0.8 & 0.7 & 0.6 \end{bmatrix} \wedge \begin{bmatrix} 0.8 & 0.7 \\ 0.7 & 0.6 \\ 0.9 & 0.8 \end{bmatrix} \right) \right\} \\
&= \begin{bmatrix} \max(0.7 \wedge 0.8, 0.8 \wedge 0.7, 0.9 \wedge 0.9) & \max(0.7 \wedge 0.8, 0.8 \wedge 0.6, 0.9 \wedge 0.8) & \max(0.7 \wedge 0.7, 0.8 \wedge 0.7, 0.9 \wedge 0.9) \\ \max(0.8 \wedge 0.8, 0.7 \wedge 0.7, 0.6 \wedge 0.9) & \max(0.8 \wedge 0.8, 0.7 \wedge 0.6, 0.6 \wedge 0.8) & \max(0.8 \wedge 0.7, 0.7 \wedge 0.7, 0.6 \wedge 0.9) \end{bmatrix} \\
&\quad \wedge \begin{bmatrix} \max(0.7 \wedge 0.8, 0.8 \wedge 0.7, 0.9 \wedge 0.9) & \max(0.7 \wedge 0.7, 0.8 \wedge 0.6, 0.9 \wedge 0.8) \\ \max(0.8 \wedge 0.8, 0.7 \wedge 0.7, 0.6 \wedge 0.9) & \max(0.8 \wedge 0.7, 0.7 \wedge 0.6, 0.6 \wedge 0.8) \end{bmatrix} \\
&= \begin{bmatrix} 0.9 & 0.8 & 0.9 \\ 0.8 & 0.8 & 0.7 \end{bmatrix} \wedge \begin{bmatrix} 0.9 & 0.8 \\ 0.8 & 0.7 \end{bmatrix} = \begin{bmatrix} 0.9 & 0.8 & 0 \\ 0.8 & 0.7 & 0 \end{bmatrix}
\end{aligned}$$

Hence, LHS = RHS, $[P \circ (Q \cap R)] = [(P \circ Q) \cap (P \circ R)]$.

Property 4

$$Q \supseteq R \Rightarrow (P \circ Q) \supseteq (P \circ R)$$

$$\begin{aligned} \text{LHS} &= (P \circ Q) = \vee_y (p_{xy} \wedge q_{yz}) \\ &= \begin{bmatrix} \max(0.7 \wedge 0.8, 0.8 \wedge 0.7, 0.9 \wedge 0.9) & \max(0.7 \wedge 0.8, 0.8 \wedge 0.6, 0.9 \wedge 0.8) & \max(0.7 \wedge 0.7, 0.8 \wedge 0.7, 0.9 \wedge 0.9) \\ \max(0.8 \wedge 0.8, 0.7 \wedge 0.7, 0.6 \wedge 0.9) & \max(0.8 \wedge 0.8, 0.7 \wedge 0.6, 0.6 \wedge 0.8) & \max(0.8 \wedge 0.7, 0.7 \wedge 0.7, 0.6 \wedge 0.9) \end{bmatrix} \\ &= \begin{bmatrix} 0.9 & 0.8 & 0.9 \\ 0.8 & 0.8 & 0.7 \end{bmatrix} \\ \text{RHS} &= (P \circ R) = \vee_{\max(Y,Z)} (p_{xy} \wedge r_{zw}) \\ &= \begin{bmatrix} \max(0.7 \wedge 0.8, 0.8 \wedge 0.7, 0.9 \wedge 0.9) & \max(0.7 \wedge 0.7, 0.8 \wedge 0.6, 0.9 \wedge 0.8) \\ \max(0.8 \wedge 0.8, 0.7 \wedge 0.7, 0.6 \wedge 0.9) & \max(0.8 \wedge 0.7, 0.7 \wedge 0.6, 0.6 \wedge 0.8) \end{bmatrix} \\ &= \begin{bmatrix} 0.9 & 0.8 \\ 0.8 & 0.7 \end{bmatrix} \end{aligned}$$

Hence, LHS $\geq$ RHS, i.e., $Q \supseteq R \Rightarrow (P \circ Q) \supseteq (P \circ R)$.

Problem 10.5 (Verifying properties of binary relations using Max-Product composition)

Repeat Problem 10.4 to verify the binary relation between the left and right hand sides of each of the three properties of Problem 10.3 using the above three fuzzy relation matrices for Max-Product composition.

Solution

Max-Product composition

Property 1

$$\begin{aligned} \text{LHS} &= [P \circ (Q \circ R)] = \vee_y [p_{xy} \cdot (Q \circ R)_{yw}] = \\ &= \vee_y \{ p_{xy} \cdot [\vee_z (q_{yz} \cdot r_{zw})] \} \\ &= \vee_y \left\{ p_{xy} \cdot \begin{bmatrix} \max(0.8 \times 0.8, 0.8 \times 0.7, 0.7 \times 0.9) & \max(0.8 \times 0.7, 0.8 \times 0.6, 0.7 \times 0.8) \\ \max(0.7 \times 0.8, 0.6 \times 0.7, 0.7 \times 0.9) & \max(0.7 \times 0.7, 0.6 \times 0.6, 0.7 \times 0.8) \\ \max(0.9 \times 0.8, 0.8 \times 0.7, 0.9 \times 0.9) & \max(0.9 \times 0.7, 0.8 \times 0.6, 0.9 \times 0.8) \end{bmatrix} \right\} \end{aligned}$$

$$\begin{aligned}
&= \vee_z \left\{ p_{xy} \cdot \begin{bmatrix} 0.64 & 0.56 \\ 0.63 & 0.56 \\ 0.81 & 0.72 \end{bmatrix} \right\} \\
&= \begin{bmatrix} \max(0.7 \times 0.64, 0.8 \times 0.63, 0.9 \times 0.81) & \max(0.7 \times 0.56, 0.8 \times 0.56, 0.9 \times 0.72) \\ \max(0.8 \times 0.64, 0.7 \times 0.63, 0.6 \times 0.81) & \max(0.8 \times 0.56, 0.7 \times 0.56, 0.6 \times 0.72) \end{bmatrix} \\
&= \begin{bmatrix} 0.729 & 0.648 \\ 0.512 & 0.448 \end{bmatrix} \\
\text{RHS} &= [(P \circ Q) \circ R] = \vee_z [(P \circ Q)_{xz} \cdot r_{zw}] \\
&= \vee_z \{ [\vee_y (p_{xy} \cdot q_{yz})] \cdot r_{zw} \} \\
&= \vee_z \left\{ \begin{bmatrix} \max(0.7 \times 0.8, 0.8 \times 0.7, 0.9 \times 0.9) & \max(0.7 \times 0.8, 0.8 \times 0.6, 0.9 \times 0.8) & \max(0.7 \times 0.7, 0.8 \times 0.7, 0.9 \times 0.9) \\ \max(0.8 \times 0.8, 0.7 \times 0.7, 0.6 \times 0.9) & \max(0.8 \times 0.8, 0.7 \times 0.6, 0.6 \times 0.8) & \max(0.8 \times 0.7, 0.7 \times 0.7, 0.6 \times 0.9) \end{bmatrix} \cdot r_{zw} \right\} \\
&= \vee_z \left\{ \begin{bmatrix} 0.81 & 0.72 & 0.81 \\ 0.64 & 0.64 & 0.56 \end{bmatrix} \cdot r_{zw} \right\} \\
&= \begin{bmatrix} \max(0.81 \times 0.8, 0.72 \times 0.7, 0.81 \times 0.9) & \max(0.81 \times 0.7, 0.72 \times 0.7, 0.81 \times 0.8) \\ \max(0.64 \times 0.8, 0.64 \times 0.7, 0.56 \times 0.9) & \max(0.64 \times 0.7, 0.64 \times 0.7, 0.56 \times 0.8) \end{bmatrix} \\
&= \begin{bmatrix} 0.729 & 0.648 \\ 0.512 & 0.448 \end{bmatrix}
\end{aligned}$$

Hence, LHS = RHS, i.e., $P \circ (Q \circ R) = (P \circ Q) \circ R$.

Property 2

$$\begin{aligned}
\text{LHS} &= [P \circ (Q \cup R)] = \vee_y [p_{xy} \cdot (q_{yz} \vee r_{zw})] \\
&= \vee_y \left\{ p_{xy} \cdot \begin{bmatrix} 0.8 \vee 0.8 & 0.8 \vee 0.7 & 0.7 \vee 0.0 \\ 0.7 \vee 0.7 & 0.6 \vee 0.6 & 0.7 \vee 0.0 \\ 0.9 \vee 0.9 & 0.8 \vee 0.8 & 0.9 \vee 0.0 \end{bmatrix} \right\} \\
&= \vee_z \left\{ p_{xy} \cdot \begin{bmatrix} 0.8 & 0.8 & 0.7 \\ 0.7 & 0.6 & 0.7 \\ 0.9 & 0.8 & 0.9 \end{bmatrix} \right\} \\
&= \begin{bmatrix} \max(0.7 \times 0.8, 0.8 \times 0.7, 0.9 \times 0.9) & \max(0.7 \times 0.8, 0.8 \times 0.6, 0.9 \times 0.8) & \max(0.7 \times 0.7, 0.8 \times 0.7, 0.9 \times 0.9) \\ \max(0.8 \times 0.8, 0.7 \times 0.7, 0.6 \times 0.9) & \max(0.8 \times 0.8, 0.7 \times 0.6, 0.6 \times 0.8) & \max(0.8 \times 0.7, 0.7 \times 0.7, 0.6 \times 0.9) \end{bmatrix} \\
&= \begin{bmatrix} 0.81 & 0.72 & 0.81 \\ 0.64 & 0.64 & 0.56 \end{bmatrix}
\end{aligned}$$

$$\text{RHS} = (P \circ Q) \cup (P \circ R) = \vee_y [(p_{xy} \cdot q_{yz})] \vee [\vee_{\min(Y,Z)} (p_{xy} \cdot r_{zw})]$$

$$(P \circ Q) = \vee_y [(p_{xy} \cdot q_{yz})]$$

$$= \begin{bmatrix} \max(0.7 \times 0.8, 0.8 \times 0.7, 0.9 \times 0.9) & \max(0.7 \times 0.8, 0.8 \times 0.6, 0.9 \times 0.8) & \max(0.7 \times 0.7, 0.8 \times 0.7, 0.9 \times 0.9) \\ \max(0.8 \times 0.8, 0.7 \times 0.7, 0.6 \times 0.9) & \max(0.8 \times 0.8, 0.7 \times 0.6, 0.6 \times 0.8) & \max(0.8 \times 0.7, 0.7 \times 0.7, 0.6 \times 0.9) \end{bmatrix}$$

$$= \begin{bmatrix} 0.81 & 0.72 & 0.81 \\ 0.64 & 0.64 & 0.56 \end{bmatrix}$$

$$(P \circ R) = \vee_{\min(Y,Z)} (p_{xy} \cdot r_{zw})$$

$$= \begin{bmatrix} \max(0.7 \times 0.8, 0.8 \times 0.7, 0.9 \times 0.9) & \max(0.7 \times 0.7, 0.8 \times 0.6, 0.9 \times 0.8) \\ \max(0.8 \times 0.8, 0.7 \times 0.7, 0.6 \times 0.9) & \max(0.8 \times 0.7, 0.7 \times 0.6, 0.6 \times 0.8) \end{bmatrix}$$

$$= \begin{bmatrix} 0.81 & 0.72 \\ 0.64 & 0.56 \end{bmatrix}$$

$$(P \circ Q) \cup (P \circ R)$$

$$= \begin{bmatrix} 0.81 & 0.72 & 0.81 \\ 0.64 & 0.64 & 0.56 \end{bmatrix} \wedge \begin{bmatrix} 0.81 & 0.72 \\ 0.64 & 0.56 \end{bmatrix} = \begin{bmatrix} 0.81 & 0.72 & 0.81 \\ 0.64 & 0.64 & 0.56 \end{bmatrix}$$

Hence, LHS = RHS, i.e., $[P \circ (Q \cup R)] = [(P \circ Q) \cup (P \circ R)]$.

Property 3

$$\text{LHS} = (P \circ Q) = \vee_y (p_{xy} \cdot q_{yz})$$

$$= \begin{bmatrix} \max(0.7 \times 0.8, 0.8 \times 0.7, 0.9 \times 0.9) & \max(0.7 \times 0.8, 0.8 \times 0.6, 0.9 \times 0.8) & \max(0.7 \times 0.7, 0.8 \times 0.7, 0.9 \times 0.9) \\ \max(0.8 \times 0.8, 0.7 \times 0.7, 0.6 \times 0.9) & \max(0.8 \times 0.8, 0.7 \times 0.6, 0.6 \times 0.8) & \max(0.8 \times 0.7, 0.7 \times 0.7, 0.6 \times 0.9) \end{bmatrix}$$

$$= \begin{bmatrix} 0.81 & 0.72 & 0.81 \\ 0.64 & 0.64 & 0.56 \end{bmatrix}$$

$$\text{RHS} = (P \circ R) = \vee_{\min(Y,Z)} (p_{xy} \cdot r_{zw})$$

$$= \begin{bmatrix} \max(0.7 \times 0.8, 0.8 \times 0.7, 0.9 \times 0.9) & \max(0.7 \times 0.7, 0.8 \times 0.6, 0.9 \times 0.8) \\ \max(0.8 \times 0.8, 0.7 \times 0.7, 0.6 \times 0.9) & \max(0.8 \times 0.7, 0.7 \times 0.6, 0.6 \times 0.8) \end{bmatrix}$$

$$= \begin{bmatrix} 0.81 & 0.72 \\ 0.64 & 0.56 \end{bmatrix}$$

LHS $\geq$ RHS, i.e., $Q \supseteq R \Rightarrow (P \circ Q) \supseteq (P \circ R)$.

Problem 10.6 (Computation and interpretation of fuzzy relation matrices)

Two high-school students, Student 1 and Student 2, with marks (ranging within a scale of 0 to 1) for Languages, Sciences, and Mathematics, are applying for admissions to Humanities, Business, and Engineering, in University 1 and University 2.

Let $X = \{x_1, x_2\} = \{\text{Student 1, Student 2}\}$, $Y = \{y_1, y_2, y_3\} = \{\text{Languages, Sciences, Mathematics}\}$, $Z = \{z_1, z_2, z_3\} = \{\text{Humanities, Business, Engineering}\}$, $W = \{w_1, w_2\} = \{\text{University 1, University 2}\}$, the “students to marks” are represented by the fuzzy relation $P(X, Y)$, the “marks to programs” are represented by the fuzzy relation $Q(Y, Z)$, and the “programs to universities” are represented by the fuzzy relation $R(Z, W)$.

Compute and interpret the fuzzy relation matrices $S(X, W)$ between “students to universities” using:

- (a) Max-Min composition; and
- (b) Max-Product composition.

Solution

- (a) Max-Min composition

$$\begin{aligned}
 S &= [(P \circ Q) \circ R] = \vee_z [(P \circ Q)_{xz} \wedge r_{zw}] \\
 &= \vee_z \{ [\vee_y (p_{xy} \wedge q_{yz})] \wedge r_{zw} \} \\
 &= \vee_z \left\{ \begin{bmatrix} \max(0.7 \wedge 0.8, 0.8 \wedge 0.7, 0.9 \wedge 0.9) & \max(0.7 \wedge 0.8, 0.8 \wedge 0.6, 0.9 \wedge 0.8) & \max(0.7 \wedge 0.7, 0.8 \wedge 0.7, 0.9 \wedge 0.9) \\ \max(0.8 \wedge 0.8, 0.7 \wedge 0.7, 0.6 \wedge 0.9) & \max(0.8 \wedge 0.8, 0.7 \wedge 0.6, 0.6 \wedge 0.8) & \max(0.8 \wedge 0.7, 0.7 \wedge 0.7, 0.6 \wedge 0.9) \end{bmatrix} \wedge r_{zw} \right\} \\
 &= \vee_z \left\{ \begin{bmatrix} 0.9 & 0.8 & 0.9 \\ 0.8 & 0.8 & 0.7 \end{bmatrix} \wedge r_{zw} \right\} \\
 &= \begin{bmatrix} \max(0.9 \wedge 0.8, 0.8 \wedge 0.7, 0.9 \wedge 0.9) & \max(0.9 \wedge 0.7, 0.8 \wedge 0.6, 0.9 \wedge 0.8) \\ \max(0.8 \wedge 0.8, 0.8 \wedge 0.7, 0.7 \wedge 0.9) & \max(0.8 \wedge 0.7, 0.8 \wedge 0.6, 0.7 \wedge 0.8) \end{bmatrix} \\
 &= \begin{bmatrix} 0.9 & 0.8 \\ 0.8 & 0.7 \end{bmatrix}
 \end{aligned}$$

Interpretations:

Student 1 has a fuzzy relation of 0.9 with University 1 and 0.8 with University 2.

Student 2 has a fuzzy relation of 0.8 with University 1 and 0.7 with University 2.

(b) Max-Product composition

$$\begin{aligned}
 S &= [P \circ (Q \circ R)] = \vee_y [p_{xy} \cdot (Q \circ R)_{yw}] = \vee_y \{p_{xy} \cdot [\vee_z (q_{yz} \cdot r_{zw})]\} \\
 &= \vee_y \left\{ p_{xy} \cdot \begin{bmatrix} \max(0.8 \times 0.8, 0.8 \times 0.7, 0.7 \times 0.9) & \max(0.8 \times 0.7, 0.8 \times 0.6, 0.7 \times 0.8) \\ \max(0.7 \times 0.8, 0.6 \times 0.7, 0.7 \times 0.9) & \max(0.7 \times 0.7, 0.6 \times 0.6, 0.7 \times 0.8) \\ \max(0.9 \times 0.8, 0.8 \times 0.7, 0.9 \times 0.9) & \max(0.9 \times 0.7, 0.8 \times 0.6, 0.9 \times 0.8) \end{bmatrix} \right\} \\
 &= \vee_z \left\{ p_{xy} \cdot \begin{bmatrix} 0.64 & 0.56 \\ 0.63 & 0.56 \\ 0.81 & 0.72 \end{bmatrix} \right\} \\
 &= \begin{bmatrix} \max(0.7 \times 0.64, 0.8 \times 0.63, 0.9 \times 0.81) & \max(0.7 \times 0.56, 0.8 \times 0.56, 0.9 \times 0.72) \\ \max(0.8 \times 0.64, 0.7 \times 0.63, 0.6 \times 0.81) & \max(0.8 \times 0.56, 0.7 \times 0.56, 0.6 \times 0.72) \end{bmatrix} \\
 &= \begin{bmatrix} 0.729 & 0.648 \\ 0.512 & 0.448 \end{bmatrix}
 \end{aligned}$$

Interpretations:

Student 1 has a fuzzy relation of 0.729 with University 1 and 0.648 with University 2.

Student 2 has a fuzzy relation of 0.512 with University 1 and 0.448 with University 2.

Problem 10.7 (Fuzzy relation matrix computation using different compositions)

Assume that $R_1(X, Y)$ and $R_2(Y, Z)$ can be expressed as the following fuzzy relation matrices

$$R_1 = \begin{bmatrix} 0.1 & 0.7 \\ 0.6 & 0.1 \end{bmatrix} \quad R_2 = \begin{bmatrix} 0.5 & 0.9 & 0.4 \\ 0.1 & 0.2 & 0.8 \end{bmatrix}$$

Compute $R_1 \circ R_2$ using each of the following three compositions:

- (a) Max-Min composition;
- (b) Max-Product composition;
- (c) Max-Average composition.
- (d) Compare the results of the above three compositions.
- (e) Write a Matlab program to implement (a), (b), and (c).

Solution

(a) Max-Min composition

$$\begin{aligned} \mu_{R_1 \circ R_2}(x, z) &= \max_y [\mu_{R_1}(x, y), \mu_{R_2}(y, z)] \\ &= \begin{bmatrix} \max(0.1 \wedge 0.5, 0.7 \wedge 0.1) & \max(0.1 \wedge 0.9, 0.7 \wedge 0.2) & \max(0.1 \wedge 0.4, 0.7 \wedge 0.8) \\ \max(0.6 \wedge 0.5, 0.1 \wedge 0.1) & \max(0.6 \wedge 0.9, 0.1 \wedge 0.2) & \max(0.6 \wedge 0.4, 0.1 \wedge 0.8) \end{bmatrix} \\ &= \begin{bmatrix} \max(0.1, 0.1) & \max(0.1, 0.2) & \max(0.1, 0.7) \\ \max(0.5, 0.1) & \max(0.6, 0.1) & \max(0.4, 0.1) \end{bmatrix} \\ &= \begin{bmatrix} 0.1 & 0.2 & 0.7 \\ 0.5 & 0.6 & 0.4 \end{bmatrix} \end{aligned}$$

(b) Max-Product composition

$$\begin{aligned} \mu_{R_1 \circ R_2}(x, z) &= \max_y [\mu_{R_1}(x, y) \times \mu_{R_2}(y, z)] \\ &= \begin{bmatrix} \max(0.1 * 0.5, 0.7 * 0.1) & \max(0.1 * 0.9, 0.7 * 0.2) & \max(0.1 * 0.4, 0.7 * 0.8) \\ \max(0.6 * 0.5, 0.1 * 0.1) & \max(0.6 * 0.9, 0.1 * 0.2) & \max(0.6 * 0.4, 0.1 * 0.8) \end{bmatrix} \\ &= \begin{bmatrix} \max(0.05, 0.07) & \max(0.09, 0.14) & \max(0.04, 0.56) \\ \max(0.3, 0.01) & \max(0.54, 0.02) & \max(0.24, 0.08) \end{bmatrix} \end{aligned}$$

$$= \begin{bmatrix} 0.07 & 0.14 & 0.56 \\ 0.30 & 0.54 & 0.24 \end{bmatrix}$$

(c) Max-Average composition

$$\begin{aligned} \mu_{R_1 \circ R_2}(x, z) &= 0.5 \times \max_y [\mu_{R_1}(x, y) + \mu_{R_2}(y, z)] \\ &= 0.5 \times \\ &\quad \begin{bmatrix} \max(0.1 + 0.5, 0.7 + 0.1) & \max(0.1 + 0.9, 0.7 + 0.2) & \max(0.1 + 0.4, 0.7 + 0.8) \\ \max(0.6 + 0.5, 0.1 + 0.1) & \max(0.6 + 0.9, 0.1 + 0.2) & \max(0.6 + 0.4, 0.1 + 0.8) \end{bmatrix} \\ &= 0.5 \times \begin{bmatrix} \max(0.6, 0.8) & \max(1, 0.9) & \max(0.5, 1.5) \\ \max(1.1, 0.2) & \max(1.5, 0.3) & \max(1.0, 0.9) \end{bmatrix} \\ &= \begin{bmatrix} 0.40 & 0.50 & 0.75 \\ 0.55 & 0.75 & 0.50 \end{bmatrix} \end{aligned}$$

(d) In comparison, the result of the Max-Average composition is the most fuzzy (with most similar output values), followed by that of the Max-Product composition, and the Max-Min composition is the least fuzzy (with least similar output values).

(e)

Matlab program

```
% Problem 10.7
clear all; clc;
nproblem=input('Enter 1: P10.10; 2: E10.2');
if nproblem==1
    %Problem 10.10
    R1 = [0.1,0.7;0.6,0.1];
    R2 = [0.5,0.9,0.4;0.1,0.2,0.8];
elseif nproblem==2
    %Example 10.2
    R1 = [0.9,0.7,0.5;0.6,0.8,0.9];
    R2 = [1.0,0.6;0.6,0.8;0.3,1.0];
elseif nproblem==3
    R1 = input('Enter 1st fuzzy relation matrix');
    R2 = input('Enter 2nd fuzzy relation matrix');
```

```

    %R1 = [0.1,0.3;0.7,0.2;0.5,0.4];
    %R2 = [0.9,0.1,0.3;0.4,0.2,0.9];
end
%Enter 1 for Max-Min; 2 for Max-Product;
%3 for Max-Average:
option=input('Enter 1 or 2 or 3');
[x,y1]=size(R1);
[y2,z]=size(R2);
if (y1==y2) %Check R1 & R2 cmpatibility
    for i=1:x
        for j=1:z
            A=R1(i,:);
            B=R2(:,j);
            [a,b]=size(A);
            [c,d]=size(B);
            for k=1:b
                if option==1
                    C(1,k)=min(A(1,k),B(k,1));
                elseif option==2
                    C(1,k)=A(1,k)*B(k,1);
                elseif option==3
                    C(1,k)=(A(1,k)+B(k,1))/2;
                end
            end
            R(i,j)=max(C);
        end
    end
    disp('Composition(R1,R2)=' );
    disp(R)
else
    disp('Incompatible (R1,R2)');
end

```

Problem 10.8 (Output MF derivation of 1-rule 1-input 1-output fuzzy reasoning system)

Based on the Mamdani's fuzzy implication functions and the Max-Min composition rule of inference, derive the membership function at the output of a one-rule one-input one-output fuzzy reasoning system as shown in Figure P10.8.1.

Given

$$A = \text{TriMF}(x, [10, 40, 70])$$

$$A' = \text{TriMF}(x, [20, 30, 40])$$

$$B = \text{TrapMF}(y, [-6, -2, 2])$$

The output fuzzy set B' can be determined graphically as follows:

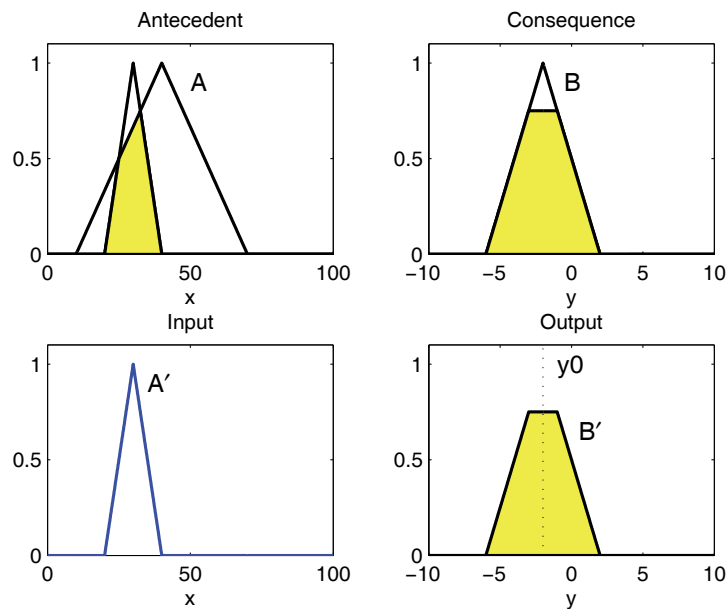


Figure P10.8.1 1-rule 1-input 1-output fuzzy reasoning ($w = 0.75$; centroid defuzzified output, $y_0 = -2.0$).

Solution

For one-rule one-input one-output fuzzy reasoning as shown in Figure P10.8.1.

Fact: x is A'

Rule: If x is A then y is B

Conclusion: y is B'

In the operations shown in Figure P10.7.1, the degree of belief w of the antecedent part of a rule is first determined as the peak of $\mu_{A'}(x) \wedge \mu_A(x)$; then the membership function of the resulting B' is equal to the membership function of B clipped by w .

According to the fuzzy implication of (10.7.1) for the two-input case, the compositional rule of inference for the one-input case can be expressed as

$$B' = A' \circ R = A' \circ (A \rightarrow B)$$

In a similar manner to the Max-Product composition as specified in (10.7.2), for the Max-Min composition, we have

$$\begin{aligned}\mu_{B'}(y) &= \vee_x \{\mu_{A'}(x) \wedge \mu_R(x, y)\} = \vee_x \{\mu_{A'}(x) \wedge [\mu_A(x) \wedge \mu_B(y)]\} \\ &= \{\vee_x [\mu_{A'}(x) \wedge \mu_A(x)]\} \wedge \mu_B(y) = w \wedge \mu_B(y)\end{aligned}$$

In so doing, $w = 0.75$. Using this weight $w = 0.75$, B' can be derived from taking $\min [0.75, \mu_B(y)]$.

The resultant trapezoidal area encompassing B' gives the fuzzy output and the centroid defuzzified output y_0 is equal to -2.0 .

Problem 10.9 (Output MF derivation of 1-rule 2-input 1-output fuzzy reasoning system)

Based on the Mamdani's fuzzy implication functions and the Max-Min composition rule of inference, derive the membership function at the output of a one-rule two-input one-output fuzzy reasoning system as shown in Figure P10.9.1.

Given

$$A_1 = \text{TriMF}(x_1, [10, 40, 70])$$

$$A'_1 = \text{TriMF}(x_1, [20, 30, 40])$$

$$A_2 = \text{TriMF}(x_2, [-20, 10, 40])$$

$$A'_2 = \text{TriMF}(x_2, [-25, -15, -5])$$

$$B = \text{TrapMF}(y, [-6, -2, 2])$$

Using the minimum of the two firing strengths, the output fuzzy set B'_1 can be determined graphically as shown in Figure P10.9.1.

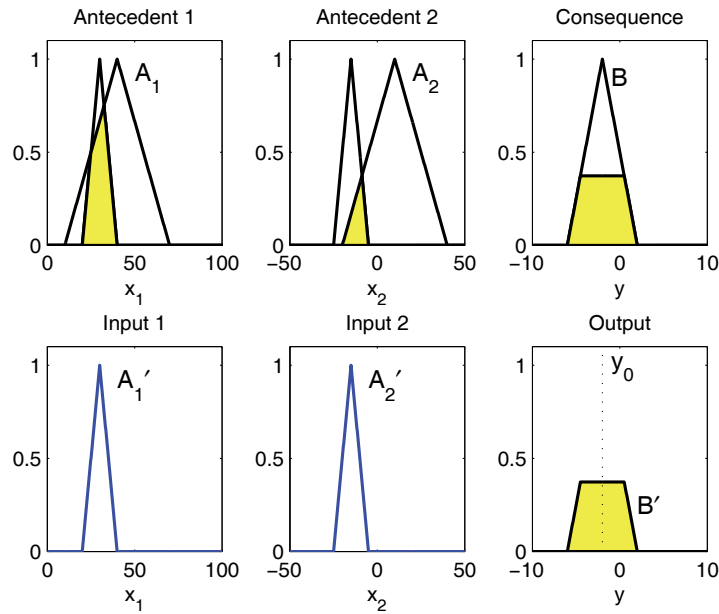


Figure P10.9.1 1-rule 2-input 1-output fuzzy reasoning (minimum operator for weights, $w_1 = 0.7500$, $w_2 = 0.3733$, $w = \min(w_1, w_2) = 0.3733$; centroid defuzzified output, $y_0 = -2.0000$).

Solution

For one-rule two-input one-output fuzzy reasoning as shown in Figure P10.9.1.

Fact: x_1 is A'_1 and x_2 is A'_2

Rule: If x_1 is A_1 and x_2 is A_2 then y is B

Conclusion: y is B'

T-norm operators include Minimum and Algebraic Product. In Figure P10.9.1, Minimum is used as the T-norm operator.

Based on the Mamdani's fuzzy implication function,

$$\begin{aligned} R_m(A_1, A_2, B) &= (A_1 \times A_2) \times B \\ &= \int_{X_1 \times X_2 \times Y} \mu_{A_1}(x_1) \wedge \mu_{A_2}(x_2) \wedge \mu_B(y) / (x_1, x_2, y) \end{aligned}$$

According to the fuzzy implication of (10.7.1) for the two-input case, the resulting B' can be obtained as

$$B' = (A'_1 \times A'_2) \circ (A_1 \times A_2 \rightarrow B)$$

In a similar manner to the Max-Product composition in (10.7.2), for Max-Min composition, we have

$$\begin{aligned} \mu_{B'}(y) &= \vee_{x_1, x_2} [\mu_{A'_1}(x_1) \wedge \mu_{A'_2}(x_2)] \wedge [\mu_{A_1}(x_1) \wedge \mu_{A_2}(x_2) \wedge \mu_B(y)] \\ &= \vee_{x_1, x_2} [\mu_{A'_1}(x_1) \wedge \mu_{A'_2}(x_2) \wedge \mu_{A_1}(x_1) \wedge \mu_{A_2}(x_2)] \wedge \mu_B(y) \\ &= \{ \vee_{x_1} [\mu_{A'_1}(x_1) \wedge \mu_{A_1}(x_1)] \wedge \vee_{x_2} [\mu_{A'_2}(x_2) \wedge \mu_{A_2}(x_2)] \} \wedge \mu_B(y) \\ &= (w_1 \wedge w_2) \wedge \mu_B(y) \end{aligned}$$

w_1 denotes the degree of compatibility between A_1 and A'_1 ; w_2 denotes the degree of compatibility between A_2 and A'_2 ; and $w = w_1 \wedge w_2$ is called the firing strength of the fuzzy rule.

The weight w_1 is determined from $\vee_{x_1} (\mu_{A'_1}(x_1) \wedge \mu_{A_1}(x_1))$ as 0.75 and w_2 is determined from $\vee_{x_2} (\mu_{A'_2}(x_2) \wedge \mu_{A_2}(x_2))$ as 0.3733, therefore, the

firing strength is determined as 0.3733, which is the minimum of [0.75, 0.3733].

After the firing strength is determined, the fuzzy output is obtained as the shaped trapezoidal area shown in Figure P10.9.1 and the centroid defuzzified output y_0 is equal to -2.0 .

Problem 10.10 (Output MF derivation of 3-rule 3-input 3-output fuzzy reasoning system)

Given a three-rule three-input one-output fuzzy reasoning system described by

Fact: x_1 is A'_1 , x_2 is A'_2 , and x_3 is A'_3

Rule 1: If x_1 is A_{11} , x_2 is A_{12} , and x_3 is A_{13} , then $y = B_1$

Rule 2: If x_1 is A_{21} , x_2 is A_{22} , and x_3 is A_{23} , then $y = B_2$

Rule 3: If x_1 is A_{31} , x_2 is A_{32} , and x_3 is A_{33} , then $y = B_3$

Conclusion: y is B'

(a) Using the Max-Min compositional rule of inference, derive the general output membership function $\mu_{B'_r}(y)$ ($r = 1, 2, 3$) for each of the three rules.

(b) Using the Max-Product compositional rule of inference, derive the general output membership function $\mu_{B'_r}(y)$ ($r = 1, 2, 3$) for each of the three rules.

(c) Derive the membership function $\mu_{B'}(y)$ at the overall output of the fuzzy reasoning system of both (a) and (b).

Solution

For each of the r th rule ($r = 1, 2, 3$), according to fuzzy implication of the two-input case as specified in (10.7.1), the compositional rule of inference for the three-input case can be expressed as

$$B'_r = (A'_1 \times A'_2 \times A'_3) \circ (A_{r1} \times A_{r2} \times A_{r3} \rightarrow B_r)$$

(a) According to the Max-Min compositional rule of inference, similar to the Max-Product case as specified in (10.7.2), the general output membership function $\mu_{B'_r}(y)$ ($r = 1, 2, 3$) for each of the three rules is given by

$$\begin{aligned} \mu_{B'_r}(y) &= \vee_{x_1, x_2, x_3} [\mu_{A'_1}(x_1) \wedge \mu_{A'_2}(x_2) \wedge \mu_{A'_3}(x_3)] \\ &\quad \wedge [\mu_{A_{r1}}(x_1) \wedge \mu_{A_{r2}}(x_2) \wedge \mu_{A_{r3}}(x_3) \wedge \mu_{B_r}(y)] \\ &= \vee_{x_1, x_2, x_3} [\mu_{A'_1}(x_1) \wedge \mu_{A'_2}(x_2) \wedge \mu_{A'_3}(x_3) \wedge \mu_{A_{r1}}(x_1) \wedge \mu_{A_{r2}}(x_2) \\ &\quad \wedge \mu_{A_{r3}}(x_3)] \wedge \mu_{B_r}(y) \\ &= \{\vee_{x_1} [\mu_{A'_1}(x_1) \wedge \mu_{A_{r1}}(x_1)] \wedge \vee_{x_2} [\mu_{A'_2}(x_2) \wedge \mu_{A_{r2}}(x_2)] \\ &\quad \wedge \vee_{x_3} [\mu_{A'_3}(x_3) \wedge \mu_{A_{r3}}(x_3)]\} \wedge \mu_{B_r}(y) \\ &= (w_{r1} \wedge w_{r2} \wedge w_{r3}) \wedge \mu_{B_r}(y) \end{aligned}$$

(b) According to the Max-Product compositional rule of inference of (10.7.2) for the two-input case, the general output membership function $\mu_{B'_r}(y)$ ($r = 1, 2, 3$) for each of the three rules is given by

$$\begin{aligned} \mu_{B'_r}(y) &= \vee_{x_1, x_2, x_3} [\mu_{A'_1}(x_1) \wedge \mu_{A'_2}(x_2) \wedge \mu_{A'_3}(x_3)] \\ &\quad \times [\mu_{A_{r1}}(x_1) \wedge \mu_{A_{r2}}(x_2) \wedge \mu_{A_{r3}}(x_3) \wedge \mu_{B_r}(y)] \\ &= \vee_{x_1, x_2, x_3} \{[\mu_{A'_1}(x_1) \wedge \mu_{A'_2}(x_2) \wedge \mu_{A'_3}(x_3)] \\ &\quad \times [\mu_{A_{r1}}(x_1) \wedge \mu_{A_{r2}}(x_2) \wedge \mu_{A_{r3}}(x_3)]\} \wedge \mu_{B_r}(y) \\ &= \{\vee_{x_1} [\mu_{A'_1}(x_1) \wedge \mu_{A_{r1}}(x_1)] \times \vee_{x_2} [\mu_{A'_2}(x_2) \wedge \mu_{A_{r2}}(x_2)] \\ &\quad \times \vee_{x_3} [\mu_{A'_3}(x_3) \wedge \mu_{A_{r3}}(x_3)]\} \wedge \mu_{B_r}(y) \\ &= (w_{r1} \times w_{r2} \times w_{r3}) \wedge \mu_{B_r}(y) \end{aligned}$$

(c) For the overall output, we have

$$B' = (A'_1 \times A'_2 \times A'_3) \circ (R_1 \cup R_2 \cup R_3)$$

Hence,

$$B' = [(A'_1 \times A'_2 \times A'_3) \circ R_1] \cup [(A'_1 \times A'_2 \times A'_3) \circ R_2] \\ \cup [(A'_1 \times A'_2 \times A'_3) \circ R_3] = B'_1 \cup B'_2 \cup B'_3$$

In other words,

$$\mu_{B'}(y) = \mu_{B'_1}(y) \cup \mu_{B'_2}(y) \cup \mu_{B'_3}(y)$$

Problem 10.11 (Centroid computation of aggregated output MF of 2-rule 2-input 1-output Mamdani fuzzy System)

Consider a two-rule two-input single-output Mamdani fuzzy system, given $w_{11} = 0.75$; $w_{12} = 0.3733$; $w_{21} = 0.25$; and $w_{22} = 0.8733$; where the weight w_{ij} denotes the weight of rule i of input j .

The consequent membership functions of rules 1 and 2 are, respectively, $B_1 = \text{TriMF}(y, [-6, -2, 2])$; and $B_2 = \text{TriMF}(y, [0, 4, 8])$ with an output universe of -10 to 10 ; and the minimum operator is used to determine the firing strength of each of the two rules.

(a) Write a Matlab program to compute the centroid of the aggregated output membership function using the weights w_{ij} for $1 \leq i, j \leq 2$; the complete areas of B_1 and B_2 ; and the centroids of B_1 and B_2 .

(a) Given the weight-clipped output membership functions of rules 1 and 2 as $B_{1c} = \text{TrapMF}(y, [-6, -4.88, 0.88, 2])$ and $B_{2c} = \text{TrapMF}(y, [0, 0.8732, 7.1268, 8])$, write a Matlab program to compute the centroid of the aggregated output membership function using the areas and centroids of the of B_{1c} and B_{2c} .

(b) Repeat (a) using algebraic product to determine the firing strength of each of the two rules.

(c) Repeat (b) using algebraic product to determine the firing strength of each of the two rules.

Solution

(a) Enter weights of 2-rule 2-input Mamdani fuzzy system

$[w_{11}, w_{12}, w_{21}, w_{22}]$: [0.75, 0.3733, 0.25, 0.8733]

Enter weight operator, 1 for min; 2 for algebraic product: 1

Enter 1 for weight-area average; 2 for clip-area average: 1

$y_c = 0.406545804588481$

(b) Enter weights of 2-rule 2-input Mamdani fuzzy system

$[w_{11}, w_{12}, w_{21}, w_{22}]$: [0.75, 0.3733, 0.25, 0.8733]

Enter weight operator, 1 for min; 2 for algebraic product: 1

Enter 1 for weight-area average; 2 for clip-area average: 2

$y_c = 0.457514981595419$

(c) Enter weights of 2-rule 2-input Mamdani fuzzy system

$[w_{11}, w_{12}, w_{21}, w_{22}]$: [0.75, 0.3733, 0.25, 0.8733]

Enter weight operator, 1 for min; 2 for algebraic product: 2

Enter 1 for weight-area average; 2 for clip-area average: 1

$y_c = 0.628838049367850$

(d) Enter weights of 2-rule 2-input Mamdani fuzzy system

$[w_{11}, w_{12}, w_{21}, w_{22}]$: [0.75, 0.3733, 0.25, 0.8733]

Enter weight operator, 1 for min; 2 for algebraic product: 2

Enter 1 for weight-area average; 2 for clip-area average: 2

$y_c = 0.681002687561308$

Matlab program

% Problem 10.11

clear all; clc;

%%%%%%%%%

%Option to enter input MF to compute weights from antecedent MFs

%A1p=input('Enter [a1,b1,c1] of trimf input1: ');

```

%A2p=input('Enter [a2,b2,c2] of trimf input2: ');
%%Example
%%A1p = [20,30,40]
%%A2p = [-25,-15,-5]
%A1= trimf(x1,A1p);
%A2= trimf(x2,A2p);
%x1 = 0:100; %Input 1 universe
%x2 = -50:50; %Input 2 universe
%A11= trimf(x1,[10,40,70]);
%Rule 1 antecedent MF for input 1
%A21= trimf(x1,[30,60,90]);
%Rule 1 antecedent MF for input 2
%A12= trimf(x2,[-20,10,40]);
%Rule 2 antecedent MF for input 1
%A22= trimf(x2,[-40,-10,20]);
%Rule 2 antecedent MF for input 2
%%%%%%%%%%
%Option to enter weights (directly)
%Enter weights of 2-rule 2-input 1-output
%Mamdani fuzzy system [w11,w12,w21,w22]:
%Example
%[w11,w12,w21,w22]=[0.75, 0.3733, 0.25, 0.8733];
W=input('Enter weights [w11,w12,w21,w22]: ');
w11=W(1,1);
w12=W(1,2);
w21=W(1,3);
w22=W(1,4);
%Enter weight operator, 1 for min;
%2 for algebraic product:
wt_operator=input('Enter wt operator, 1 or 2');
if wt_operator==1
    w1 = min(w11,w12);
    w2 = min(w21,w22);
elseif wt_operator==2
    w1 = w11*w12;
    w2 = w21*w22;

```

```

end
y = -10:0.01:10; %Output universe
B1 = trimf (y,[-6,-2,2]); %Rule 1 consequent MF
B2 = trimf (y,[0,4,8]); %Rule 2 consequent MF
y1=-6+(2-(-6))/2;
%Centroid of complete & clipped consequent MF
%of Rule 1
y2=0+(8-(0))/2;
%Centroid of complete & clipped consequent MF
%of Rule 2
noption=input('Enter 1 for weight-area average;...
2 for clip-area average: ');
if noption==1
    areaB1=(2-(-6))*1/2;
    %Area of complete consequent MF of Rule 1
    areaB2=(8-0)*1/2;
    %Area of complete consequent MF of Rule 2
    yc=(w1*areaB1*y1+w2*areaB2*y2)/(w1*areaB1+w2*areaB2);
elseif noption==2
    output_trape_R1=[-6 -4.88 0.88 2];
    areaB1c=((2-(-6))+(0.88-(-4.88)))*w1/2;
    %Area of clipped consequent MF of Rule 1
    output_trape_R2=[0 0.8732 7.1268 8];
    areaB2c=((8-(0))+(7.1268-0.8732))*w2/2;
    %Area of clipped consequent MF of Rule 2
    yc=(areaB1c*y1+areaB2c*y2)/(areaB1c+areaB2c);
end

```


Chapter 11

Fuzzy Systems

Problem 11.1 (Derivation of inverse Gaussian membership function)

The definition of the Gaussian membership function is defined as

$$\text{GaussMF}(x, [c, \sigma]) = e^{\frac{-1}{2} \left(\frac{x-c}{\sigma} \right)^2}$$

Define a discrete universe of discourse X of a set of positive integers from 1 to 100.

W is the corresponding set of membership values, and $c = 50$, $\sigma = 10$. Derive the inverse GaussMF membership function for X as a function of W , c , and σ .

Write a Matlab program to plot and verify the correctness of forward and backward (inverse) transforms.

Solution

$$W = e^{\frac{-1}{2} \left(\frac{x-c}{\sigma} \right)^2}$$

$$-(1/2) * [(X - c)/\sigma]^2 = \ln(W)$$

$$(X - c)^2 / \sigma^2 = -2 \ln(W)$$

$$(X - c)^2 = -2 \sigma^2 \ln(W)$$

$$(X - c) = \pm \sqrt{-2 \sigma^2 \ln(W)}$$

$$X = c \pm \sqrt{-2 \sigma^2 \ln(W)}$$

$$X = c - \sqrt{2 \sigma^2 \ln(1/W)} \quad \text{for } X < c$$

$$X = c + \sqrt{2 \sigma^2 \ln(1/W)} \quad \text{for } X \geq c$$

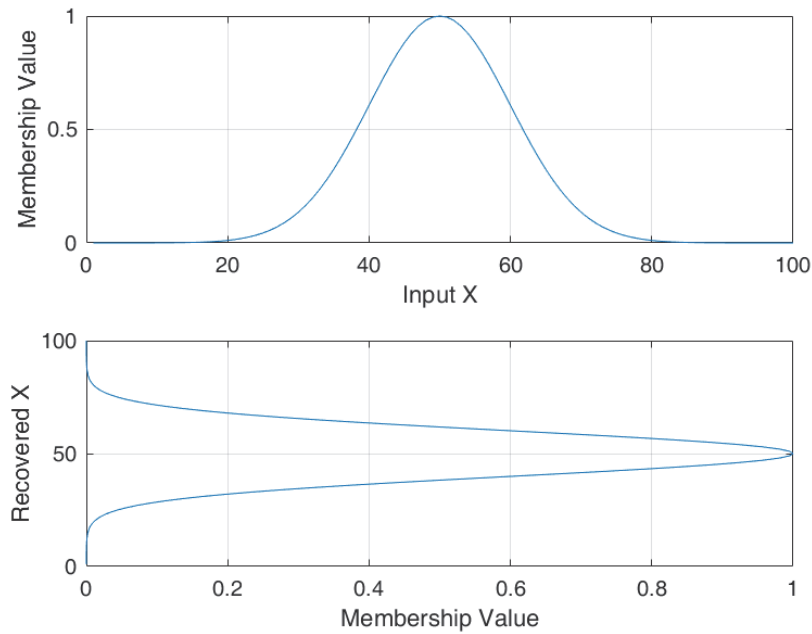


Figure P11.1.1

Matlab program

% Problem 11.1

close all; clear all;

NPt = 100;

x = linspace(1, 100, NPt);

x_inverse = zeros(1, NPt);

w = zeros(1, NPt);

c = 50;

sigma = 10;

for k=1:NPt

 %w(1,k) = gauss_mf(x(1,k), [sigma, c]);

 w(1,k) = exp(-((x(1,k)-c)/sigma)^2/2);

 if x(1,k) < c

 x_inverse(1,k) = c - sqrt(2*(sigma^2)*log(1/w(1,k)));

 elseif x(1,k) >= c

 x_inverse(1,k) = c + sqrt(2*(sigma^2)*log(1/w(1,k)));

```

    end
end
figure
subplot(2,1,1); plot(x,w); grid; ylabel('Membership Value');
xlabel('Input X');
subplot(2,1,2); plot(w,x_inverse); grid; ylabel('Recovered X');
xlabel('Membership Value');

```

Problem 11.2 (Derivation of inverse generalized bell membership function)

The generalized bell membership function is defined as

$$\text{GBellMF}(X, [a, b, c]) = \frac{1}{1 + \left[\left(\frac{X - c}{a} \right)^2 \right]^b}$$

Define a discrete universe of discourse X of a set of positive integers from 1 to 100. W is the corresponding set of membership values, and $a = 20, b = 5, c = 50$.

Derive the inverse GBellMF membership function for X as a function of W, a, b , and c .

Write a Matlab program to plot and verify the correctness of forward and backward (inverse) transforms.

Solution

$$W = 1 / (1 + [(X - c)/a]^2)^b$$

$$[(X - c)/a]^2 = (1/W) - 1$$

$$((X - c)/a)^2 = ((1/W) - 1)^{1/b}$$

$$(X - c)/a = \pm \sqrt{((1/W) - 1)^{1/b}}$$

$$X = c \pm a \sqrt{((1/W) - 1)^{1/b}}$$

$$X = c - a\sqrt{\left(\left(\frac{1}{W}\right) - 1\right)^{1/b}} \quad \text{for } X < c$$

$$X = c + a\sqrt{\left(\left(\frac{1}{W}\right) - 1\right)^{1/b}} \quad \text{for } X \geq c$$

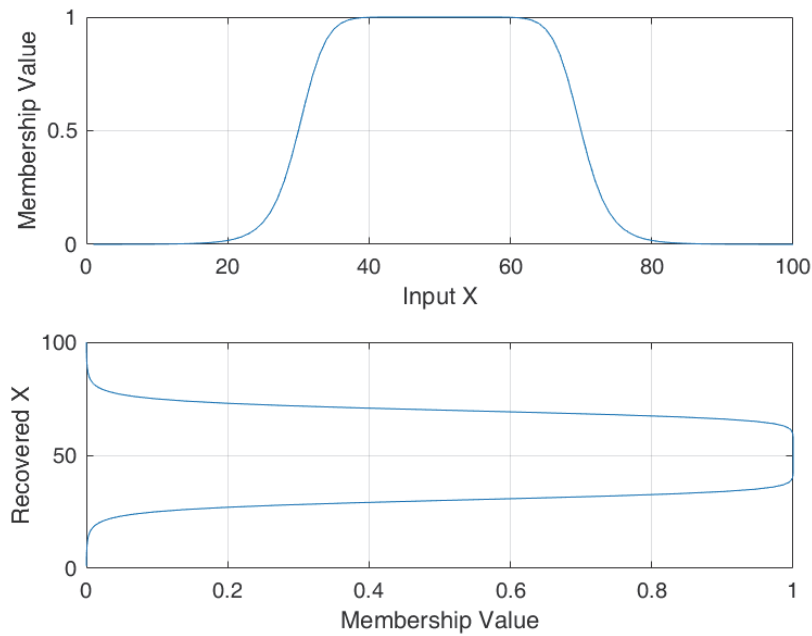


Figure P11.2.1

Matlab Program

% Problem 11.2

close **all**; clear **all**;

NPt = 100;

x = linspace(1, 100, NPt);

x_inverse = zeros(1, NPt);

w = zeros(1, NPt);

a = 20;

b = 5;

c = 50;

for k=1:NPt

```

%w(1,k) = gbell_mf(x(1,k),[a b c]);
w(1,k) = 1/(1+(((x(1,k)-c)/a)^2)^b);
if x(1,k) < c
    x_inverse(1,k) = c - a*sqrt( ((1/w(1,k))-1)^(1/b) );
elseif x(1,k) >= c
    x_inverse(1,k) = c + a*sqrt( ((1/w(1,k))-1)^(1/b) );
end
end
figure
subplot(2,1,1); plot(x,w); grid; ylabel('Membership Value');
xlabel('Input X');
subplot(2,1,2); plot(w,x_inverse); grid; ylabel('Recovered X');
xlabel('Membership Value');

```

Problem 11.3 (Inverses and gradients of monotonic membership functions)

Given two consequent membership functions $\mu_{C1}(z_1)$ and $\mu_{C2}(z_2)$ and the ranges of their input values are given by $0 \leq z_1, z_2 \leq 10$ as

$$\mu_{C1}(z_1) = \min \left[1, \max \left(\frac{z_1 - 0}{2 - 0}, 0 \right) \right]$$

$$\mu_{C2}(z_2) = \min \left[1, \max \left(\frac{z_2 - 6}{9 - 6}, 0 \right) \right]$$

- Determine their inverses.
- State the key difficulty of a monotonic membership function for gradient learning; and state a practical solution.
- Give labelled plots for $\mu_{C1}(z_1)$ and $\mu_{C2}(z_2)$ and their inverses.

Solution

(a)

$$z_1 = \min \left[10, \max(2\mu_{c1}(z_1), 0) \right]$$

$$z_2 = \min \left[10, \max(3(1 - \mu_{c2}(z_2)) + 6, 0) \right]$$

(b) The key difficulty of a monotonic membership function for gradient learning is its absence of a derivative.

To overcome this difficulty, a practical solution is to use an approximation of its derivative.

(c) Plot is shown in Figure P11.3.1.

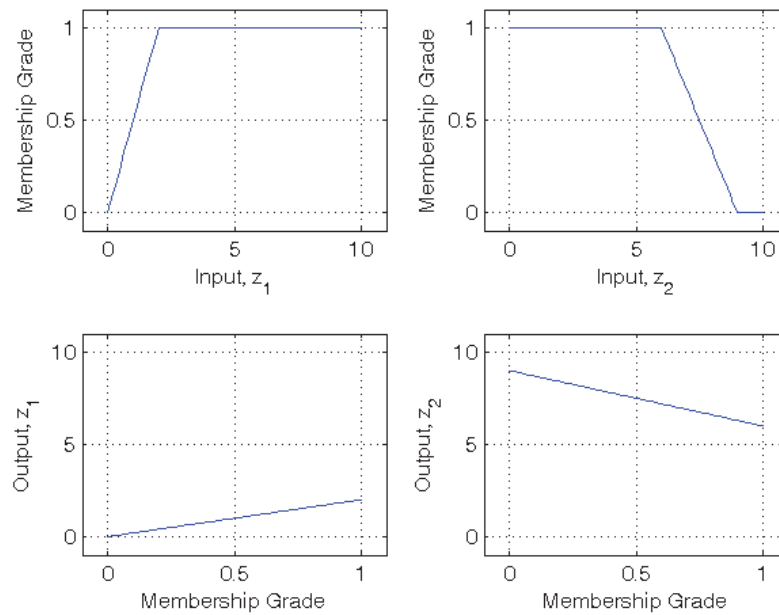


Figure P11.3.1

Problem 11.4 (Defuzzified output computation of Mamdani fuzzy system with fuzzy inputs)

Consider four-rule one-input one-output Mamdani fuzzy system of Example 11.1 in which the four rules of the fuzzy system are defined by:

Rule 1: If X is very young then Y implies fair experience

Rule 2: If X is young then Y implies good experience

Rule 3: If X is old then Y implies very good experience

Rule 4: If X is very old then Y implies excellent experience

The antecedent membership functions as shown in Figure P11.4.1 are defined by

$$\mu_{very\ young}(x) = \text{GBellMF}(x, [10, 3, 0])$$

$$\mu_{young}(x) = \text{GBellMF}(x, [20, 5, 30])$$

$$\mu_{old}(x) = \text{GBellMF}(x, [20, 5, 70])$$

$$\mu_{very\ old}(x) = \text{GBellMF}(x, [10, 3, 100])$$

where $x \in [0, 100]$.

The consequent membership functions as shown in Figure P11.4.1 are defined by

$$\mu_{fair}(y) = \text{GBellMF}(y, [1, 8, 0])$$

$$\mu_{good}(y) = \text{GBellMF}(y, [1, 6, 2])$$

$$\mu_{very\ good}(y) = \text{GBellMF}(y, [2, 6, 5])$$

$$\mu_{excellent}(y) = \text{GBellMF}(y, [2, 8, 9])$$

where $y \in [0, 10]$.

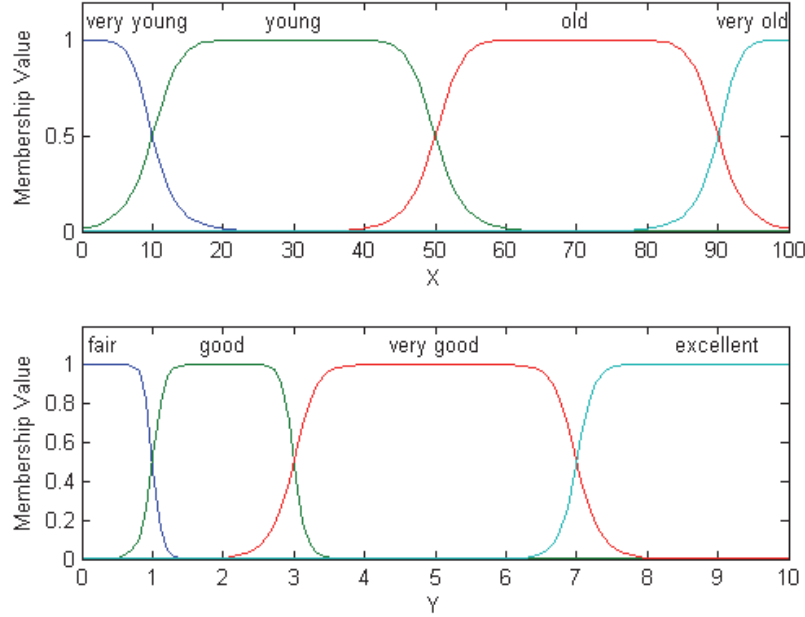


Figure P11.4.1

Assuming that fuzzy inputs (instead of crisp inputs) are applied. Using the Max-Min composition, the Larsen's algebraic product implication function (for binary fuzzy relation), and the centroid defuzzification, determine the defuzzified output of each of the following four fuzzy inputs as shown in Figure P11.4.2.

(a) $\mu_{I_1}(x) = \text{GBellMF}(x, [1, 1, 10])$

(b) $\mu_{I_2}(x) = \text{GBellMF}(x, [1, 1, 45])$

(c) $\mu_{I_3}(x) = \text{GBellMF}(x, [1, 1, 70])$

(d) $\mu_{I_4}(x) = \text{GBellMF}(x, [1, 1, 95])$

For convenience, the area (a_j) and centroid (z_{Cj}) of each of the four consequent membership functions ($j = 1$ to 4) are given by

$a_1 = 1.0048, a_2 = 2.0212, a_3 = 4.0460, a_4 = 3.0128;$

$$z_{c1} = 0.4491, z_{c2} = 2.0000, z_{c3} = 5.0000, z_{c4} = 8.5468$$

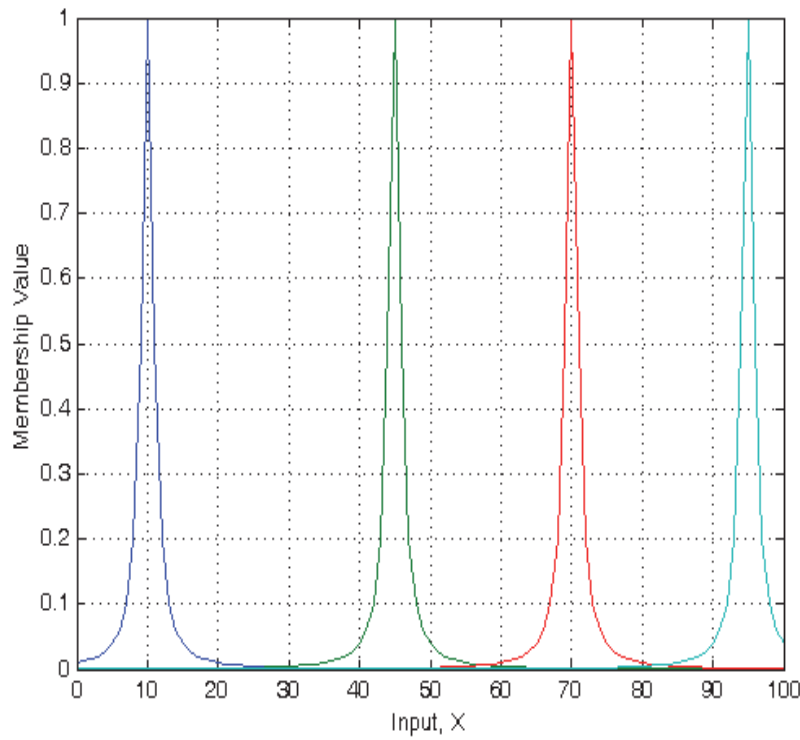


Figure P11.4.2

Solution

For illustration and simplicity, define the discrete universe of discourse X for all the four inputs and the four antecedent fuzzy sets as

$$\begin{aligned} X &= \{0, 6, 8, 9, 10, 11, 12, 14, 20, 30, 41, 43, 44, 45, 46, 47, 49, 60, 66, \\ &68, 69, 70, 71, 72, 74, 80, 91, 93, 94, 95, 96, 97, 99, 100\} \\ &= \{x_1, x_2, x_3, \dots, x_{34}\} \end{aligned}$$

Such a fuzzy set F in X can be expressed as

$$F = \mu_F(x_1)/x_1 + \mu_F(x_2)/x_2 + \mu_F(x_3)/x_3 + \dots + \mu_F(x_{34})/x_{34}$$

For simplicity, let A_1, A_2, A_3, A_4 denote, respectively, the antecedent fuzzy sets of very young, young, old, very old as

$$\mu_{A_1}(x) = \text{GBellMF}(x, [10, 3, 0])$$

$$\mu_{A_2}(x) = \text{GBellMF}(x, [20, 5, 30])$$

$$\mu_{A_3}(x) = \text{GBellMF}(x, [20, 5, 70])$$

$$\mu_{A_4}(x) = \text{GBellMF}(x, [10, 3, 100])$$

The values of antecedent membership function $\mu_{A_1}(x)$ for $X = \{x_1, x_2, x_3, \dots, x_{34}\}$ are

```
{1.0000  0.9554  0.7923  0.6530  0.5000  0.3608  0.2509
0.1172  0.0154  0.0014  0.0002  0.0002  0.0001  0.0001
0.0001  0.0001  0.0001  0.0000  0.0000  0.0000  0.0000
0.0000  0.0000  0.0000  0.0000  0.0000  0.0000  0.0000
0.0000  0.0000  0.0000  0.0000  0.0000  0.0000  0.0000}
```

The values of antecedent membership function $\mu_{A_2}(x)$ for $X = \{x_1, x_2, x_3, \dots, x_{34}\}$ are

```
{0.0170  0.1390  0.2783  0.3804  0.5000  0.6255  0.7415
0.9030  0.9990  1.0000  0.9975  0.9867  0.9725  0.9467
0.9030  0.8355  0.6255  0.0170  0.0028  0.0016  0.0013
0.0010  0.0008  0.0006  0.0004  0.0001  0.0000  0.0000
0.0000  0.0000  0.0000  0.0000  0.0000  0.0000  0.0000}
```

The values of antecedent membership function $\mu_{A_3}(x)$ for $X = \{x_1, x_2, x_3, \dots, x_{34}\}$ are

```
{0.0000  0.0000  0.0000  0.0000  0.0000  0.0000  0.0000
0.0000  0.0001  0.0010  0.0238  0.0474  0.0676  0.0970
0.1390  0.1982  0.3804  0.9990  1.0000  1.0000  1.0000}
```

1.0000 1.0000 1.0000 1.0000 0.9990 0.3804 0.1982
 0.1390 0.0970 0.0676 0.0474 0.0238 0.0170}

The values of antecedent membership function $\mu_{A4}(x)$ for $X = \{x_1, x_2, x_3, \dots x_{34}\}$ are

{0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000
 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000 0.0000
 0.0000 0.0000 0.0001 0.0002 0.0006 0.0009 0.0011
 0.0014 0.0017 0.0021 0.0032 0.0154 0.6530 0.8947
 0.9554 0.9846 0.9959 0.9993 1.0000 1.0000}

With the compositional rule of inference, the fuzzy output of the j th rule due to the i th fuzzy input is

$$C'_{ij} = I_i \circ R_j = I_i \circ (A_j \rightarrow C_j) \text{ for } i, j = 1 \text{ to } 4$$

Equivalently,

$$\mu_{C'_{ij}}(y) = \vee_x [\mu_{I_i}(x) \wedge \mu_{R_j}(x, y)] \text{ for } i, j = 1 \text{ to } 4$$

Adopting the Larsen's algebraic product implication function

$$\begin{aligned} \mu_{C'_{ij}}(y) &= \vee_x [\mu_{I_i}(x) \wedge (\mu_{A_j}(x) \cdot \mu_{C_j}(y))] \\ &= [\vee_x (\mu_{I_i}(x) \wedge \mu_{A_j}(x))] \cdot \mu_{C_j}(y) = w_{ij} \cdot \mu_{C_j}(y) \end{aligned}$$

for $i, j = 1 \text{ to } 4$

Based on (11.10)-(11.12), the aggregated defuzzified output of the four fuzzy rules due to the i th fuzzy input is

$$y_i = \frac{(w_{i1}a_1z_{C1} + w_{i2}a_2z_{C2} + w_{i3}a_3z_{C3} + w_{i4}a_4z_{C4})}{(w_{i1}a_1 + w_{i2}a_2 + w_{i3}a_3 + w_{i4}a_4)} \text{ for } i = 1 \text{ to } 4$$

(a) The fuzzy set of the 1st fuzzy input can be expressed as

$$\begin{aligned}
 I_1 &= \mu_{I_1}(6)/6 + \mu_{I_1}(8)/8 + \mu_{I_1}(9)/9 + \mu_{I_1}(10)/10 + \mu_{I_1}(11)/11 \\
 &\quad + \mu_{I_1}(12)/12 + \mu_{I_1}(14)/14 \\
 &= 0.0588/6 + 0.2000/8 + 0.5000/9 + 1.0000/10 + 0.5000/11 \\
 &\quad + 0.2000/12 + 0.0588/14
 \end{aligned}$$

The firing strength of each of the four antecedent fuzzy rules due to the 1st fuzzy input is

$$w_{1j} = \left[\vee_x \left(\mu_{I_1}(x) \wedge \mu_{A_j}(x) \right) \right] \text{ for } j\text{th antecedent fuzzy rule, } j = 1 \text{ to } 4$$

$$w_{11} = \vee_x \left[\begin{array}{c} \frac{0.0588 \wedge 0.9554}{6}, \frac{0.2000 \wedge 0.7923}{8}, \frac{0.5000 \wedge 0.6530}{9}, \\ \frac{1.0000 \wedge 0.5000}{10}, \frac{0.5000 \wedge 0.3608}{11}, \frac{0.2000 \wedge 0.2509}{12}, \\ \frac{0.0588 \wedge 0.1172}{14} \end{array} \right]$$

$$= 0.5000$$

$$w_{12} = \vee_x \left[\begin{array}{c} \frac{0.0588 \wedge 0.1390}{6}, \frac{0.2000 \wedge 0.2783}{8}, \frac{0.5000 \wedge 0.3804}{9}, \\ \frac{1.0000 \wedge 0.5000}{10}, \frac{0.5000 \wedge 0.6255}{11}, \frac{0.2000 \wedge 0.7415}{12}, \\ \frac{0.0588 \wedge 0.9030}{14} \end{array} \right]$$

$$= 0.5000$$

$$w_{13} = \vee_x \left[\begin{array}{c} \frac{0.0588 \wedge 0.0000}{6}, \frac{0.2000 \wedge 0.0000}{8}, \frac{0.5000 \wedge 0.0000}{9}, \\ \frac{1.0000 \wedge 0.0000}{10}, \frac{0.5000 \wedge 0.0000}{11}, \frac{0.2000 \wedge 0.0000}{12}, \\ \frac{0.0588 \wedge 0.0000}{14} \end{array} \right]$$

$$= 0.0000$$

$$w_{14} = V_x \left[\begin{array}{c} \frac{0.0588 \wedge 0.0000}{6}, \frac{0.2000 \wedge 0.0000}{8}, \frac{0.5000 \wedge 0.0000}{9}, \\ \frac{1.0000 \wedge 0.0000}{10}, \frac{0.5000 \wedge 0.0000}{11}, \frac{0.2000 \wedge 0.0000}{12}, \\ \frac{0.0588 \wedge 0.0000}{14} \end{array} \right]$$

$$= 0.0000$$

Hence, the defuzzified output due to the 1st fuzzy input can be calculated as

$$\begin{aligned} y_1 &= \frac{w_{11}a_1z_{C1} + w_{12}a_2z_{C2} + w_{13}a_3z_{C3} + w_{14}a_4z_{C4}}{w_{11}a_1 + w_{12}a_2 + w_{13}a_3 + w_{14}a_4} \\ &= (0.5000 * 1.0048 * 0.4491 + 0.5000 * 2.0212 * 2.0000 \\ &\quad + 0.0000 * 4.0460 * 5.0000 + 0.0000 * 3.0128 \\ &\quad * 8.5468) / (0.5000 * 1.0048 + 0.5000 * 2.0212 \\ &\quad + 0.0000 * 4.0460 + 0.0000 * 3.0128) = 1.4854 \end{aligned}$$

(b) The fuzzy set of the 2nd fuzzy input can be expressed as

$$\begin{aligned} I_2 &= \mu_{I_2}(41)/41 + \mu_{I_2}(43)/43 + \mu_{I_1}(44)/44 + \mu_{I_2}(45)/45 \\ &\quad + \mu_{I_1}(46)/46 + \mu_{I_2}(47)/47 + \mu_{I_2}(49)/49 \\ &= 0.0588/41 + 0.2000/43 + 0.5000/44 + 1.0000/45 \\ &\quad + 0.5000/46 + 0.2000/47 + 0.0588/49 \end{aligned}$$

The firing strength of each of the four antecedent fuzzy rules due to the 2nd fuzzy input is

$$w_{2j} = \left[V_x \left(\mu_{I_2}(x) \wedge \mu_{A_j}(x) \right) \right] \text{ for } j\text{th antecedent fuzzy rule, } j = 1 \text{ to } 4.$$

$$w_{21} = V_x \left[\begin{array}{c} \frac{0.0588 \wedge 0.0002}{41}, \frac{0.2000 \wedge 0.0002}{43}, \frac{0.5000 \wedge 0.0001}{44}, \\ \frac{1.0000 \wedge 0.0001}{45}, \frac{0.5000 \wedge 0.0001}{46}, \frac{0.2000 \wedge 0.0001}{47}, \\ \frac{0.0588 \wedge 0.0001}{49} \end{array} \right]$$

$$= 0.0002$$

$$w_{22} = V_x \left[\begin{array}{c} \frac{0.0588 \wedge 0.9975}{41}, \frac{0.2000 \wedge 0.9867}{43}, \frac{0.5000 \wedge 0.9725}{44}, \\ \frac{1.0000 \wedge 0.9467}{45}, \frac{0.5000 \wedge 0.9030}{46}, \frac{0.2000 \wedge 0.8355}{47}, \\ \frac{0.0588 \wedge 0.6255}{49} \end{array} \right]$$

$$= 0.9467$$

$$w_{23} = V_x \left[\begin{array}{c} \frac{0.0588 \wedge 0.0238}{41}, \frac{0.2000 \wedge 0.0474}{43}, \frac{0.5000 \wedge 0.0676}{44}, \\ \frac{1.0000 \wedge 0.0970}{45}, \frac{0.5000 \wedge 0.1390}{46}, \frac{0.2000 \wedge 0.1982}{47}, \\ \frac{0.0588 \wedge 0.3804}{49} \end{array} \right]$$

$$= 0.1982$$

$$w_{24} = V_x \left[\begin{array}{c} \frac{0.0588 \wedge 0.0000}{41}, \frac{0.2000 \wedge 0.0000}{43}, \frac{0.5000 \wedge 0.0000}{44}, \\ \frac{1.0000 \wedge 0.0000}{45}, \frac{0.5000 \wedge 0.0000}{46}, \frac{0.2000 \wedge 0.0000}{47}, \\ \frac{0.0588 \wedge 0.0000}{49} \end{array} \right]$$

$$= 0.0000$$

Hence, the defuzzified output due to the 2nd fuzzy input can be calculated as

$$y_2 = \frac{w_{21}a_1z_{C1} + w_{22}a_2z_{C2} + w_{23}a_3z_{C3} + w_{24}a_4z_{C4}}{w_{21}a_1 + w_{22}a_2 + w_{23}a_3 + w_{24}a_4}$$

$$\begin{aligned} &= (0.0002 * 1.0048 * 0.4491 + 0.9467 * 2.0212 * 2.0000 \\ &\quad + 0.1982 * 4.0460 * 5.0000 + 0.0001 * 3.0128 \\ &\quad * 8.5468) / (0.0002 * 1.0048 + 0.9467 * 2.0212 \\ &\quad + 0.1982 * 4.0460 + 0.0001 * 3.0128) = 2.8861 \end{aligned}$$

(c) The fuzzy set of the 3rd fuzzy input can be expressed as

$$\begin{aligned}
 I_3 &= \mu_{I_3}(66)/66 + \mu_{I_3}(68)/68 + \mu_{I_1}(69)/69 + \mu_{I_3}(70)/70 \\
 &\quad + \mu_{I_1}(71)/71 + \mu_{I_3}(72)/72 + \mu_{I_3}(74)/74 \\
 &= 0.0588/66 + 0.2000/68 + 0.5000/69 + 1.0000/70 \\
 &\quad + 0.5000/71 + 0.2000/72 + 0.0588/74
 \end{aligned}$$

The firing strength of each of the four antecedent fuzzy rules due to the 3rd fuzzy input is

$$w_{3j} = \left[\vee_x \left(\mu_{I_3}(x) \wedge \mu_{A_j}(x) \right) \right] \text{ for } j\text{th antecedent fuzzy rule, } j = 1 \text{ to } 4.$$

$$w_{31} = \vee_x \left[\begin{array}{c} \frac{0.0588 \wedge 0.0000}{66}, \frac{0.2000 \wedge 0.0000}{68}, \frac{0.5000 \wedge 0.0000}{69}, \\ \frac{1.0000 \wedge 0.0000}{70}, \frac{0.5000 \wedge 0.0000}{71}, \frac{0.2000 \wedge 0.0000}{72}, \\ \frac{0.0588 \wedge 0.0000}{74} \end{array} \right]$$

$$= 0.0000$$

$$w_{32} = \vee_x \left[\begin{array}{c} \frac{0.0588 \wedge 0.0028}{66}, \frac{0.2000 \wedge 0.0016}{68}, \frac{0.5000 \wedge 0.0013}{69}, \\ \frac{1.0000 \wedge 0.0010}{70}, \frac{0.5000 \wedge 0.0008}{71}, \frac{0.2000 \wedge 0.0006}{72}, \\ \frac{0.0588 \wedge 0.0004}{74} \end{array} \right]$$

$$= 0.0028$$

$$w_{33} = \vee_x \left[\begin{array}{c} \frac{0.0588 \wedge 1.0000}{66}, \frac{0.2000 \wedge 1.0000}{68}, \frac{0.5000 \wedge 1.0000}{69}, \\ \frac{1.0000 \wedge 1.0000}{70}, \frac{0.5000 \wedge 1.0000}{71}, \frac{0.2000 \wedge 1.0000}{72}, \\ \frac{0.0588 \wedge 1.0000}{74} \end{array} \right]$$

$$= 1.0000$$

$$w_{34} = v_x \left[\begin{array}{c} \frac{0.0588 \wedge 0.0006}{66}, \frac{0.2000 \wedge 0.0009}{68}, \frac{0.5000 \wedge 0.0011}{69}, \\ \frac{1.0000 \wedge 0.0014}{70}, \frac{0.5000 \wedge 0.0017}{71}, \frac{0.2000 \wedge 0.0021}{72}, \\ \frac{0.0588 \wedge 0.0032}{74} \end{array} \right]$$

$$= 0.0032$$

Hence, the defuzzified output due to the 3rd fuzzy input can be calculated as

$$\begin{aligned} y_3 &= \frac{w_{31}a_1z_{C1} + w_{32}a_2z_{C2} + w_{33}a_3z_{C3} + w_{34}a_4z_{C4}}{w_{31}a_1 + w_{32}a_2 + w_{33}a_3 + w_{34}a_4} \\ &= (0.0000 * 1.0048 * 0.4491 + 0.0028 * 2.0212 * 2.0000 \\ &\quad + 1.0000 * 4.0460 * 5.0000 + 0.0032 * 3.0128 \\ &\quad * 8.5468) / (0.0000 * 1.0048 + 0.0028 * 2.0212 \\ &\quad + 1.0000 * 4.0460 + 0.0032 * 3.0128) = 5.0043 \end{aligned}$$

(d) The fuzzy set of the 4th fuzzy input can be expressed as

$$\begin{aligned} I_4 &= \mu_{I_4}(91)/91 + \mu_{I_4}(93)/93 + \mu_{I_4}(94)/94 + \mu_{I_4}(95)/95 \\ &\quad + \mu_{I_4}(96)/96 + \mu_{I_4}(97)/97 + \mu_{I_4}(99)/99 \\ &= 0.0588/91 + 0.2000/93 + 0.5000/94 + 1.0000/95 \\ &\quad + 0.5000/96 + 0.2000/97 + 0.0588/99 \end{aligned}$$

The firing strength of each of the four antecedent fuzzy rules due to the 4th fuzzy input is

$$w_{4j} = \left[\vee_x \left(\mu_{I_4}(x) \wedge \mu_{A_j}(x) \right) \right] \text{ for } j\text{th antecedent fuzzy rule, } j = 1 \text{ to } 4$$

$$w_{41} = v_x \left[\begin{array}{c} \frac{0.0588 \wedge 0.0000}{91}, \frac{0.2000 \wedge 0.0000}{93}, \frac{0.5000 \wedge 0.0000}{94}, \\ \frac{1.0000 \wedge 0.0000}{95}, \frac{0.5000 \wedge 0.0000}{96}, \frac{0.2000 \wedge 0.0000}{97}, \\ \frac{0.0588 \wedge 0.0000}{99} \end{array} \right]$$

$$= 0.0000$$

$$w_{42} = V_x \left[\begin{array}{c} \frac{0.0588 \wedge 0.0000}{91}, \frac{0.2000 \wedge 0.0000}{93}, \frac{0.5000 \wedge 0.0000}{94}, \\ \frac{1.0000 \wedge 0.0000}{95}, \frac{0.5000 \wedge 0.0000}{96}, \frac{0.2000 \wedge 0.0000}{97}, \\ \frac{0.0588 \wedge 0.0000}{99} \end{array} \right]$$

$$= 0.0000$$

$$w_{43} = V_x \left[\begin{array}{c} \frac{0.0588 \wedge 0.3804}{91}, \frac{0.2000 \wedge 0.1982}{93}, \frac{0.5000 \wedge 0.1390}{94}, \\ \frac{1.0000 \wedge 0.0970}{95}, \frac{0.5000 \wedge 0.0676}{96}, \frac{0.2000 \wedge 0.0474}{97}, \\ \frac{0.0588 \wedge 0.0238}{99} \end{array} \right]$$

$$= 0.1982$$

$$w_{44} = V_x \left[\begin{array}{c} \frac{0.0588 \wedge 0.6530}{91}, \frac{0.2000 \wedge 0.8947}{93}, \frac{0.5000 \wedge 0.9554}{94}, \\ \frac{1.0000 \wedge 0.9846}{95}, \frac{0.5000 \wedge 0.9959}{96}, \frac{0.2000 \wedge 0.9993}{97}, \\ \frac{0.0588 \wedge 1.0000}{99} \end{array} \right]$$

$$= 0.9846$$

Hence, the defuzzified output due to the 4th fuzzy input can be calculated as

$$\begin{aligned} y_4 &= \frac{w_{41}a_1z_{C1} + w_{42}a_2z_{C2} + w_{43}a_3z_{C3} + w_{44}a_4z_{C4}}{w_{41}a_1 + w_{42}a_2 + w_{43}a_3 + w_{44}a_4} \\ &= (0.0000 * 1.0048 * 0.4491 + 0.0000 * 2.0212 * 2.0000 \\ &\quad + 0.1982 * 4.0460 * 5.0000 + 0.9846 * 3.0128 \\ &\quad * 8.5468) / (0.0000 * 1.0048 + 0.0000 * 2.0212 \\ &\quad + 0.1982 * 4.0460 + 0.9846 * 3.0128) = 7.7920 \end{aligned}$$

Chapter 13

Genetic Algorithms

Problem 13.1 (Average fitness value calculation of schemas)

Given that the fitness value of a binary string consisting of 3 integer bits is defined by its value, calculate the average fitness value of each of the following three schemas: (a) $**0$; (b) $*1*$; (c) $0**$.

Solution

(a) Average fitness value of $[000\ 010\ 100\ 110] = (0+2+4+6)/4 = 3$

(b) Average fitness value of $[010\ 011\ 110\ 111] = (2+3+6+7)/4 = 4.5$

(c) Average fitness value of $[000\ 001\ 010\ 011] = (0+1+2+3)/4 = 1.5$

Problem 13.2 (Formula for average fitness value calculation of schemas)

Given that the fitness value of a binary string consisting of L integer bits is defined by the number of zero bits in the string, derive a formula for the average fitness of such a schema consisting of K defined zero bits.

Solution

There are $L - K$ different schemas each consisting of a minimum of K defined zero bits. In other words, these $L - K$ schemas consist of $K, K + 1, K + 2, \dots, L - 1, L$ numbers of zero bits.

For each of these $L - K$ schemas consisting of the $K + k$ zero bits, there are $N_k = {}_{L-K}C_k$ possible combinations of k extra zero bits on top of the K defined zero bits. Therefore the average fitness is

$$K + \frac{\sum_{k=K}^L k N_k}{\sum_{k=K}^L N_k} = K + \frac{\sum_{k=K}^L k {}_{L-K}C_k}{\sum_{k=K}^L {}_{L-K}C_k}$$

Problem 13.3 (Maximum number of distinct strings in population of N binary strings of L -bit length)

Consider a population of N binary strings each of L binary bits, under what condition will the population consists of a maximum of $N \times 2^L$ different strings?

Solution

Each binary string has L binary bits with a maximum of 2^L different combinations. These 2^L different combinations define the maximum number N of different strings that is possible.

If N is greater than 2^L , for $N = 2^L + n$, any n of the string has to repeat n of the remaining 2^L strings. Therefore, the maximum N is $N \leq 2^L$.

Problem 13.4 (Union and interception operations on schemas)

For each of the following union and interception operations of two schemas, state whether the result is correct or incorrect.

$$*1* \cup *0* = ***$$

$$*0* \cup *1* = ***$$

$$*0* \cup *0* = *0*$$

$$*1* \cup *1* = *1*$$

$$*1* \cap *0* = ***$$

$$*0* \cap *1* = ***$$

$$*0* \cap *0* = *0*$$

$$*1* \cap *1* = *1*$$

Solution

$$*1* \cup *0* = *** \text{ correct}$$

$$*0* \cup *1* = *** \text{ correct}$$

$$*0* \cup *0* = *0* \text{ correct}$$

$$*1* \cup *1* = *1* \text{ correct}$$

$$*1* \cap *0* = *** \text{ incorrect}$$

$$*0* \cap *1* = *** \text{ incorrect}$$

$$*0* \cap *0* = *0* \text{ correct}$$

$$*1* \cap *1* = *1* \text{ correct}$$

Problem 13.5 (GA with basic operators)

Given the initial population size $P = 4$ with randomly selected positive integers $x = 2, 6, 17, 25$, $p_c = 1$, and $p_m = 0.001$, use a GA to illustrate how to maximize the function $f(x) = x^2 - x$ with x defined within the interval $[0, 31]$ in a single generation of reproduction, crossover, and mutation.

Solution

The interval $[0, 31]$ can be coded by 5 unsigned binary integers. For such a maximization problem, the fitness function is simply defined as $f(x)$.

Since GA is a probabilistic algorithm, for illustration, the following assumptions are made.

Selection is carried out by spinning the roulette wheel 4 times, assume $x = 2$ selected zero time, both $x = 6$ and $x = 17$ selected one time, and $x = 25$ selected two times.

Crossover is carried out by random pairing of two parents; assume $x = 6$ is paired to $x = 25$, and $x = 17$ is paired to $x = 25$; and after bits 3 and 1 are respectively the randomly selected crossover bit positions.

Mutation is carried out by mutating randomly selected bits among the 4×5 bits; assume the bit 4 of the 3rd chromosome is mutated.

The results obtained by one generation of reproduction, crossover, and mutation are shown in Table P13.5.1.

Table P13.5.1 Reproduction, crossover, and mutation calculation.

Reproduction Process (Fitness Sum=904; Fitness Average=226; Fitness maximum=600)					
i	Initial Population	Value x_i	Fitness $f(x_i)$	$\frac{f(x_i)}{\sum_{i=1}^4 f(x_i)}$	# Roulette Wheel Copies
1	00010	2	2	0.002	0
2	00110	6	30	0.033	1
3	10001	17	272	0.301	1
4	11001	25	600	0.664	2
Crossover Process (Fitness Sum=1542; Fitness Average=385.5; Fitness maximum=650)					
i	Mating Pool	Site	New Population	Value x'_i	Fitness $f(x'_i)$
2	001 10	3	00101	5	20
4	110 01	3	11010	26	650
3	1 0001	1	11001	25	600
4	1 1001	1	10001	17	272
Mutation Process (Fitness Sum=1644; Fitness Average=411; Fitness maximum=702)					

	Population	Site	Mutated Population	Value x'_i	Fitness $f(x'_i)$
1	00101	-	00101	5	20
2	11010	-	11010	26	650
3	11001	4	11011	27	702
4	10001	-	10001	17	272

The expected mutation is $4 \times 5 \times 0.001 = 0.02$ bits per generation, and hence the probability of mutation is small but in fact 1 mutation happens in this generation. As shown above, the population average fitness has improved from 226 to 411 and maximum fitness has increased from 600 to 702 in one generation.

Problem 13.6 (Roulette-wheel selection method)

In the Roulette-wheel selection method, the probabilities of selection are calculated using all the individual fitness values f_i for a population of N chromosomes as

$$P_{select_i} = \frac{f_i}{\sum_{j=1}^N f_j} \quad (\text{P13.6.1})$$

The actual number of copies of each chromosome in a new population will be determined by turning the roulette-wheel (with N slots proportional to the probabilities of selection of the N chromosomes) N times to select N winners.

As a result, some chromosomes may be selected more than 1 copy and some chromosomes may not be selected.

Given ten chromosomes with the following fitness values: 70, 23, 9, 35, 55, 18, 46, 68, 87, and 94, use and comment on the roulette-wheel selection method to compute the probability of selection of each chromosome in the next generation.

Solution

The sum of the 10 given fitness values is

$$70 + 23 + 9 + 35 + 55 + 18 + 46 + 68 + 87 + 94 = 505$$

The probability of selection of each chromosome in the mating pool is

chromosome 1: $70/505 = 0.13861$

chromosome 2: $23/505 = 0.04554$

chromosome 3: $9/505 = 0.01782$

chromosome 4: $35/505 = 0.06931$

chromosome 5: $55/505 = 0.10891$

chromosome 6: $18/505 = 0.03564$

chromosome 7: $46/505 = 0.09109$

chromosome 8: $68/505 = 0.13465$

chromosome 9: $87/505 = 0.17228$

chromosome 10: $94/505 = 0.18614$

The actual number of copies of each chromosome in the new population will be determined by turning the roulette-wheel (with 10 slots proportional to the probabilities of selection of the 10 chromosomes) 10 times to select 10 winners.

In a roulette-wheel selection, the potential problem that the best member of the population from failing to produce offsprings in the next generation can be avoided by adopting an elitist strategy, such that the best member of each generation is copied into the next generation.

This strategy improves the local search at the expense of a global perspective. But on balance, it appears to improve the GA performance. Two other elite strategies are described in Problems 13.7-13.8.

Problem 13.7 (Elite strategy by deterministic sampling method)

In the deterministic sampling method [1], the probabilities of selection are calculated using all the individual fitness values f_i for a population of N chromosomes as $P_{select_i} = f_i / \sum_{j=1}^N f_j$.

Then each chromosome i is allocated offsprings according to the “integer part” of e_i defined by

$$e_i = NP_{select_i} = \frac{Nf_i}{\sum_{j=1}^N f_j} \quad (\text{P13.7.1})$$

The remaining chromosomes needed to fill out the population are drawn from the top chromosome of the sorted list.

Consider ten chromosomes with the following fitness values: 70, 23, 9, 35, 55, 18, 46, 68, 87, and 94, compute the expected number of copies of each chromosome in the next generation.

Solution

The expected number of copies of each chromosome is

chromosome 1: $10 \cdot 70 / 505 = 1.3861$

chromosome 2: $10 \cdot 23 / 505 = 0.4554$

chromosome 3: $10 \cdot 9 / 505 = 0.1782$

chromosome 4: $10 \cdot 35 / 505 = 0.6931$

chromosome 5: $10 \cdot 55 / 505 = 1.0891$

chromosome 6: $10 \cdot 18 / 505 = 0.3564$

chromosome 7: $10 \cdot 46 / 505 = 0.9109$

chromosome 8: $10 \cdot 68 / 505 = 1.3465$

chromosome 9: $10 \cdot 87 / 505 = 1.7228$

chromosome 10: $10 \cdot 94 / 505 = 1.8614$

Hence, the new population will have 1 copy from chromosomes 1, 5, 8, 9, 10 and additional 5 copies from chromosome 10 (the chromosome with the top fitness value).

This method applies the elitist strategy to the best chromosome.

Problem 13.8 (Elite strategy by remainder stochastic sampling with replacement)

In the remainder stochastic sampling with replacement strategy [1], each chromosome i of a population of N chromosomes is allocated offsprings according to the integer part of e_i from $e_i = NP_{select_i} = Nf_i/\sum_{j=1}^N f_j$ where f_i denotes the fitness value of chromosome i .

The fractional parts r_i of the expected number values are used to calculate weights ($r_i/\sum_{j=1}^N r_j$) or the probabilities of selection) in a roulette-wheel selection procedure to fill the remaining population slots.

Consider ten chromosomes with the following fitness values: 70, 23, 9, 35, 55, 18, 46, 68, 87, and 94 compute the expected number of copies of each chromosome in the next generation.

Solution

The expected number of copies of each chromosome in the next generation is

chromosome 1: $10 \cdot 70/505 = 1.3861$, copy: 1,

chromosome 2: $10 \cdot 23/505 = 0.4554$, copy: 0,

chromosome 3: $10 \cdot 9/505 = 0.1782$, copy: 0,

chromosome 4: $10 \cdot 35/505 = 0.6931$, copy: 0,

chromosome 5: $10 \cdot 55/505 = 1.0891$, copy: 1,

chromosome 6: $10 \cdot 18/505 = 0.3564$, copy: 0,

chromosome 7: $10 \cdot 46 / 505 = 0.9109$, copy: 0,

chromosome 8: $10 \cdot 68 / 505 = 1.3465$, copy: 1,

chromosome 9: $10 \cdot 87 / 505 = 1.7228$, copy: 1,

chromosome 10: $10 \cdot 94 / 505 = 1.8614$, copy: 1.

Chromosome 1, 5, 8, 9, 10 are selected to have one copy in the new population.

The remaining 5 chromosomes of the new population are selected based on the roulette-wheel principle with the fractional parts of the expected numbers of copies as their probabilities for selections.

Sum of the 10 fractional copies gives

$$0.3861 + 0.4554 + 0.1782 + 0.6931 + 0.0891 + 0.3564 + 0.9109 \\ + 0.3465 + 0.7228 + 0.8614 = 5$$

chromosome 1: $0.3861 / 5 = 0.07723$,

chromosome 2: $0.4554 / 5 = 0.09109$,

chromosome 3: $0.1782 / 5 = 0.03564$,

chromosome 4: $0.6931 / 5 = 0.13861$,

chromosome 5: $0.0891 / 5 = 0.01782$,

chromosome 6: $0.3564 / 5 = 0.07129$,

chromosome 7: $0.9109 / 5 = 0.18218$,

chromosome 8: $0.3465 / 5 = 0.06931$,

chromosome 9: $0.7228 / 5 = 0.14455$,

chromosome 10: $0.8614 / 5 = 0.17228$.

The actual number of copies of each chromosome in the remaining 5 copies of the new population will be determined by turning the roulette-wheel (with 10 slots proportional to the probabilities of selection of the 10 chromosomes) 5 times to select 5 winners.

Problem 13.9 (Comparisons of improved reproduction methods)

Given a comparison among:

- (a) the roulette-wheel method,
- (b) the deterministic sampling method,
- (c) the remainder stochastic sampling with replacement, and
- (d) remainder stochastic sampling without replacement.

Solution

Guaranteed copy in new population

In the method (a), each chromosome is not guaranteed to have a copy in the new population. The methods (b)-(d) ensure that the chromosome has at least one copy if its expected number of copies is more than 1.

Similarity

The similarity between the methods (a) and (c) is that the set of all the chromosomes in the next generation will be the same if the expected numbers of copies of all the chromosomes are less than 1.0.

Differences

The methods (b) to (d) use different strategies to fill the remaining population slots.

The method (b) simply fills the remaining slots by the chromosome with the top fitness value.

The method (c) fills the remaining slots based on the roulette-wheel principle with the fractional parts of the expected numbers of copies as their probabilities for selections.

The method (d) uses the fractional part of each expected number value as a probability for choosing another offspring of the chromosome i up until a complete new population is formed. It has been widely used due to its superiority over the expected value schemes as in (a) to (c).

Problem 13.10 (Multiple-point crossover operation for retaining quality schemas)

One point crossover operator has the disadvantage of positional bias due to unequal lengths of two cut segments.

Multiple-point crossover is a micro genetic operator which operates in the chromosome level.

In particular, two-point crossover can reduce a positional bias by taking two locations and swap the middle segments.

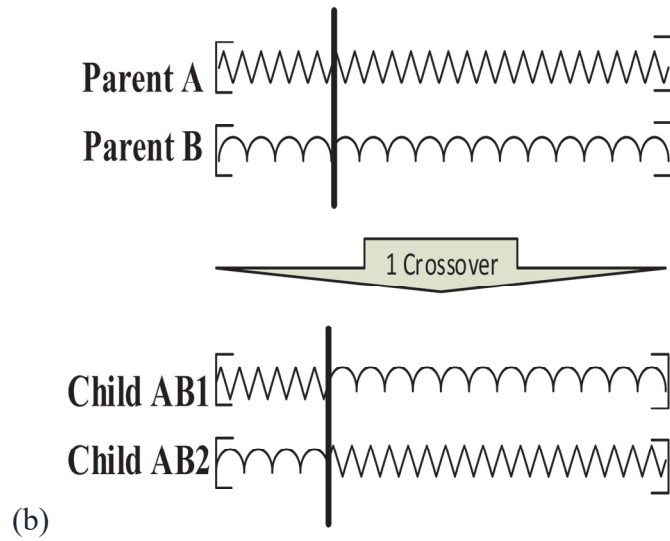
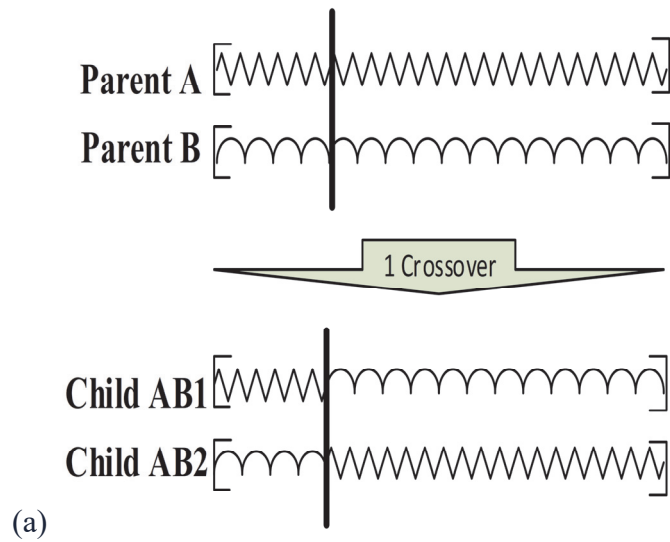
This can be generalized to N_c crossover points in which the offspring inherits alternative segments from the parents.

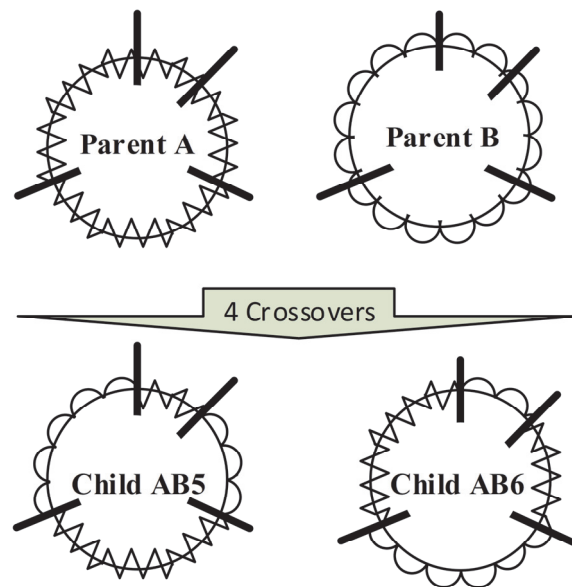
Draw a figure and give an example to illustrate the operation of multiple-point crossover.

Solution

An illustration of multiple-point crossover is shown in Figure P13.10.1. With even N_c , the string is treated as a ring with no start and no end. With odd N_c , a default crossing point is always assumed at the start.

Empirical studies have shown that as N_c increases, the multiple-point crossover degrades the performance resulting from more mixing and less structure.





(c)

Figure P13.10.1 Multiple-point crossover operations with random crossover points: (a) Biased Single-Point crossover; (b) Odd-numbered crossover ($CP = 3$); (c) Even-numbered crossover ($CP = 4$, in cyclical form). Key: Each solid bold black line represents a random crossover location.

Multiple-point crossover can solve certain combinations of features encoded on chromosomes that 1-point crossover cannot solve. Consider the two schemas as shown in Table P13.10.1:

Table P13.10.1 Retaining quality schema using multiple crossover.

Schema 1	<u>100</u> 100011 <u>00</u>		
Schema 2	0110 <u>1</u> 10 <u>1</u> 001		
Parent 1	<u>100</u> 1	0001	100
Parent 2	0110	<u>1</u> 10 <u>1</u>	001
Child 1	<u>100</u> 1	<u>1</u> 10 <u>1</u>	100
Child 2	0110	0001	001

In each of the two schemas, the underlined bits form a set of bits which has high-performance when these bits are grouped together.

Any 1-point crossover will break at least the schema 1 (if not the schema 2 as well) into two parts and at least one of the schemas (if not both) will not be passed on.

With the two-point crossover, the two quality schemas are retained in Child 1.

Problem 13.11 (Inversion operator for better codes and better sets)

Reordering operator is a micro genetic operator which operates in the chromosomal level.

Different from the operators which search for better sets, reordering operators search for better codes as well as better sets.

These operators are appropriate for a problem with its fitness value depends on string arrangement, such as where fitness $f(v, o)$ depends on some combination of allele value v and order o .

One commonly used reordering operator in GAs is the inversion operator.

In this operator, two points are chosen along the length of a chromosome, the chromosome is then cut at these points, and the end points of the cut section switch places.

Give an example to illustrate its operations.

Solution

For example, consider the nine-position string where two inversion sites are chosen at random as shown in Figure P13.11.1.

In this example, the two inversion sites 3 and 6 are chosen randomly.

After having been cut at the inversion sites, the end points of the cut section switch places.

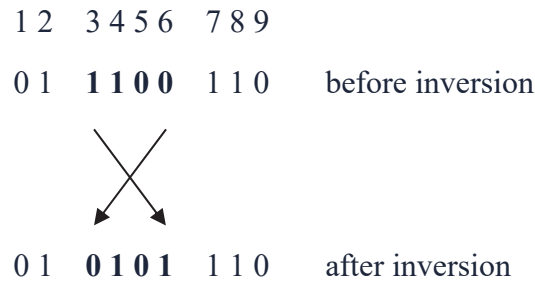


Figure P13.11.1 Inversion operator.

Problem 13.12 (Ranking procedure to limit uncontrolled spread of top chromosomes I)

Given a population of N chromosomes, an individual chromosome k is ranked according to its fitness value with a ranking order $r(k)$ from 1 to N .

According to the ranking procedure, the expected number of offsprings, $n(k)$, of an individual chromosome k is defined [2] by

$$n(k) = (\lambda_{max} - 1) \left[\frac{N + 1 - 2r(k)}{N - 1} \right] + 1 \quad (\text{P13.12.1})$$

- (a) What is λ_{max} ?
- (b) What is the range of $n(k)$ in term of λ_{max} ?
- (c) Given a population of $N = 3$, the individual fitness values of the chromosomes are 15, 45, 35; and $\lambda_{max} = 2$.
 - (i) Calculate the expected number of offsprings for each chromosome in the next population.
 - (ii) Plot and label the ranking curve.
- (d) State the purpose of the ranking procedure.
- (e) State the two major steps of the ranking procedure.

Solution

(a) λ_{max} is defined as the maximum expected number of offsprings in a new population.

(b) The range of $n(k)$ is $[\lambda_{min}, \lambda_{max}] = [2 - \lambda_{max}, \lambda_{max}]$.

(c)

$$n(1) = (\lambda_{max} - 1) \left\lfloor \frac{N + 1 - 2(1)}{N - 1} \right\rfloor + 1 = (1) \left\lfloor \frac{3 + 1 - 2}{3 - 1} \right\rfloor + 1 = 2$$

$$n(2) = (\lambda_{max} - 1) \left\lfloor \frac{N + 1 - 2(2)}{N - 1} \right\rfloor + 1 = (1) \left\lfloor \frac{3 + 1 - 4}{3 - 1} \right\rfloor + 1 = 1$$

$$n(3) = (\lambda_{max} - 1) \left\lfloor \frac{N + 1 - 2(3)}{N - 1} \right\rfloor + 1 = (1) \left\lfloor \frac{3 + 1 - 6}{3 - 1} \right\rfloor + 1 = 0$$

A plot of the ranking curve is shown in Figure P13.11.1.

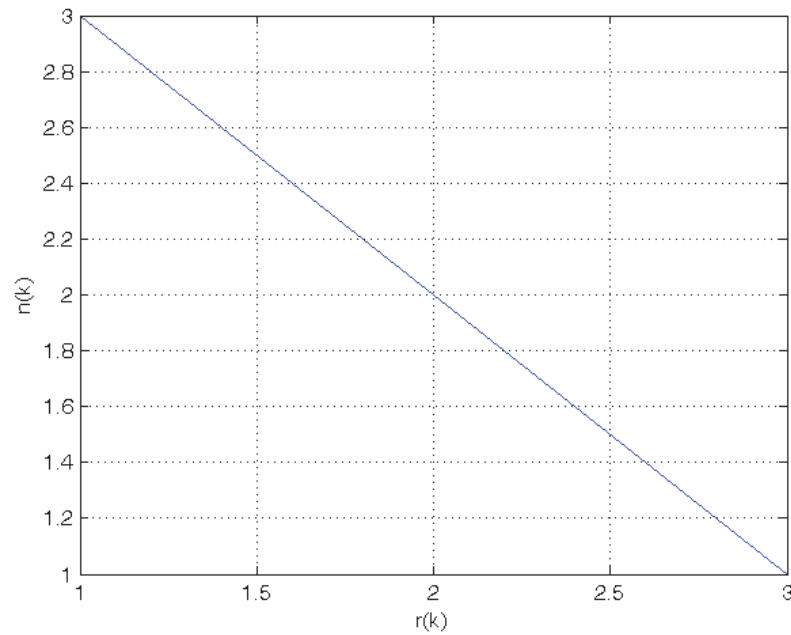


Figure P13.12.1 Plot of ranking curve.

- (d) Purpose: The purpose of the ranking procedure is to control the spread of the chromosomes with top fitness values.
- (e) Two major steps:
 - (i) Within a population, all the chromosomes are sorted and ranked according to their fitness values.
 - (ii) Each individual chromosome is assigned a number of offspring $n(k)$ as a linear decreasing function of their rank $r(k)$.

Problem 13.13 (Ranking procedure to limit uncontrolled spread of top chromosomes II)

In the ranking procedure, the spread of the chromosomes with top fitness values can be controlled by fixing a maximum expected number of offsprings λ_{max} in a new population of P .

In this procedure, the population is sorted and ranked according to fitness.

Each individual chromosome is then assigned a number of offspring $n(k)$ as a linear decreasing function of its rank $r(k)$.

One such function is shown in Figure P13.13.1.

Specifically, the expected number $n(k)$ of an individual k is computed [2] according to

$$n(k) = (\lambda_{max} - 1) \left\lceil \frac{N + 1 - 2r(k)}{N - 1} \right\rceil + 1 \quad (\text{P13.13.1})$$

where $1 \leq \lambda_{max} \leq 2$ and N is the population size. The range of $n(k)$ will then be $[\lambda_{min}, \lambda_{max}] = [2 - \lambda_{max}, \lambda_{max}]$.

Given a population of $N=10$ chromosomes with individual fitness values as 70, 23, 9, 35, 55, 18, 46, 68, 87, 94, and $\lambda_{max}=2$.

Determine the expected number of individual chromosome in the new population of a fixed size of 10, and plot the ranking curve defined by the above equation.

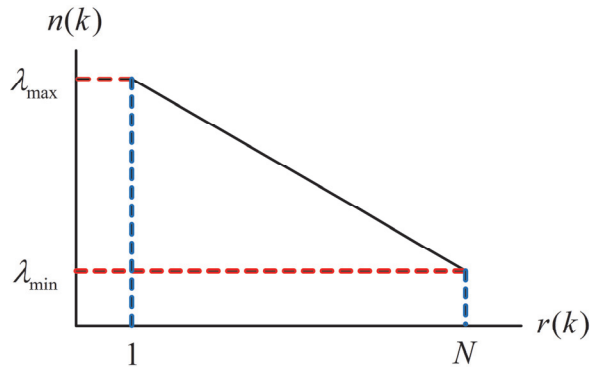


Figure P13.13.1 Ranking curve function.

Solution

The “Ranking (Fitness)” order of the 10 chromosomes are: 3 (70), 8 (23), 10 (9), 7 (35), 5 (55), 9 (18), 6 (46), 4 (68), 2 (87), 1 (94).

From the ranking curve shown in Figure P13.15.1, their expected numbers of “Offsprings (Fitness)” are 1.5556 (70), 0.4444 (23), 0.0000 (9), 0.6667 (35), 1.1111 (55), 0.2222 (18), 0.8889 (46), 1.3333 (68), 1.7778 (87), 2.0000 (94), respectively which are summarized in Table P13.13.1.

Table P13.13.1 Ranking fitness calculation of chromosomes.

Chromosome	10	9	1	8	5	7	4	2	6	3
Fitness	94	87	70	68	55	46	35	23	18	9
Ranking	1	2	3	4	5	6	7	8	9	10
Offspring	2.0000	1.7778	1.5556	1.3333	1.1111	0.8889	0.6667	0.4444	0.2222	0.0000

A plot of the ranking curve is shown in Figure P13.13.2.

It should be noted that the ranking procedure is a simple procedure based on simple calculation.

However, the procedure depends solely on the ranking of fitness values and disregards the difference between any two nearest ranked fitness values.

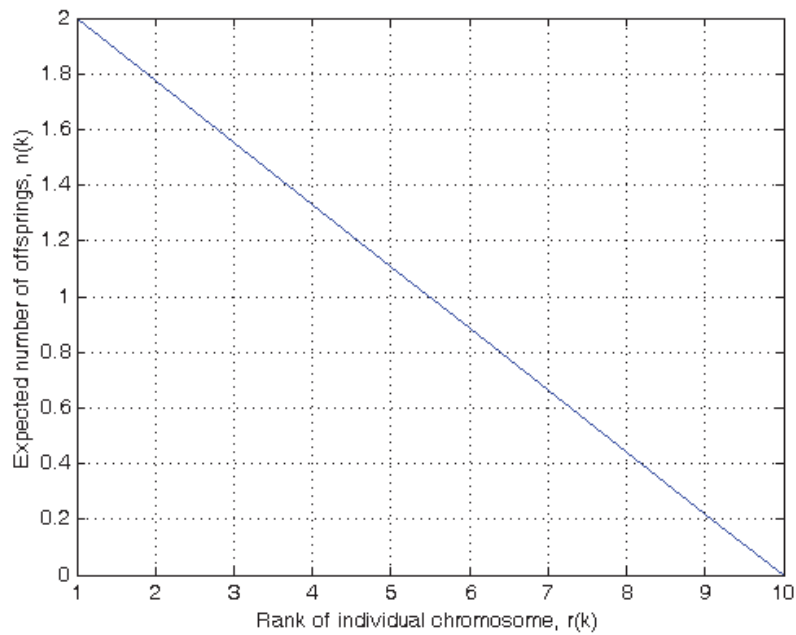


Figure P13.13.2 Plot of ranking curve.

Problem 13.14 (Sharing function to limit uncontrolled growth of similar strings)

A sharing function [3] serves to limit the uncontrolled growth of particular species within a population of P .

If a string x_i has many similar strings, the Hamming distances d_{ij} from the string x_i to its similar strings x_j are smaller.

A smaller value of d_{ij} gives rise to a larger value sharing function $s(d_{ij})$, and the sum of $s(d_{ij})$ becomes larger.

If many similar strings exist, they derate one another's fitness values, thus avoiding an uncontrolled growth.

Define the Hamming distance, d_{ij} , between two strings, x_i and x_j , as the number of places in which the two strings differ.

For binary-represented strings, d_{ij} is just equal to the number of binary bits in which the two strings differ.

Let $d_{\max}$ denote the maximum d_{ij} among all i and j , a simple sharing function can be defined as

$$s(d_{ij}) = 1 - \frac{d_{ij}}{d_{\max}} \quad \text{for } i, j = 1 \text{ to } P \quad (\text{P13.14.1})$$

A plot of (P13.14.1) is shown in Figure P13.14.1.

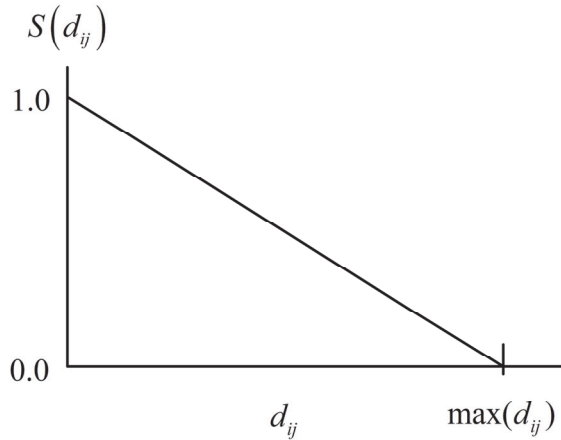


Figure P13.14.1 Sharing function.

The derated fitness function, f'_i , of a string i is obtained by taking its fitness function f_i over the sum of all its sharing functions $s(d_{ij})$ as

$$f'_i = \frac{f_i}{\sum_{j=1}^P s(d_{ij})} \quad \text{for } i = 1 \text{ to } P \quad (\text{P13.14.2})$$

If a string x_i has many similar strings, the Hamming distances d_{ij} from the string x_i to its similar strings x_j are smaller.

From (P13.14.1), a smaller value of d_{ij} gives rise to a larger value of $s(d_{ij})$; consequently from (P13.14.2), the sum of $s(d_{ij})$ becomes larger.

If many similar strings exist, they derate one another's fitness values, resulting in a reduced number of offsprings.

Define the fitness value f_i of a string as its number of '1' bits. Given three strings in which each string consists of three binary bits as shown in Table P13.14.1.

(a) Determine and write down all the values of Hamming distance between any two strings in Table P13.14.2.

(b) Show how the maximum value of Hamming distance can be determined.

(c) Show how the sharing functions can be calculated and write down the results in Table P13.14.3.

(d) Determine and write down the fitness values of the three strings in Table P13.14.4.

(e) Show how the derated fitness values can be calculated and also write down the results in Table P13.14.4.

Table P13.14.1 Three strings represented by binary bits.

String i	Binary representation
1	100
2	101
3	110

Solution

(a)

Table P13.14.2 Hamming distance between any two strings.

d_{ij}		j		
		1	2	3
i	1	0	1	1
	2	1	0	2
	3	1	2	0

(b) Maximum of d_{ij} is

$$\begin{aligned}
 d_{max} &= \max(d_{ij}) = \max(d_{11}, d_{12}, d_{13}, d_{21}, d_{22}, d_{23}, d_{31}, d_{32}, d_{33}) \\
 &= \max(0, 1, 1, 1, 0, 2, 1, 2, 0) = 2
 \end{aligned}$$

(c) From (P13.14.1), the sharing function of d_{ij} can be calculated by

$$s(d_{ij}) = 1 - \frac{d_{ij}}{d_{max}} \quad \text{for } i, j = 1 \text{ to } P$$

Hence

$$s(d_{11}) = 1 - (0/2) = 1$$

$$s(d_{12}) = 1 - (1/2) = 0.5$$

$$s(d_{13}) = 1 - (1/2) = 0.5$$

$$s(d_{21}) = 1 - (1/2) = 0.5$$

$$s(d_{22}) = 1 - (0/2) = 1$$

$$s(d_{23}) = 1 - (2/2) = 0$$

$$s(d_{31}) = 1 - (1/2) = 0.5$$

$$s(d_{32}) = 1 - (2/2) = 0$$

$$s(d_{33}) = 1 - (0/2) = 1$$

Table P13.14.3 Calculated sharing function values.

$s(d_{ij})$		j		
		1	2	3
i	1	1.0	0.5	0.5
	2	0.5	1.0	0.0
	3	0.5	0.0	1.0

(d) From (P13.14.2), the derated fitness value of the i th string is defined as

$$f'_i = \frac{f_i}{\sum_{j=1}^P s(d_{ij})} \quad \text{for } i = 1 \text{ to } P$$

Hence

$$f'_1 = f_1 / [s(d_{11}) + s(d_{12}) + s(d_{13})] = 1 / (1.0 + 0.5 + 0.5) = 0.5$$

$$f'_2 = f_2 / [s(d_{21}) + s(d_{22}) + s(d_{23})] = 2 / (0.5 + 1.0 + 0.0) = 1.3333$$

$$f'_3 = f_3 / [s(d_{31}) + s(d_{32}) + s(d_{33})] = 2 / (0.5 + 0.0 + 1.0) = 1.3333$$

Table P13.14.4 Calculated fitness and derated fitness values.

String i	Fitness values f_i	Derated fitness values f'_i
1	1	0.5000
2	2	1.3333
3	2	1.3333

References

- [1] Chin-Teng Lin and C. S. George Lee, Neural Fuzzy Systems - a Neuro-Fuzzy Synergism to Intelligent Systems, Prentice-Hall, New Jersey, 1996, Chapter 14.

- [2] J. E. Baker, “Adaptive selection methods for genetic algorithms,” Proceedings of International Conference on Genetic Algorithms and Their Applications, Pittsburgh, July 24-26, 1985, page 101-111.
- [3] D. E. Goldberg and J. Richardson, “Genetic algorithms with sharing for multimodal function optimization,” Proceedings of 2nd International Conference on Genetic Algorithms, Cambridge, MA, July 28-31, 1987, pages 41-49.

Chapter 14

Differential Evolution

Problem 14.1 (2-variable sphere function minimization using DE)

This example serves to illustrate the procedure of differential evolution to minimize the following objective function $f(\mathbf{X})$ for real values $\mathbf{X} = [x_1, x_2]$.

$$f(\mathbf{X}) = x_1^2 + x_2^2$$

The parameters are given by $D = 2$; $P = 4$; $m = 0.7$; and $p_c = 0.5$.

A parameter $\varepsilon = 10^{-5}$ is set as the convergence criterion; when $f(\mathbf{X}) \leq \varepsilon$, iteration stops and optimization result is obtained.

Solution

If the optimal solution or its region in the search space of a problem is known, one could randomly assign a population of initial solutions within a specific region.

In this example, the optimal solution for $\mathbf{X} = [x_1, x_2] = [0, 0]$, random initial vectors can be assigned within the following ranges, $-1 < x_1 < 1$ and $-1 < x_2 < 1$.

Generation 1

Assume 4 ($= P$) initial vectors are randomly assigned as

$\mathbf{X}_1$	$[-0.6, \quad -0.1]$
$\mathbf{X}_2$	$[0.4, \quad 0.1]$
$\mathbf{X}_3$	$[0.3, \quad 0.4]$
$\mathbf{X}_4$	$[-1, \quad 0.8]$
$\sum_{p=1}^4 f(\mathbf{X}_p) = 2.430$	

Compute the sum of objective function values as

$$\sum_{p=1}^4 f(\mathbf{X}_p) = ((-0.6)^2 + (-0.1)^2) + (0.4^2 + 0.1^2) + (0.3^2 + 0.4^2) + ((-1)^2 + 0.8^2) = 2.430$$

Mutation

Generate (with randomly chosen $\mathbf{X}_{r1}$ and $\mathbf{X}_{r2}$) to compute $\mathbf{Y}_p = \mathbf{X}_p + m(\mathbf{X}_{r1} - \mathbf{X}_{r2})$ for $p = \{1, 2, 3, 4\}$ as

$\mathbf{Y}_1 = \mathbf{X}_1 + m(\mathbf{X}_4 - \mathbf{X}_2)$	$[-1.58, \quad 0.39]$
$\mathbf{Y}_2 = \mathbf{X}_2 + m(\mathbf{X}_1 - \mathbf{X}_4)$	$[\quad 0.68, \quad -0.53]$
$\mathbf{Y}_3 = \mathbf{X}_3 + m(\mathbf{X}_1 - \mathbf{X}_4)$	$[\quad 0.58, \quad -0.23]$
$\mathbf{Y}_4 = \mathbf{X}_4 + m(\mathbf{X}_3 - \mathbf{X}_2)$	$[-1.07, \quad 1.01]$

Crossover

Select $\mathbf{Z}_p = \mathbf{X}_p$ if $rand_p \geq p_c$ or $\mathbf{Z}_p = \mathbf{Y}_p$ if $rand_p < p_c$ for $p = \{1, 2, 3, 4\}$ as

$\mathbf{Z}_1 = \mathbf{X}_1 (rand_1 = 0.88823)$	$[-0.6, \quad -0.10]$
$\mathbf{Z}_2 = \mathbf{Y}_2 (rand_2 = 0.38302)$	$[\quad 0.68, \quad -0.53]$
$\mathbf{Z}_3 = \mathbf{X}_3 (rand_3 = 0.91649)$	$[\quad 0.30, \quad 0.40]$
$\mathbf{Z}_4 = \mathbf{X}_4 (rand_4 = 0.80217)$	$[-1.0, \quad 0.80]$

Selection

Compute the sum of the objective function values of the $\mathbf{Z}_p$ population as

$$\sum_{p=1}^4 f(\mathbf{Z}_p) = ((-0.6)^2 + (-0.1)^2) + (0.68^2 + (-0.53)^2) + (0.3^2 + 0.4^2) + ((-1)^2 + 0.8^2) \approx 3.003$$

Since $\min \{ \sum_{p=1}^4 f(\mathbf{X}_p), \sum_{p=1}^4 f(\mathbf{Z}_p) \} = 2.430$, select $\mathbf{X}_p$ for $p = \{1, 2, 3, 4\}$ for generation 2.

Generation 2

$\mathbf{X}_1$	$[-0.6, -0.1]$
$\mathbf{X}_2$	$[0.4, 0.1]$
$\mathbf{X}_3$	$[0.3, 0.4]$
$\mathbf{X}_4$	$[-1, 0.8]$
$\sum_{p=1}^4 f(\mathbf{X}_p) = 2.43$	

Mutation

Generate (with randomly chosen $\mathbf{X}_{r1}$ and $\mathbf{X}_{r2}$) to compute $\mathbf{Y}_p = \mathbf{X}_p + m(\mathbf{X}_{r1} - \mathbf{X}_{r2})$ for p in $\{1, \dots, 4\}$ as

$\mathbf{Y}_1 = \mathbf{X}_1 + m(\mathbf{X}_2 - \mathbf{X}_3)$	$[-0.53, 0.31]$
$\mathbf{Y}_2 = \mathbf{X}_2 + m(\mathbf{X}_1 - \mathbf{X}_3)$	$[-0.23, -0.25]$
$\mathbf{Y}_3 = \mathbf{X}_3 + m(\mathbf{X}_2 - \mathbf{X}_4)$	$[1.28, -0.09]$
$\mathbf{Y}_4 = \mathbf{X}_4 + m(\mathbf{X}_1 - \mathbf{X}_3)$	$[-1.63, 0.45]$

Crossover

Select $\mathbf{Z}_p = \mathbf{X}_p$ if $rand_p \geq p_c$ or $\mathbf{Z}_p = \mathbf{Y}_p$ if $rand_p < p_c$ for $p = \{1, 2, 3, 4\}$ as

$\mathbf{Z}_1 = \mathbf{Y}_1 (rand_1 = 0.34736)$	$[-0.53, -0.31]$
$\mathbf{Z}_2 = \mathbf{Y}_2 (rand_2 = 0.15124)$	$[-0.23, -0.25]$
$\mathbf{Z}_3 = \mathbf{X}_3 (rand_3 = 0.54223)$	$[0.3, 0.4]$
$\mathbf{Z}_4 = \mathbf{X}_4 (rand_4 = 0.95598)$	$[-1, 0.8]$

Selection

Compute the sum of the objective function values of the $\mathbf{z}_p$ population as

$$\begin{aligned} \sum_{p=1}^4 f(\mathbf{Z}_p) &= ((-0.53)^2 + (-0.31)^2) + ((-0.23)^2 + (-0.25)^2) \\ &\quad + (0.3^2 + 0.4^2) + ((-1)^2 + 0.8^2) \approx 2.382 \end{aligned}$$

Since $\min \{ \sum_{p=1}^4 f(\mathbf{X}_p), \sum_{p=1}^4 f(\mathbf{Z}_p) \} = 2.382$, select $\mathbf{Z}_p$ for $p = \{1, 2, 3, 4\}$ for generation 3.

Generation 3

$\mathbf{X}_1$	$[-0.53, -0.31]$
$\mathbf{X}_2$	$[-0.23, -0.25]$
$\mathbf{X}_3$	$[0.3, 0.4]$
$\mathbf{X}_4$	$[-1, 0.8]$
$\sum_{p=1}^4 f(\mathbf{X}_p) = 2.3824$	

Mutation

Generate (with randomly chosen $\mathbf{X}_{r1}$ and $\mathbf{X}_{r2}$) to compute $\mathbf{Y}_p = \mathbf{X}_p + m(\mathbf{X}_{r1} - \mathbf{X}_{r2})$ for $p = \{1, 2, 3, 4\}$ as

$\mathbf{Y}_1 = \mathbf{X}_1 + m(\mathbf{X}_4 - \mathbf{X}_2)$	$[-1.069, 0.425]$
$\mathbf{Y}_2 = \mathbf{X}_2 + m(\mathbf{X}_4 - \mathbf{X}_1)$	$[-0.559, 0.527]$
$\mathbf{Y}_3 = \mathbf{X}_3 + m(\mathbf{X}_1 - \mathbf{X}_4)$	$[0.629, -0.377]$
$\mathbf{Y}_4 = \mathbf{X}_4 + m(\mathbf{X}_3 - \mathbf{X}_2)$	$[-0.629, 1.255]$

Crossover

Select $\mathbf{Z}_p = \mathbf{X}_p$ if $rand_p \geq p_c$ or $\mathbf{Z}_p = \mathbf{Y}_p$ if $rand_p < p_c$ for $p = \{1, 2, 3, 4\}$ as

$\mathbf{Z}_1 = \mathbf{X}_1$ ($rand_1 = 0.66489$)	$[-0.53, -0.31]$
$\mathbf{Z}_2 = \mathbf{Y}_2$ ($rand_2 = 0.13269$)	$[-0.559, 0.527]$
$\mathbf{Z}_3 = \mathbf{X}_3$ ($rand_3 = 0.51330$)	$[0.3, 0.4]$
$\mathbf{Z}_4 = \mathbf{Y}_4$ ($rand_4 = 0.29447$)	$[-0.629, 1.255]$

Selection

Compute the sum of the objective function values of the $\mathbf{z}_p$ population as

$$\begin{aligned}\sum_{p=1}^4 f(\mathbf{Z}_p) &= ((-0.53)^2 + (-0.31)^2) \\ &\quad + ((-0.559)^2 + (0.527)^2) + (0.3^2 + 0.4^2) \\ &\quad + ((-0.629)^2 + 1.255^2) \approx 3.1879\end{aligned}$$

Since $\min \{\sum_{p=1}^4 f(\mathbf{X}_p), \sum_{p=1}^4 f(\mathbf{Z}_p)\} = 2.3824$, select $\mathbf{Z}_p$ for $p = \{1, 2, 3, 4\}$ for generation 4.

Generation 4

$\mathbf{X}_1$	$[-0.53, -0.31]$
$\mathbf{X}_2$	$[-0.23, -0.25]$
$\mathbf{X}_3$	$[0.3, 0.4]$
$\mathbf{X}_4$	$[-1, 0.8]$
$\sum_{p=1}^4 f(\mathbf{X}_p) = 2.3824$	

...

Generation 523 (DE optimization converged)

$\mathbf{X}_1$	$[-3.9854 \times 10^{-4}, -9.1718 \times 10^{-4}]$
$\mathbf{X}_2$	$[1.041 \times 10^{-3}, 1.1281 \times 10^{-3}]$
$\mathbf{X}_3$	$[7.59 \times 10^{-5}, 4.4394 \times 10^{-5}]$
$\mathbf{X}_4$	$[9.0703 \times 10^{-4}, 8.3934 \times 10^{-4}]$
$\sum_{p=1}^4 f(\mathbf{X}_p) = 4.8912 \times 10^{-6}$	

As $\sum_{p=1}^4 f(\mathbf{X}_p) \leq \varepsilon$, the algorithm stops; and the sum of objective function values has been reduced from initial 2.430 to 4.8912×10^{-6} .

Problem 14.2 (1-dimensional sine function interpolation using DE)

This example serves to illustrate the procedure of differential evolution to determine the best set of coefficients $[a_0, a_1, a_2]$ of a specified sine function to fit a given set of N input to output data pairs as denoted by $\mathbf{X} = [x_1, x_2, x_3, \dots, x_N]$ and $\mathbf{Y} = g_o(\mathbf{X}) = [y_1, y_2, y_3, \dots, y_N]$.

The sine function is given by

$$y = g(x) = a_0 - a_1 x \sin(6\pi x + a_2 \pi)$$

The coefficient of the optimal function is $\mathbf{A}_o = [a_0, a_1, a_2] = [1, 0.5, 0.25]$.

The differential evolution parameters are given as

$$\mathbf{X} = [-0.5000 \quad -0.2500 \quad 0.0000 \quad 0.2500 \quad 0.5000]$$

$$\mathbf{Y} = [0.8232 \quad 1.0884 \quad 1.0000 \quad 1.0884 \quad 1.1768]$$

$$D = 3; P = 6; m = 0.7; p_c = 0.5.$$

Evaluate the interpolation performance of the optimized solution using 40 uniformly distributed values of inputs.

Solution

Denote a population of P chromosomes as $[\mathbf{A}_1, \mathbf{A}_2, \mathbf{A}_3, \dots, \mathbf{A}_P]$ and the p th chromosome as $\mathbf{A}_p = [a_{0p} \quad a_{1p} \quad a_{2p}]$, define a fitness function $f(\mathbf{A}_p)$ based on input $\mathbf{X}$ and desired output $\mathbf{Y}$ as

$$f(\mathbf{A}_p) = \sum_{n=1}^N (g_p(x_n) - y_n)^2 / N$$

where

$$g_p(x_n) = a_{0p} - a_{1p} \sin(6\pi x_n + a_{2p} \pi)$$

Define the convergence criterion ε ($\varepsilon = 10^{-4}$), which should be a small value, such that if in a generation there is a population member $\mathbf{A}_p$ with $f(\mathbf{A}_p) \leq \varepsilon$, then evolution stops and a satisfactory optimized

solution $\mathbf{A}_p$ is said to have been found.

If the optimal solution or its region in the search space of an optimization problem is known, one could randomly assign a population of initial solutions within a specific space.

In this example, given the optimal solution = $[a_0 \ a_1 \ a_2] = [1, 0.5, 0.25]$, random initial vectors can be assigned within an appropriate range of each coefficient.

Generation 1

Randomly assign 6 ($= P$) initial vectors as

$\mathbf{A}_1$	[0.7 − 0.5 − 0.9]
$\mathbf{A}_2$	[−0.3 0.2 0.1]
$\mathbf{A}_3$	[−0.8 0.2 0]
$\mathbf{A}_4$	[−0.9 − 0.2 − 0.3]
$\mathbf{A}_5$	[0.4 0.7 − 0.4]
$\mathbf{A}_6$	[0.3 0.1 0.2]
$\sum_{p=1}^6 f(\mathbf{A}_p) = 9.9548$	

The objective function values of the generation 1 are

$$\left\{ \begin{array}{l} f(\mathbf{A}_1) = 0.08702 \\ f(\mathbf{A}_2) = 1.74160 \\ f(\mathbf{A}_3) = 3.30840 \\ f(\mathbf{A}_4) = 3.79820 \\ f(\mathbf{A}_5) = 0.48084 \\ f(\mathbf{A}_6) = 0.53872 \end{array} \right.$$

The minimum objective function value of the generation 1 is $f(\mathbf{A}_1) = 0.08702 \geq \varepsilon = 10^{-4}$, thus evolution has to be continued.

Compute the sum of objective function values as

$$\sum_{p=1}^6 f(\mathbf{A}_p) = f(\mathbf{A}_1) + f(\mathbf{A}_2) + f(\mathbf{A}_3) + f(\mathbf{A}_4) + f(\mathbf{A}_5) + f(\mathbf{A}_6) = 9.9548$$

Mutation

Generate (with randomly chosen $\mathbf{A}_{r1}$ and $\mathbf{A}_{r2}$) to compute $\mathbf{B}_p = \mathbf{A}_p + m(\mathbf{A}_{r1} - \mathbf{A}_{r2})$ for $p = \{1, 2, \dots, 6\}$ as

$\mathbf{B}_1 = \mathbf{A}_1 + m(\mathbf{A}_2 - \mathbf{A}_3)$	[1.05 - 0.5 - 0.83]
$\mathbf{B}_2 = \mathbf{A}_2 + m(\mathbf{A}_6 - \mathbf{A}_5)$	[-0.37 - 0.22 0.52]
$\mathbf{B}_3 = \mathbf{A}_3 + m(\mathbf{A}_6 - \mathbf{A}_5)$	[-0.87 - 0.22 0.42]
$\mathbf{B}_4 = \mathbf{A}_4 + m(\mathbf{A}_1 - \mathbf{A}_6)$	[-0.62 - 0.62 - 1.07]
$\mathbf{B}_5 = \mathbf{A}_5 + m(\mathbf{A}_4 - \mathbf{A}_2)$	[-0.02 0.42 - 0.68]
$\mathbf{B}_6 = \mathbf{A}_6 + m(\mathbf{A}_3 - \mathbf{A}_5)$	[-0.54 - 0.25 0.48]

Crossover

Select $\mathbf{Z}_p = \mathbf{A}_p$ if $rand_p \geq p_c$ or $\mathbf{Z}_p = \mathbf{B}_p$ if $rand_p < p_c$ for $p = \{1, 2, \dots, 6\}$ as

$\mathbf{Z}_1 = \mathbf{B}_1 (rand_1 = 0.12703)$	[1.05 - 0.5 - 0.83]
$\mathbf{Z}_2 = \mathbf{A}_2 (rand_2 = 0.55978)$	[-0.3 0.2 0.1]
$\mathbf{Z}_3 = \mathbf{A}_3 (rand_3 = 0.82228)$	[-0.8 0.2 0]
$\mathbf{Z}_4 = \mathbf{B}_4 (rand_4 = 0.36721)$	[-0.62 - 0.62 - 1.07]
$\mathbf{Z}_5 = \mathbf{A}_5 (rand_5 = 0.50135)$	[0.4 0.7 - 0.4]
$\mathbf{Z}_6 = \mathbf{A}_6 (rand_6 = 0.88760)$	[0.3 0.1 0.2]

Selection

Compute the sum of the objective function values of the $\mathbf{z}_p$ population as

$$\begin{aligned} \sum_{p=1}^6 f(\mathbf{Z}_p) &= f(\mathbf{Z}_1) + f(\mathbf{Z}_2) + f(\mathbf{Z}_3) + f(\mathbf{Z}_4) + f(\mathbf{Z}_5) + f(\mathbf{Z}_6) \\ &= 4.3964 \times 10^{-3} + 1.7416 + 3.3084 + 2.5684 + 0.48084 \\ &\quad + 0.53872 = 8.6424 \end{aligned}$$

Since $\min \{ \sum_{p=1}^6 f(\mathbf{A}_p), \sum_{p=1}^6 f(\mathbf{Z}_p) \} = 8.6424$, select $\mathbf{Z}_p$ for $p = \{1, 2, \dots, 6\}$ for generation 2.

Generation 2

$\mathbf{A}_1$	[1.05 - 0.5 - 0.83]
$\mathbf{A}_2$	[-0.3 0.2 0.1]
$\mathbf{A}_3$	[-0.8 0.2 0]
$\mathbf{A}_4$	[-0.62 - 0.62 - 1.07]
$\mathbf{A}_5$	[0.4 0.7 - 0.4]
$\mathbf{A}_6$	[0.3 0.1 0.2]
$\sum_{p=1}^6 f(\mathbf{A}_p) = 8.6424$	

The objective function values of the generation 2 are

$$\left\{ \begin{array}{l} f(\mathbf{A}_1) = 4.3964 \times 10^{-3} \\ f(\mathbf{A}_2) = 1.7416 \\ f(\mathbf{A}_3) = 3.3084 \\ f(\mathbf{A}_4) = 2.5684 \\ f(\mathbf{A}_5) = 0.48084 \\ f(\mathbf{A}_6) = 0.53872 \end{array} \right.$$

The minimum objective function value of the generation 2 is $f(\mathbf{A}_1) = 4.3964 \times 10^{-3} \geq \varepsilon = 10^{-4}$, thus evolution has to be continued.

Mutation

Generate (with randomly chosen $\mathbf{A}_{r1}$ and $\mathbf{A}_{r2}$) to compute $\mathbf{B}_p = \mathbf{A}_p + m(\mathbf{A}_{r1} - \mathbf{A}_{r2})$ for $p = \{1, 2, \dots, 6\}$ as

$\mathbf{B}_1 = \mathbf{A}_1 + m(\mathbf{A}_2 - \mathbf{A}_5)$	[0.56 - 0.85 - 0.48]
$\mathbf{B}_2 = \mathbf{A}_2 + m(\mathbf{A}_6 - \mathbf{A}_1)$	[-0.825 0.62 0.821]
$\mathbf{B}_3 = \mathbf{A}_3 + m(\mathbf{A}_6 - \mathbf{A}_5)$	[-0.87 - 0.22 0.42]
$\mathbf{B}_4 = \mathbf{A}_4 + m(\mathbf{A}_3 - \mathbf{A}_1)$	[-1.915 - 0.13 - 0.489]
$\mathbf{B}_5 = \mathbf{A}_5 + m(\mathbf{A}_1 - \mathbf{A}_6)$	[0.925 0.28 - 1.121]
$\mathbf{B}_6 = \mathbf{A}_6 + m(\mathbf{A}_5 - \mathbf{A}_2)$	[0.79 0.45 - 0.15]

Crossover

Select $\mathbf{Z}_p = \mathbf{A}_p$ if $rand_p \geq p_c$ or $\mathbf{Z}_p = \mathbf{B}_p$ if $rand_p < p_c$ for $p = \{1, 2, \dots, 6\}$ as

$\mathbf{Z}_1 = \mathbf{A}_1$ ($rand_1 = 0.90042$)	[1.05 - 0.5 - 0.83]
$\mathbf{Z}_2 = \mathbf{A}_2$ ($rand_2 = 0.75759$)	[-0.3 0.2 0.1]
$\mathbf{Z}_3 = \mathbf{A}_3$ ($rand_3 = 0.77128$)	[-0.8 0.2 0]
$\mathbf{Z}_4 = \mathbf{A}_4$ ($rand_4 = 0.85840$)	[-0.62 - 0.62 - 1.07]
$\mathbf{Z}_5 = \mathbf{A}_5$ ($rand_5 = 0.58389$)	[0.4 0.7 - 0.4]
$\mathbf{Z}_6 = \mathbf{A}_6$ ($rand_6 = 0.56570$)	[0.3 0.1 0.2]

Notice that in this case $\mathbf{Z}_p = \mathbf{A}_p$, so we force the difference between $\mathbf{Z}_p$ and $\mathbf{A}_p$ by choosing a random index, $rand = 5$, thus selecting $\mathbf{Z}_5 = \mathbf{B}_5$ instead of $\mathbf{Z}_5 = \mathbf{A}_5$.

Selection

Compute the sum of the objective function values of the $\mathbf{z}_p$ population as

$$\begin{aligned}
 \sum_{p=1}^6 f(\mathbf{Z}_p) &= f(\mathbf{Z}_1) + f(\mathbf{Z}_2) + f(\mathbf{Z}_3) + f(\mathbf{Z}_4) + f(\mathbf{Z}_5) + f(\mathbf{Z}_6) \\
 &= 4.3964 \times 10^{-3} + 1.7416 + 3.3084 + 2.5684 + 0.030473 \\
 &\quad + 0.53872 = 8.192
 \end{aligned}$$

Since $\min \{ \sum_{p=1}^6 f(\mathbf{A}_p), \sum_{p=1}^6 f(\mathbf{Z}_p) \} = 8.192$, select $\mathbf{Z}_p$ for $p = \{1, 2, \dots, 6\}$ for generation 3.

Generation 3

$\mathbf{A}_1$	[1.05 - 0.5 - 0.83]
$\mathbf{A}_2$	[-0.3 0.2 0.1]
$\mathbf{A}_3$	[-0.8 0.2 0]
$\mathbf{A}_4$	[-0.62 - 0.62 - 1.07]
$\mathbf{A}_5$	[0.925 0.28 - 1.121]
$\mathbf{A}_6$	[0.3 0.1 0.2]
$\sum_{p=1}^6 f(\mathbf{A}_p) = 8.192$	

The objective function values of the generation 3 are

$$\left\{ \begin{array}{l} f(\mathbf{A}_1) = 4.3964 \times 10^{-3} \\ f(\mathbf{A}_2) = 1.7416 \\ f(\mathbf{A}_3) = 3.3084 \\ f(\mathbf{A}_4) = 2.5684 \\ f(\mathbf{A}_5) = 3.0473 \times 10^{-2} \\ f(\mathbf{A}_6) = 0.53872 \end{array} \right.$$

The minimum objective function value of generation 3 is $f(\mathbf{A}_1) = 4.3964 \times 10^{-3} \geq \varepsilon = 10^{-4}$, thus evolution has to continue.

Compute the sum of objective function values as

$$\begin{aligned} \sum_{p=1}^6 f(\mathbf{A}_p) &= f(\mathbf{A}_1) + f(\mathbf{A}_2) + f(\mathbf{A}_3) + f(\mathbf{A}_4) + f(\mathbf{A}_5) + f(\mathbf{A}_6) \\ &= 8.192 \end{aligned}$$

Mutation

Generate (with randomly chosen $\mathbf{A}_{r1}$ and $\mathbf{A}_{r2}$) to compute $\mathbf{B}_p = \mathbf{A}_p + m(\mathbf{A}_{r1} - \mathbf{A}_{r2})$ for $p = \{1, 2, \dots, 6\}$ as

$\mathbf{B}_1 = \mathbf{A}_1 + m(\mathbf{A}_4 - \mathbf{A}_5)$	$[-0.0315 \quad -1.13 \quad -0.7943]$
$\mathbf{B}_2 = \mathbf{A}_2 + m(\mathbf{A}_5 - \mathbf{A}_3)$	$[0.9075 \quad 0.256 \quad -0.6847]$
$\mathbf{B}_3 = \mathbf{A}_3 + m(\mathbf{A}_5 - \mathbf{A}_2)$	$[0.0575 \quad 0.256 \quad -0.8547]$
$\mathbf{B}_4 = \mathbf{A}_4 + m(\mathbf{A}_5 - \mathbf{A}_3)$	$[0.5875 \quad -0.564 \quad -1.8547]$
$\mathbf{B}_5 = \mathbf{A}_5 + m(\mathbf{A}_6 - \mathbf{A}_1)$	$[0.4 \quad 0.7 \quad -0.4]$
$\mathbf{B}_6 = \mathbf{A}_6 + m(\mathbf{A}_3 - \mathbf{A}_1)$	$[-0.995 \quad 0.59 \quad 0.781]$

Crossover

Select $\mathbf{Z}_p = \mathbf{A}_p$ if $rand_p \geq p_c$ or $\mathbf{Z}_p = \mathbf{B}_p$ if $rand_p < p_c$ for $p = \{1, 2, \dots, 6\}$ as

$\mathbf{Z}_1 = \mathbf{Y}_1 (rand_1 = 0.49269)$	$[-0.0315 \quad -1.13 \quad -0.7943]$
$\mathbf{Z}_2 = \mathbf{Y}_2 (rand_2 = 0.20981)$	$[0.9075 \quad 0.256 \quad -0.6847]$
$\mathbf{Z}_3 = \mathbf{Y}_3 (rand_3 = 0.37838)$	$[0.0575 \quad 0.256 \quad -0.8547]$
$\mathbf{Z}_4 = \mathbf{Y}_4 (rand_4 = 0.13462)$	$[0.5875 \quad -0.564 \quad -1.8547]$
$\mathbf{Z}_5 = \mathbf{A}_5 (rand_5 = 0.58834)$	$[0.925 \quad 0.28 \quad -1.121]$
$\mathbf{Z}_6 = \mathbf{A}_6 (rand_6 = 0.66071)$	$[0.3 \quad 0.1 \quad 0.2]$

Selection

Compute the sum of the objective function values of the $\mathbf{z}_p$ population as

$$\begin{aligned}
 \sum_{p=1}^6 f(\mathbf{Z}_p) &= f(\mathbf{Z}_1) + f(\mathbf{Z}_2) + f(\mathbf{Z}_3) + f(\mathbf{Z}_4) + f(\mathbf{Z}_5) + f(\mathbf{Z}_6) \\
 &= 0.96903 + 0.056015 + 1.0285 + 0.29583 + 0.030473 \\
 &\quad + 0.53872 = 2.9186
 \end{aligned}$$

Since $\min \{\sum_{p=1}^6 f(\mathbf{A}_p), \sum_{p=1}^6 f(\mathbf{Z}_p)\} = 2.9186$, select $\mathbf{Z}_p$ for $p = \{1, 2, \dots, 6\}$ for generation 2.

...

Generation 3491 (DE optimization converged)

$\mathbf{A}_1$	[0.997959 – 0.62423 – 8.79688]
$\mathbf{A}_2$	[1.00254 0.462094 10.2506]
$\mathbf{A}_3$	[1.01145 – 0.450102 – 8.59021]
$\mathbf{A}_4$	[0.968537 – 0.622985 – 8.78809]
$\mathbf{A}_5$	[1.06784 – 0.459238 5.72162]
$\mathbf{A}_6$	[0.975528 – 0.588504 – 8.78091]
$\sum_{p=1}^6 f(\mathbf{A}_j) = 9.5314 \times 10^{-3}$	

The objective function values of the generation 3491 are

$$\left\{ \begin{array}{l} f(\mathbf{A}_1) = 5.2367 \times 10^{-4} \\ f(\mathbf{A}_2) = 7.9910 \times 10^{-5} \\ f(\mathbf{A}_3) = 1.5235 \times 10^{-3} \\ f(\mathbf{A}_4) = 6.9422 \times 10^{-4} \\ f(\mathbf{A}_5) = 6.3094 \times 10^{-3} \\ f(\mathbf{A}_6) = 4.0071 \times 10^{-4} \end{array} \right.$$

As $\min_{1 \text{ to } P} f(\mathbf{A}_p) = f(\mathbf{A}_2) = 7.9910 \times 10^{-5} \leq \varepsilon = 10^{-4}$, the algorithm stops, and sum of fitness has been reduced from initial 9.9548×10^0 to 9.5314×10^{-3} .

The coefficients of the optimal function are $\mathbf{A}_o = [a_0 \ a_1 \ a_2] = [1 \ 0.5 \ 0.25]$, which is equivalent to $[1 \ 0.5 \ 0.25 + 2k]$ or $[1 \ -0.5 \ 0.25 + 2k + 1]$, where k is an integer. This is because

$$\begin{aligned} a_0 - a_1 x \sin(w\pi x + a_2 \pi) &= a_0 - a_1 x \sin(w\pi x + a_2 \pi + 2k \pi) \\ &= -a_1 x \sin(w\pi x + (a_2 + 2k) \pi) \end{aligned}$$

also

$$\begin{aligned}
a_0 - a_1 x \sin(w\pi x + a_2 \pi) &= a_0 - (-a_1)x \sin(w\pi x + a_2 \pi + \pi) \\
&= a_0 - (-a_1)x \sin(w\pi x + (a_2 + 1)\pi) \\
&= a_0 - (-a_1)x \sin(w\pi x + (a_2 + 1)\pi + 2k\pi) \\
&= a_0 - (-a_1)x \sin(w\pi x + (a_2 + 1 + 2k)\pi)
\end{aligned}$$

Thus from the above

$$[a_0 \ a_1 \ a_2] \equiv [a_0 \ a_1 \ a_2 + 2k] \equiv [a_0 \ -a_1 \ a_2 + 2k + 1]$$

It can be seen that at last $\mathbf{A}_1$, $\mathbf{A}_2$, $\mathbf{A}_3$, $\mathbf{A}_4$ and $\mathbf{A}_6$ are approaching the optimal polynomial coefficients. As for $\mathbf{A}_1$, $\mathbf{A}_3$, $\mathbf{A}_4$ and $\mathbf{A}_6$, their solutions are close to

$$[a_0 \ -a_1 \ a_2 + 2k + 1], \text{ where } k = -5$$

For the optimized solution $\mathbf{A}_5 = [1.06784, -0.459238, 5.72162]$, it is close to

$$[a_0 \ -a_1 \ a_2 + 2k + 1], \text{ where } k = 2$$

For the optimized solution $\mathbf{A}_2 = [1.00254, 0.462094, 10.2506]$, it is close to

$$[a_0 \ a_1 \ a_2 + 2k], \text{ where } k = 5$$

Therefore the optimized solution $\mathbf{A}_2$ is equivalent to $[1.00254, 0.462094, 0.2506]$, which is close to the optimal solution $\mathbf{A}_o = [1, 0.5, 0.25]$. The diagram of optimal sine function and optimized function are plotted in Figure P14.2.1.

To evaluate the performance of the optimized function, forty uniformly spread values of x in between any two nearest neighbors of the five fixed values of input x are used to calculate the mean squared error MSE_q between the optimal function $g_{optimal}(x_q)$ and the optimized function $g_{optimized}(x_q)$ as

$$\begin{aligned}
MSE_q &= \sum_{q=1}^{40} (g_{optimal}(x_q) - g_{optimized}(x_q))^2 / 40 \\
&= 7.5474 \times 10^{-5}
\end{aligned}$$

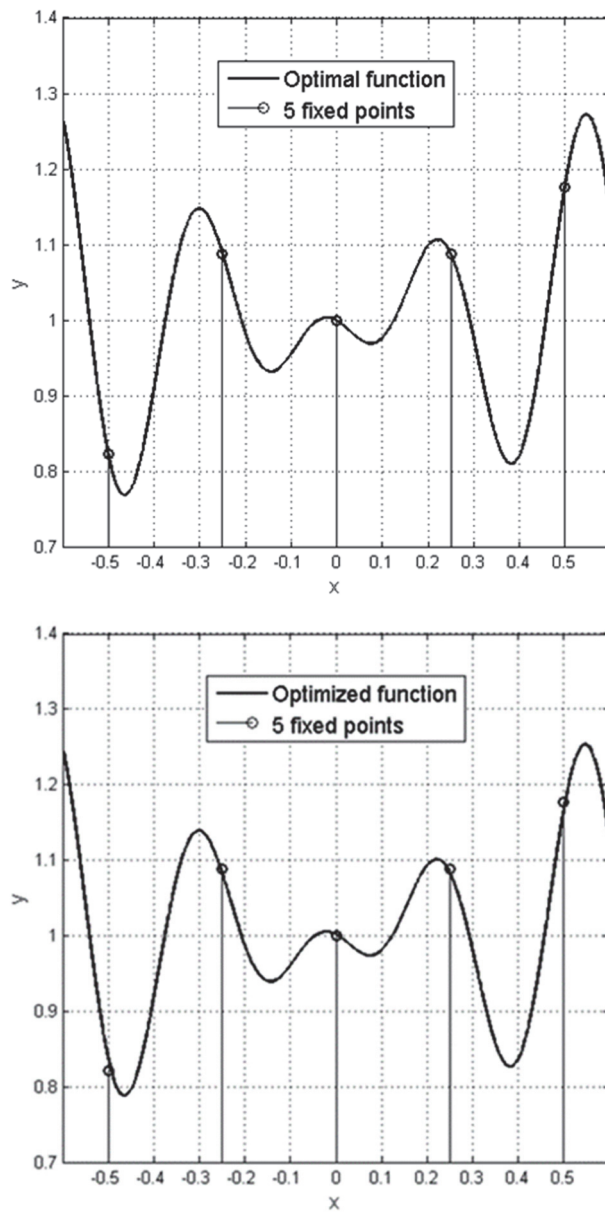


Figure P14.2.1 Optimal sine function (Top); Optimized sine function (Bottom).

The MSE_q value obtained indicates a satisfactory optimized solution for the interpolation problem has been obtained.

Depending on the requirements of an application, a smaller or larger value of the convergence criterion δ could be used.

Chapter 15

Particle Swarm Optimization

Problem 15.1 (Second-order polynomial function interpolation using PSO)

Consider the use of the PSO to determine the continuous coefficients of a second-order polynomial function $y(x) = x^2 + 4x - 5$ to fit N number of input and output data pairs.

For simplicity, a population size of four (4) is taken and the calculation steps of three generations plus the last generation are shown.

The error function is taken as the mean squared error difference between the output of the target function and the output from each particle.

The fitness function is defined by the inverse of the sum of the error function and a small constant. c_1 and c_2 are chosen to be 1 and the term ‘rand’ represents a random value between 0 and 1.

The local best (LBest) ρ_{pd} of a particle is taken as the best of the particle among the present and all the past generations.

Due to the small size of the population, the neighbourhood best (GBest) ρ_{gd} is taken as the global best which is the best of all particles among the present and all the past generations.

The particle velocity and position updates within each generation are carried out by implementing the following two equations in which $x_{pd}(k)$ denotes the position and $v_{pd}(k)$ denotes velocity of the parameter d of a particle p at time (or generation) k .

$$v_{pd}(k) = v_{pd}(k-1) + \varphi_1 (\rho_{pd} - x_{pd}(k-1)) \\ + \varphi_2 (\rho_{gd} - x_{pd}(k-1))$$

$$x_{pd}(k) = x_{pd}(k-1) + v_{pd}(k)$$

where $\varphi_1 = c_1 \times rand_p()$; and $\varphi_2 = c_2 \times rand_p()$

Solution

For generation k , the error function of a particle p is defined as the mean squared error (MSE) $error_p(k)$ between the optimal output $d_k(\mathbf{X})$ and the present output $f_{p,k}(\mathbf{X})$ due to an input vector, $\mathbf{X} (= [x_1, x_2, \dots x_N])$ such that

$$error_p(k) = \sum_{n=1}^N (d_k(x_n) - f_{p,k}(x_n))^2 / N$$

If the input $\mathbf{X}$ is kept the same in all generations, the optimal output $d_k(\mathbf{X})$ remains unchanged and let denote it as $d(\mathbf{X})$; the output mean squared error becomes

$$error_p(k) = \sum_{n=1}^N (d(x_n) - f_{p,k}(x_n))^2 / N$$

The particle's fitness function is defined as the inverse of the mean squared error plus a small positive constant to avoid a zero-valued denominator. Hence the fitness function $F_p(k)$ becomes

$$F_p(k) = \frac{1}{\varepsilon + error_p(k)}$$

During the generation k , the best among all the particles GBest is defined as the particle in record that has the highest fitness or the lowest mean squared error as

$$GBest(k) = \max_{F_p} \{particle\} = \min_{error_p} \{particle\}$$

Define a small positive constant δ such that if $error_{GBest}(k) \leq \delta$, then the algorithm stops, and it could be assumed that a satisfying solution has been obtained in the generation k .

Initialization

$$c_1 = 1$$

$$c_2 = 1$$

$$\varepsilon = 0.01$$

$$\text{Convergence criterion: } \delta = 10^{-4}$$

$$\text{Target function: } y(x) = x^2 + 4x - 5$$

$$\text{Number of particles (population size), } P = 4$$

$$\text{Number of parameters in a particle, } D = 3$$

$$\text{Initializing GBest} = 0$$

$$\text{Initializing GBest fitness} = 0$$

$$\text{Number of fixed inputs, } N = 3$$

At the start, a set of three ($N = 3$) randomly selected fixed inputs $[x_1, x_2, x_3] = [-0.5, 0.1, 0.4]$ are to be used in all generations.

Therefore, the corresponding three desired outputs $[d(x_1), d(x_2), d(x_3)]$ are

$$\begin{cases} d(-0.5) = 1 \cdot (-0.5)^2 + 4 \cdot (-0.5) - 5 = -6.75 \\ d(0.1) = 1 \cdot (0.1)^2 + 4 \cdot (0.1) - 5 = -4.59 \\ d(0.4) = 1 \cdot (0.4)^2 + 4 \cdot (0.4) - 5 = -3.24 \end{cases}$$

In other words, the optimal outputs for all the particles are $[-6.75, -4.59, -3.24]$ under fixed inputs $[-0.5, 0.1, 0.4]$.

$$\text{Initializing LBest} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

$$\text{Initializing LBest fitness} = [0 \quad 0 \quad 0 \quad 0]^T$$

$$\text{Initializing random population} = \begin{bmatrix} -0.26 & 0.48 & 0.47 \\ 0.98 & -0.51 & 0.09 \\ 0.27 & 0.62 & 0.13 \\ -0.10 & -0.08 & 0.77 \end{bmatrix}$$

Initializing velocity (for each particle in the population)

$$= [0 \quad 0 \quad 0 \quad 0]^T$$

Generation 1

Particle 1 output:

$$\begin{cases} f_{1,1}(-0.5) = -0.26 \cdot (-0.5)^2 + 0.48 \cdot (-0.5) + 0.47 = 0.165 \\ f_{1,1}(0.1) = -0.26 \cdot (0.1)^2 + 0.48 \cdot (0.1) + 0.47 = 0.5154 \\ f_{1,1}(0.4) = -0.26 \cdot (0.4)^2 + 0.48 \cdot (0.4) + 0.47 = 0.6204 \end{cases}$$

In vector format: $f_{1,1}(\mathbf{X}) = [0.165 \quad 0.5154 \quad 0.6204]$

Particle 1 error:

$$\begin{aligned} error_1(1) &= \sum_{n=1}^3 (d(x_n) - f_{1,1}(x_n))^2 / 3 \\ &= \frac{1}{3} [(-6.75 - 0.165)^2 + (-4.59 - 0.5154)^2 \\ &\quad + (-3.24 - 0.6204)^2] = 29.595 \end{aligned}$$

Particle 1 fitness:

$$F_1(1) = \frac{1}{\varepsilon + error_1(k)} = \frac{1}{0.01 + error_1(1)} = 0.03377$$

Similarly,

Particle 2 output: $f_{2,1}(\mathbf{X}) = [0.59 \quad 0.0488 \quad 0.0428]$

Particle 2 error:

$$error_2(1) = \sum_{n=1}^3 (d(x_n) - f_{2,1}(x_n))^2 / 3 = 28.7236$$

Particle 2 fitness: $F_2(1) = 1/(\varepsilon + error_2(1)) = 0.03480$

Particle 3 output: $f_{3,1}(\mathbf{X}) = [-0.1125 \ 0.1947 \ 0.4212]$

Particle 3 error

$$error_3(1) = \sum_{n=1}^3 (d(x_n) - f_{3,1}(x_n))^2 / 3 = 26.7847$$

Particle 3 fitness: $F_3(1) = 1/(\varepsilon + error_3(1)) = 0.03732$

Particle 4 output: $f_{4,1}(\mathbf{X}) = [0.785 \ 0.761 \ 0.722]$

Particle 4 error

$$error_4(1) = \sum_{n=1}^3 (d(x_n) - f_{4,1}(x_n))^2 / 3 = 33.7023$$

Particle 4 fitness: $F_4(1) = 1/(\varepsilon + error_4(1)) = 0.02966$

As this is the first generation, the LBest is the current population and the fitness is the current fitness.

$$\text{LBest} = \begin{bmatrix} -0.26 & 0.48 & 0.47 \\ 0.98 & -0.51 & 0.09 \\ 0.27 & 0.62 & 0.13 \\ -0.10 & -0.08 & 0.77 \end{bmatrix}$$

$$\text{LBest fitness} = [0.03377 \ 0.03480 \ 0.03732 \ 0.02966]^T$$

Among the 4 fitness values, the particle 3 has the highest fitness value 0.03732. Consequently, GBest fitness is obtained by particle 3.

$$\text{GBest particle} = [0.27 \ 0.62 \ 0.13]$$

$$\text{GBest fitness} = 0.03732$$

With GBest, $\min error_{1 \text{ to } P}(1) = error_3(1) = 26.7847 > \delta = 1 \times 10^{-4}$, so particle updates need to be continued and more generations are required.

Particle velocity update:

$$\begin{aligned}
 \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix} &= \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} + 1 \times 0.22 \\
 &\times \left(\begin{bmatrix} -0.26 & 0.48 & 0.47 \\ 0.98 & -0.51 & 0.09 \\ 0.27 & 0.62 & 0.13 \\ -0.10 & -0.08 & 0.77 \end{bmatrix} - \begin{bmatrix} -0.26 & 0.48 & 0.47 \\ 0.98 & -0.51 & 0.09 \\ 0.27 & 0.62 & 0.13 \\ -0.10 & -0.08 & 0.77 \end{bmatrix} \right) \\
 &+ 1 \times 0.61 \times \left(\begin{bmatrix} 0.27 & 0.62 & 0.13 \\ 0.27 & 0.62 & 0.13 \\ 0.27 & 0.62 & 0.13 \\ 0.27 & 0.62 & 0.13 \end{bmatrix} - \begin{bmatrix} -0.26 & 0.48 & 0.47 \\ 0.98 & -0.51 & 0.09 \\ 0.27 & 0.62 & 0.13 \\ -0.10 & -0.08 & 0.77 \end{bmatrix} \right)
 \end{aligned}$$

Hence,

$$\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix} = \begin{bmatrix} 0.32461 & 0.08575 & -0.20824 \\ -0.43485 & 0.69208 & 0.024499 \\ 0 & 0 & 0 \\ 0.22661 & 0.42872 & -0.39198 \end{bmatrix}$$

Particle position update:

$$\begin{aligned}
 \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} &= \begin{bmatrix} -0.26 & 0.48 & 0.47 \\ 0.98 & -0.51 & 0.09 \\ 0.27 & 0.62 & 0.13 \\ -0.10 & -0.08 & 0.77 \end{bmatrix} \\
 &+ \begin{bmatrix} 0.32461 & 0.08575 & -0.20824 \\ -0.43485 & 0.69208 & 0.024499 \\ 0 & 0 & 0 \\ 0.22661 & 0.42872 & -0.39198 \end{bmatrix} \\
 &= \begin{bmatrix} 0.06461 & 0.56574 & 0.26176 \\ 0.54515 & 0.18208 & 0.1145 \\ 0.27 & 0.62 & 0.13 \\ 0.12661 & 0.34872 & 0.37802 \end{bmatrix}
 \end{aligned}$$

Generation 2

From generation 1, a new population is produced. So the new particle positions are

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 0.06461 & 0.56574 & 0.26176 \\ 0.54515 & 0.18208 & 0.1145 \\ 0.27 & 0.62 & 0.13 \\ 0.12661 & 0.34872 & 0.37802 \end{bmatrix}$$

Particle 1 output: $f_{1,2}(\mathbf{X}) = [-0.00496 \ 0.31898 \ 0.49840]$

Particle 1 error:

$$error_1(2) = \sum_{n=1}^3 (d(x_n) - f_{1,2}(x_n))^2 / 3 = 27.8564$$

Particle 1 fitness: $F_1(2) = 1/(\varepsilon + error_1(2)) = 0.035886$

Particle 2 output: $f_{2,2}(\mathbf{X}) = [0.15974 \ 0.13816 \ 0.27456]$

Particle 2 error

$$error_2(2) = \sum_{n=1}^3 (d(x_n) - f_{2,2}(x_n))^2 / 3 = 27.48400$$

Particle 2 fitness: $F_2(2) = 1/(\varepsilon + error_2(2)) = 0.036372$

Particle 3 output: $f_{3,2}(\mathbf{X}) = [-0.1125 \ 0.1947 \ 0.4212]$

Particle 3 error:

$$error_3(2) = \sum_{n=1}^3 (d(x_n) - f_{3,2}(x_n))^2 / 3 = 26.78467$$

Particle 3 fitness: $F_3(2) = 1/(\varepsilon + error_3(2)) = 0.037321$

Particle 4 output: $f_{4,2}(\mathbf{X}) = [0.23531 \ 0.41416 \ 0.53777]$

Particle 4 error:

$$error_4(2) = \sum_{n=1}^3 (d(x_n) - f_{4,2}(x_n))^2 / 3 = 29.36933$$

$$\text{Particle 4 fitness: } F_4(2) = 1/(\varepsilon + error_4(2)) = 0.034038$$

$$Lbest = \begin{bmatrix} 0.06461 & 0.56574 & 0.26176 \\ 0.54515 & 0.18208 & 0.11450 \\ 0.27 & 0.62 & 0.13 \\ 0.12661 & 0.34872 & 0.37802 \end{bmatrix}$$

$$LBest \text{ fitness} = \begin{bmatrix} 0.035886 \\ 0.036372 \\ 0.037321 \\ 0.034038 \end{bmatrix}$$

Among the fitness values, the particle 3 has again the highest fitness value among all the generations. Therefore, the GBest fitness is equal to the particle 3's LBest fitness.

$$GBest \text{ particle} = [0.27 \ 0.62 \ 0.13]$$

$$GBest \text{ fitness} = 0.037321$$

With GBest, $\min error_{1 \text{ to } P}(2) = error_3(2) = 26.78467 > \delta = 1 \times 10^{-4}$, so particle updates need to be continued and more generations are required.

Particle velocity update:

$$\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix} = \begin{bmatrix} 0.32461 & 0.08575 & -0.20824 \\ -0.43485 & 0.69208 & 0.024499 \\ 0 & 0 & 0 \\ 0.22661 & 0.42872 & -0.39198 \end{bmatrix} + 1 \times 0.22 \times \left(\begin{bmatrix} 0.06461 & 0.56574 & 0.26176 \\ 0.54515 & 0.18208 & 0.11450 \\ 0.27 & 0.62 & 0.13 \\ 0.12661 & 0.34872 & 0.37802 \end{bmatrix} - \begin{bmatrix} 0.06461 & 0.56574 & 0.26176 \\ 0.54515 & 0.18208 & 0.1145 \\ 0.27 & 0.62 & 0.13 \\ 0.12661 & 0.34872 & 0.37802 \end{bmatrix} \right)$$

$$+1 \times 0.76 \times \left(\begin{bmatrix} 0.27 & 0.62 & 0.13 \\ 0.27 & 0.62 & 0.13 \\ 0.27 & 0.62 & 0.13 \\ 0.27 & 0.62 & 0.13 \end{bmatrix} - \begin{bmatrix} 0.06461 & 0.56574 & 0.26176 \\ 0.54515 & 0.18208 & 0.1145 \\ 0.27 & 0.62 & 0.13 \\ 0.12661 & 0.34872 & 0.37802 \end{bmatrix} \right)$$

Hence,

$$\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix} = \begin{bmatrix} 0.48170 & 0.12724 & -0.30902 \\ -0.6453 & 1.027 & 0.036355 \\ 0 & 0 & 0 \\ 0.33628 & 0.63621 & -0.58168 \end{bmatrix}$$

Particle position update:

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 0.06461 & 0.56574 & 0.26176 \\ 0.54515 & 0.18208 & 0.1145 \\ 0.27 & 0.62 & 0.13 \\ 0.12661 & 0.34872 & 0.37802 \end{bmatrix} + \begin{bmatrix} 0.48170 & 0.12724 & -0.30902 \\ -0.6453 & 1.027 & 0.036355 \\ 0 & 0 & 0 \\ 0.33628 & 0.63621 & -0.58168 \end{bmatrix} = \begin{bmatrix} 0.54631 & 0.69299 & -0.04725 \\ -0.10015 & 1.2091 & 0.15085 \\ 0.27 & 0.62 & 0.13 \\ 0.46289 & 0.98493 & -0.20365 \end{bmatrix}$$

Generation 3

The new particle positions at the start of generation 3 are

$$\begin{bmatrix} 0.54631 & 0.69299 & -0.04725 \\ -0.10015 & 1.2091 & 0.15085 \\ 0.27 & 0.62 & 0.13 \\ 0.46289 & 0.98493 & -0.20365 \end{bmatrix}$$

Particle 1 output: $f_{1,3}(\mathbf{X}) = [-0.25717 \ 0.027508 \ 0.31735]$

Particle 1 error:

$$error_1(3) = \sum_{n=1}^3 (d(x_n) - f_{1,3}(x_n))^2 / 3 = 25.37760$$

Particle 1 fitness: $F_1(3) = 1/(\varepsilon + error_1(3)) = 0.039389$

Particle 2 output: $f_{2,3}(\mathbf{X}) = [-0.47874 \ 0.27076 \ 0.61847]$

Particle 2 error

$$error_2(3) = \sum_{n=1}^3 (d(x_n) - f_{2,3}(x_n))^2 / 3 = 25.94787$$

Particle 2 fitness: $F_2(3) = 1/(\varepsilon + error_2(3)) = 0.038523$

Particle 3 output: $f_{3,3}(\mathbf{X}) = [-0.1125 \ 0.1947 \ 0.4212]$

Particle 3 error:

$$error_3(3) = \sum_{n=1}^3 (d(x_n) - f_{3,3}(x_n))^2 / 3 = 26.78467$$

Particle 3 fitness: $F_3(3) = 1/(\varepsilon + error_3(3)) = 0.037321$

Particle 4 output: $f_{4,3}(\mathbf{X}) = [-0.5804 \ -0.10053 \ 0.26438]$

Particle 4 error:

$$error_4(3) = \sum_{n=1}^3 (d(x_n) - f_{4,3}(x_n))^2 / 3 = 23.50000$$

Particle 4 fitness: $F_4(3) = 1/(\varepsilon + error_4(3)) = 0.042535$

$$Lbest = \begin{bmatrix} 0.54631 & 0.69299 & -0.04725 \\ -0.10015 & 1.2091 & 0.15085 \\ 0.27 & 0.62 & 0.13 \\ 0.46289 & 0.98493 & -0.20365 \end{bmatrix}$$

$$\text{LBest fitness} = \begin{bmatrix} 0.039389 \\ 0.038523 \\ 0.037321 \\ 0.042535 \end{bmatrix}$$

At generation 3, the particle 4 has the highest fitness value among all the generations. Therefore, the GBest fitness is equal to the particle 4's fitness.

$$\text{GBest particle} = [0.46289 \quad 0.98493 \quad -0.20365]$$

$$\text{GBest fitness} = 0.042535$$

With GBest, $\min error_{1 \text{ to } P}(3) = error_4(3) = 23.50000 > \delta = 1 \times 10^{-4}$, so particle updates need to be continued and more generations are required.

Particle velocity update:

$$\begin{aligned} \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix} &= \begin{bmatrix} 0.48170 & 0.12724 & -0.30902 \\ -0.6453 & 1.027 & 0.036355 \\ 0 & 0 & 0 \\ 0.33628 & 0.63621 & -0.58168 \end{bmatrix} \\ &+ 1 \times 0.73 \times \left(\begin{bmatrix} 0.54631 & 0.69299 & -0.04725 \\ -0.10015 & 1.2091 & 0.15085 \\ 0.27 & 0.62 & 0.13 \\ 0.46289 & 0.98493 & -0.20365 \end{bmatrix} \right. \\ &\quad \left. - \begin{bmatrix} 0.54631 & 0.69299 & -0.04725 \\ -0.10015 & 1.2091 & 0.15085 \\ 0.27 & 0.62 & 0.13 \\ 0.46289 & 0.98493 & -0.20365 \end{bmatrix} \right) \\ &+ 1 \times 0.68 \times \left(\begin{bmatrix} 0.46289 & 0.98493 & -0.20365 \\ 0.46289 & 0.98493 & -0.20365 \\ 0.46289 & 0.98493 & -0.20365 \\ 0.46289 & 0.98493 & -0.20365 \end{bmatrix} \right. \\ &\quad \left. - \begin{bmatrix} 0.54631 & 0.69299 & -0.04725 \\ -0.10015 & 1.2091 & 0.15085 \\ 0.27 & 0.62 & 0.13 \\ 0.46289 & 0.98493 & -0.20365 \end{bmatrix} \right) \end{aligned}$$

Hence,

$$\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix} = \begin{bmatrix} 0.42525 & 0.32484 & -0.41487 \\ -0.26422 & 0.87530 & -0.20358 \\ 0.13055 & 0.24699 & -0.22582 \\ 0.33628 & 0.63621 & -0.58168 \end{bmatrix}$$

Particle position update:

$$\begin{aligned} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} &= \begin{bmatrix} 0.54631 & 0.69299 & -0.04725 \\ -0.10015 & 1.2091 & 0.15085 \\ 0.27 & 0.62 & 0.13 \\ 0.46289 & 0.98493 & -0.20365 \end{bmatrix} \\ &+ \begin{bmatrix} 0.42525 & 0.32484 & -0.41487 \\ -0.26422 & 0.87530 & -0.20358 \\ 0.13055 & 0.24699 & -0.22582 \\ 0.33628 & 0.63621 & -0.58168 \end{bmatrix} \\ &= \begin{bmatrix} 0.97155 & 1.0178 & -0.46212 \\ -0.36437 & 2.0844 & 0.052728 \\ 0.40055 & 0.8670 & -0.09582 \\ 0.79918 & 1.6211 & -0.78533 \end{bmatrix} \end{aligned}$$

This is the end of generation 3. As the optimization goes on, the particle positions evolve and approach the optimal solution. The PSO algorithm continues until generation 979.

...

Generation 979 (PSO converged)

The new particle positions at the start of generation 979 are

$$\begin{bmatrix} 1.0448 & 4.0008 & -5.0021 \\ 1.0338 & 4.0012 & -5.0000 \\ 1.0345 & 3.9955 & -5.0004 \\ 1.0412 & 3.9958 & -5.0018 \end{bmatrix}$$

Particle 1 output, $f_{1,979}(\mathbf{X}) = [-6.7414, -4.5916, -3.2346]$

Particle 1 error:

$$error_1(979) = \sum_{n=1}^3 (d(x_n) - f_{1,979}(x_n))^2 / 3 = 3.53520 \times 10^{-5}$$

Particle 1 fitness: $F_1(979) = 1/(\varepsilon + error_1(979)) = 99.6477$

Particle 2 output, $f_{2,979}(\mathbf{X}) = [-6.7422 \quad -4.5896 \quad -3.2341]$

Particle 2 error:

$$error_2(979) = \sum_{n=1}^3 (d(x_n) - f_{2,979}(x_n))^2 / 3 = 3.1828 \times 10^{-5}$$

Particle 2 fitness: $F_2(979) = 1/(\varepsilon + error_2(979)) = 99.6827$

Particle 3 output: $f_{3,979}(\mathbf{X}) = [-6.7396 \quad -4.5905 \quad -3.2367]$

Particle 3 error:

$$error_3(979) = \sum_{n=1}^3 (d(x_n) - f_{3,979}(x_n))^2 / 3 = 4.0131 \times 10^{-5}$$

Particle 3 fitness: $F_3(979) = 1/(\varepsilon + error_3(979)) = 99.6003$

Particle 4 output: $f_{4,979}(\mathbf{X}) = [-6.7395 \quad -4.5919 \quad -3.2369]$

Particle 4 error:

$$error_4(979) = \sum_{n=1}^3 (d(x_n) - f_{4,979}(x_n))^2 / 3 = 4.1360 \times 10^{-5}$$

Particle 4 fitness: $F_4(979) = 1/(\varepsilon + error_4(979)) = 99.5881$

$$Lbest = \begin{bmatrix} 1.0448 & 4.0008 & -5.0021 \\ 1.0338 & 4.0012 & -5.0000 \\ 1.0345 & 3.9955 & -5.0004 \\ 1.0412 & 3.9958 & -5.0018 \end{bmatrix}$$

$$\text{LBest fitness} = \begin{bmatrix} 99.6477 \\ 99.6827 \\ 99.6003 \\ 99.5881 \end{bmatrix}$$

At generation 979, the particle 2 has again the highest fitness value among all the generations. Therefore, the GBest fitness is equal to the particle 2's fitness.

$$\text{GBest particle} = [1.0338 \quad 4.0012 \quad -5.0000]$$

$$\text{GBest fitness} = 99.6827$$

With GBest, $\min error_{1 \text{ to } P}(979) = error_2(979) = 3.1828 \times 10^{-5} < \delta = 1 \times 10^{-4}$, convergence criterion has been met, no more generation is required.

Thus the optimized polynomial coefficients are $[1.0338 \quad 4.0012 \quad -5]$. The convergence property of the PSO can be observed in Figure P15.1.1 as the optimized polynomial function gradually approaches the optimal polynomial function.

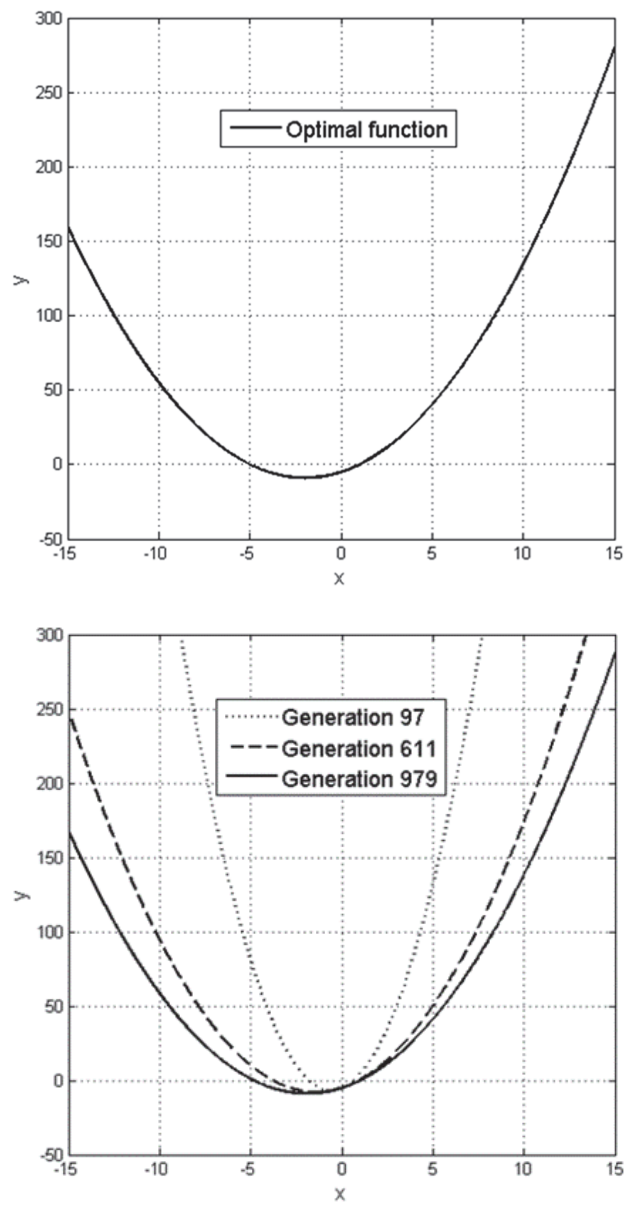


Figure P15.1.1 Optimal polynomial function (Top). Optimized polynomial function at generations 97, 611, and 979 (Bottom).

Problem 15.2 (Third-order polynomial function interpolation using PSO)

Consider the use of the PSO to optimize the coefficients of a third-order polynomial function: $y(x) = a_3x^3 + a_2x^2 + a_1x + a_0$ to fit N number of input and output data pairs.

For simplicity, a population size of six (6) is taken and the calculation steps of three (3) generations plus the last generation are shown.

The error function is taken as the mean squared error difference between the output of the target function and the output from each particle.

The fitness function is defined by the inverse of the sum of the error function plus a small positive constant. c_1 and c_2 are chosen to be 1 and the term ‘rand’ represents a random value between 0 and 1.

The local best (LBest) ρ_{pd} of a particle is taken as the best of the particle among the present and all the past generations.

Due to the small size of the population, the neighbourhood best (GBest) ρ_{gd} is taken as the global best which is the best of all particles among the present and all the past generations.

The particle velocity and position updates within each generation are carried out by implementing the following two equations in which $x_{pd}(k)$ denotes the position and $v_{pd}(k)$ denotes velocity of the parameter d of a particle p at time (or generation) k .

$$v_{pd}(k) = v_{pd}(k-1) + \varphi_1 (\rho_{pd} - x_{pd}(k-1)) + \varphi_2 (\rho_{gd} - x_{pd}(k-1))$$

$$x_{pd}(k) = x_{pd}(k-1) + v_{pd}(k)$$

where $\varphi_1 = c_1 \times rand_p()$; and $\varphi_2 = c_2 \times rand_p()$

Solution

For generation k , the error function of a particle p is defined as the mean

squared error (MSE) $error_p(k)$ between the desired output $d_k(\mathbf{X})$ and the present output $f_{p,k}(\mathbf{X})$ due to an input vector, $\mathbf{X} (= [x_1, x_2, \dots, x_N])$ such that

$$error_p(k) = \sum_{n=1}^N (d_k(x_n) - f_{p,k}(x_n))^2 / N$$

If the input $\mathbf{X}$ is kept the same in all generations, the optimal output $d_k(\mathbf{X})$ remains unchanged and let denote it as $d(\mathbf{X})$; the output mean squared error becomes

$$error_p(k) = \sum_{n=1}^N (d(x_n) - f_{p,k}(x_n))^2 / N$$

The particle's fitness function is defined as the inverse of the sum of mean squared error and a small positive constant to avoid a zero-valued denominator. Hence the fitness function $F_p(k)$ becomes

$$F_p(k) = \frac{1}{\varepsilon + error_p(k)}$$

During the generation k , the best among all the particles $GBest$ is defined as the particle in record that has the highest fitness or the lowest mean squared error as

$$GBest(k) = \max_{F_p} \{particle\} = \min_{error_p} \{particle\}$$

Define a small positive constant δ such that if $error_{GBest}(k) \leq \delta$, then the algorithm stops, and it could be assumed that a satisfying solution has been obtained in the generation k .

Initialization

$$c_1 = 1$$

$$c_2 = 1$$

$$\varepsilon = 0.01$$

Convergence criterion: $\delta = 10^{-4}$

Target function: $y(x) = 10x^3 - 3x^2 - 2.4x + 1$

Number of particles (population size), $P = 6$

Number of parameters in a particle, $D = 4$

Initializing GBest = 0

Initializing GBest fitness = 0

Number of fixed inputs, $N = 5$

At the start, a set of five ($N = 5$) uniformly spaced fixed inputs $[x_1, x_2, x_3, x_4, x_5] = [0.5, 0.25, 0, -0.25, -0.5]$ are to be used in all generations. The corresponding five desired outputs $[d(x_1), d(x_2), d(x_3), d(x_4), d(x_5)]$ are

$$\begin{cases} d(0.5) = 10 \cdot 0.5^3 - 3 \cdot 0.5^2 - 2.4 \cdot 0.5 + 1 = 0.30000 \\ d(0.25) = 10 \cdot 0.25^3 - 3 \cdot 0.25^2 - 2.4 \cdot 0.25 + 1 = 0.36875 \\ d(0) = 10 \cdot 0^3 - 3 \cdot 0^2 - 2.4 \cdot 0 + 1 = 1.0000 \\ d(-0.25) = 10(-0.25)^3 - 3(-0.25)^2 - 2.4(-0.25) + 1 = 1.2563 \\ d(-0.5) = 10(-0.5)^3 - 3(-0.5)^2 - 2.4(-0.5) + 1 = 0.2000 \end{cases}$$

In other words, the optimal outputs for all the particles are $[0.30, 0.36875, 1.0000, 1.2563, 0.2000]$ under fixed inputs $[0.5, 0.25, 0, -0.25, -0.5]$.

$$\text{Initializing LBest (6} \times \text{4 matrix)} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\text{Initializing LBest fitness} = [0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0]^T$$

Initializing random population

$$= \begin{bmatrix} 0.69 & -0.38 & 0.03 & -0.77 \\ 0.23 & -0.38 & -0.1 & -0.22 \\ 0.57 & 0.84 & 0.66 & 0.03 \\ 0.25 & -0.89 & -0.06 & 0.40 \\ -0.77 & 0.39 & 0.45 & -0.92 \\ -0.54 & 0.00 & -0.20 & -0.55 \end{bmatrix}$$

Initializing velocity (for each particle in the population)

$$= [0 \ 0 \ 0 \ 0 \ 0 \ 0]^T$$

Generation 1

Particle 1 output:

$$\begin{cases} f_{1,1}(0.5) = 0.69 + (-0.38) \cdot 0.5 + 0.03 \cdot 0.5^2 + (-0.77) \cdot 0.5^3 = 0.41125 \\ f_{1,1}(0.25) = 0.69 + (-0.38) \cdot 0.25 + 0.03 \cdot 0.25^2 + (-0.77) \cdot 0.25^3 = 0.58484 \\ f_{1,1}(0) = 0.69 + (-0.38) \cdot 0 + 0.03 \cdot 0^2 + (-0.77) \cdot 0^3 = 0.69 \\ f_{1,1}(-0.25) = 0.69 + (-0.38)(-0.25) + 0.03(-0.25)^2 + (-0.77)(-0.25)^3 = 0.79891 \\ f_{1,1}(-0.5) = 0.69 + (-0.38)(-0.5) + 0.03(-0.5)^2 + (-0.77)(-0.5)^3 = 0.98375 \end{cases}$$

In vector format: $f_{1,1}(\mathbf{x}) = [0.41125, 0.58484, 0.69, 0.79891, 0.98375]$

Particle 1 error:

$$\begin{aligned} error_1(1) &= \sum_{n=1}^5 (d(x_n) - f_{1,1}(x_n))^2 / 5 \\ &= \frac{1}{5} [(0.300 - 0.411)^2 + (0.3687 - 0.5848)^2 + \dots \\ &\quad + (0.200 - 0.9838)^2] = 0.19572 \end{aligned}$$

Particle 1 fitness:

$$F_1(1) = \frac{1}{\varepsilon + error_p(k)} = \frac{1}{0.01 + error_1(1)} = 4.861$$

Particle 2 output:

$$f_{2,1}(\mathbf{X}) = [-0.0125, 0.12531, 0.23, 0.32219, 0.4225]$$

Particle 2 error

$$error_2(1) = \sum_{n=1}^5 (d(x_n) - f_{2,1}(x_n))^2 / 5 = 0.33436$$

$$\text{Particle 2 fitness: } F_2(1) = 1/(\varepsilon + error_2(1)) = 2.9039$$

Particle 3 output:

$$f_{3,1}(\mathbf{X}) = [1.1587, 0.82172, 0.57, 0.40078, 0.31125]$$

Particle 3 error

$$error_3(1) = \sum_{n=1}^5 (d(x_n) - f_{3,1}(x_n))^2 / 5 = 0.37435$$

$$\text{Particle 3 fitness: } F_3(1) = 1/(\varepsilon + error_3(1)) = 2.6018$$

$$\text{Particle 4 output: } f_{4,1}(\mathbf{X}) = [-0.16, 0.03, 0.25, 0.4625, 0.63]$$

Particle 4 error

$$error_4(1) = \sum_{n=1}^5 (d(x_n) - f_{4,1}(x_n))^2 / 5 = 0.34076$$

$$\text{Particle 4 fitness: } F_4(1) = 1/(\varepsilon + error_4(1)) = 2.851$$

Particle 5 output:

$$f_{5,1}(\mathbf{X}) = [-0.5775, -0.65875, -0.77, -0.825, -0.7375]$$

Particle 5 error:

$$error_5(1) = \sum_{n=1}^5 (d(x_n) - f_{5,1}(x_n))^2 / 5 = 2.0338$$

$$\text{Particle 5 fitness: } F_5(1) = 1/(\varepsilon + error_5(1)) = 0.48928$$

Particle 6 output:

$$f_{6,1}(\mathbf{X}) = [-0.65875, -0.56109, -0.54, -0.54391, -0.52125]$$

Particle 6 error:

$$error_6(1) = \sum_{n=1}^5 (d(x_n) - f_{6,1}(x_n))^2 / 5 = 1.5832$$

$$\text{Particle 6 fitness: } F_6(1) = 1/(\varepsilon + error_6(1)) = 0.62765$$

As this is the first generation, the LBest is the current population and the fitness is the current fitness.

$$\text{LBest} = \begin{bmatrix} 0.69 & -0.38 & 0.03 & -0.77 \\ 0.23 & -0.38 & -0.1 & -0.22 \\ 0.57 & 0.84 & 0.66 & 0.03 \\ 0.25 & -0.89 & -0.06 & 0.40 \\ -0.77 & 0.39 & 0.45 & -0.92 \\ -0.54 & 0.00 & -0.20 & -0.55 \end{bmatrix}$$

LBest fitness

$$= [4.861 \quad 2.9039 \quad 2.6018 \quad 2.851 \quad 0.48928 \quad 0.62765]^T$$

Among the 4 fitness values, the particle 1 has the highest fitness value: 4.861. Consequently, GBest fitness is obtained by particle 1.

$$\text{GBest particle} = [0.69, -0.38, 0.03, -0.77]$$

$$\text{GBest fitness} = 4.861$$

With GBest, $\min error_{1 \text{ to } P}(1) = error_1(1) = 0.19572 > \delta = 1 \times 10^{-4}$, so particle updates need to be continued and more generations are required.

Particle velocity update:

$$\begin{aligned}
 \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{bmatrix} &= \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \\
 &+ 1 \times 0.82973 \times \left(\begin{bmatrix} 0.69 & -0.38 & \dots & -0.77 \\ 0.23 & -0.38 & \dots & -0.22 \\ \vdots & & \ddots & \vdots \\ -0.54 & 0.00 & \dots & -0.55 \end{bmatrix} \right. \\
 &\quad \left. - \begin{bmatrix} 0.69 & -0.38 & \dots & -0.77 \\ 0.23 & -0.38 & \dots & -0.22 \\ \vdots & & \ddots & \vdots \\ -0.54 & 0.00 & \dots & -0.55 \end{bmatrix} \right) \\
 &+ 1 \times 0.47303 \times \left(\begin{bmatrix} 0.69 & -0.38 & \dots & -0.77 \\ 0.69 & -0.38 & \dots & -0.77 \\ \vdots & & \ddots & \vdots \\ 0.69 & -0.38 & \dots & -0.77 \end{bmatrix} \right. \\
 &\quad \left. - \begin{bmatrix} 0.69 & -0.38 & \dots & -0.77 \\ 0.23 & -0.38 & \dots & -0.22 \\ \vdots & & \ddots & \vdots \\ -0.54 & 0.00 & \dots & -0.55 \end{bmatrix} \right)
 \end{aligned}$$

Hence,

$$\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0.2175 & 0 & 0.0615 & -0.2602 \\ 0.0568 & -0.5771 & -0.2980 & -0.3784 \\ 0.2081 & 0.2412 & 0.04257 & -0.5534 \\ 0.6906 & -0.3642 & -0.1987 & 0.0710 \\ 0.5818 & -0.1798 & 0.1088 & -0.1041 \end{bmatrix}$$

Particle position update:

$$\begin{aligned}
 \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix} &= \begin{bmatrix} 0.69 & -0.38 & 0.03 & -0.77 \\ 0.23 & -0.38 & -0.1 & -0.22 \\ 0.57 & 0.84 & 0.66 & 0.03 \\ 0.25 & -0.89 & -0.06 & 0.40 \\ -0.77 & 0.39 & 0.45 & -0.92 \\ -0.54 & 0.00 & -0.20 & -0.55 \end{bmatrix} \\
 &+ \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0.2175 & 0 & 0.0615 & -0.2602 \\ 0.0568 & -0.5771 & -0.2980 & -0.3784 \\ 0.2081 & 0.2412 & 0.04257 & -0.5534 \\ 0.6906 & -0.3642 & -0.1987 & 0.0710 \\ 0.5818 & -0.1798 & 0.1088 & -0.1041 \end{bmatrix} \\
 &= \begin{bmatrix} 0.69 & -0.38 & 0.03 & -0.77 \\ 0.4476 & -0.38 & -0.0385 & -0.4802 \\ 0.6268 & 0.2629 & 0.3620 & -0.3484 \\ 0.4581 & -0.649 & -0.0174 & -0.1534 \\ -0.0794 & 0.0258 & 0.2513 & -0.8491 \\ 0.0418 & -0.1798 & -0.0912 & -0.6541 \end{bmatrix}
 \end{aligned}$$

Generation 2

From generation 1, a new population is produced. So the new particle positions are

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix} = \begin{bmatrix} 0.69 & -0.38 & 0.03 & -0.77 \\ 0.4476 & -0.38 & -0.0385 & -0.4802 \\ 0.6268 & 0.2629 & 0.3620 & -0.3484 \\ 0.4581 & -0.649 & -0.0174 & -0.1534 \\ -0.0794 & 0.0258 & 0.2513 & -0.8491 \\ 0.0418 & -0.1798 & -0.0912 & -0.6541 \end{bmatrix}$$

Particle 1 output:

$$f_{1,2}(\mathbf{X}) = [0.41125, 0.58484, 0.69, 0.79891, 0.98375]$$

Particle 1 error:

$$error_1(2) = \sum_{n=1}^5 (d(x_n) - f_{1,2}(x_n))^2 / 5 = 0.19572$$

$$\text{Particle 1 fitness: } F_1(2) = 1/(\varepsilon + error_1(2)) = 4.861$$

Particle 2 output:

$$f_{2,2}(\mathbf{X}) = [0.18795, 0.34268, 0.44759, 0.54769, 0.68799]$$

Particle 2 error:

$$error_2(2) = \sum_{n=1}^5 (d(x_n) - f_{2,2}(n))^2 / 5 = 0.21172$$

$$\text{Particle 2 fitness: } F_2(2) = 1/(\varepsilon + error_2(2)) = 4.5103$$

Particle 3 output:

$$f_{3,2}(\mathbf{X}) = [0.80516, 0.70967, 0.62676, 0.58911, 0.62936]$$

Particle 3 error:

$$error_3(2) = \sum_{n=1}^5 (d(x_n) - f_{3,2}(n))^2 / 5 = 0.22803$$

$$\text{Particle 3 fitness: } F_3(2) = 1/(\varepsilon + error_3(2)) = 4.2011$$

Particle 4 output:

$$f_{4,2}(\mathbf{X}) = [0.11022, 0.29246, 0.45813, 0.62163, 0.79733]$$

Particle 4 error:

$$error_4(2) = \sum_{n=1}^5 (d(x_n) - f_{4,2}(n))^2 / 5 = 0.21900$$

$$\text{Particle 4 fitness: } F_4(2) = 1/(\varepsilon + error_4(2)) = 4.3668$$

Particle 5 output:

$$f_{5,2}(\mathbf{X}) = [-0.1098, -0.07049, -0.07938, -0.05685, 0.0767]$$

Particle 5 error:

$$error_5(2) = \sum_{n=1}^5 (d(x_n) - f_{5,2}(n))^2 / 5 = 0.65307$$

$$\text{Particle 5 fitness: } F_5(2) = 1/(\varepsilon + error_5(2)) = 1.5081$$

Particle 6 output:

$$f_{6,2}(\mathbf{X}) = [-0.1526, -0.01903, 0.041825, 0.091283, 0.19066]$$

Particle 6 error:

$$error_6(2) = \sum_{n=1}^5 (d(x_n) - f_{6,2}(n))^2 / 5 = 0.52611$$

$$\text{Particle 6 fitness: } F_6(2) = 1/(\varepsilon + error_6(2)) = 1.8653$$

$$\text{Lbest} = \begin{bmatrix} 0.69 & -0.38 & 0.03 & -0.77 \\ 0.4476 & -0.38 & -0.0385 & -0.4802 \\ 0.6268 & 0.2629 & 0.3620 & -0.3484 \\ 0.4581 & -0.649 & -0.0174 & -0.1534 \\ -0.0794 & 0.0258 & 0.2513 & -0.8491 \\ 0.0418 & -0.1798 & -0.0912 & -0.6541 \end{bmatrix}$$

$$\text{LBest fitness} = \begin{bmatrix} 4.8610 \\ 4.5103 \\ 4.2011 \\ 4.3668 \\ 1.5081 \\ 1.8653 \end{bmatrix}$$

Among the fitness values, the particle 1 has again the highest fitness value among all the generations. Therefore, the GBest fitness is equal to the particle 1's LBest fitness.

GBest particle [0.69, -0.38, 0.03, -0.77]

GBest fitness = 4.8610

With GBest, $\min error_{1 to P} = error_1(2) = 0.19572 > \delta = 1 \times 10^{-4}$, so particle updates need to be continued and more generations are required.

Particle velocity update:

$$\begin{aligned}
 \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{bmatrix} &= \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0.2175 & 0 & 0.0615 & -0.2602 \\ 0.0568 & -0.5771 & -0.2980 & -0.3784 \\ 0.2081 & 0.2412 & 0.04257 & -0.5534 \\ 0.6906 & -0.3642 & -0.1987 & 0.0710 \\ 0.5818 & -0.1798 & 0.1088 & -0.1041 \end{bmatrix} \\
 &+ 1 \times 0.37639 \times \left(\begin{bmatrix} 0.69 & -0.38 & \dots & -0.77 \\ 0.448 & -0.38 & \dots & -0.48 \\ \vdots & \vdots & \ddots & \vdots \\ 0.0418 & -0.1798 & \dots & -0.654 \end{bmatrix} \right. \\
 &\quad \left. - \begin{bmatrix} 0.69 & -0.38 & \dots & -0.77 \\ 0.448 & -0.38 & \dots & -0.48 \\ \vdots & \vdots & \ddots & \vdots \\ 0.0418 & -0.1798 & \dots & -0.654 \end{bmatrix} \right) \\
 &+ 1 \times 0.42563 \times \left(\begin{bmatrix} 0.69 & -0.38 & \dots & -0.77 \\ 0.69 & -0.38 & \dots & -0.77 \\ \vdots & \vdots & \ddots & \vdots \\ 0.69 & -0.38 & \dots & -0.77 \end{bmatrix} \right. \\
 &\quad \left. - \begin{bmatrix} 0.69 & -0.38 & \dots & -0.77 \\ 0.448 & -0.38 & \dots & -0.48 \\ \vdots & \vdots & \ddots & \vdots \\ 0.0418 & -0.1798 & \dots & -0.654 \end{bmatrix} \right)
 \end{aligned}$$

Hence,

$$\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0.3207 & 0 & 0.0907 & -0.3835 \\ 0.0837 & -0.8507 & -0.4393 & -0.5579 \\ 0.3068 & 0.3556 & 0.06276 & -0.8159 \\ 1.0181 & -0.5369 & -0.2929 & 0.1046 \\ 0.8577 & -0.2650 & 0.1604 & -0.1534 \end{bmatrix}$$

Particle position update:

$$\begin{aligned}
 \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix} &= \begin{bmatrix} 0.69 & -0.38 & \cdots & -0.77 \\ 0.448 & -0.38 & \cdots & -0.48 \\ \vdots & \vdots & \ddots & \vdots \\ 0.0418 & -0.1798 & \cdots & -0.654 \end{bmatrix} \\
 &+ \begin{bmatrix} 0 & 0 & \cdots & 0 \\ 0.3207 & 0 & \cdots & -0.3835 \\ \vdots & \vdots & \ddots & \vdots \\ 0.8577 & -0.2650 & \cdots & -0.1534 \end{bmatrix} \Rightarrow \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix} \\
 &= \begin{bmatrix} 0.69 & -0.38 & 0.03 & -0.77 \\ 0.7684 & -0.38 & 0.0521 & -0.8637 \\ 0.7104 & -0.5878 & -0.0773 & -0.9063 \\ 0.7650 & -0.2931 & 0.04533 & -0.9693 \\ 0.9387 & -0.5112 & -0.04155 & -0.7445 \\ 0.8995 & -0.4447 & 0.0692 & -0.8075 \end{bmatrix}
 \end{aligned}$$

...

Generation 8

The new particle positions at the start of generation 8 are

$$\begin{bmatrix} 0.6682 & -0.3800 & 0.0238 & -0.7440 \\ 0.4658 & -0.3800 & -0.0334 & -0.5019 \\ 0.5848 & 0.4685 & 0.4620 & -0.1876 \\ 0.3622 & -0.7347 & -0.0388 & 0.0698 \\ -0.3472 & 0.1556 & 0.3160 & -0.8483 \\ -0.1873 & -0.1157 & -0.1361 & -0.5910 \end{bmatrix}$$

Particle 1 output:

$$f_{1,8}(\mathbf{X}) = [0.39119, 0.56309, 0.66822, 0.77634, 0.95718]$$

Particle 1 error:

$$error_1(8) = \sum_{n=1}^5 (d(x_n) - f_{1,8}(x_n))^2 / 5 = 0.19196$$

Particle 1 fitness: $F_1(8) = 1/(\varepsilon + error_1(8)) = 4.9515$

Particle 2 output:

$$f_{2,8}(\mathbf{X}) = [0.20469, 0.36084, 0.46577, 0.56653, 0.71017]$$

Particle 2 error:

$$error_2(8) = \sum_{n=1}^5 (d(x_n) - f_{2,8}(x_n))^2 / 5 = 0.20611$$

Particle 2 fitness: $F_2(8) = 1/(\varepsilon + error_2(8)) = 4.6273$

Particle 3 output:

$$f_{3,8}(\mathbf{X}) = [0.91108, 0.72784, 0.58476, 0.49943, 0.48944]$$

Particle 3 error:

$$error_3(8) = \sum_{n=1}^5 (d(x_n) - f_{3,8}(x_n))^2 / 5 = 0.26627$$

Particle 3 fitness: $F_3(8) = 1/(\varepsilon + error_3(8)) = 3.6197$

Particle 4 output:

$$f_{4,8}(\mathbf{X}) = [-6.1257 \times 10^{-3}, 0.1772, 0.3622, 0.54236, 0.71114]$$

Particle 4 error:

$$error_4(8) = \sum_{n=1}^5 (d(x_n) - f_{4,8}(x_n))^2 / 5 = 0.26162$$

Particle 4 fitness: $F_4(8) = 1/(\varepsilon + error_4(8)) = 3.6816$

Particle 5 output:

$$f_{5,8}(\mathbf{X}) = [-0.2965, -0.30185, -0.3472, -0.3531, -0.23998]$$

Particle 5 error:

$$error_5(8) = \sum_{n=1}^5 \left(d(x_n) - f_{5,8}(x_n) \right)^2 / 5 = 1.0808$$

$$\text{Particle 5 fitness: } F_5(8) = 1/(\varepsilon + error_5(8)) = 0.91673$$

Particle 6 output:

$$f_{6,8}(\mathbf{X}) = [-0.353, -0.2339, -0.18726, -0.1576, -0.089573]$$

Particle 6 error:

$$error_6(8) = \sum_{n=1}^5 \left(d(x_n) - f_{6,8}(x_n) \right)^2 / 5 = 0.85642$$

$$\text{Particle 6 fitness: } F_6(8) = 1/(\varepsilon + error_6(8)) = 1.1542$$

$$\text{Lbest} = \begin{bmatrix} 0.6682 & -0.3800 & 0.0238 & -0.7440 \\ 0.5778 & -0.3800 & -0.0017 & -0.6358 \\ 0.6583 & -0.0585 & 0.1960 & -0.5591 \\ 0.5740 & -0.5144 & 0.0063 & -0.4616 \\ 0.3051 & -0.1771 & 0.1407 & -0.8094 \\ 0.3658 & -0.2799 & -0.0306 & -0.7119 \end{bmatrix}$$

$$\text{LBest fitness} = \begin{bmatrix} 4.9515 \\ 5.0679 \\ 4.8864 \\ 4.9367 \\ 3.4235 \\ 3.8555 \end{bmatrix}$$

At generation 8, the particle 2 has the highest fitness value among all the generations. Therefore, the GBest fitness is equal to the particle 2's fitness.

$$\text{GBest particle} = [0.57777, -0.3800, -1.7161 \times 10^{-3}, -0.63582]$$

$$\text{GBest fitness} = 5.0679$$

$$\text{With GBest, } \min error_{1 \text{ to } P}(8) = error_2(8) = 0.18732 > \delta = 1 \times$$

10^{-4} , so particle updates need to be continued and more generations are required.

Particle velocity update:

$$\begin{aligned}
 \begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{bmatrix} &= \begin{bmatrix} -0.0217 & 0 & -0.061 & 0.0259 \\ -0.1120 & 0 & -0.0317 & 0.1339 \\ -0.0735 & 0.5270 & 0.2660 & 0.3715 \\ -0.2118 & -0.2203 & -0.0450 & 0.5314 \\ -0.6524 & 0.3326 & 0.1753 & -0.0389 \\ -0.5530 & 0.1642 & -0.1055 & 0.1210 \end{bmatrix} \\
 &+ 1 \times 0.9469 \times \left(\begin{bmatrix} 0.6882 & -0.3800 & \dots & -0.7440 \\ 0.5778 & -0.3800 & \dots & -0.6358 \\ \vdots & \vdots & \ddots & \vdots \\ 0.3658 & -0.2799 & \dots & -0.7119 \end{bmatrix} \right. \\
 &\quad \left. - \begin{bmatrix} 0.6682 & -0.3800 & \dots & -0.7440 \\ 0.4658 & -0.3800 & \dots & -0.5019 \\ \vdots & \vdots & \ddots & \vdots \\ -0.1873 & -0.1157 & \dots & -0.5910 \end{bmatrix} \right) \\
 &+ 1 \times 0.80771 \times \left(\begin{bmatrix} 0.5778 & -0.3800 & \dots & -0.6358 \\ 0.5778 & -0.3800 & \dots & -0.6358 \\ \vdots & \vdots & \ddots & \vdots \\ 0.5778 & -0.3800 & \dots & -0.6358 \end{bmatrix} \right. \\
 &\quad \left. - \begin{bmatrix} 0.6682 & -0.3800 & \dots & -0.7440 \\ 0.4658 & -0.3800 & \dots & -0.5019 \\ \vdots & \vdots & \ddots & \vdots \\ -0.1873 & -0.1157 & \dots & -0.5910 \end{bmatrix} \right)
 \end{aligned}$$

Hence,

$$\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \\ v_6 \end{bmatrix} = \begin{bmatrix} -0.0948 & 0 & -0.0268 & 0.1133 \\ 0.08452 & 0 & 0.0239 & -0.1011 \\ -0.0095 & -0.6574 & -0.3605 & -0.3424 \\ 0.16289 & 0.27482 & 0.0275 & -0.5417 \\ 0.71252 & -0.4149 & -0.2473 & 0.1696 \\ 0.58859 & -0.2048 & 0.1030 & -0.0298 \end{bmatrix}$$

Particle position update:

$$\begin{aligned}
 \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix} &= \begin{bmatrix} 0.6682 & -0.3800 & \cdots & -0.7440 \\ 0.4658 & -0.3800 & \cdots & -0.5019 \\ \vdots & \vdots & \ddots & \vdots \\ -0.1873 & -0.1157 & \cdots & -0.5910 \end{bmatrix} \\
 &+ \begin{bmatrix} -0.0948 & 0 & \cdots & 0.1133 \\ 0.08452 & 0 & \cdots & -0.1011 \\ \vdots & \vdots & \ddots & \vdots \\ 0.58859 & -0.2048 & \cdots & -0.0298 \end{bmatrix} \\
 &= \begin{bmatrix} 0.5735 & -0.3800 & -0.0029 & -0.6307 \\ 0.5503 & -0.3800 & -0.0095 & -0.6030 \\ 0.5752 & -0.1889 & 0.1016 & -0.5299 \\ 0.5251 & -0.4599 & -0.0112 & -0.4719 \\ 0.3653 & -0.2594 & 0.0687 & -0.6787 \\ 0.4013 & -0.3205 & -0.0332 & -0.6208 \end{bmatrix}
 \end{aligned}$$

This is the end of generation 8. As the optimization goes on, the particle positions evolve and approach the optimal solution. The PSO algorithm continues until generation 1579.

...

Generation 1579 (PSO converged)

The new particle positions at the start of generation 1579 are

$$\begin{bmatrix} 1.0002 & -2.3650 & -2.9899 & 9.8362 \\ 1.0010 & -2.3738 & -2.9891 & 9.8657 \\ 1.0004 & -2.3664 & -2.9898 & 9.8405 \\ 1.0009 & -2.3698 & -2.9919 & 9.8488 \\ 1.0009 & -2.3732 & -2.9891 & 9.8644 \\ 1.0009 & -2.3732 & -2.9891 & 9.8644 \end{bmatrix}$$

Particle 1 output:

$$f_{1,1579}(\mathbf{X}) = [0.29977, 0.37579, 1.0002, 1.2509, 0.20568]$$

Particle 1 error:

$$error_1(1579) = \sum_{n=1}^5 (d(x_n) - f_{1,1579}(x_n))^2 / 5 = 2.2123 \times 10^{-5}$$

$$\text{Particle 1 fitness: } F_1(1579) = 1/(\varepsilon + error_1(1579)) = 99.7793$$

Particle 2 output:

$$f_{2,1579}(\mathbf{X}) = [0.30002, 0.37486, 1.001, 1.2534, 0.20736]$$

Particle 2 error:

$$error_2(1579) = \sum_{n=1}^5 (d(x_n) - f_{2,1579}(x_n))^2 / 5 = 2.0069 \times 10^{-5}$$

$$\text{Particle 2 fitness: } F_2(1579) = 1/(\varepsilon + error_2(1579)) = 99.7997$$

Particle 3 output:

$$f_{3,1579}(\mathbf{X}) = [0.29977, 0.37566, 1.0004, 1.2514, 0.20607]$$

Particle 3 error:

$$error_3(1579) = \sum_{n=1}^5 (d(x_n) - f_{3,1579}(x_n))^2 / 5 = 2.1747 \times 10^{-5}$$

$$\text{Particle 3 fitness: } F_3(1579) = 1/(\varepsilon + error_3(1579)) = 99.7830$$

Particle 4 output:

$$f_{4,1579}(\mathbf{X}) = [0.29918, 0.37539, 1.0009, 1.2525, 0.20675]$$

Particle 4 error:

$$error_4(1579) = \sum_{n=1}^5 (d(x_n) - f_{4,1579}(x_n))^2 / 5 = 2.1065 \times 10^{-5}$$

$$\text{Particle 4 fitness: } F_4(1579) = 1/(\varepsilon + error_4(1579)) = 99.7898$$

Particle 5 output:

$$f_{5,1579}(\mathbf{X}) = [0.30003, 0.37489, 1.0009, 1.2532, 0.20718]$$

Particle 5 error:

$$error_5(1579) = \sum_{n=1}^5 (d(x_n) - f_{5,1579}(x_n))^2 / 5 = 1.9800 \times 10^{-5}$$

$$\text{Particle 5 fitness: } F_5(1579) = 1/(\varepsilon + error_5(1579)) = 99.8024$$

Particle 6 output:

$$f_{6,1579}(\mathbf{X}) = [0.30003, 0.37489, 1.0009, 1.2532, 0.20718]$$

Particle 6 error:

$$error_6(1579) = \sum_{n=1}^5 (d(x_n) - f_{6,1579}(x_n))^2 / 5 = 1.9800 \times 10^{-5}$$

$$\text{Particle 6 fitness: } F_6(1579) = 1/(\varepsilon + error_6(1579)) = 99.8024$$

$$Lbest = \begin{bmatrix} 1.0004 & -2.3687 & -2.9869 & 9.8499 \\ 1.0010 & -2.3738 & -2.9891 & 9.8657 \\ 1.0004 & -2.3664 & -2.9898 & 9.8405 \\ 1.0009 & -2.3698 & -2.9919 & 9.8488 \\ 1.0009 & -2.3732 & -2.9891 & 9.8644 \\ 1.0009 & -2.3732 & -2.9891 & 9.8644 \end{bmatrix}$$

$$LBest \text{ fitness} = \begin{bmatrix} 99.7837 \\ 99.7997 \\ 99.7830 \\ 99.7898 \\ 99.8024 \\ 99.8024 \end{bmatrix}$$

At generation 1579, the particle 5 and 6 has the highest fitness value among all the generations. Therefore, the GBest fitness is equal to the particle 5 and 6's fitness.

$$GBest \text{ particle} = [1.0009, -2.3732, -2.9891, 9.8644]$$

GBest fitness = 99.8024

With GBest, $\min error_{1 to P}(1579) = error_{5,6}(1579) = 1.9800 \times 10^{-5} < \delta = 1 \times 10^{-4}$, convergence criterion has been met, no more generation is required.

Thus the optimized polynomial coefficients are $[1.0009, -2.3732, -2.9891, 9.8644]$.

The convergence property of the PSO can be observed in Figure P15.2.1 as the optimized polynomial function gradually approaches the optimal polynomial function.

To evaluate the performance of the optimized function, forty uniformly spread values of x in between any two nearest neighbors of the five fixed values of input x are used to calculate the mean squared error MSE_q between the optimal function $g_{optimal}(x_q)$ and the optimized function $g_{optimized}(x_q)$ as

$$MSE_q = \sum_{q=1}^{40} (g_{optimal}(x_q) - g_{optimized}(x_q))^2 / 40 = 1.349 \times 10^{-5}$$

The MSE_q value obtained indicates a satisfactory optimized solution for the interpolation problem has been obtained. Depending on the requirements of an application, a smaller or larger value of the convergence criterion δ could be used.

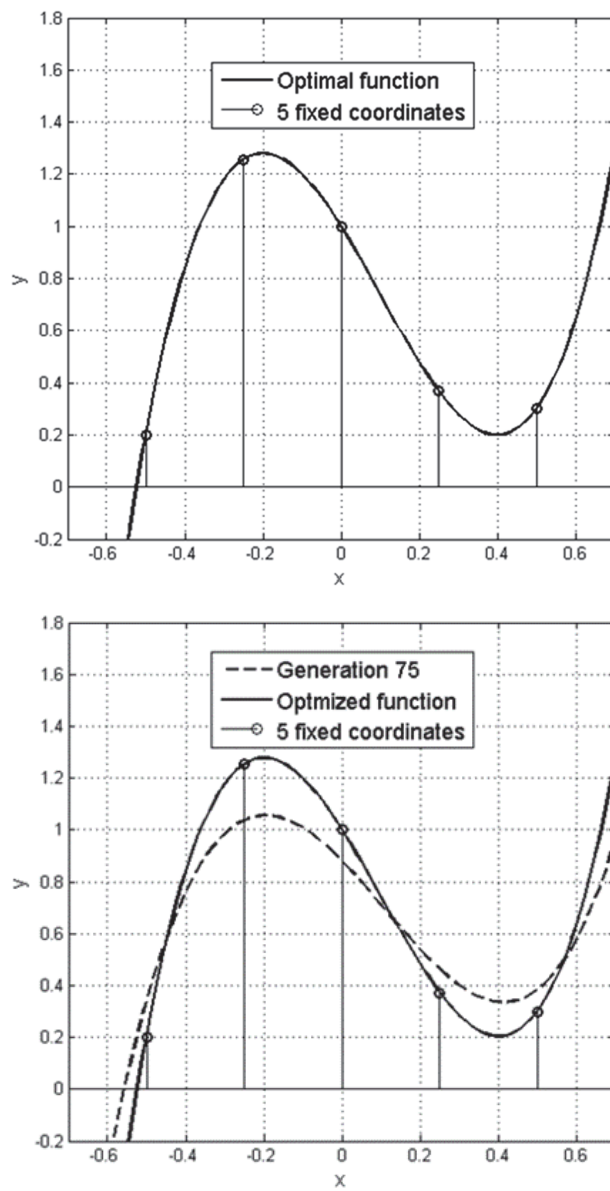


Figure P15.2.1 Optimal polynomial function (Top). Optimized polynomial function at generations 75 and 1579 (Bottom).

Chapter 17

Coding, Initialization, and Constraints

Problem 17.1 (Calculation of maximum 5-parameter codes of bipolar bits)

Consider a 5-parameter coding of 5 single-parameters $\mathbf{V}_1$ to $\mathbf{V}_5$, assume each single-parameter is of the same length $L = 6$ and the same set of $\mathbf{V}_{max}$ and $\mathbf{V}_{min}$ in which $\mathbf{V}_{min} = [-1, -1, -1, -1, -1, -1]$ and $\mathbf{V}_{max} = [+1, +1, +1, +1, +1, +1]$ is used, show one possible 5-parameter coding and calculate the total number of possible codes.

Solution

Table P17.1.1 5-parameter code example.

$\mathbf{V}$	$[\mathbf{V}_1, \mathbf{V}_2, \mathbf{V}_3, \mathbf{V}_4, \mathbf{V}_5]$
$\mathbf{V}_1$	$[+1, -1, -1, -1, +1, -1]$
$\mathbf{V}_2$	$[+1, -1, +1, -1, +1, -1]$
$\mathbf{V}_3$	$[+1, -1, -1, -1, +1, +1]$
$\mathbf{V}_4$	$[+1, +1, -1, +1, -1, +1]$
$\mathbf{V}_5$	$[-1, -1, +1, +1, +1, -1]$

Note that each parameter can have $2^6 = 64$ codes. The total number of possible codes for such 5-parameter coding is $64^5 = 1073741824$.

Problem 17.2 (Improved parameter initialization method with continuous and discrete values)

For optimization methods including backpropagation algorithms, evolutionary algorithms, and gradient-based algorithms, a method to determine an initial solution or the initial set of solutions is required.

Suggest an improved parameter initialization method suitable to be used for initial continuous and discrete parameters and describe its features.

Problem 17.3 (Violating-index-based objective function)

Let $\mathbf{x}$ represent a parameter vector to be optimized. Consider a constrained minimization problem in which the objective function (or error function) $e(\mathbf{x})$ is to be minimized subject to constraints

$$c_i(\mathbf{x}) \leq 0 \quad \text{for } i = 1, 2, \dots, C \quad (\text{P17.3.1})$$

Depending on the extent to which its given constraints are violated, a violation index $v(\mathbf{x}_p)$ can be defined for an individual parameter vector $\mathbf{x}_p$ as

$$v(\mathbf{x}_p) = \sum_{i=1}^C c'_i(\mathbf{x}_p) \quad \text{for } c'_i(\mathbf{x}_p) = \begin{cases} c_i(\mathbf{x}_p) & \text{if } c_i(\mathbf{x}_p) > 0 \\ 0 & \text{if } c_i(\mathbf{x}_p) \leq 0 \end{cases} \quad (\text{P17.3.2})$$

A smaller value for $v(\mathbf{x}_p)$ implies a better parameter vector.

Let v_{max} and v_{min} represent respectively the maximum and minimum violation indices in a population, and e_{max} be the worst objective value among those acceptable parameter vectors.

If there exists no acceptable parameter vector in a population, then $e_{max} = v_{min}$.

Suggest an adjusted objective function to balance the selection between acceptable and unacceptable parameter vectors.

Derive and comment the adjusted objective functions for $v(\mathbf{x}_p) = v_{min}$, $v(\mathbf{x}_p) = v_{max}$, and $v(\mathbf{x}_p) = 0$.

Explain the condition $s_2 > s_1 > 1$.

Solution

One adjusted objective function incorporating the violation index $v(\mathbf{x}_p)$ is

$$e'(\mathbf{x}_p) = e(\mathbf{x}_p) + e_{max} \left[s_1 + (s_2 - s_1) \frac{v(\mathbf{x}_p) - v_{min}}{v_{max} - v_{min}} \right] \quad (\text{P17.3.3})$$

for $s_2 > s_1 > 1$

Hence, from (P17.3.3), we have

$$e'(\mathbf{x}_p) = e(\mathbf{x}_p) + s_1 e_{max} \quad (\text{P17.3.4})$$

for $v(\mathbf{x}_p) = v_{min}$

$$e'(\mathbf{x}_p) = e(\mathbf{x}_p) + s_2 e_{max} \quad (\text{P17.3.5})$$

for $v(\mathbf{x}_p) = v_{max}$

The minimum and maximum of the adjusted objective functions $e'(\mathbf{x}_p)$ of unacceptable parameter vectors will be normalized within a range of $[s_1 e_{max}, s_2 e_{max}]$ corresponding to $v(\mathbf{x}_p) = v_{min}$ and $v(\mathbf{x}_p) = v_{max}$.

Larger s_1 and s_2 values result a larger $e'(\mathbf{x}_p)$ and hence a smaller fitness value $f(\mathbf{x}_p)$, which reduces the probability of selecting unacceptable parameter vectors.

$$e'(\mathbf{x}_p) = e(\mathbf{x}_p) + e_{max} \left[\frac{s_1 v_{max} - s_2 v_{min}}{v_{max} - v_{min}} \right] \quad (\text{P17.3.6})$$

for $v(\mathbf{x}_p) = 0$

The adjusted objective functions $e'(\mathbf{x}_p)$ of acceptable parameter vectors will be normalized and depends on the values selected for s_1 and s_2 .

By trial and error, optimal s_1 and s_2 can be determined to offer a balance for selecting both acceptable and unacceptable parameter vectors.

For a minimization problem, we have $v_{max} < s_1 e_{max} < s_2 e_{max}$, this condition implies $s_2 > s_1 > 1$.

Chapter 18

Multiplierless Neural Networks

Problem 18.1 (Second-order nonlinear function derivations I)

Consider the second-order nonlinear function $H_{s1}(z)$ for constructing a sigmoid function as defined by

$$H_{s1}(z) = \begin{cases} z(\beta - \theta z) & \text{for } 0 \leq z \leq L \\ z(\beta + \theta z) & \text{for } -L \leq z < 0 \end{cases} \quad \begin{matrix} \text{(P18.1.1a)} \\ \text{(P18.1.1b)} \end{matrix}$$

Using (P18.1.1a), show the step-by-step derivations of the following relationships:

- (a) $\beta = 2\theta L$;
- (b) $\theta = 1/L^2$;
- (c) $\beta = 2/L$;
- (d) $L = 2$ for 45° slope at $z = 0$.

Solution

(a) Differentiating $H_{s1}(z)$ of (P18.1.1a) with respect to z , we have

$$\frac{\partial z(\beta - \theta z)}{\partial z} = \beta - 2\theta z$$

Since the derivative $\beta - 2\theta z = 0$ at $z = L$, we have

$$\beta - 2\theta L = 0$$

Hence

$$\beta = 2\theta L \quad \text{(P18.1.2)}$$

(b) Since $H_{s1}(z)$ of (P18.1.1a) equal to 1 at $z = L$, we have

$$L(\beta - \theta L) = 1 \quad (\text{P18.1.3})$$

Substituting (P18.1.2) into (P18.1.3), we obtain

$$L(2\theta L - \theta L) = 1$$

Hence

$$\theta = 1/L^2 \quad (\text{P18.1.4})$$

(c) Substituting (P18.1.4) into (P18.1.2), we obtain

$$\beta = 2/L \quad (\text{P18.1.5})$$

(d) Differentiating $H_{s1}(z)$ of (P18.1.1a) with respect to z , we have

$$\frac{\partial z(\beta - \theta z)}{\partial z} = \beta - 2\theta z$$

Since the derivative $\beta - 2\theta z = 1$ at $z = 0$, we have

$$\beta - 2\theta(0) = 1$$

Hence

$$\beta = 1 \quad (\text{P18.1.6})$$

Substituting (P18.1.6) into (P18.1.5), we obtain

$$L = 2 \quad (\text{P18.1.7})$$

Problem 18.2 (Second-order nonlinear function derivations II)

Consider the second-order nonlinear function $H_{s1}(z)$ for constructing a sigmoid function as defined by

$$H_{s1}(z) = \begin{cases} z(\beta - \theta z) & \text{for } 0 \leq z \leq L \\ z(\beta + \theta z) & \text{for } -L \leq z < 0 \end{cases} \quad (\text{P18.2.1a})$$

$$(\text{P18.2.1b})$$

Using (P18.2.1b), show the step-by-step derivations of the following relationships:

$$(a) \beta = 2\theta L;$$

(b) $\theta = 1/L^2$;

(c) $\beta = 2/L$;

(d) $L = 2$ for 45° slope at $z = 0$.

Solution

(a) Differentiating $H_{s1}(z)$ of (P18.2.1b) with respect to z , we have

$$\frac{\partial z(\beta + \theta z)}{\partial z} = \beta + 2\theta z$$

Since the derivative $\beta + 2\theta z = 0$ at $z = -L$, we have

$$\beta + 2\theta(-L) = 0$$

Hence

$$\beta = 2\theta L \quad (\text{P18.2.2})$$

(b) Since $H_{s1}(z)$ of (P18.2.1b) equal to -1 at $z = -L$, we have

$$(-L)(\beta + \theta(-L)) = -1 \quad (\text{P18.2.3})$$

Substituting (P18.2.2) into (P18.2.3), we obtain

$$(-L)(2\theta L + \theta(-L)) = -1$$

Hence

$$\theta = 1/L^2 \quad (\text{P18.2.4})$$

(c) Substituting (P18.2.4) into (P18.2.2), we obtain

$$\beta = 2/L \quad (\text{P18.2.5})$$

(d) Differentiating $H_{s1}(z)$ of (P18.2.1b) with respect to z , we have

$$\frac{\partial z(\beta + \theta z)}{\partial z} = \beta + 2\theta z$$

Since the derivative $\beta + 2\theta z = 1$ at $z = 0$, we have

$$\beta + 2\theta(0) = 1$$

Hence

$$\beta = 1 \quad (\text{P18.2.6})$$

Substituting (P18.2.6) into (P18.2.5), we obtain

$$L = 2 \quad (\text{P18.2.7})$$

Description

“Matlab and Worked Problems in Intelligent Computing and System Design” is an accompanying eBook for the “Intelligent Computing and System Design” eBook written by the same author and are designed to be used together to reinforce learning.

These two eBooks form a set of senior undergraduate and graduate textbooks on Intelligent Computing and System Design for students in engineering, computer science, science, and business, and other programs; and contain fundamental and advanced topics to meet the curriculum requirements of both undergraduate and graduate courses.

These two eBooks are also suitable for engineers and computing professionals who want to self-learn this interesting and useful subject.

No specific prerequisites are required but a background of elementary courses in linear algebra and calculus would be useful.

Key Features

1. A comprehensive coverage of the essential topics on intelligent computing plus system design.
2. Topics are clearly described and explained with the aid of step-by-step hand-calculation problems, Matlab problems, equations, and self-explanatory figures and tables.
3. The eBook contains 68 Matlab and worked problems.
4. Supplement the main eBook on “Intelligent Computing and System Design” to reinforce learning.
5. A clickable table of contents allows a quick link of a specific chapter and problem.
6. The eBook has been designed and formatted to be readable and usable directly by an instructor in lecture and tutorial classes.
7. The contents of this eBook have been developed under research and classroom environments for courses taught by the author.

dfisp.org

